МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Направление подготовки: 10.03.01 – Информационная безопасность.
Образовательная программа: Технологии защиты информации.

Дисциплина:
«*Информационная безопасность баз данных*»

КУРСОВАЯ РАБОТА.

на тему *«Информационная система управления доступом в IT-компании»*

Выполнил студент:
группа/поток: N3348 / ИББД_ТЗИ_N3 1.2
Жук Анастасия Валерьевна / ____________
ФИО *Подпись*

Проверил:
Салихов Максим Русланович / ____________
ФИО *Подпись*

____
Отметка о выполнении (один из вариантов: отлично, хорошо, удовлетворительно)

____
Дата

Санкт-Петербург
2026г.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ЗАДАНИЕ НА КУРСОВОЙ ПРОЕКТ (РАБОТУ)

Студент Жук А. В.
(Фамилия, И., О.)
Факультет Безопасности информационных технологий (БИТ)
Группа N3348
Направление (специальность) 10.03.01 Информационная безопасность
Руководитель Салихов М. Р., преподаватель факультета БИТ
(Фамилия, И.О., должность, ученое звание, степень)
Дисциплина Информационная безопасность баз данных

Наименование темы Информационная система управления доступом в IT-компании

Задание Осуществить инфологическое моделирование базы данных, реализовать базу данных в системе управления базами данных, разработать систему защиты, многопользовательский интерфейс, реализовать веб-интерфейс настроить резервное копирование и восстановление.

Краткие методические указания Проанализировать и смоделировать базу данных; реализовать базу данных в системе управления базами данных PostgreSQL; обеспечить безопасность данных с помощью хэширования и логирования; разработать роли для взаимодействия с базой данных; написать веб-приложение на FastAPI; выполнить настройку резервного копирования.

Содержание пояснительной записки Анализ предметной области и проектирование базы данных, реализация базы данных в рамках системы управления базами данных, разработка примитивных систем логирования и защиты, реализация уровня приложения многопользовательским доступом, настройка резервирования и восстановления.

Рекомендуемая литература 1. Основы технологий баз данных: учебное пособие / Б. А. Новиков, Е. А. Горшкова, Н. Г. Графеева; под ред. Е. В. Рогова. — 2-е изд. — М.: ДМК Пресс, 2020. — 582 с.
2. Заколдаев Д.А., Кармановский Н.С., Комаров И.И., Филькова И.А. Методические указания по выполнению научно-исследовательской и выпускной квалификационной работы магистрантов: Учебно-методическое пособие. - Санкт-Петербург: Университет ИТМО, 2024. - 41 с. - экз. Режим доступа: https://books.ifmo.ru/book/2728/metodicheskie_ukazaniya_po_vypolneniyu_nauchno-issledovatelskoy_i_vypusknoy_kvalifikacionnoy_raboty_magistrantov_:_uchebno-metodicheskoe_posobie..htm.

Руководитель
Подпись, дата

Студент 08.01.2026
Подпись, дата

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ГРАФИК ВЫПОЛНЕНИЯ КУРСОВОГО ПРОЕКТА (РАБОТЫ)

Студент Жук А. В.
(Фамилия, И.О.)
Факультет Безопасности информационных технологий (БИТ)
Группа N3348
Направление (специальность) 10.03.01 Информационная безопасность
Руководитель Салихов М. Р., преподаватель факультета БИТ
(Фамилия, И.О., место работы, должность, ученое звание, степень)
Дисциплина Информационная безопасность баз данных

Наименование темы Информационная система управления доступом в IT-компании

№ п/п	Наименование этапа	Дата завершения		Оценка и подпись руководителя
		Планируемая	Фактическая	
1	Инфологическое моделирование баз данных по методу «сущность-связь»	28.09.2025	30.12.2025	
2	Реализация БД в рамках СУБД	12.10.2025	31.12.2025	
3	Защита базы данных	02.11.2025	02.01.2026	
4	Реализация сервиса для взаимодействия с разработанной базой данных	23.11.2025	08.01.2026	
5	Резервирование БД и восстановление по контрольным точкам	14.12.2025	08.01.2026	

Руководитель ______________________
подпись, дата

Студент 08.01.2026
подпись, дата

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

АННОТАЦИЯ НА КУРСОВОЙ ПРОЕКТ (РАБОТУ)

Студент Жук А. В.
(Фамилия, И.О.)
Факультет Безопасности информационных технологий (БИТ)
Группа N3348
Направление (специальность) 10.03.01 Информационная безопасность
Руководитель Салихов М. Р., преподаватель факультета БИТ
(Фамилия, И.О., место работы, должность, ученое звание, степень)
Дисциплина Информационная безопасность баз данных

Наименование темы Информационная система управления доступом в IT-компании

ХАРАКТЕРИСТИКА КУРСОВОГО ПРОЕКТА (РАБОТЫ)

1. Цель и задачи работы
■ Предложены студентом
■ Сформулированы при участии студента
☐ Определены руководителем

Цель работы: разработать защищенную, отказоустойчивую информационную систему управления доступом для IT-компании с полным жизненным циклом: от инфологического проектирования и реализации базы данных до создания веб-интерфейса с ролевой моделью и механизмами резервного копирования.
Задачи: Проанализировать и смоделировать базу данных; реализовать базу данных в системе управления базами данных PostgreSQL; обеспечить безопасность данных с помощью хэширования и логирования; разработать роли для взаимодействия с базой данных; написать веб-приложение на FastAPI; выполнить настройку резервного копирования.

2. Характер работы
☐ Расчет
■ Конструирование
■ Моделирование
☐ Другое, ____________________

4. Содержание работы

Анализ предметной области и проектирование базы данных, реализация базы данных в рамках системы управления базами данных, разработка примитивных систем логирования и защиты, реализация уровня приложения многопользовательским доступом, настройка резервирования и восстановления.

5. Выводы

В результате курсовой работы был успешно создан и протестирован комплексный, многоуровневый прототип защищенной системы управления доступом на основе PostgreSQL и FastAPI, демонстрирующий практическую применимость интегрированного подхода к проектированию безопасности данных – от инфологической модели и встроенных механизмов защиты СУБД до реализации сквозной ролевой авторизации в веб-интерфейсе, что подтверждает возможность построения отказоустойчивых корпоративных решений с использованием современных открытых технологий.

Студент ____________________
(подпись)

Руководитель ____________________
(подпись)

«08» января 2026 г.

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ.............................. 7

ВВЕДЕНИЕ.. 8

ХОД РАБОТЫ.. 11

1 ИНФОЛОГИЧЕСКОЕ МОДЕЛИРОВАНИЕ БАЗ ДАННЫХ ПО МЕТОДУ «СУЩНОСТЬ-СВЯЗЬ».. 11

1.1 Описание ограничений на сущности моделируемой базы данных... 13

1.2 Выделение сущностей и построение ER-диаграммы........................ 15

1.3 Преобразование ER-диаграммы в схему отношений......................... 21

1.4 Приведение схемы предварительных отношений к 3НФ.................. 24

1.5 Моделирование уровня представлений ИС IT-компании.................. 26

1.6 Вывод.. 31

2 РЕАЛИЗАЦИЯ БД В РАМКАХ СУБД.. 32

2.1 Выбор СУБД... 32

2.2 Создание разработанной БД... 33

2.2.1 Создание базы данных... 33

2.2.2 Создание таблиц согласно схеме отношений............................... 34

2.2.3 Установка взаимосвязей таблиц. Индексирование таблиц......... 42

2.2.4 Заполнение таблиц данными.. 48

2.3 Создание представлений... 53

2.4 Вывод.. 63

3 ЗАЩИТА БД... 64

3.1 Задачи по мониторингу операций в БД... 64

3.2 Задачи по шифрованию данных... 69

3.3 Задачи по разграничению доступа в БД.. 72

3.4 Вывод.. 114

4 РЕАЛИЗАЦИЯ СЕРВИСА ДЛЯ ВЗАИМОДЕЙСТВИЯ С

РАЗРАБОТАННОЙ БД.. 115
4.1 Технологический стек.. 115
4.2 Архитектура приложения... 117
4.3 Реализация многопользовательского доступа..................................118
4.4 Реализация сложных SQL-запросов с GROUP BY.......................... 129
4.5 Запуск приложение и использование интерфейса............................138
4.6 Вывод..144
5 РЕЗЕРВИРОВАНИЕ БД И ВОССТАНОВЛЕНИЕ ПО КОНТРОЛЬНЫМ ТОЧКАМ.. 145
5.1 Создание резервной копии БД...145
5.2 Внесение изменений в таблицы...148
5.3 Процесс отката к контрольной точке..150
5.4 Анализ откатанных изменений через логирование..........................153
5.5 Вывод..154
ЗАКЛЮЧЕНИЕ...155
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....................................157

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

БД – база данных;

СУБД – система управления базами данных;

ИС – информационная система;

IT – Information Technology (информационные технологии);

ER-диаграмма (ERD) – Entity-Relationship Diagram (диаграмма «сущность-связь»);

1НФ – первая нормальная форма;

2НФ – вторая нормальная форма;

3НФ – третья нормальная форма;

DML – Data Manipulation Language;

AES-256 – Advanced Encryption Standard;

ИБ – информационная безопасность;

API – Application Programming Interface;

SQL – Structured Query Language;

JWT – JSON Web Token;

HTTP – Hypertext Transfer Protocol;

CSS – Cascading Style Sheets (каскадные таблицы стилей);

JS – JavaScript;

ORM – Object-Relational Mapping (объектно-реляционное отображение).

ВВЕДЕНИЕ

В условиях цифровой трансформации и роста киберугроз информационная безопасность становится критически важным аспектом функционирования любой организации, особенно в IT-секторе. Управление доступом к корпоративным ресурсам – физическим и информационным – является фундаментом системы защиты данных. Традиционные подходы, основанные на ручном управлении правами и статических списках доступа, не справляются с динамикой современных бизнес-процессов, частыми изменениями в штате, ролевой структурой и статусами сотрудников. Это приводит к возникновению «теневых» привилегий, нарушению принципа минимальных необходимых прав и, как следствие, к повышению рисков утечек данных и инсайдерских атак.

Разработка специализированной информационной системы, автоматизирующей процессы управления доступом на основе актуальных ролей, командных принадлежностей и текущих статусов сотрудников, является насущной практической задачей. Такая система должна обеспечивать не только гибкое и точное разграничение прав, но и обладать встроенными механизмами безопасности на всех уровнях: от архитектуры базы данных и криптографической защиты до безопасного веб-интерфейса и процедур восстановления после сбоев. Интеграция принципов информационной безопасности на этапе проектирования, а не как надстройки, является современным и наиболее эффективным подходом (Security by Design).

Целью данной курсовой работы является разработка комплексной, защищенной и отказоустойчивой информационной системы управления доступом для IT-компании. Работа направлена на сквозную практическую реализацию: от инфологического моделирования и создания безопасной базы данных в СУБД PostgreSQL до построения веб-приложения с ролевой моделью и механизмами оперативного восстановления данных. Ключевым результатом должен стать функционирующий прототип системы, демонстрирующий целостный подход к обеспечению конфиденциальности, целостности и доступности данных.

Для достижения поставленной цели в работе решаются следующие **задачи**:

- проектирование структуры данных: проведение анализа предметной области, выделение сущностей и связей, построение ER-диаграммы и ее трансформация в нормализованную (до 3НФ) реляционную схему с разработкой специализированных представлений для различных категорий пользователей;
- физическая реализация и администрирование БД: создание базы данных в СУБД PostgreSQL, реализация схемы с использованием DDL-скриптов, настройка индексов, ограничений целостности и заполнение тестовыми данными с применением безопасного хэширования паролей;
- внедрение механизмов защиты на уровне СУБД: реализация системы аудита DML-операций с помощью триггеров, шифрование конфиденциальных данных средствами pgcrypto, построение детализированной ролевой модели доступа на основе групповых ролей и принципа минимальных привилегий;
- разработка клиентского веб-приложения: создание back-end на Python с использованием фреймворка FastAPI и ORM SQLAlchemy, реализация аутентификации на основе JWT-токенов и авторизации с интеграцией ролевой модели БД, а также разработка интуитивного веб-интерфейса с персонализированными дашбордами и аналитическими отчетами для пяти категорий пользователей;
- обеспечение отказоустойчивости: разработка и апробация процедур резервного копирования и восстановления базы данных с помощью утилит pg_dump/pg_restore, анализ возможностей системы логирования для расследования инцидентов;
- интеграция и тестирование: сборка и проверка работоспособности всех компонентов системы, формулировка выводов об эффективности реализованных решений и оценка перспектив развития системы.

Работа носит прикладной характер и объединяет знания и навыки в областях проектирования баз данных, системного администрирования СУБД, разработки безопасного программного обеспечения и обеспечения непрерывности бизнеса.

ХОД РАБОТЫ

1 ИНФОЛОГИЧЕСКОЕ МОДЕЛИРОВАНИЕ БАЗ ДАННЫХ ПО МЕТОДУ «СУЩНОСТЬ-СВЯЗЬ»

В качестве информационной системы, для которой будет составлена БД, выбрана «Информационная система управления доступом в IT-компании». Разрабатываемая БД является ядром системы управления доступом и **автоматизирует ключевые процессы безопасности**:

- централизованный учет сотрудников с поддержкой смены ролей и команд;
- динамический контроль доступа на основе текущих статусов сотрудников;
- гибкое управление ролевыми и командными политиками доступа;
- мониторинг активных и архивных команд/ролей;
- хранение статусов пользователей;
- полный аудит всех операций доступа.

Источниками данных для разрабатываемой БД являются:

- HR-система: персональные данные сотрудников, должности;
- IT-отдел: информация о ресурсах, системах, помещениях;
- администратор ИБ: политики безопасности, шаблоны доступов;
- руководители подразделений: назначение в команды, запросы на доступ;
- системы мониторинга: автоматическое логирование действий доступа.

Потребителями информации из разрабатываемой БД являются:

- администратор ИБ – полный контроль над системой, управление политиками;

- руководители команд – владение всеми ресурсами, необходимыми для работы команды, и регулирование состава команды;
- сотрудники – просмотр своих прав, текущего статуса;
- аудиторы – просмотр логов и отчетов для проверки соответствия требованиям безопасности;
- служба безопасности – мониторинг подозрительной активности через статусный контроль.

На выходе пользователи ИС, включающей БД, получают информацию в виде:

- персональные рабочие панели сотрудников с организованным представлением информации о ролях, статусах и доступных ресурсах;
- интерактивные таблицы с сортировкой и фильтрацией по параметрам системы доступа: текущим статусам сотрудников, типам ресурсов, уровням ролевых и командных прав, срокам действия временных доступов;
- отчеты по статистике доступов с учетом статусных ограничений и ролевых политик.

1.1 Описание ограничений на сущности моделируемой базы данных

Каждый сотрудник IT-компании работает в рамках одной основной роли, которая определяет его должностные обязанности и базовый уровень доступа к корпоративным ресурсам. Роли в компании иерархически организованы по уровням безопасности (мандатная модель разграничения доступа). Сотрудники могут объединяться в рабочие команды для выполнения проектных задач, при этом каждый сотрудник может состоять только в одной активной команде одновременно, что обеспечивает фокус на текущих проектных обязанностях.

Команды характеризуются следующими параметрами: уникальное название команды, идентификатор руководителя команды и статус активности. Ресурсы компании, включая физические помещения, информационные системы и оборудование, классифицируются по типам и уровням безопасности, что позволяет устанавливать гранулярную систему контроля доступа.

Критическим компонентом системы является контроль текущих статусов сотрудников. Каждый сотрудник имеет актуальный статус, который динамически влияет на его права доступа. Система поддерживает следующие основные статусы: активная работа, отпуск, больничный, уволен, отсутствие на рабочем месте. При смене статуса автоматически пересчитываются права доступа, что обеспечивает безопасность компании. Например, сотрудник в статусе «отпуск» не может получить доступ к критически важным системам.

Можно выделить следующие ограничения на связи и информацию для разрабатываемой БД:

- один сотрудник не может иметь несколько активных ролей одновременно, но обязательно должен иметь какую-нибудь одну;
- сотрудник не может состоять в нескольких активных командах в один момент времени, но может работать один;
- неактивные сотрудники автоматически лишаются всех прав доступа;

- доступ к ресурсу предоставляется только в том случае, если уровень безопасности роли сотрудника не ниже уровня безопасности ресурса;
- попытка доступа к ресурсу блокируется если текущий статус сотрудника не позволяет такой доступ;
- расформированные команды сохраняются в архиве, но не могут иметь новых членов или получать доступ к ресурсам;
- каждый сотрудник может иметь только один текущий статус в любой момент времени.

В результате БД должна обеспечивать решение следующих основных задач в рамках ИС управления доступом:

- централизованный учет и управление правами доступа сотрудников;
- динамический контроль доступа на основе актуальных статусов сотрудников;
- автоматическое предоставление и отзыв прав при изменениях в ролях, командах или статусах;
- полный аудит всех операций доступа с возможностью расследования инцидентов безопасности;
- обеспечение соответствия политикам информационной безопасности компании;
- оперативное предоставление актуальной информации о правах доступа в удобном для различных категорий пользователей виде.

1.2 Выделение сущностей и построение ER-диаграммы

На основе проведенного анализа ИС управления доступом для IT-компании можно выделить следующие сущности:

- «Сотрудник»,
- «Роль»,
- «Команда»,
- «Статус сотрудника»,
- «Физический ресурс»,
- «Информационный ресурс»,
- «Журнал доступа»,
- «Журнал статусов».

Сущность «Сотрудник» характеризуется атрибутами «ID сотрудника», «Логин», «Пароль», «ФИО сотрудника», «Электронная почта» и «Номер телефона». Ключом этой сущности является атрибут «ID сотрудника», так как каждый сотрудник имеет уникальный идентификатор в системе. Кроме того, каждый сотрудник связан с одной основной ролью и может быть прикреплен к одной команде.

Сущность «Роль» моделирует должностные позиции в компании и определяет уровень доступа сотрудников. Каждая роль имеет уникальные идентификатор и название, уровень безопасности и статус активности. Ключом этой сущности является атрибут «ID роли». Роли образуют иерархическую структуру, где уровень безопасности определяет максимально возможный доступ к ресурсам компании. Статус активности позволяет изменять ролевую концепцию компании, делая неактивными роли, которые заменяются новыми.

Сущность «Команда» представляет рабочие группы, создаваемые для выполнения проектных задач. Каждая команда имеет уникальные идентификатор и название, а также статус активности. Ключом этой сущности является атрибут «ID команды». Команды могут быть активными или расформированными, что влияет на возможность предоставления доступов к ресурсам. Также за каждой командой обязательно закрепляется единственный руководитель. Стоит также отметить, что нет фиксированного числа для количества членов команды.

Сущность «Статус сотрудника» отражает текущее состояние каждого сотрудника и динамически влияет на его права доступа. Эта сущность характеризуется атрибутами «Код актуального статуса», «Тип статуса», «Активность сотрудника». Ключом является «Код статуса». «Тип статуса» позволяет нам предвидеть отсутствие сотрудника на рабочем месте (то есть отсутствие сотрудника по уважительной причине). Этот атрибут может принимать значения «Отпуск», «Больничный», «Командировка», «Декретный отпуск», «Уволен» и «Активная работа». Будем считать, что эти данные заполняются автоматически из онлайн-календаря с отметками всех запланированных отсутствий сотрудников. Атрибут «Активность сотрудника» нужен для вычисления прогулов. Если атрибут «Тип статуса» принимает значение «Активная работа», а «Активность сотрудника» – 0 (1 – сотрудник в текущую дату активен, 0 – сотрудник не активен), то можно сделать вывод о прогуле сотрудника.

Сущность «Журнал статусов» хранит все статусы сотрудников. То есть сущность «Статус сотрудника» отражает лишь текущий статус, действующий на сегодняшний день. А вот журнал позволяет сохранять всю историю статусов. Эта сущность характеризуется атрибутами «Код статуса», «Дата», «Тип статуса» и «Активность сотрудника». Ключом является «Код статуса». В эту сущность переносятся данные из сущности «Статус сотрудника» каждый рабочий день.

Сущность «Физический ресурс» представляет все физические защищаемые активы компании: помещения, лаборатории, оборудование, материальные ценности. Ресурсы характеризуются атрибутами «ID физического ресурса», «Название физического ресурса» и «Уровень безопасности». Ключом является «ID физического ресурса», а уровень безопасности определяет необходимый минимальный уровень роли для доступа к данному ресурсу.

Сущность «Информационный ресурс» представляет все информационные защищаемые активы компании: базы данных, электронные документы, цифровые файлы, физические документы (бумажные носители), программные ресурсы. Ресурсы характеризуются атрибутами «ID ресурса», «Название ресурса» и «Уровень безопасности». Ключом является «ID ресурса», а уровень безопасности определяет необходимый минимальный уровень роли для доступа к данному ресурсу.

Сущность «Журнал доступа» фиксирует все попытки доступа к ресурсам компании. Эта сущность включает атрибуты «Код события», «Дата записи», «Действие», «Результат», «Причина». Ключом является «ID записи», причем каждая запись обязательно связана с сотрудником и ресурсом.

Проанализируем возможное взаимодействие объектов ИС, которое моделируется сущностями в разрабатываемой БД.

Между сущностями «Сотрудник» и «Роль» существует связь «Имеет» степени M:1. Это объясняется тем, что каждый сотрудник занимает одну основную должность в компании, но одна и та же роль может быть назначена нескольким сотрудникам. При этом класс принадлежности со стороны сотрудника является обязательным, так как каждый сотрудник должен иметь определенную роль, в то время как роль может существовать в системе без привязки к конкретному сотруднику (например, при создании новой позиции).

Сущности «Сотрудник» и «Команда» объединены связью «Состоит в» степени M:1. Это обусловлено тем, что сотрудник может работать в одной команде или индивидуально, но не может одновременно состоять в нескольких командах. Вместе с тем одна команда может включать нескольких сотрудников. Класс принадлежности со стороны команды является обязательным для активных команд, в то время как сотрудник может работать без прикрепления к команде. Также эти две сущности объединены связью «Руководит» степени 1:1. Это обусловлено тем, что сотрудник может руководить только одной командой.

Класс принадлежности со стороны команды является обязательным, так как каждая команда должна иметь руководителя, но со стороны сотрудника она не является обязательной, так как не каждый сотрудник в принципе состоит в команде, а тем более руководит ей.

Между сущностями «Сотрудник» и «Статус сотрудника» наблюдается связь «Имеет статус» степени 1:1. Это означает, что каждый сотрудник имеет только один актуальный статус на текущий момент времени. Класс принадлежности со стороны сотрудника является обязательным, так как каждый сотрудник должен иметь как минимум один статус в системе. Связь 1:1 выбрана, потому что в любой момент времени у сотрудника может быть только один действующий статус, который определяет его права доступа.

Сущность «Статус сотрудника» связана с сущностью «Журнал статусов» связью «Хранится» степени М:1. Это объясняется тем, что каждый актуальный статус сотрудника в течение своего жизненного цикла может меняться несколько раз, и каждое изменение фиксируется в журнале. Класс принадлежности со стороны «Журнала статусов» является обязательным, так как каждая запись в журнале должна относиться к конкретному статусу сотрудника. Со стороны «Статуса сотрудника» класс принадлежности также является обязательным, так как журнал существует для хранения истории статусов, а не для произвольных записей.

Сущность «Сотрудник» связана с сущностями «Физический ресурс» и «Информационный ресурс» через связь «Пытается использовать», которая реализуется через сущность «Журнал доступа». Это означает, что любая попытка доступа к ресурсу (физическому или информационному) фиксируется в журнале. Связь между «Сотрудник» и «Журнал доступа» имеет степень 1:М, так как один сотрудник может совершать множество попыток доступа. Аналогично, связь между «Физический ресурс» и «Журнал доступа» и между «Информационный ресурс» и «Журнал доступа» также имеет степень 1:М, так как один ресурс может быть объектом множества попыток доступа со стороны различных сотрудников. Классы принадлежности со стороны всех трех сущностей являются обязательными, так как каждая запись в журнале должна быть ассоциирована с конкретным сотрудником и конкретным ресурсом.

Наконец, сущность «Команда» связана с сущностями «Физический ресурс» и «Информационный ресурс» связью «Имеет доступ к» степени M:N. Это означает, что одна команда может иметь доступ к множеству ресурсов, а также один и тот же ресурс может быть доступен множеству команд. Данную связь нужно будет реализовывать через отдельное отношение. Класс принадлежности со стороны команды является необязательным, так как не все команды имеют прямой доступ к специфическим ресурсам (например, команды, работающие с общими корпоративными системами, не нуждаются в отдельных разрешениях). Однако, если запись существует в связующем отношении, то она обязательно должна быть связана с существующей командой и ресурсом. Класс принадлежности со стороны ресурса также является необязательным, так как некоторые ресурсы могут быть общими и не требовать явного разрешения для каждой команды (например, внутренний портал компании).

В соответствии с выделенными сущностями и связями между ними, ER-диаграмма для рассматриваемой предметной области будет выглядеть так, как показано на рисунке 1.

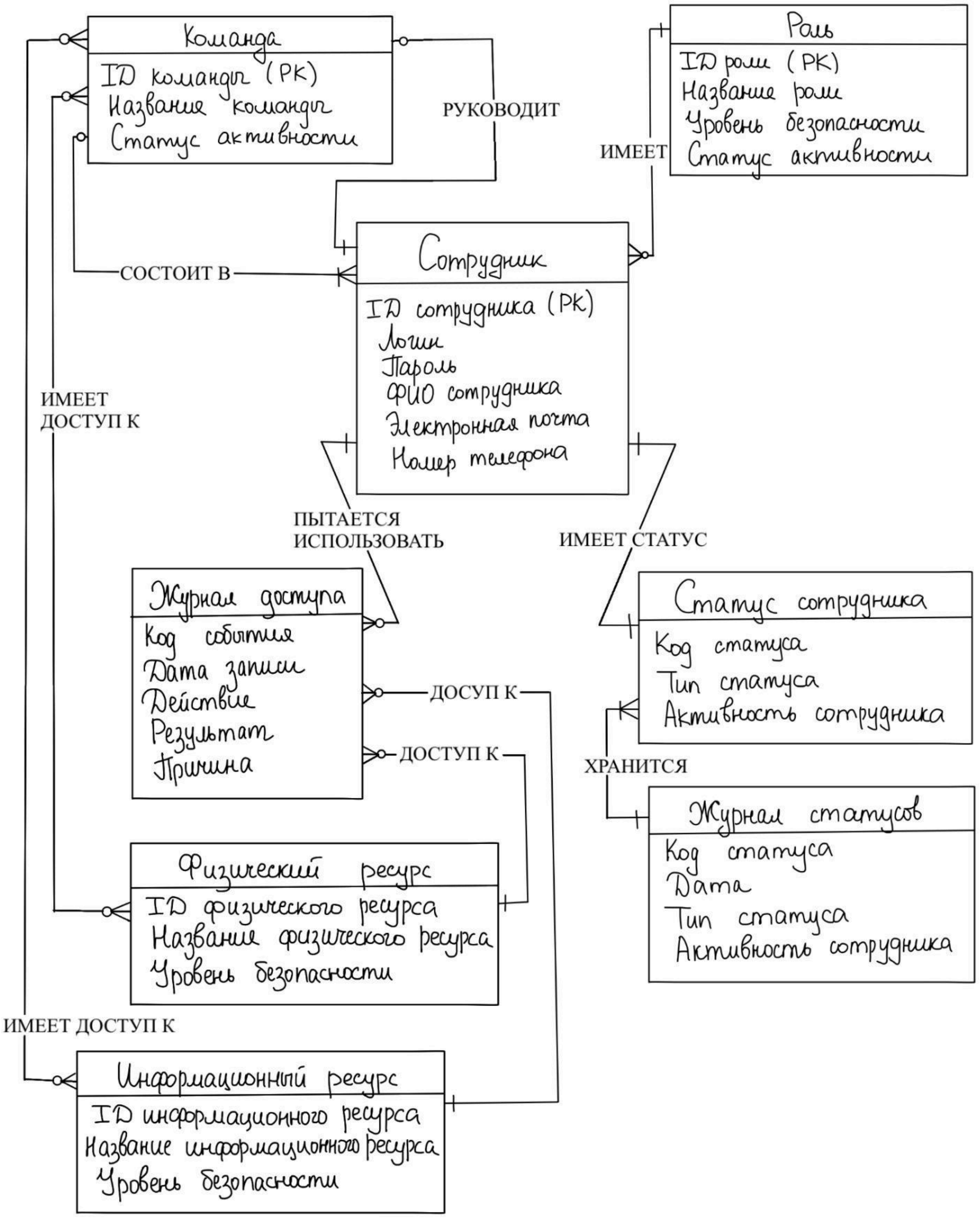

Команда
ID команды (PK)
Название команды
Статус активности
РУКОВОДИТ
Роль
ID роли (PK)
Название роли
Уровень безопасности
Статус активности
ИМЕЕТ
Сотрудник
СОСТОИТ В
ID сотрудника (PK)
Логин
Пароль
ФИО сотрудника
Электронная почта
Номер телефона
ИМЕЕТ
ДОСТУП К
ПЫТАЕТСЯ
ИСПОЛЬЗОВАТЬ
ИМЕЕТ СТАТУС
Журнал доступа
Код события
Дата записи
Действие
Результат
Причина
ДОСУП К
ДОСТУП К
Статус сотрудника
Код статуса
Тип статуса
Активность сотрудника
ХРАНИТСЯ
Журнал статусов
Код статуса
Дата
Тип статуса
Активность сотрудника
Физический ресурс
ID физического ресурса
Название физического ресурса
Уровень безопасности
ИМЕЕТ ДОСТУП К
Информационный ресурс
ID информационного ресурса
Название информационного ресурса
Уровень безопасности

Рисунок 1 – ER-диаграмма разрабатываемой БД

1.3 Преобразование ER-диаграммы в схему отношений

В соответствии с правилами формирования предварительных отношений, каждая сущность ER-диаграммы преобразуется в отдельное отношение (таблицу), а связи между сущностями реализуются путём добавления внешних ключей или создания отдельных связующих таблиц.

Связь «Имеет» между сущностями «Сотрудник» и «Роль» имеет степень 1:М и обязательный класс принадлежности со стороны многосвязной сущности «Сотрудник». Следовательно, в таблицу «Сотрудник», являющуюся зависимой, добавляется внешний ключ «ID роли», ссылающийся на первичный ключ таблицы «Роль».

Связь «Состоит в» между сущностями «Сотрудник» и «Команда» также имеет степень 1:М. Поскольку сотрудник может принадлежать только одной команде (или не принадлежать ни к какой), а команда может включать множество сотрудников, в таблицу «Сотрудник» добавляется внешний ключ «ID команды», ссылающийся на первичный ключ таблицы «Команда».

Дополнительно в таблице «Команда» присутствует атрибут «Руководитель», являющийся внешним ключом, ссылающимся на «ID сотрудника» в таблице «Сотрудник». Это отражает связь «Руководит» степени 1:1, где каждая команда имеет ровно одного руководителя, но не каждый сотрудник является руководителем.

Связь «Имеет статус» между сущностями «Сотрудник» и «Статус сотрудника» имеет степень 1:1. В этом случае в таблицу «Статус сотрудника» добавляется внешний ключ «ID сотрудника», ссылающийся на первичный ключ таблицы «Сотрудник», что позволяет однозначно связать текущий статус с конкретным сотрудником

Связь между сущностями «Статус сотрудника» и «Журнал статусов» имеет степень 1:М, так как один и тот же статус (в виде записи с кодом) может фиксироваться в журнале несколько раз (например, при ежедневном обновлении). Поэтому в таблицу «Журнал статусов» добавляется внешний ключ «Код статуса», ссылающийся на первичный ключ таблицы «Статус сотрудника».

Стоит также отметить, что первичные ключи этих отношений совпадаю. При обновлении статусов в таблице «Статус сотрудника» индексация не сбрасывается, а продолжается, поэтому можно гарантировать уникальность данных полей в обоих отношениях.

Связи между сущностями «Сотрудник», «Физический ресурс», «Информационный ресурс» и «Журнал доступа» реализуются через таблицу «Журнал доступа», которая содержит два внешних ключа: «ID сотрудника», ссылающийся на таблицу «Сотрудник», и «ID ресурса» — в зависимости от типа ресурса, ссылающийся либо на «ID физического ресурса», либо на «ID информационного ресурса». Для обеспечения корректной ссылочной целостности при использовании двух типов ресурсов в таблицу «Журнал доступа» добавляется атрибут «Тип ресурса» (physical или information).

Наконец, связь M:N между сущностями «Команда» и «Ресурс» (включая оба типа ресурсов) преобразуется в отдельное связующее отношение «Доступ команды к ресурсу». Данное отношение содержит первичный ключ «ID разрешения», а также два внешних ключа: «ID команды», ссылающийся на таблицу «Команда», и «ID ресурса», который может ссылаться на «ID физического ресурса» или «ID информационного ресурса». Аналогично предыдущей связи, здесь добавляется атрибут «Тип ресурса», который помогает идентифицировать ресурс.

Итоговая схема отношений представлена на рисунке 2.

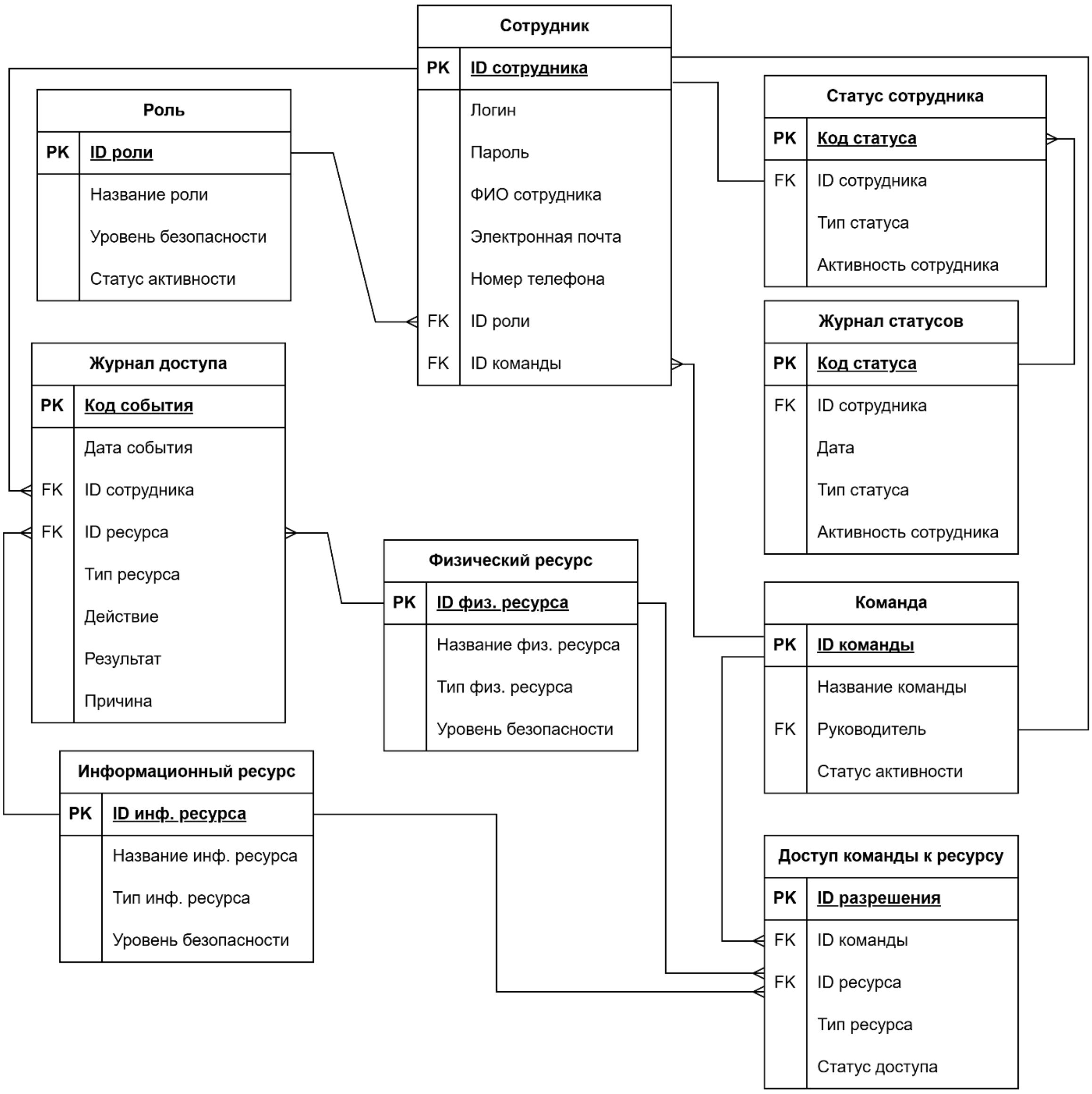

Сотрудник
PK ID сотрудника
Логин
Пароль
ФИО сотрудника
Электронная почта
Номер телефона
FK ID роли
FK ID команды
Роль
PK ID роли
Название роли
Уровень безопасности
Статус активности
Статус сотрудника
PK Код статуса
FK ID сотрудника
Тип статуса
Активность сотрудника
Журнал статусов
PK Код статуса
FK ID сотрудника
Дата
Тип статуса
Активность сотрудника
Журнал доступа
PK Код события
Дата события
FK ID сотрудника
FK ID ресурса
Тип ресурса
Действие
Результат
Причина
Физический ресурс
PK ID физ. ресурса
Название физ. ресурса
Тип физ. ресурса
Уровень безопасности
Команда
PK ID команды
Название команды
FK Руководитель
Статус активности
Информационный ресурс
PK ID инф. ресурса
Название инф. ресурса
Тип инф. ресурса
Уровень безопасности
Доступ команды к ресурсу
PK ID разрешения
FK ID команды
FK ID ресурса
Тип ресурса
Статус доступа

Рисунок 2 – Схема предварительных отношений

1.4 Приведение схемы предварительных отношений к 3НФ

Так как для проверки на нахождение отношения в одной из нормальных форм необходимо последовательно убедиться в выполнении условий всех предыдущих нормальных форм, начнем с первой нормальной формы (1НФ).

Отношение находится в первой нормальной форме, если все его атрибуты являются простыми (атомарными), то есть не содержат составных или повторяющихся значений.

Все отношения, представленные в схеме («Сотрудник», «Роль», «Команда», «Статус сотрудника», «Журнал статусов», «Физический ресурс», «Информационный ресурс», «Журнал доступа», «Доступ команды к ресурсу»), содержат только атомарные атрибуты:

- текстовые поля (например, «ФИО сотрудника», «Название ресурса») используются как единые значения, поскольку поиск и/или фильтрация по частям этих полей (например, по фамилии отдельно от имени) не требуется в рамках поставленных задач;
- даты и временные метки хранятся в стандартном формате СУБД;
- числовые идентификаторы являются скалярными значениями.

Следовательно, все отношения находятся в 1НФ.

Отношение находится во второй нормальной форме (2НФ), если оно находится в 1НФ и каждый неключевой атрибут функционально полно зависит от всего первичного ключа, а не от его части.

Во всех отношениях нашей схемы первичные ключи являются простыми (состоят из одного атрибута: «ID сотрудника», «ID роли», «ID команды», «Код статуса», «ID физического ресурса», «ID информационного ресурса», «Код события», «ID разрешения»). При простом первичном ключе любая функциональная зависимость автоматически является полной, так как отсутствуют «части» ключа. Таким образом, все отношения находятся во 2НФ.

Отношение находится в третьей нормальной форме (3НФ), если оно находится во 2НФ и каждый неключевой атрибут нетранзитивно зависит от первичного ключа, то есть отсутствуют зависимости вида PK — A — B, где A и B – неключевые атрибуты.

Проанализируем каждую таблицу:

– «Сотрудник»: неключевые атрибуты («Логин», «Пароль», «ФИО», и т.д.) зависят только от «ID сотрудника» и не определяют друг друга.

– «Роль»: атрибуты «Название роли», «Уровень безопасности», «Статус активности» зависят только от «ID роли».

– «Команда»: «Название команды», «Статус активности зависят» от ID команды; атрибут «Руководитель» – это внешний ключ, а не неключевой атрибут, определяющий другие поля.

– «Статус сотрудника» и «Журнал статусов»: все атрибуты («Тип статуса», «Активность сотрудника», «Дата») зависят напрямую от первичного ключа и не образуют транзитивных зависимостей.

– «Физический ресурс» и «Информационный ресурс»: атрибуты «Название» и «Уровень безопасности» зависят только от соответствующего ID.

– «Журнал доступа» и «Доступ команды к ресурсу»: неключевые атрибуты («Дата записи», «Действие», «Результат», «Причина», «Статус доступа» и др.) зависят от первичного ключа записи и не определяют друг друга.

Во всех таблицах отсутствуют транзитивные зависимости между неключевыми атрибутами. Следовательно, все отношения находятся в 3НФ.

Таким образом, разработанная схема отношений полностью удовлетворяет требованиям 3НФ и является нормализованной. Дополнительная декомпозиция не требуется.

1.5 Моделирование уровня представлений ИС IT-компании — Администратор ИБ

Представление 1. «Общий обзор прав доступа»

Общий обзор всех активных сотрудников, их ролей, команд и связанных ресурсов.

Список атрибутов, отображаемых в рамках представления:

- Логин (Сотрудник)
- ФИО (Сотрудник)
- Название роли (Роль)
- Уровень безопасности роли (Роль)
- Название команды (Команда)
- Тип ресурса (Физический / Информационный)
- Название ресурса (Ресурс)
- Уровень безопасности ресурса (Ресурс)

Позволяет администратору быстро оценить соответствие ролевой модели политике: «уровень роли ≥ уровня ресурса».

Представление 2. «Попытки нарушения политик безопасности»

Попытки доступа, нарушающие политики безопасности (например, роль ниже уровня ресурса или статус «отпуск»).

Список атрибутов, отображаемых в рамках представления:

- Логин (Сотрудник)
- Название роли (Роль)
- Уровень безопасности роли (Роль)
- Название ресурса (Ресурс)
- Уровень безопасности ресурса (Ресурс)
- Тип статуса (Статус сотрудника)

- Дата записи (Журнал доступа)
- Результат (Журнал доступа)
- Причина (Журнал доступа)

Используется для выявления системных недостатков в настройке доступа.

- **Руководители команд**

Представление 3. «Состав и статусы команды»

Состав команды с текущими статусами и активностью.

Список атрибутов, отображаемых в рамках представления:

- Название команды (Команда)
- ФИО сотрудника (Сотрудник)
- Логин (Сотрудник)
- Название роли (Роль)
- Тип статуса (Статус сотрудника)
- Активность (Статус сотрудника)
- Признак прогула (CASE: статус = «Активная работа» AND активность = 0)

Помогает контролировать явку и планировать задачи.

Представление 4. «Ресурсы, доступные команде»

Все ресурсы, доступные команде (физические и информационные).

Список атрибутов, отображаемых в рамках представления:

- Название команды (Команда)
- Тип ресурса (Физический / Информационный)
- Название ресурса (Ресурс)
- Уровень безопасности ресурса (Ресурс)
- Статус доступа (Доступ команды к ресурсу)

Для управления доступами: запрос новых ресурсов, отзыв уволенных сотрудников.

- **Сотрудники**

Представление 5. «Мои текущие права доступа»

Мои текущие права: роль, команда, доступные ресурсы.

Список атрибутов, отображаемых в рамках представления:

- Логин (Сотрудник)
- ФИО (Сотрудник)
- Название роли (Роль)
- Уровень безопасности роли (Роль)
- Название команды (Команда)
- Тип ресурса (Физический / Информационный)
- Название ресурса (Ресурс)

Персональная панель: «что мне доступно и почему».

Представление 6. «Мой статус и активность»

Мой текущий статус и активность + история статусов.

Список атрибутов, отображаемых в рамках представления:

- Логин (Сотрудник)
- Текущий тип статуса (Статус сотрудника)
- Активность (Статус сотрудника)
- История: Дата — Тип статуса (Журнал статусов)

Позволяет сотруднику видеть, как ИБ-система «видит» его активность.

- **Аудиторы**

Представление 7. «Подозрительные попытки доступа»

Подозрительные попытки доступа (статус ≠ «Активная работа»).

Список атрибутов, отображаемых в рамках представления:

- Логин (Сотрудник)

- Тип статуса (Статус сотрудника)
- Название ресурса (Ресурс)
- Тип ресурса (Ресурс)
- Дата записи (Журнал доступа)
- Действие (Журнал доступа)
- Результат (Журнал доступа)
- Причина (Журнал доступа)

Фокус на инцидентах: доступ из отпуска, после увольнения и т.д.

Представление 8. «Статистика доступов по статусам»

Статистика по статусам.

Список атрибутов, отображаемых в рамках представления:

- Тип статуса (Статус сотрудника)
- Общее число попыток (Журнал доступа)
- Успешных попыток (Журнал доступа)
- Неуспешных попыток (Журнал доступа)

Агрегированный отчет для аудита эффективности политики статус-контроля.

- **Служба безопасности**

Представление 9. «Потенциальные прогулы»

Сотрудники с признаками прогула (статус = «Активная работа», активность = 0).

Список атрибутов, отображаемых в рамках представления:

- ФИО (Сотрудник)
- Логин (Сотрудник)
- Название команды (Команда)
- Тип статуса (Статус сотрудника)

- Активность (Статус сотрудника)
- Последняя запись в Журнале доступа (дата, ресурс)

Выявление прогулов и несанкционированного отсутствия.

Представление 10. «Доступы после смены статуса»

Доступы, совершенные сразу после смены статуса на «уволен» / «отпуск».

Список атрибутов, отображаемых в рамках представления:

- Логин (Сотрудник)
- Старый статус — Новый статус (Журнала статусов)
- Дата смены статуса
- Первая попытка доступа после смены (дата, ресурс, результат)

Для расследования возможных утечек: «уволенный сотрудник все еще в системе».

1.6 Вывод

В ходе выполнения первого этапа работы была успешно спроектирована инфологическая модель БД для ИС управления доступом в IT-компании. Проведен системный анализ предметной области, выделены ключевые сущности и связи, построена ER-диаграмма, корректно преобразованная в схему отношений. Разработанная модель приведена к третьей нормальной форме, что подтверждает ее структурную целостность и отсутствие избыточности. Для всех выделенных классов потребителей созданы релевантные представления, обеспечивающие безопасный и удобный доступ к информации.

2 РЕАЛИЗАЦИЯ БД В РАМКАХ СУБД

2.1 Выбор СУБД

Для реализации разработанной базы данных выбрана СУБД PostgreSQL.

Обоснование выбора:

- поддержка расширенных функций безопасности (роли, шифрование, аудит), что соответствует тематике дисциплины;
- наличие встроенных средств для создания триггеров, представлений, индексов;
- широкая распространённость в индустрии и поддержка со стороны сообщества;
- совместимость с графическими инструментами администрирования (pgAdmin, DBeaver);
- возможность использования в последующих лабораторных работах (логирование, шифрование, разграничение доступа).

Дистрибутив PostgreSQL можно скачать с официального сайта https://www.postgresql.org/download/ вместе с графическим клиентом pgAdmin.

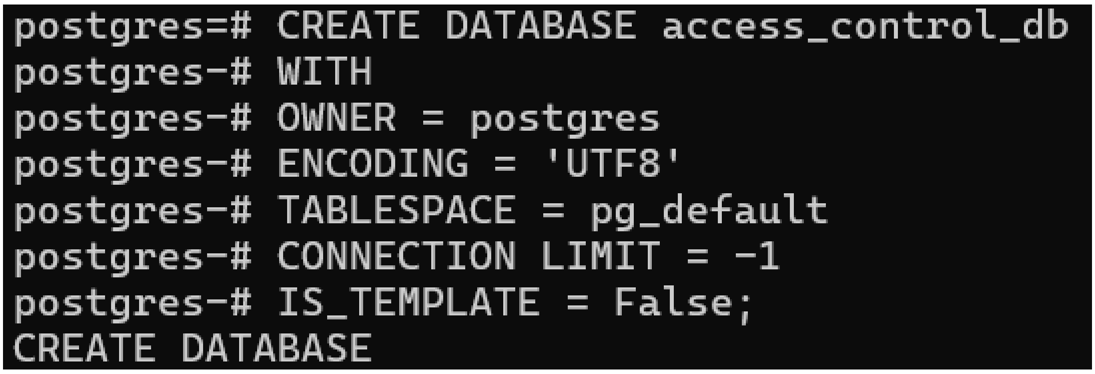
2.2 Создание разработанной БД

2.2.1 Создание базы данных

Код для создания БД выглядит следующим образом:

```
CREATE DATABASE access_control_db
   WITH
   OWNER = postgres
   ENCODING = 'UTF8'
   TABLESPACE = pg_default
   CONNECTION LIMIT = -1
   IS_TEMPLATE = False;
```

Процесс создания БД в psql и ответ системы представлены на рисунке 3.



```
postgres=# CREATE DATABASE access_control_db
postgres-# WITH
postgres-# OWNER = postgres
postgres-# ENCODING = 'UTF8'
postgres-# TABLESPACE = pg_default
postgres-# CONNECTION LIMIT = -1
postgres-# IS_TEMPLATE = False;
CREATE DATABASE
```

Рисунок 3 – Успешное создание БД в psql

2.2.2 Создание таблиц согласно схеме отношений

Напомню как выглядит схема предварительных отношений, полученная на первом этапе разработки (рисунок 4).

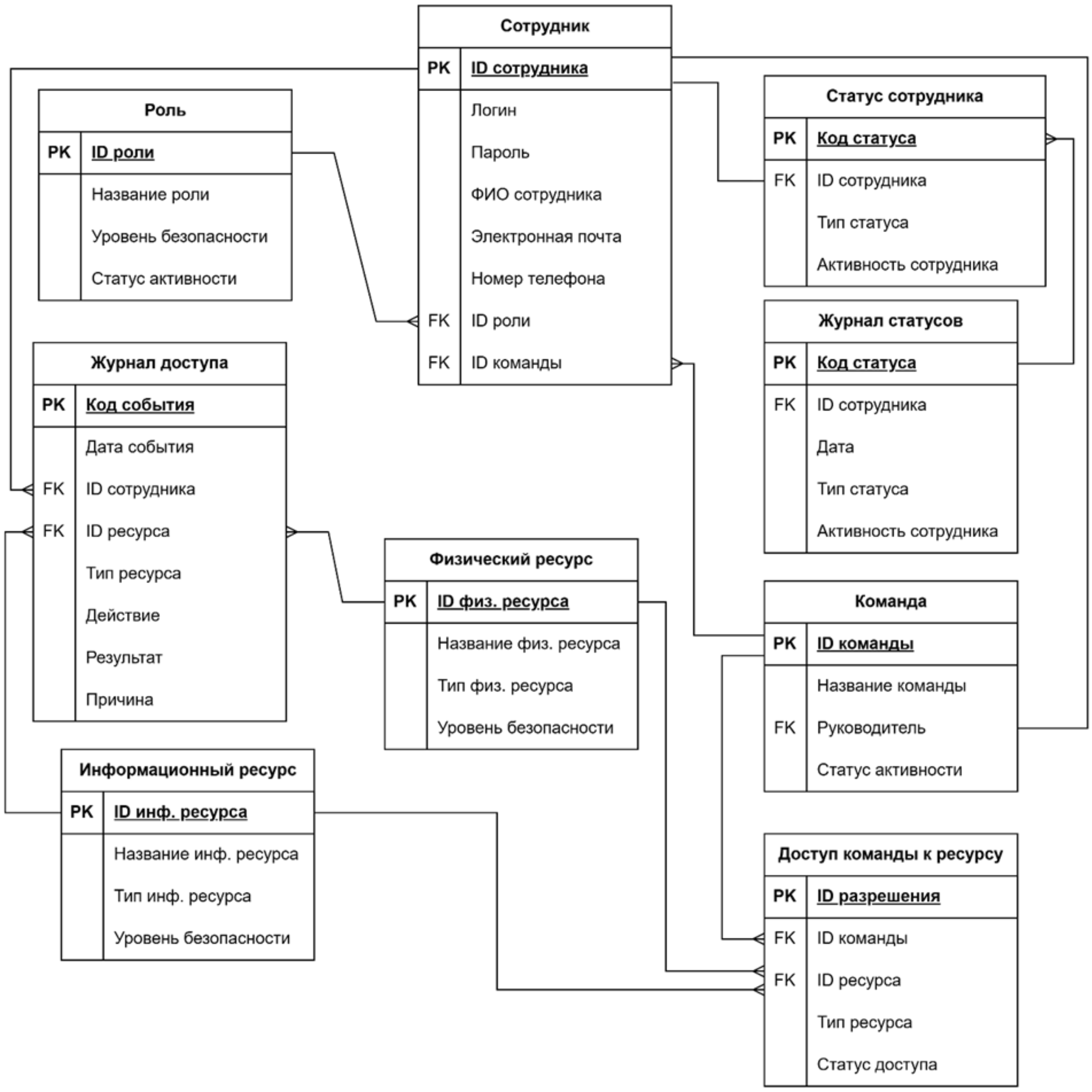
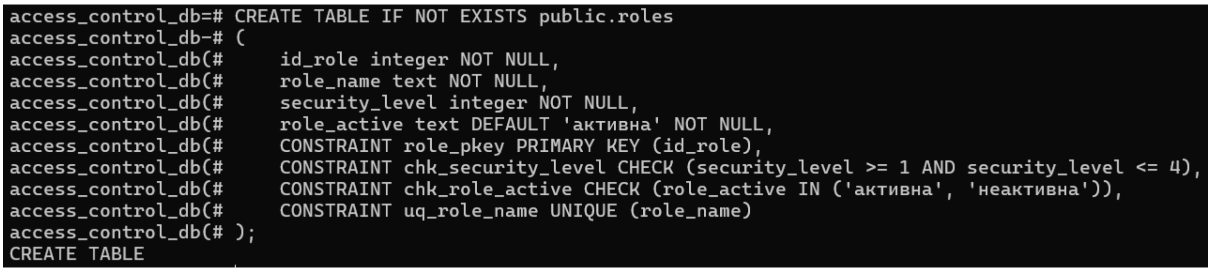

Сотрудник
PK
ID сотрудника
Логин
Пароль
ФИО сотрудника
Электронная почта
Номер телефона
FK
ID роли
FK
ID команды
Роль
PK
ID роли
Название роли
Уровень безопасности
Статус активности
Статус сотрудника
PK
Код статуса
FK
ID сотрудника
Тип статуса
Активность сотрудника
Журнал статусов
PK
Код статуса
FK
ID сотрудника
Дата
Тип статуса
Активность сотрудника
Журнал доступа
PK
Код события
Дата события
FK
ID сотрудника
FK
ID ресурса
Тип ресурса
Действие
Результат
Причина
Физический ресурс
PK
ID физ. ресурса
Название физ. ресурса
Тип физ. ресурса
Уровень безопасности
Команда
PK
ID команды
Название команды
FK
Руководитель
Статус активности
Информационный ресурс
PK
ID инф. ресурса
Название инф. ресурса
Тип инф. ресурса
Уровень безопасности
Доступ команды к ресурсу
PK
ID разрешения
FK
ID команды
FK
ID ресурса
Тип ресурса
Статус доступа

Рисунок 4 – Схема предварительных отношений

Таблица «Роли»

```
CREATE TABLE IF NOT EXISTS public.roles
(
    id_role integer NOT NULL,
    role_name text NOT NULL,
    security_level integer NOT NULL,
    role_active text DEFAULT 'активна' NOT NULL,
    CONSTRAINT role_pkey PRIMARY KEY (id_role),
    CONSTRAINT chk_security_level CHECK (security_level >= 1 AND security_level <= 4),
    CONSTRAINT chk_role_active CHECK (role_active IN ('активна', 'неактивна')),
    CONSTRAINT uq_role_name UNIQUE (role_name)
);
```

Процесс создания таблицы «Роли» в psql и ответ системы представлены на рисунке 5.



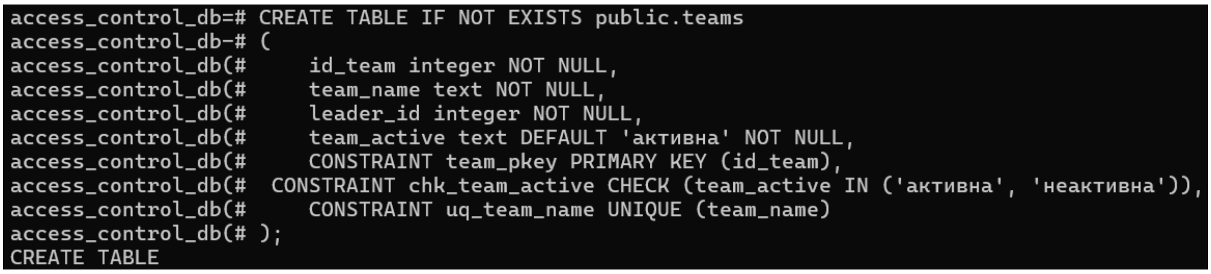
```
access_control_db=# CREATE TABLE IF NOT EXISTS public.roles
access_control_db-# (
access_control_db(#     id_role integer NOT NULL,
access_control_db(#     role_name text NOT NULL,
access_control_db(#     security_level integer NOT NULL,
access_control_db(#     role_active text DEFAULT 'активна' NOT NULL,
access_control_db(#     CONSTRAINT role_pkey PRIMARY KEY (id_role),
access_control_db(#     CONSTRAINT chk_security_level CHECK (security_level >= 1 AND security_level <= 4),
access_control_db(#     CONSTRAINT chk_role_active CHECK (role_active IN ('активна', 'неактивна')),
access_control_db(#     CONSTRAINT uq_role_name UNIQUE (role_name)
access_control_db(# );
CREATE TABLE
```

Рисунок 5 – Успешное создание таблицы «Роли»

Таблица «Команды»

```
CREATE TABLE IF NOT EXISTS public.teams
(
    id_team integer NOT NULL,
    team_name text NOT NULL,
    leader_id integer,
    team_active text DEFAULT 'активна' NOT NULL,
    CONSTRAINT team_pkey PRIMARY KEY (id_team),
    CONSTRAINT chk_team_active CHECK (team_active IN ('активна', 'неактивна')),
    CONSTRAINT uq_team_name UNIQUE (team_name)
);
```

Процесс создания таблицы «Команды» в psql и ответ системы представлены на рисунке 6.



```
access_control_db=# CREATE TABLE IF NOT EXISTS public.teams
access_control_db-# (
access_control_db(#     id_team integer NOT NULL,
access_control_db(#     team_name text NOT NULL,
access_control_db(#     leader_id integer NOT NULL,
access_control_db(#     team_active text DEFAULT 'активна' NOT NULL,
access_control_db(#     CONSTRAINT team_pkey PRIMARY KEY (id_team),
access_control_db(#  CONSTRAINT chk_team_active CHECK (team_active IN ('активна', 'неактивна')),
access_control_db(#     CONSTRAINT uq_team_name UNIQUE (team_name)
access_control_db(# );
CREATE TABLE
```

Рисунок 6 – Успешное создание таблицы «Команды»

Таблица «Сотрудники»

CREATE TABLE IF NOT EXISTS public.employees

(

id_employee integer NOT NULL,

login text NOT NULL,

password text NOT NULL,

full_name text NOT NULL,

email text,

phone text,

id_role integer NOT NULL,

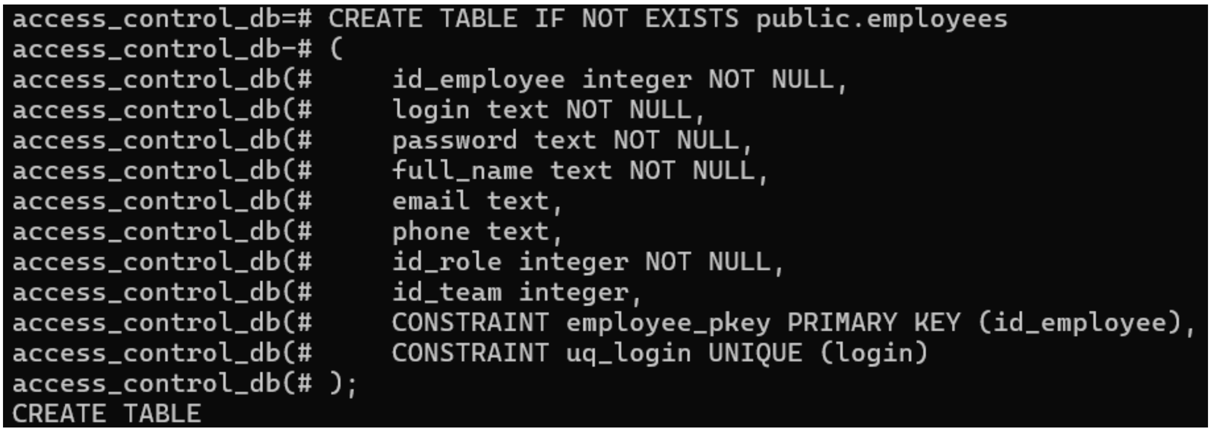
id_team integer,

CONSTRAINT employee_pkey PRIMARY KEY (id_employee),

CONSTRAINT uq_login UNIQUE (login)

);

Процесс создания таблицы «Сотрудники» в psql и ответ системы представлены на рисунке 7.



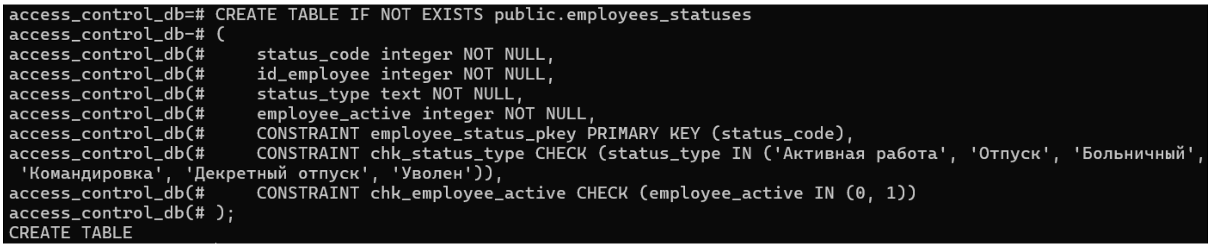
```
access_control_db=# CREATE TABLE IF NOT EXISTS public.employees
access_control_db-# (
access_control_db(#     id_employee integer NOT NULL,
access_control_db(#     login text NOT NULL,
access_control_db(#     password text NOT NULL,
access_control_db(#     full_name text NOT NULL,
access_control_db(#     email text,
access_control_db(#     phone text,
access_control_db(#     id_role integer NOT NULL,
access_control_db(#     id_team integer,
access_control_db(#     CONSTRAINT employee_pkey PRIMARY KEY (id_employee),
access_control_db(#     CONSTRAINT uq_login UNIQUE (login)
access_control_db(# );
CREATE TABLE
```

Рисунок 7 – Успешное создание таблицы «Сотрудники»

Таблица «Статусы сотрудников»

```
CREATE TABLE IF NOT EXISTS public.employees_statuses
(
    status_code integer NOT NULL,
    id_employee integer NOT NULL,
    status_type text NOT NULL,
    employee_active integer NOT NULL,
    CONSTRAINT employee_status_pkey PRIMARY KEY (status_code),
    CONSTRAINT chk_status_type CHECK (status_type IN ('Активная работа', 'Отпуск', 'Больничный', 'Командировка', 'Декретный отпуск', 'Уволен')),
    CONSTRAINT chk_employee_active CHECK (employee_active IN (0, 1))
);
```

Процесс создания таблицы «Статусы сотрудников» в psql и ответ системы представлены на рисунке 8.



```
access_control_db=# CREATE TABLE IF NOT EXISTS public.employees_statuses
access_control_db-# (
access_control_db(#     status_code integer NOT NULL,
access_control_db(#     id_employee integer NOT NULL,
access_control_db(#     status_type text NOT NULL,
access_control_db(#     employee_active integer NOT NULL,
access_control_db(#     CONSTRAINT employee_status_pkey PRIMARY KEY (status_code),
access_control_db(#     CONSTRAINT chk_status_type CHECK (status_type IN ('Активная работа', 'Отпуск', 'Больничный',
 'Командировка', 'Декретный отпуск', 'Уволен')),
access_control_db(#     CONSTRAINT chk_employee_active CHECK (employee_active IN (0, 1))
access_control_db(# );
CREATE TABLE
```

Рисунок 8 – Успешное создание таблицы «Статусы сотрудников»

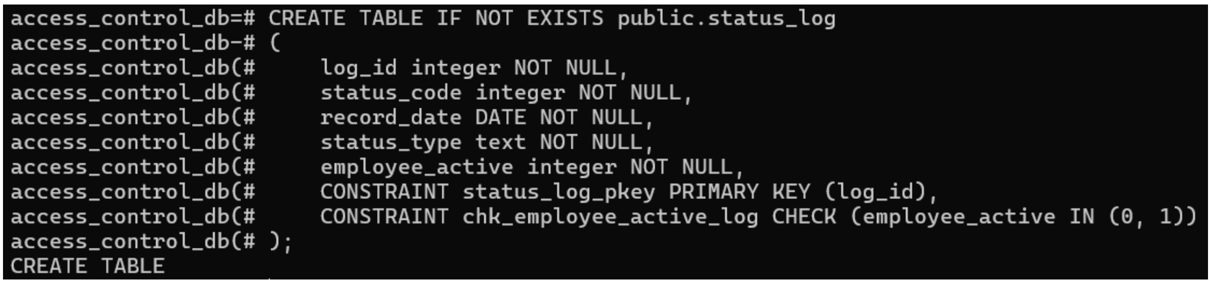
Таблица «Журнал статусов»

```
CREATE TABLE IF NOT EXISTS public.status_log
(
    log_id integer NOT NULL,
    status_code integer NOT NULL,
    record_date DATE NOT NULL,
    status_type text NOT NULL,
    employee_active integer NOT NULL,
    CONSTRAINT status_log_pkey PRIMARY KEY (log_id),
```

CONSTRAINT chk_employee_active_log CHECK (employee_active IN (0, 1))

);

Процесс создания таблицы «Журнал статусов» в psql и ответ системы представлены на рисунке 9.



```
access_control_db=# CREATE TABLE IF NOT EXISTS public.status_log
access_control_db-# (
access_control_db(#     log_id integer NOT NULL,
access_control_db(#     status_code integer NOT NULL,
access_control_db(#     record_date DATE NOT NULL,
access_control_db(#     status_type text NOT NULL,
access_control_db(#     employee_active integer NOT NULL,
access_control_db(#     CONSTRAINT status_log_pkey PRIMARY KEY (log_id),
access_control_db(#     CONSTRAINT chk_employee_active_log CHECK (employee_active IN (0, 1))
access_control_db(# );
CREATE TABLE
```

Рисунок 9 – Успешное создание таблицы «Журнал статусов»

Таблица «Физические ресурсы»

CREATE TABLE IF NOT EXISTS public.physical_resources

(

id_physical_resource integer NOT NULL,

resource_name text NOT NULL,

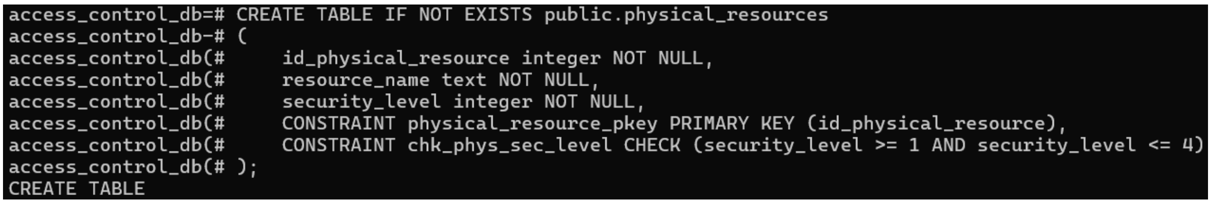
security_level integer NOT NULL,

CONSTRAINT physical_resource_pkey PRIMARY KEY (id_physical_resource),

CONSTRAINT chk_phys_sec_level CHECK (security_level >= 1 AND security_level <= 4)

);

Процесс создания таблицы «Физические ресурсы» в psql и ответ системы представлены на рисунке 10.



```
access_control_db=# CREATE TABLE IF NOT EXISTS public.physical_resources
access_control_db-# (
access_control_db(#     id_physical_resource integer NOT NULL,
access_control_db(#     resource_name text NOT NULL,
access_control_db(#     security_level integer NOT NULL,
access_control_db(#     CONSTRAINT physical_resource_pkey PRIMARY KEY (id_physical_resource),
access_control_db(#     CONSTRAINT chk_phys_sec_level CHECK (security_level >= 1 AND security_level <= 4)
access_control_db(# );
CREATE TABLE
```

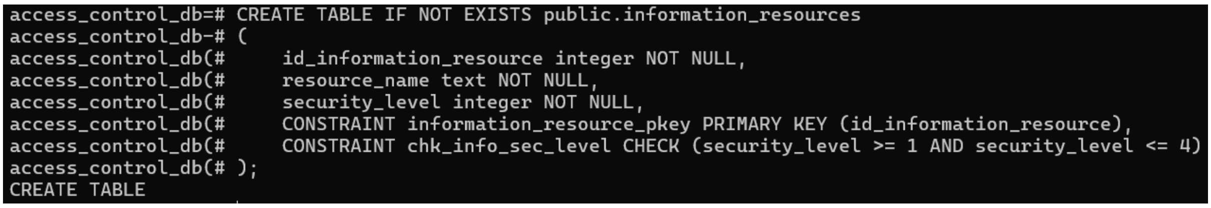
Рисунок 10 – Успешное создание таблицы «Физические ресурсы»

Таблица «Информационные ресурсы»

CREATE TABLE IF NOT EXISTS public.information_resources

```
(
    id_information_resource integer NOT NULL,
    resource_name text NOT NULL,
    security_level integer NOT NULL,
    CONSTRAINT information_resource_pkey PRIMARY KEY (id_information_resource),
    CONSTRAINT chk_info_sec_level CHECK (security_level >= 1 AND security_level <= 4)
);
```

Процесс создания таблицы «Информационные ресурсы» в psql и ответ системы представлены на рисунке 11.



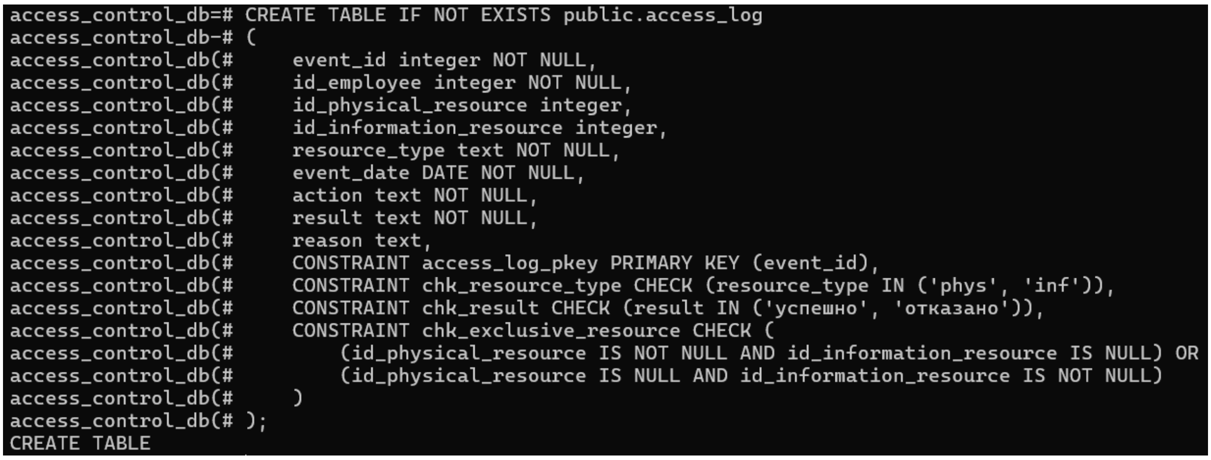
```
access_control_db=# CREATE TABLE IF NOT EXISTS public.information_resources
access_control_db-# (
access_control_db(#     id_information_resource integer NOT NULL,
access_control_db(#     resource_name text NOT NULL,
access_control_db(#     security_level integer NOT NULL,
access_control_db(#     CONSTRAINT information_resource_pkey PRIMARY KEY (id_information_resource),
access_control_db(#     CONSTRAINT chk_info_sec_level CHECK (security_level >= 1 AND security_level <= 4)
access_control_db(# );
CREATE TABLE
```

Рисунок 11 – Успешное создание таблицы «Информационные ресурсы»

Таблица «Журнал доступа»

```
CREATE TABLE IF NOT EXISTS public.access_log
(
    event_id integer NOT NULL,
    id_employee integer NOT NULL,
    id_physical_resource integer,
    id_information_resource integer,
    resource_type text NOT NULL,
    event_date DATE NOT NULL,
    action text NOT NULL,
    result text NOT NULL,
    reason text,
    CONSTRAINT access_log_pkey PRIMARY KEY (event_id),
    CONSTRAINT chk_resource_type CHECK (resource_type IN ('phys', 'inf')),
    CONSTRAINT chk_result CHECK (result IN ('успешно', 'отказано')),
    CONSTRAINT chk_exclusive_resource CHECK (
        (id_physical_resource IS NOT NULL AND id_information_resource IS NULL) OR
        (id_physical_resource IS NULL AND id_information_resource IS NOT NULL)
    )
);
```

Процесс создания таблицы «Журнал доступа» в psql и ответ системы представлены на рисунке 12.



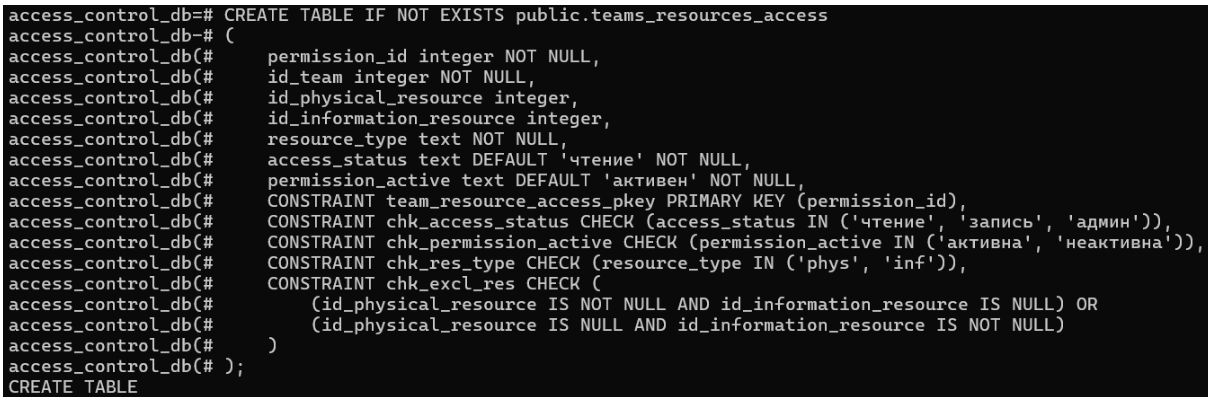
```
access_control_db=# CREATE TABLE IF NOT EXISTS public.access_log
access_control_db-# (
access_control_db(#     event_id integer NOT NULL,
access_control_db(#     id_employee integer NOT NULL,
access_control_db(#     id_physical_resource integer,
access_control_db(#     id_information_resource integer,
access_control_db(#     resource_type text NOT NULL,
access_control_db(#     event_date DATE NOT NULL,
access_control_db(#     action text NOT NULL,
access_control_db(#     result text NOT NULL,
access_control_db(#     reason text,
access_control_db(#     CONSTRAINT access_log_pkey PRIMARY KEY (event_id),
access_control_db(#     CONSTRAINT chk_resource_type CHECK (resource_type IN ('phys', 'inf')),
access_control_db(#     CONSTRAINT chk_result CHECK (result IN ('успешно', 'отказано')),
access_control_db(#     CONSTRAINT chk_exclusive_resource CHECK (
access_control_db(#         (id_physical_resource IS NOT NULL AND id_information_resource IS NULL) OR
access_control_db(#         (id_physical_resource IS NULL AND id_information_resource IS NOT NULL)
access_control_db(#     )
access_control_db(# );
CREATE TABLE
```

Рисунок 12 – Успешное создание таблицы «Журнал доступа»

Таблица «Доступ команд к ресурсам»

```
CREATE TABLE IF NOT EXISTS public.teams_resources_access
(
    permission_id integer NOT NULL,
    id_team integer NOT NULL,
    id_physical_resource integer,
    id_information_resource integer,
    resource_type text NOT NULL,
    access_status text DEFAULT 'чтение' NOT NULL,
    permission_active text DEFAULT 'активен' NOT NULL,
    CONSTRAINT team_resource_access_pkey PRIMARY KEY (permission_id),
        CONSTRAINT chk_access_status CHECK (access_status IN ('чтение', 'запись', 'админ')),
        CONSTRAINT chk_permission_active CHECK (permission_active IN ('активна', 'неактивна')),
    CONSTRAINT chk_res_type CHECK (resource_type IN ('phys', 'inf')),
    CONSTRAINT chk_excl_res CHECK (
        (id_physical_resource IS NOT NULL AND id_information_resource IS NULL) OR
        (id_physical_resource IS NULL AND id_information_resource IS NOT NULL)
    )
);
```

Процесс создания таблицы «Доступ команд к ресурсам» в psql и ответ системы представлены на рисунке 13.



```
access_control_db=# CREATE TABLE IF NOT EXISTS public.teams_resources_access
access_control_db-# (
access_control_db(#     permission_id integer NOT NULL,
access_control_db(#     id_team integer NOT NULL,
access_control_db(#     id_physical_resource integer,
access_control_db(#     id_information_resource integer,
access_control_db(#     resource_type text NOT NULL,
access_control_db(#     access_status text DEFAULT 'чтение' NOT NULL,
access_control_db(#     permission_active text DEFAULT 'активен' NOT NULL,
access_control_db(#     CONSTRAINT team_resource_access_pkey PRIMARY KEY (permission_id),
access_control_db(#     CONSTRAINT chk_access_status CHECK (access_status IN ('чтение', 'запись', 'админ')),
access_control_db(#     CONSTRAINT chk_permission_active CHECK (permission_active IN ('активна', 'неактивна')),
access_control_db(#     CONSTRAINT chk_res_type CHECK (resource_type IN ('phys', 'inf')),
access_control_db(#     CONSTRAINT chk_excl_res CHECK (
access_control_db(#         (id_physical_resource IS NOT NULL AND id_information_resource IS NULL) OR
access_control_db(#         (id_physical_resource IS NULL AND id_information_resource IS NOT NULL)
access_control_db(#     )
access_control_db(# );
CREATE TABLE
```

Рисунок 13 – Успешное создание таблицы «Доступ команд к ресурсам»

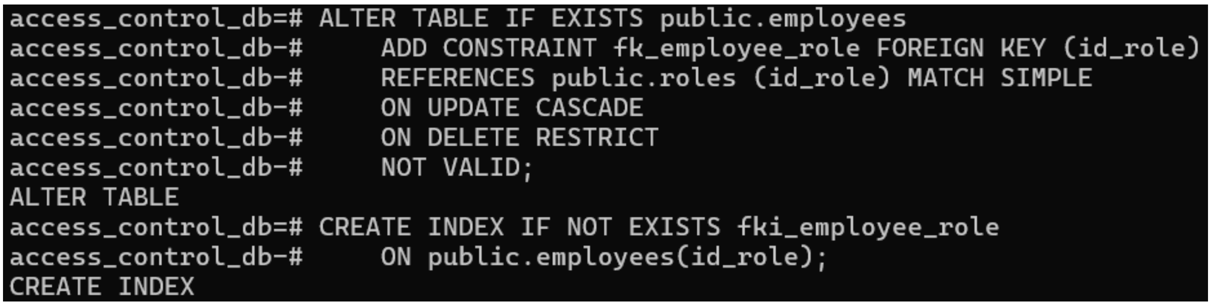
2.2.3 Установка взаимосвязей таблиц. Индексирование таблиц

Добавим дополнительные ограничения на отношения, соответствующие внешним ключам и проиндексируем их, чтобы запросы при объединении таблиц выполнялись быстрее.

Сотрудник – Роль

```
ALTER TABLE IF EXISTS public.employees
    ADD CONSTRAINT fk_employee_role FOREIGN KEY (id_role)
    REFERENCES public.roles (id_role) MATCH SIMPLE
    ON UPDATE CASCADE
    ON DELETE RESTRICT
    NOT VALID;
CREATE INDEX IF NOT EXISTS fki_employee_role
    ON public.employees(id_role);
```

Процесс создания взаимосвязи Сотрудник – Роль в psql и ответ системы представлены на рисунке 14.



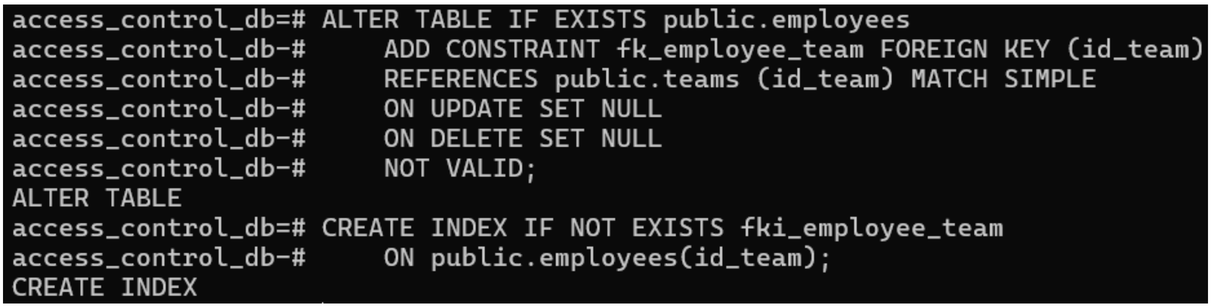
```
access_control_db=# ALTER TABLE IF EXISTS public.employees
access_control_db-#     ADD CONSTRAINT fk_employee_role FOREIGN KEY (id_role)
access_control_db-#     REFERENCES public.roles (id_role) MATCH SIMPLE
access_control_db-#     ON UPDATE CASCADE
access_control_db-#     ON DELETE RESTRICT
access_control_db-#     NOT VALID;
ALTER TABLE
access_control_db=# CREATE INDEX IF NOT EXISTS fki_employee_role
access_control_db-#     ON public.employees(id_role);
CREATE INDEX
```

Рисунок 14 – Внешний ключ + индекс. Сотрудник – Роль

Сотрудник – Команда

```
ALTER TABLE IF EXISTS public.employees
    ADD CONSTRAINT fk_employee_team FOREIGN KEY (id_team)
    REFERENCES public.teams (id_team) MATCH SIMPLE
    ON UPDATE SET NULL
    ON DELETE SET NULL
    NOT VALID;
CREATE INDEX IF NOT EXISTS fki_employee_team
    ON public.employees(id_team);
```

Процесс создания взаимосвязи Сотрудник – Команда в psql и ответ системы представлены на рисунке 15.



```
access_control_db=# ALTER TABLE IF EXISTS public.employees
access_control_db-#     ADD CONSTRAINT fk_employee_team FOREIGN KEY (id_team)
access_control_db-#     REFERENCES public.teams (id_team) MATCH SIMPLE
access_control_db-#     ON UPDATE SET NULL
access_control_db-#     ON DELETE SET NULL
access_control_db-#     NOT VALID;
ALTER TABLE
access_control_db=# CREATE INDEX IF NOT EXISTS fki_employee_team
access_control_db-#     ON public.employees(id_team);
CREATE INDEX
```

Рисунок 13 – Внешний ключ + индекс. Сотрудник – Команда

Команда – Сотрудник (Руководитель)

ALTER TABLE IF EXISTS public.teams

ADD CONSTRAINT fk_team_leader FOREIGN KEY (leader_id)

REFERENCES public.employees (id_employee) MATCH SIMPLE

ON UPDATE SET NULL

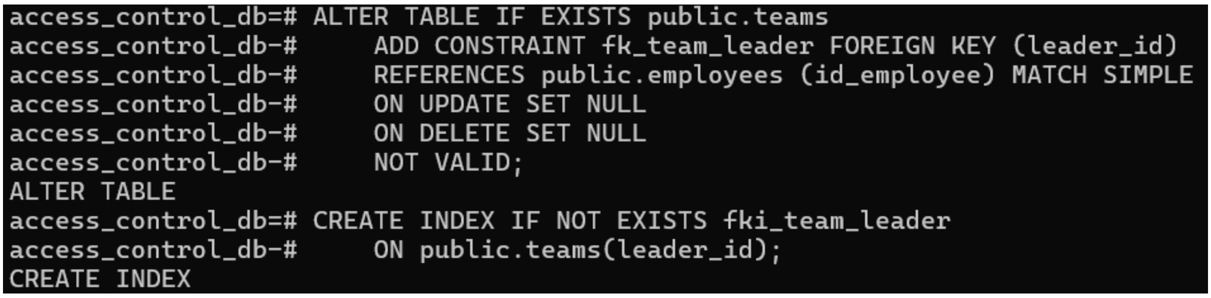
ON DELETE SET NULL

NOT VALID;

CREATE INDEX IF NOT EXISTS fki_team_leader

ON public.teams(leader_id);

Процесс создания взаимосвязи Команда – Сотрудник (Руководитель) в psql и ответ системы представлены на рисунке 16.



```
access_control_db=# ALTER TABLE IF EXISTS public.teams
access_control_db-#     ADD CONSTRAINT fk_team_leader FOREIGN KEY (leader_id)
access_control_db-#     REFERENCES public.employees (id_employee) MATCH SIMPLE
access_control_db-#     ON UPDATE SET NULL
access_control_db-#     ON DELETE SET NULL
access_control_db-#     NOT VALID;
ALTER TABLE
access_control_db=# CREATE INDEX IF NOT EXISTS fki_team_leader
access_control_db-#     ON public.teams(leader_id);
CREATE INDEX
```

Рисунок 16 – Внешний ключ + индекс. Команда – Сотрудник (Руководитель)

Статус сотрудника – Сотрудник

ALTER TABLE IF EXISTS public.employees_statuses

ADD CONSTRAINT fk_status_employee FOREIGN KEY (id_employee)

REFERENCES public.employees (id_employee) MATCH SIMPLE

ON UPDATE CASCADE

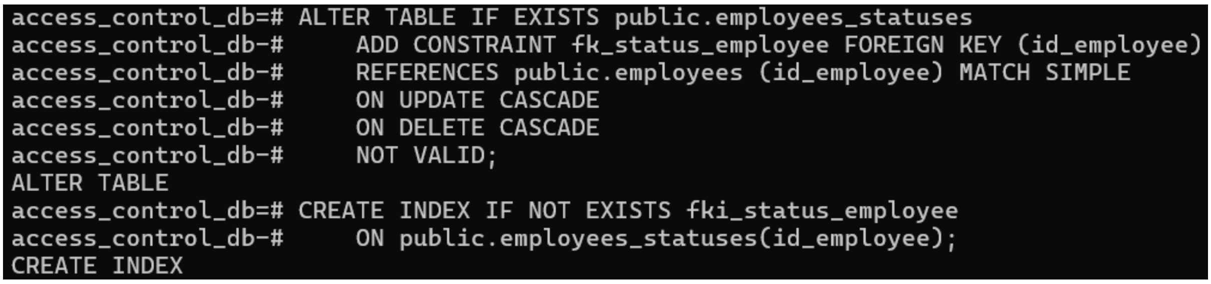
ON DELETE CASCADE

NOT VALID;

CREATE INDEX IF NOT EXISTS fki_status_employee

ON public.employees_statuses(id_employee);

Процесс создания взаимосвязи Статус сотрудника – Сотрудник в psql и ответ системы представлены на рисунке 17.



```
access_control_db=# ALTER TABLE IF EXISTS public.employees_statuses
access_control_db-#     ADD CONSTRAINT fk_status_employee FOREIGN KEY (id_employee)
access_control_db-#     REFERENCES public.employees (id_employee) MATCH SIMPLE
access_control_db-#     ON UPDATE CASCADE
access_control_db-#     ON DELETE CASCADE
access_control_db-#     NOT VALID;
ALTER TABLE
access_control_db=# CREATE INDEX IF NOT EXISTS fki_status_employee
access_control_db-#     ON public.employees_statuses(id_employee);
CREATE INDEX
```

Рисунок 17 – Внешний ключ + индекс. Статус сотрудника – Сотрудник

Журнал статусов – Статус сотрудника

ALTER TABLE IF EXISTS public.status_log

ADD CONSTRAINT fk_log_status FOREIGN KEY (status_code)

REFERENCES public.employees_statuses (status_code) MATCH SIMPLE

ON UPDATE CASCADE

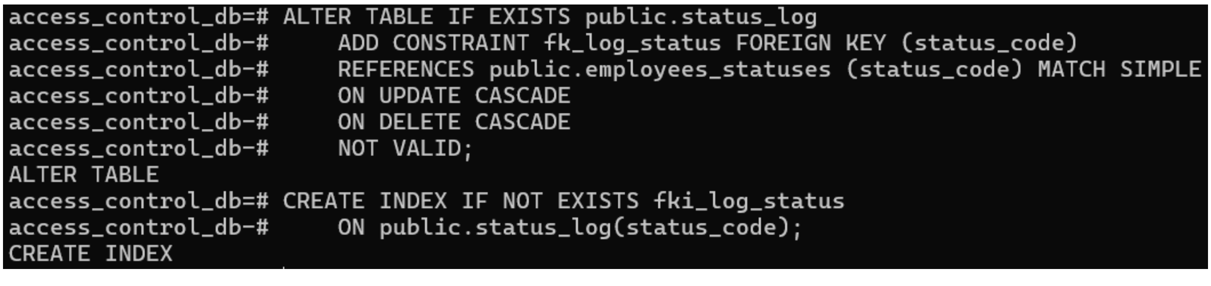
ON DELETE CASCADE

NOT VALID;

CREATE INDEX IF NOT EXISTS fki_log_status

ON public.status_log(status_code);

Процесс создания взаимосвязи Журнал статусов – Статус сотрудника в psql и ответ системы представлены на рисунке 18.



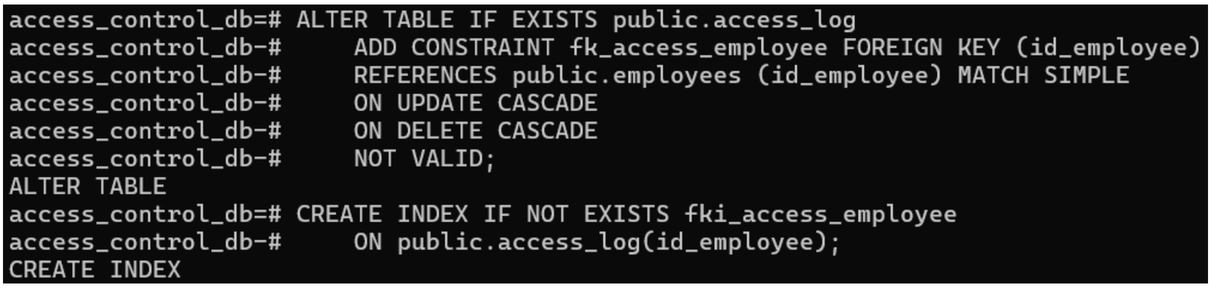
```
access_control_db=# ALTER TABLE IF EXISTS public.status_log
access_control_db-#     ADD CONSTRAINT fk_log_status FOREIGN KEY (status_code)
access_control_db-#     REFERENCES public.employees_statuses (status_code) MATCH SIMPLE
access_control_db-#     ON UPDATE CASCADE
access_control_db-#     ON DELETE CASCADE
access_control_db-#     NOT VALID;
ALTER TABLE
access_control_db=# CREATE INDEX IF NOT EXISTS fki_log_status
access_control_db-#     ON public.status_log(status_code);
CREATE INDEX
```

Рисунок 18 – Внешний ключ + индекс. Журнал статусов – Статус сотрудника

Журнал доступа – Сотрудник

```
ALTER TABLE IF EXISTS public.access_log
    ADD CONSTRAINT fk_access_employee FOREIGN KEY (id_employee)
    REFERENCES public.employees (id_employee) MATCH SIMPLE
    ON UPDATE CASCADE
    ON DELETE CASCADE
    NOT VALID;
CREATE INDEX IF NOT EXISTS fki_access_employee
    ON public.access_log(id_employee);
```

Процесс создания взаимосвязи Журнал доступа – Сотрудник в psql и ответ системы представлены на рисунке 19.



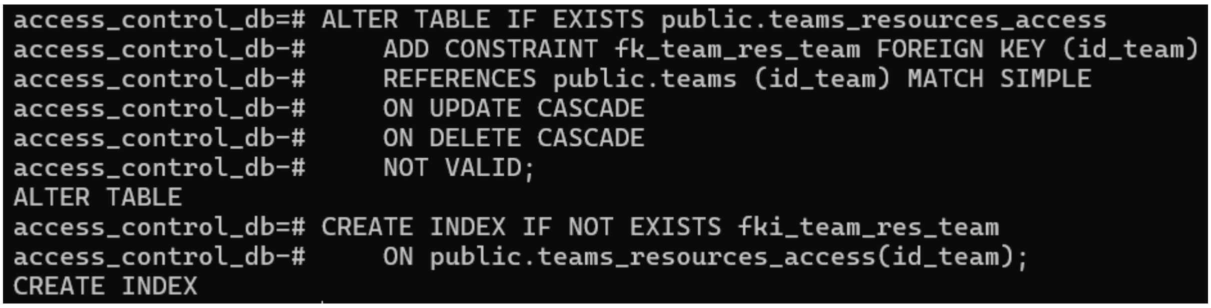
```
access_control_db=# ALTER TABLE IF EXISTS public.access_log
access_control_db-#     ADD CONSTRAINT fk_access_employee FOREIGN KEY (id_employee)
access_control_db-#     REFERENCES public.employees (id_employee) MATCH SIMPLE
access_control_db-#     ON UPDATE CASCADE
access_control_db-#     ON DELETE CASCADE
access_control_db-#     NOT VALID;
ALTER TABLE
access_control_db=# CREATE INDEX IF NOT EXISTS fki_access_employee
access_control_db-#     ON public.access_log(id_employee);
CREATE INDEX
```

Рисунок 19 – Внешний ключ + индекс. Журнал доступа – Сотрудник

Доступ команды к ресурсам – Команда

```
ALTER TABLE IF EXISTS public.teams_resources_access
    ADD CONSTRAINT fk_team_res_team FOREIGN KEY (id_team)
    REFERENCES public.teams (id_team) MATCH SIMPLE
    ON UPDATE CASCADE
    ON DELETE CASCADE
    NOT VALID;
CREATE INDEX IF NOT EXISTS fki_team_res_team
    ON public.teams_resources_access(id_team);
```

Процесс создания взаимосвязи Доступ команды к ресурсам – Команда в psql и ответ системы представлены на рисунке 20.



```
access_control_db=# ALTER TABLE IF EXISTS public.teams_resources_access
access_control_db-#     ADD CONSTRAINT fk_team_res_team FOREIGN KEY (id_team)
access_control_db-#     REFERENCES public.teams (id_team) MATCH SIMPLE
access_control_db-#     ON UPDATE CASCADE
access_control_db-#     ON DELETE CASCADE
access_control_db-#     NOT VALID;
ALTER TABLE
access_control_db=# CREATE INDEX IF NOT EXISTS fki_team_res_team
access_control_db-#     ON public.teams_resources_access(id_team);
CREATE INDEX
```

Рисунок 20 – Внешний ключ + индекс. Доступ команды к ресурсам – Команда

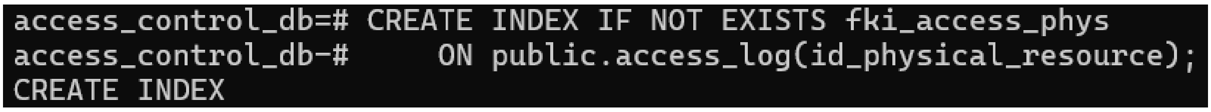
Журнал доступа – Физический ресурс (условный FK – только если ресурс типа 'phys')

Нужно отметить, что из-за разделения на два типа ресурсов, полный FK невозможен. Вместо этого создаем индексы для ускорения JOIN.

CREATE INDEX IF NOT EXISTS fki_access_phys

ON public.access_log(id_physical_resource);

Процесс создания взаимосвязи Журнал доступа – Физический ресурс в psql и ответ системы представлены на рисунке 21.



```
access_control_db=# CREATE INDEX IF NOT EXISTS fki_access_phys
access_control_db-#     ON public.access_log(id_physical_resource);
CREATE INDEX
```

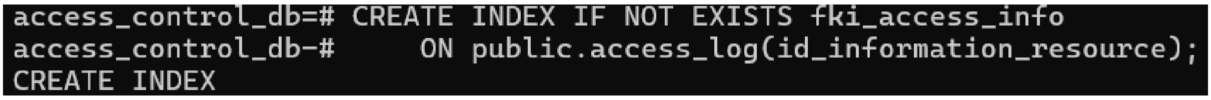
Рисунок 21 – Внешний ключ + индекс. Журнал доступа – Физический ресурс

Журнал доступа – Информационный ресурс (условный FK — только если ресурс типа 'inf')

CREATE INDEX IF NOT EXISTS fki_access_info

ON public.access_log(id_information_resource);

Процесс создания взаимосвязи Журнал доступа – Информационный ресурс в psql и ответ системы представлены на рисунке 22.



```
access_control_db=# CREATE INDEX IF NOT EXISTS fki_access_info
access_control_db-#     ON public.access_log(id_information_resource);
CREATE INDEX
```

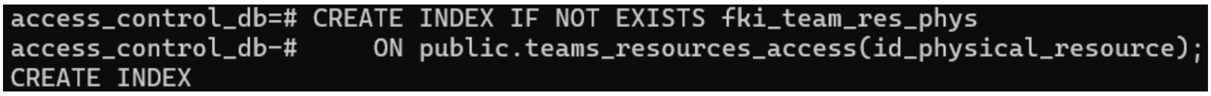
Рисунок 22 – Внешний ключ + индекс. Журнал доступа – Информационный ресурс

Доступ команды к ресурсам – Физический ресурс

CREATE INDEX IF NOT EXISTS fki_team_res_phys

ON public.teams_resources_access(id_physical_resource);

Процесс создания взаимосвязи Доступ команды к ресурсам – Физический ресурс в psql и ответ системы представлены на рисунке 23.



```
access_control_db=# CREATE INDEX IF NOT EXISTS fki_team_res_phys
access_control_db-#     ON public.teams_resources_access(id_physical_resource);
CREATE INDEX
```

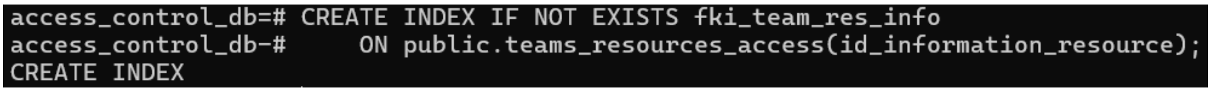
Рисунок 23 – Внешний ключ + индекс. Доступ команды к ресурсам – Физический ресурс

Доступ команды к ресурсам – Информационный ресурс

CREATE INDEX IF NOT EXISTS fki_team_res_info

ON public.teams_resources_access(id_information_resource);

Процесс создания взаимосвязи Доступ команды к ресурсам – Информационный ресурс в psql и ответ системы представлены на рисунке 24.



```
access_control_db=# CREATE INDEX IF NOT EXISTS fki_team_res_info
access_control_db-#     ON public.teams_resources_access(id_information_resource);
CREATE INDEX
```

Рисунок 24 – Внешний ключ + индекс. Доступ команды к ресурсам – Информационный ресурс

2.2.4 Заполнение таблиц данными

Важно отметить значения некоторых столбцов:

- security_level от 1 до 4 (где 1 – низкий, 2 – средний, 3 – высокий, 4 – критический),
- resource_type – «phys» или «inf»,
- role_active, team_active – «активна» / «неактивна»,
- employee_active – 0 или 1 (где 0 – сотрудник в определенный день работает, 1 – не работает),
- access_status – «чтение», «запись», «админ» («админ» – чтение + запись, а также для физических ресурсов доступно только «чтение», что означает доступ),
- permission_active — «активна» / «неактивна».

Таблица public.roles

```
INSERT INTO public.roles (id_role, role_name, security_level, role_active) VALUES
(1, 'Стажер', 1, 'активна'),
(2, 'Разработчик', 2, 'активна'),
(3, 'Тимлид', 3, 'активна'),
(4, 'DevOps-инженер', 3, 'активна'),
(5, 'Администратор ИБ', 4, 'активна'),
(6, 'Служба безопасности', 4, 'активна'),
(7, 'Аудитор', 3, 'активна'),
(8, 'Руководитель отдела', 3, 'активна');
```

Таблица public.teams

```
INSERT INTO public.teams (id_team, team_name, leader_id, team_active) VALUES
(1, 'Frontend Team', NULL, 'активна'),
(2, 'Backend Team', NULL, 'активна'),
(3, 'QA Team', NULL, 'активна'),
(4, 'DevSecOps', NULL, 'активна'),
(5, 'Инфраструктура', NULL, 'активна'),
(6, 'Аналитика', NULL, 'активна'),
(7, 'Архивная команда', NULL, 'неактивна');
```

Таблица public.employees

В этой таблице хранятся пароли, но так как пароли не могут храниться в БД в открытом виде, необходимо их захэшировать. Сделать это можно, например, с помощью функции crypt(). Сначала нужно активировать модуль pgcrypto (выполняется один раз):

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
INSERT INTO public.employees (id_employee, login, password, full_name, email, phone, id_role, id_team) VALUES
(1, 'ivanov_i', crypt('password1', gen_salt('bf')), 'Иванов Иван Иванович', 'ivanov@company.com', '+79001112233', 2, 1),
(2, 'petrov_p', crypt('password2', gen_salt('bf')), 'Петров Петр Петрович', 'petrov@company.com', '+79001112234', 3, 2),
(3, 'sidorov_s', crypt('password3', gen_salt('bf')), 'Сидоров Сидор Сидорович', 'sidorov@company.com', '+79001112235', 4, 4),
(4, 'admin_ib', crypt('password4', gen_salt('bf')), 'Козлов Геннадий Андреевич', 'admin_ib@company.com', '+79001112236', 5, NULL),
(5, 'sec_officer', crypt('password5', gen_salt('bf')), 'Васьков Владимир Васильевич', 'sec@company.com', '+79001112237', 6, NULL),
(6, 'auditor1', crypt('password6', gen_salt('bf')), 'Дементьев Сергей Данилович', 'audit@company.com', '+79001112238', 7, NULL),
(7, 'manager_a', crypt('password7', gen_salt('bf')), 'Кривова Анна Валерьевна', 'manager@company.com', '+79001112239', 8, 1),
(8, 'vacation_user', crypt('password8', gen_salt('bf')), 'Прокофьев Денис Иванович', 'ex@company.com', NULL, 2, NULL);
```

Обновление leader_id в public.teams

```
UPDATE public.teams SET leader_id = 7 WHERE id_team = 1; UPDATE
public.teams SET leader_id = 2 WHERE id_team = 2;  UPDATE public.teams SET
leader_id = 3 WHERE id_team = 4;
```

Таблица public.employees_statuses

```
INSERT INTO public.employees_statuses (status_code, id_employee,
status_type, employee_active) VALUES
(1, 1, 'Активная работа', 1),
(2, 2, 'Активная работа', 1),
(3, 3, 'Активная работа', 1),
(4, 4, 'Активная работа', 1),
(5, 5, 'Активная работа', 1),
(6, 6, 'Активная работа', 1),
(7, 7, 'Активная работа', 1),
(8, 8, 'Уволен', 0);
```

Таблица public.status_log

```
INSERT INTO public.status_log (log_id, status_code, record_date, status_type,
employee_active) VALUES
(1, 1, '2025-04-01', 'Активная работа', 1),
(2, 8, '2025-03-15', 'Больничный', 1),
(3, 8, '2025-04-01', 'Уволен', 0),
(4, 2, '2025-04-01', 'Активная работа', 1),
(5, 3, '2025-04-01', 'Активная работа', 1),
(6, 4, '2025-04-01', 'Активная работа', 1),
(7, 5, '2025-04-01', 'Активная работа', 1),
(8, 6, '2025-04-01', 'Активная работа', 1);
```

Таблица public.physical_resources

```
INSERT INTO public.physical_resources (id_physical_resource,
resource_name, security_level) VALUES
(1, 'Серверная комната A', 4),
(2, 'Лаборатория тестирования', 2),
(3, 'Кабинет администратора ИБ', 4),
(4, 'Общее офисное помещение', 1),
(5, 'Архив документов', 3),
(6, 'Комнату для переговоров', 2),
(7, 'Сейф с ключами', 4);
```

Таблица public.information_resources

```
INSERT INTO public.information_resources (id_information_resource, resource_name, security_level) VALUES
(1, 'База данных клиентов', 4),
(2, 'CI/CD система', 3),
(3, 'Внутренний портал', 1),
(4, 'Репозиторий исходного кода', 3),
(5, 'Журнал аудита', 4),
(6, 'HR-система', 2),
(7, 'Шифрованный архив', 4);
```

Таблица public.access_log

```
INSERT INTO public.access_log (
    event_id, id_employee, id_physical_resource, id_information_resource,
    resource_type, event_date, action, result, reason
) VALUES
(1, 1, NULL, 3, 'inf', '2025-04-01', 'вход', 'успешно', NULL),
(2, 2, NULL, 1, 'inf', '2025-04-01', 'чтение', 'отказано', 'уровень роли ниже уровня ресурса'),
(3, 8, NULL, 3, 'inf', '2025-04-01', 'вход', 'отказано', 'статус "Уволен"'),
(4, 5, NULL, 5, 'inf', '2025-04-01', 'чтение', 'успешно', NULL),
(5, 1, 4, NULL, 'phys', '2025-04-01', 'вход в офис', 'успешно', NULL),
(6, 2, 1, NULL, 'phys', '2025-04-01', 'доступ к серверной', 'отказано', 'уровень роли ниже уровня ресурса'),
(7, 4, 3, NULL, 'phys', '2025-04-01', 'доступ к кабинету', 'успешно', NULL),
(8, 3, NULL, 2, 'inf', '2025-04-01', 'запуск pipeline', 'успешно', NULL);
```

Таблица public.teams_resources_access

```
INSERT INTO public.teams_resources_access (
   permission_id, id_team, id_physical_resource, id_information_resource,
   resource_type, access_status, permission_active
) VALUES
(1, 1, NULL, 4, 'inf', 'чтение', 'активна'),
(2, 2, NULL, 4, 'inf', 'чтение', 'активна'),
(3, 2, NULL, 2, 'inf', 'запись', 'активна'),
(4, 4, NULL, 1, 'inf', 'админ', 'активна'),
(5, 4, NULL, 5, 'inf', 'чтение', 'активна'),
(6, 3, NULL, 2, 'inf', 'чтение', 'активна'),
(7, 3, NULL, 3, 'inf', 'чтение', 'активна'),
(8, 1, 4, NULL, 'phys', 'чтение', 'активна');
```

Содержимое таблиц

```
SELECT * FROM access_log;
SELECT * FROM employees;
SELECT * FROM employees_statuses;
SELECT * FROM information_resources;
SELECT * FROM physical_resources;
SELECT * FROM roles;
SELECT * FROM status_log;
SELECT * FROM teams;
SELECT * FROM teams_resources_access;
```

2.3 Создание представлений

Создадим представления, составленные в пункте 5 лабораторной работы №1, для удобного просмотра информации пользователями.

Представление 1. «Общий обзор прав доступа»

Список атрибутов, отображаемых в рамках представления «Общий обзор прав доступа»:

- Логин (отношение «employees»);
- ФИО (отношение «employees»);
- Название роли (отношение «roles»);
- Уровень безопасности роли (отношение «roles»);
- Название команды (отношение «teams»);
- Тип ресурса (отношение «teams_resources_access»);
- Название ресурса (отношение «physical_resources» / «information_resources»);
- Уровень безопасности ресурса (отношение «physical_resources» / «information_resources»).

Исходя из текущих обозначений атрибутов в БД, SQL-код для создания представления может выглядеть следующим образом:

```
CREATE VIEW public.overview_access_rights
AS
SELECT
    e.login,
    e.full_name,
    r.role_name,
    r.security_level AS role_security_level,
    t.team_name,
    COALESCE('Physical', 'Informational') AS resource_type,
    COALESCE(pr.resource_name, ir.resource_name) AS resource_name,
    COALESCE(pr.security_level, ir.security_level) AS resource_security_level
FROM public.employees e
```

```sql
JOIN public.roles r ON e.id_role = r.id_role
LEFT JOIN public.teams t ON e.id_team = t.id_team
LEFT JOIN public.teams_resources_access tra ON t.id_team = tra.id_team
LEFT JOIN public.physical_resources pr ON tra.id_physical_resource = pr.id_physical_resource AND tra.resource_type = 'phys'
LEFT JOIN public.information_resources ir ON tra.id_information_resource = ir.id_information_resource AND tra.resource_type = 'inf';
```

Представление 2. «Попытки нарушения политик безопасности»

Список атрибутов, отображаемых в рамках представления «Попытки нарушения политик безопасности»:

- Логин (отношение «employees»);
- Название роли (отношение «roles»);
- Уровень безопасности роли (отношение «roles»);
- Название ресурса (отношение «physical_resources» / «information_resources»);
- Уровень безопасности ресурса (отношение «physical_resources» / «information_resources»);
- Тип статуса (отношение «employees_statuses»);
- Дата записи (отношение «access_log»);
- Результат (отношение «access_log»);
- Причина (отношение «access_log»).

```sql
CREATE VIEW public.policy_violation_attempts
AS
SELECT
    e.login,
    r.role_name,
    r.security_level AS role_sec_level,
    COALESCE(pr.resource_name, ir.resource_name) AS resource_name,
    COALESCE(pr.security_level, ir.security_level) AS resource_sec_level,
    es.status_type,
    al.event_date,
    al.result,
    al.reason
FROM public.access_log al
JOIN public.employees e ON al.id_employee = e.id_employee
JOIN public.roles r ON e.id_role = r.id_role
JOIN public.employees_statuses es ON e.id_employee = es.id_employee
LEFT JOIN public.physical_resources pr ON al.id_physical_resource = pr.id_physical_resource
LEFT JOIN public.information_resources ir ON al.id_information_resource = ir.id_information_resource
WHERE
    (r.security_level < COALESCE(pr.security_level, ir.security_level))
    OR es.status_type IN ('Отпуск', 'Больничный', 'Уволен');
```

Представление 3. «Состав и статусы команды»

Список атрибутов, отображаемых в рамках представления «Состав и статусы команды»:

- Название команды (отношение «teams»);
- ФИО сотрудника (отношение «employees»);
- Логин (отношение «employees»);
- Название роли (отношение «roles»);
- Тип статуса (отношение «employees_statuses»);
- Активность (отношение «employees_statuses»);
- Признак прогула (вычисляемый атрибут).

```
CREATE VIEW public.team_members_and_status
AS
SELECT
    t.team_name,
    e.full_name,
    e.login,
    r.role_name,
    es.status_type,
    es.employee_active,
    CASE
        WHEN es.status_type = 'Активная работа' AND es.employee_active = 0 THEN TRUE
        ELSE FALSE
    END AS is_absentee
FROM public.teams t
JOIN public.employees e ON t.id_team = e.id_team
JOIN public.roles r ON e.id_role = r.id_role
JOIN public.employees_statuses es ON e.id_employee = es.id_employee;
```

Представление 4. «Ресурсы, доступные команде»

Список атрибутов, отображаемых в рамках представления «Ресурсы, доступные команде»:

- Название команды (отношение «teams»);
- Тип ресурса (отношение «teams_resources_access»);
- Название ресурса (отношение «physical_resources» / «information_resources»);
- Уровень безопасности ресурса (отношение «physical_resources» / «information_resources»);
- Статус доступа (отношение «teams_resources_access»).

```
CREATE VIEW public.team_available_resources
AS
SELECT
    t.team_name,
    tra.resource_type,
    COALESCE(pr.resource_name, ir.resource_name) AS resource_name,
    COALESCE(pr.security_level, ir.security_level) AS security_level,
    tra.access_status
```

```
FROM public.teams t
JOIN public.teams_resources_access tra ON t.id_team = tra.id_team
LEFT JOIN public.physical_resources pr ON tra.id_physical_resource = pr.id_physical_resource
LEFT JOIN public.information_resources ir ON tra.id_information_resource = ir.id_information_resource;
```

Представление 5. «Мои текущие права доступа»

Список атрибутов, отображаемых в рамках представления «Мои текущие права доступа»:

- Логин (отношение «employees»);
- ФИО (отношение «employees»);
- Название роли (отношение «roles»);
- Уровень безопасности роли (отношение «roles»);
- Название команды (отношение «teams»);
- Тип ресурса (отношение «teams_resources_access»);
- Название ресурса (отношение «physical_resources» / «information_resources»).

```
CREATE VIEW public.my_current_access
AS
SELECT
   e.login,
   e.full_name,
   r.role_name,
   r.security_level AS role_sec_level,
   t.team_name,
   COALESCE('Physical', 'Informational') AS resource_type,
   COALESCE(pr.resource_name, ir.resource_name) AS resource_name
FROM public.employees e
JOIN public.roles r ON e.id_role = r.id_role
LEFT JOIN public.teams t ON e.id_team = t.id_team
LEFT JOIN public.teams_resources_access tra ON t.id_team = tra.id_team
LEFT JOIN public.physical_resources pr ON tra.id_physical_resource = pr.id_physical_resource AND tra.resource_type = 'phys'
LEFT JOIN public.information_resources ir ON tra.id_information_resource = ir.id_information_resource AND tra.resource_type = 'inf';
```

Представление 6. «Мой статус и активность»

Список атрибутов, отображаемых в рамках представления «Мой статус и активность»:

- Логин (отношение «employees»);
- Текущий тип статуса (отношение «employees_statuses»);
- Активность (отношение «employees_statuses»);
- История: Дата — Тип статуса (отношение «status_log»).

```
CREATE VIEW public.my_status_and_activity
AS
SELECT
   e.login,
   es.status_type AS current_status,
   es.employee_active,
   sl.record_date,
   sl.status_type AS historical_status
FROM public.employees e
JOIN public.employees_statuses es ON e.id_employee = es.id_employee
JOIN public.status_log sl ON es.status_code = sl.status_code;
```

Представление 7. «Подозрительные попытки доступа»

Список атрибутов, отображаемых в рамках представления «Подозрительные попытки доступа»:

- Логин (отношение «employees»)
- Тип статуса (отношение «employees_statuses»)
- Название ресурса (отношение «physical_resources» / «information_resources»)

- Тип ресурса (отношение «access_log»)
- Дата записи (отношение «access_log»)
- Действие (отношение «access_log»)
- Результат (отношение «access_log»)
- Причина (отношение «access_log»)

```
CREATE VIEW public.suspicious_access_attempts
AS
SELECT
   e.login,
   es.status_type,
   COALESCE(pr.resource_name, ir.resource_name) AS resource_name,
   al.resource_type,
   al.event_date,
   al.action,
   al.result,
   al.reason
FROM public.access_log al
JOIN public.employees e ON al.id_employee = e.id_employee
JOIN public.employees_statuses es ON e.id_employee = es.id_employee
LEFT JOIN public.physical_resources pr ON al.id_physical_resource = pr.id_physical_resource
LEFT JOIN public.information_resources ir ON al.id_information_resource = ir.id_information_resource
WHERE es.status_type != 'Активная работа';
```

Представление 8. «Статистика доступов по статусам»

Список атрибутов, отображаемых в рамках представления «Статистика доступов по статусам»:

- Тип статуса (отношение «employees_statuses»);
- Общее число попыток (агрегация по «access_log»);
- Успешных попыток (агрегация по «access_log»);

- Неуспешных попыток (агрегация по «access_log»).

```
CREATE VIEW public.access_statistics_by_status
AS
SELECT
   es.status_type,
   COUNT(*) AS total_attempts,
   COUNT(*) FILTER (WHERE al.result = 'успешно') AS successful,
   COUNT(*) FILTER (WHERE al.result = 'отказано') AS failed
FROM public.access_log al
JOIN public.employees e ON al.id_employee = e.id_employee
JOIN public.employees_statuses es ON e.id_employee = es.id_employee
GROUP BY es.status_type;
```

Представление 9. «Потенциальные прогулы»

Список атрибутов, отображаемых в рамках представления «Потенциальные прогулы»:

- ФИО (отношение «employees»);
- Логин (отношение «employees»);
- Название команды (отношение «teams»);
- Тип статуса (отношение «employees_statuses»);
- Активность (отношение «employees_statuses»);
- Последняя запись в Журнале доступа (дата, ресурс).

```
CREATE VIEW public.potential_absenteeism
AS
SELECT
   e.full_name,
   e.login,
   t.team_name,
   es.status_type,
   es.employee_active,
   al.event_date AS last_access,
    COALESCE(pr.resource_name, ir.resource_name) AS last_resource
FROM public.employees e
JOIN public.employees_statuses es ON e.id_employee = es.id_employee
LEFT JOIN public.teams t ON e.id_team = t.id_team
LEFT JOIN LATERAL (
    SELECT *
    FROM public.access_log al2
    WHERE al2.id_employee = e.id_employee
    ORDER BY al2.event_date DESC
    LIMIT 1
) al ON TRUE
LEFT JOIN public.physical_resources pr ON al.id_physical_resource = pr.id_physical_resource
LEFT JOIN public.information_resources ir ON al.id_information_resource = ir.id_information_resource
WHERE es.status_type = 'Активная работа' AND es.employee_active = 0;
```

Представление 10. «Доступы после смены статуса»

Список атрибутов, отображаемых в рамках представления «Доступы после смены статуса»:

- Логин (отношение «employees»);
- Старый статус — Новый статус (отношение «status_log»);
- Дата смены статуса;
- Первая попытка доступа после смены (дата, ресурс, результат).

```
CREATE VIEW public.access_after_status_change
AS
SELECT
    e.login,
    prev_sl.status_type AS old_status,
    curr_sl.status_type AS new_status,
    curr_sl.record_date AS status_change_date,
    al.event_date AS first_access_after,
    COALESCE(pr.resource_name, ir.resource_name) AS resource_name,
    al.result
FROM public.status_log curr_sl
JOIN public.status_log prev_sl
    ON curr_sl.status_code = prev_sl.status_code
    AND prev_sl.record_date < curr_sl.record_date
JOIN public.employees_statuses es ON curr_sl.status_code = es.status_code
JOIN public.employees e ON es.id_employee = e.id_employee
JOIN public.access_log al
    ON e.id_employee = al.id_employee
    AND al.event_date > curr_sl.record_date
LEFT JOIN public.physical_resources pr ON al.id_physical_resource = pr.id_physical_resource
LEFT JOIN public.information_resources ir ON al.id_information_resource = ir.id_information_resource
WHERE curr_sl.status_type IN ('Уволен', 'Отпуск', 'Больничный')
ORDER BY curr_sl.record_date DESC
LIMIT 20;
```

2.4 Вывод

В ходе второго этапа работы успешно создана база данных управления доступом в СУБД PostgreSQL. Реализованы все таблицы согласно схеме, установлены связи и индексы, данные заполнены с учетом безопасности (пароли хэшированы). Созданы 10 представлений для удобного анализа прав доступа, нарушений политик, статусов сотрудников и статистики. Работа закрепила практические навыки проектирования и администрирования БД, а также применения базовых механизмов информационной безопасности.

3 ЗАЩИТА БД

3.1 Задачи по мониторингу операций в БД

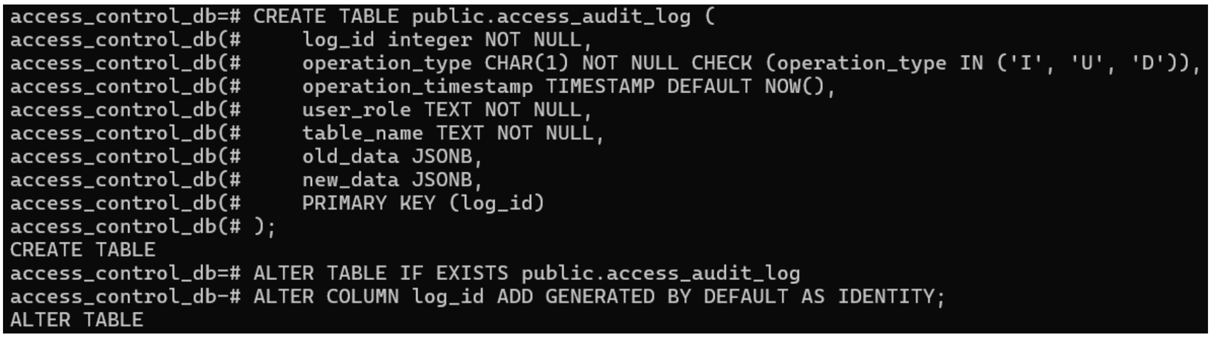
Создадим отдельную таблицу public.access_audit_log для фиксации всех операций DML (INSERT, UPDATE, DELETE) над основными таблицами БД.

```
CREATE TABLE public.access_audit_log (
   log_id integer NOT NULL,
   operation_type CHAR(1) NOT NULL CHECK (operation_type IN ('I', 'U', 'D')),
   operation_timestamp TIMESTAMP DEFAULT NOW(),
   user_role TEXT NOT NULL,
   table_name TEXT NOT NULL,
   old_data JSONB,
   new_data JSONB,
   PRIMARY KEY (log_id)
);
```

Сделаем атрибут «log_id» автогенерируемым идентификатором кортежа в таблице:

```
ALTER TABLE IF EXISTS public.access_audit_log
ALTER COLUMN log_id ADD GENERATED BY DEFAULT AS IDENTITY;
```

Процесс создания таблицы-лог в psql и ответ системы представлены на рисунке 25.



```
access_control_db=# CREATE TABLE public.access_audit_log (
access_control_db(#     log_id integer NOT NULL,
access_control_db(#     operation_type CHAR(1) NOT NULL CHECK (operation_type IN ('I', 'U', 'D')),
access_control_db(#     operation_timestamp TIMESTAMP DEFAULT NOW(),
access_control_db(#     user_role TEXT NOT NULL,
access_control_db(#     table_name TEXT NOT NULL,
access_control_db(#     old_data JSONB,
access_control_db(#     new_data JSONB,
access_control_db(#     PRIMARY KEY (log_id)
access_control_db(# );
CREATE TABLE
access_control_db=# ALTER TABLE IF EXISTS public.access_audit_log
access_control_db-# ALTER COLUMN log_id ADD GENERATED BY DEFAULT AS IDENTITY;
ALTER TABLE
```

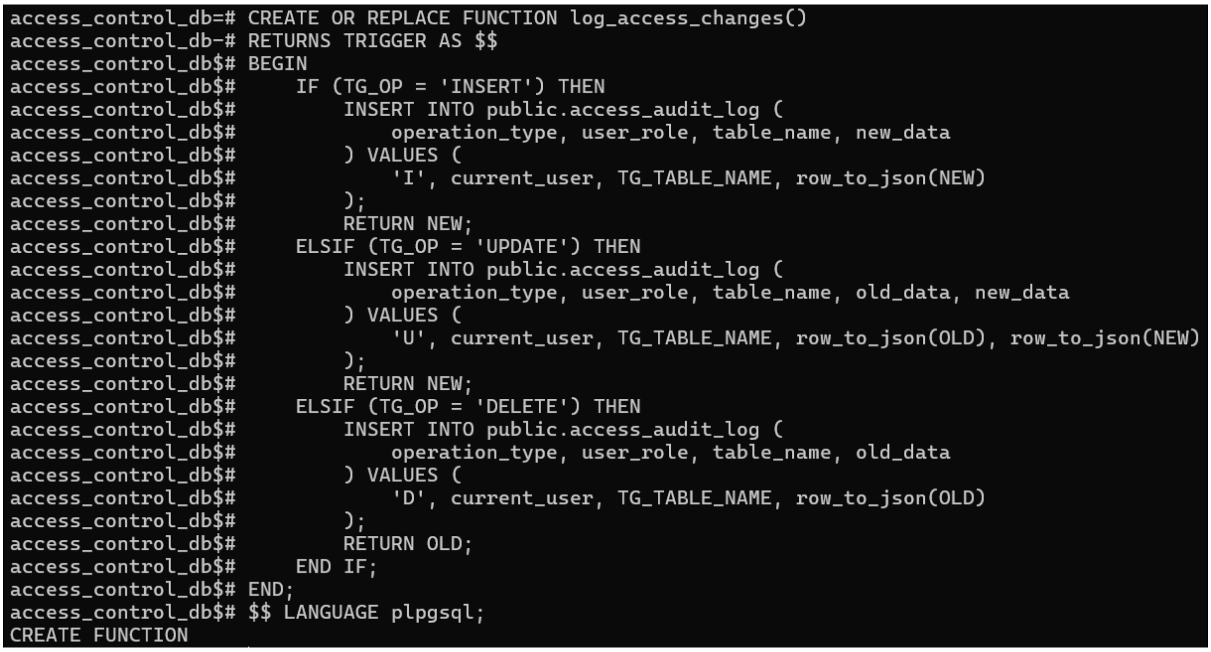
Рисунок 25 – Создание таблицы-лог

Создадим универсальную триггерную функцию log_access_changes():

```
CREATE OR REPLACE FUNCTION log_access_changes()
```

```
RETURNS TRIGGER AS $$
BEGIN
  IF (TG_OP = 'INSERT') THEN
    INSERT INTO public.access_audit_log (
      operation_type, user_role, table_name, new_data
    ) VALUES (
      'I', current_user, TG_TABLE_NAME, row_to_json(NEW)
    );
    RETURN NEW;
  ELSIF (TG_OP = 'UPDATE') THEN
    INSERT INTO public.access_audit_log (
      operation_type, user_role, table_name, old_data, new_data
    ) VALUES (
      'U', current_user, TG_TABLE_NAME, row_to_json(OLD), row_to_json(NEW)
    );
    RETURN NEW;
  ELSIF (TG_OP = 'DELETE') THEN
    INSERT INTO public.access_audit_log (
      operation_type, user_role, table_name, old_data
    ) VALUES (
      'D', current_user, TG_TABLE_NAME, row_to_json(OLD)
    );
    RETURN OLD;
  END IF;
END;
$$ LANGUAGE plpgsql;
```

Процесс создания триггерной функции в psql и ответ системы представлены на рисунке 26.



```
access_control_db=# CREATE OR REPLACE FUNCTION log_access_changes()
access_control_db-# RETURNS TRIGGER AS $$
access_control_db$# BEGIN
access_control_db$#     IF (TG_OP = 'INSERT') THEN
access_control_db$#         INSERT INTO public.access_audit_log (
access_control_db$#             operation_type, user_role, table_name, new_data
access_control_db$#         ) VALUES (
access_control_db$#             'I', current_user, TG_TABLE_NAME, row_to_json(NEW)
access_control_db$#         );
access_control_db$#         RETURN NEW;
access_control_db$#     ELSIF (TG_OP = 'UPDATE') THEN
access_control_db$#         INSERT INTO public.access_audit_log (
access_control_db$#             operation_type, user_role, table_name, old_data, new_data
access_control_db$#         ) VALUES (
access_control_db$#             'U', current_user, TG_TABLE_NAME, row_to_json(OLD), row_to_json(NEW)
access_control_db$#         );
access_control_db$#         RETURN NEW;
access_control_db$#     ELSIF (TG_OP = 'DELETE') THEN
access_control_db$#         INSERT INTO public.access_audit_log (
access_control_db$#             operation_type, user_role, table_name, old_data
access_control_db$#         ) VALUES (
access_control_db$#             'D', current_user, TG_TABLE_NAME, row_to_json(OLD)
access_control_db$#         );
access_control_db$#         RETURN OLD;
access_control_db$#     END IF;
access_control_db$# END;
access_control_db$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Рисунок 26 – Создание универсальной триггерной функции

Теперь создадим триггеры для всех основных таблиц:

- **Для таблицы сотрудников**

CREATE TRIGGER log_employees_changes

AFTER INSERT OR UPDATE OR DELETE ON public.employees

FOR EACH ROW EXECUTE FUNCTION log_access_changes();

- **Для таблицы команд**

CREATE TRIGGER log_teams_changes

AFTER INSERT OR UPDATE OR DELETE ON public.teams

FOR EACH ROW EXECUTE FUNCTION log_access_changes();

- **Для таблицы статусов**

CREATE TRIGGER log_emp_statuses_changes

AFTER INSERT OR UPDATE OR DELETE ON public.employees_statuses

FOR EACH ROW EXECUTE FUNCTION log_access_changes();

- **Для таблицы ресурсов (физических и информационных)**

CREATE TRIGGER log_phys_resources_changes

AFTER INSERT OR UPDATE OR DELETE ON public.physical_resources

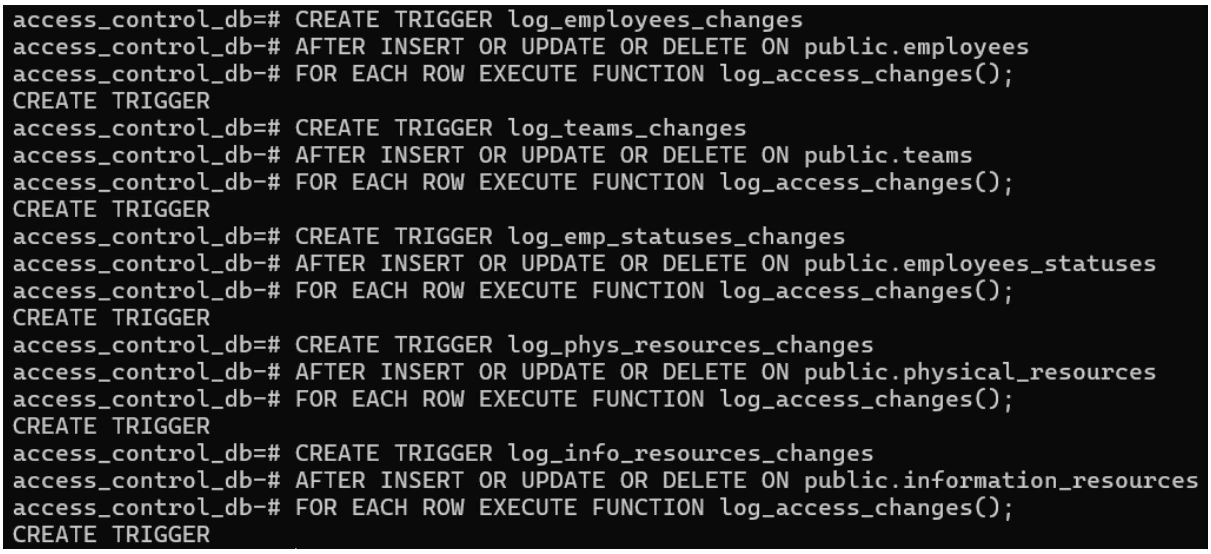
FOR EACH ROW EXECUTE FUNCTION log_access_changes();

CREATE TRIGGER log_info_resources_changes

AFTER INSERT OR UPDATE OR DELETE ON public.information_resources

FOR EACH ROW EXECUTE FUNCTION log_access_changes();

Процесс создания триггеров в psql и ответ системы представлены на рисунке 27.



```
access_control_db=# CREATE TRIGGER log_employees_changes
access_control_db-# AFTER INSERT OR UPDATE OR DELETE ON public.employees
access_control_db-# FOR EACH ROW EXECUTE FUNCTION log_access_changes();
CREATE TRIGGER
access_control_db=# CREATE TRIGGER log_teams_changes
access_control_db-# AFTER INSERT OR UPDATE OR DELETE ON public.teams
access_control_db-# FOR EACH ROW EXECUTE FUNCTION log_access_changes();
CREATE TRIGGER
access_control_db=# CREATE TRIGGER log_emp_statuses_changes
access_control_db-# AFTER INSERT OR UPDATE OR DELETE ON public.employees_statuses
access_control_db-# FOR EACH ROW EXECUTE FUNCTION log_access_changes();
CREATE TRIGGER
access_control_db=# CREATE TRIGGER log_phys_resources_changes
access_control_db-# AFTER INSERT OR UPDATE OR DELETE ON public.physical_resources
access_control_db-# FOR EACH ROW EXECUTE FUNCTION log_access_changes();
CREATE TRIGGER
access_control_db=# CREATE TRIGGER log_info_resources_changes
access_control_db-# AFTER INSERT OR UPDATE OR DELETE ON public.information_resources
access_control_db-# FOR EACH ROW EXECUTE FUNCTION log_access_changes();
CREATE TRIGGER
```

Рисунок 27 – Создание всех триггеров для основных таблиц

Выполним тестовые операции:

- **Вставка нового сотрудника**

INSERT INTO public.employees (id_employee, login, password, full_name, email, phone, id_role, id_team)

VALUES (9, 'test_user', crypt('test123', gen_salt('bf')), 'Тестовый Пользователь', 'test@company.com', '+79990000000', 1, NULL);

- **Обновление статуса**

UPDATE public.employees_statuses

SET status_type = 'Больничный'

WHERE id_employee = 1;

- **Удаление команды**

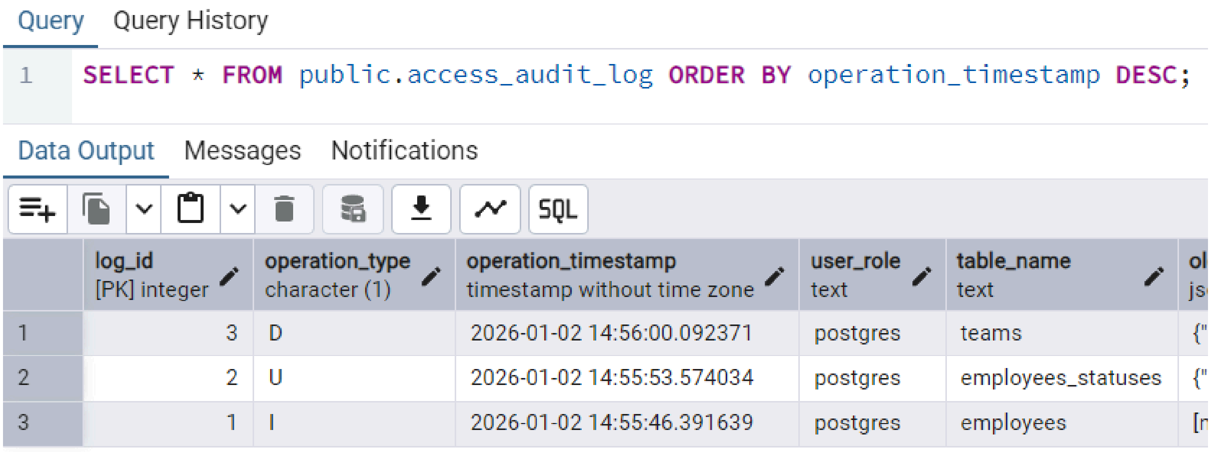
DELETE FROM public.teams

WHERE id_team = 7;

Просмотр таблицы-лог:

*SELECT * FROM public.access_audit_log ORDER BY operation_timestamp DESC;*

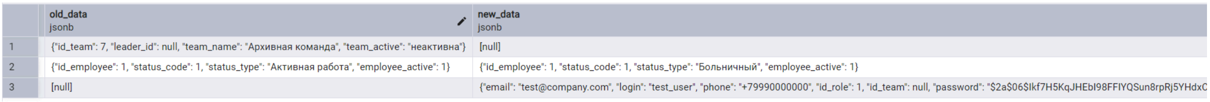
Таблица-лог после всех преобразований представлена на рисунках 28 и 29:



```sql
SELECT * FROM public.access_audit_log ORDER BY operation_timestamp DESC;
```

	log_id [PK] integer	operation_type character (1)	operation_timestamp timestamp without time zone	user_role text	table_name text	ol js
1	3	D	2026-01-02 14:56:00.092371	postgres	teams	{"
2	2	U	2026-01-02 14:55:53.574034	postgres	employees_statuses	{"
3	1	I	2026-01-02 14:55:46.391639	postgres	employees	[r

Рисунок 28 – Таблица-лог после изменений основных таблиц (логирование операций) (1)



	old_data jsonb	new_data jsonb
1	{"id_team": 7, "leader_id": null, "team_name": "Архивная команда", "team_active": "неактивна"}	[null]
2	{"id_employee": 1, "status_code": 1, "status_type": "Активная работа", "employee_active": 1}	{"id_employee": 1, "status_code": 1, "status_type": "Больничный", "employee_active": 1}
3	[null]	{"email": "test@company.com", "login": "test_user", "phone": "+79990000000", "id_role": 1, "id_team": null, "password": "$2a$06$Ikf7H5KqJHEbI98FFIYQSun8rpRj5YHdxC

Рисунок 29 – Таблица-лог после изменений основных таблиц (логирование операций) (2)

В таблице access_audit_log зафиксированы все операции с указанием пользователя, времени, типа операции и измененных данных.

3.2 Задачи по шифрованию данных

Прежде чем начинать заниматься созданием секретных данных и их шифрованием, необходимо активировать модуль pgcrypto:

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

Пусть индивидуальный пароль суперпользователя *admin_secret_key_for_tokens*. Сгенерируем из него ключ через SHA-256 (32 байта – для AES):

```
SELECT encode(digest('admin_secret_key_for_tokens', 'sha256'), 'hex');
```

Результат:

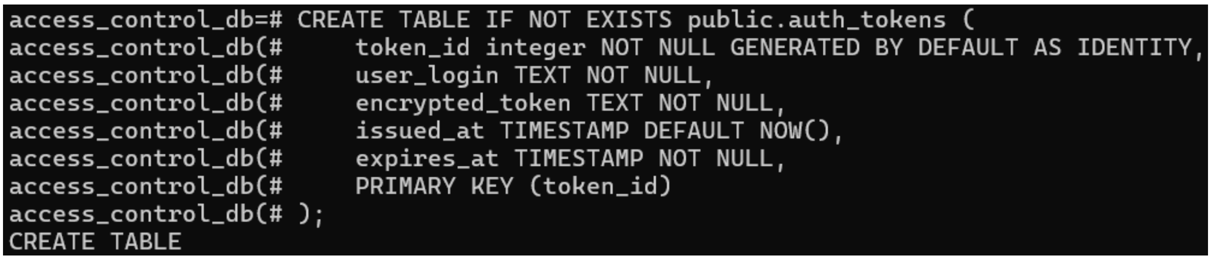
a44951a9923c54962512e4127234ff5a2f007e692a143fe67cce6e4f403478d6.

Полученный хэш будем использовать как симметричный ключ шифрования.

Создадим таблицу с секретными данными. Например, таблица с токенами аутентификации для каждого пользователей.

```
CREATE TABLE IF NOT EXISTS public.auth_tokens (
    token_id integer NOT NULL GENERATED BY DEFAULT AS IDENTITY,
    user_login TEXT NOT NULL,
    encrypted_token TEXT NOT NULL,
    issued_at TIMESTAMP DEFAULT NOW(),
    expires_at TIMESTAMP NOT NULL,
    PRIMARY KEY (token_id)
);
```

Процесс создания таблицы с секретными данными в psql и ответ системы представлены на рисунке 30.



```
access_control_db=# CREATE TABLE IF NOT EXISTS public.auth_tokens (
access_control_db(#     token_id integer NOT NULL GENERATED BY DEFAULT AS IDENTITY,
access_control_db(#     user_login TEXT NOT NULL,
access_control_db(#     encrypted_token TEXT NOT NULL,
access_control_db(#     issued_at TIMESTAMP DEFAULT NOW(),
access_control_db(#     expires_at TIMESTAMP NOT NULL,
access_control_db(#     PRIMARY KEY (token_id)
access_control_db(# );
CREATE TABLE
```

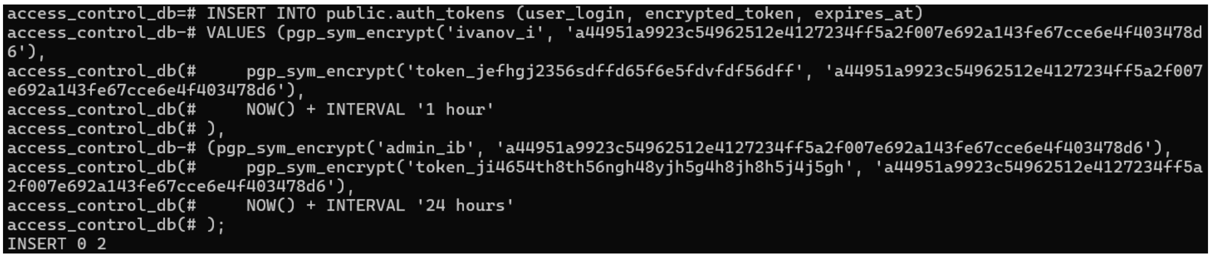
Рисунок 30 – Создание таблицы с секретными данными

Вставим данные в зашифрованном виде:

```
INSERT INTO public.auth_tokens (user_login, encrypted_token, expires_at)
```

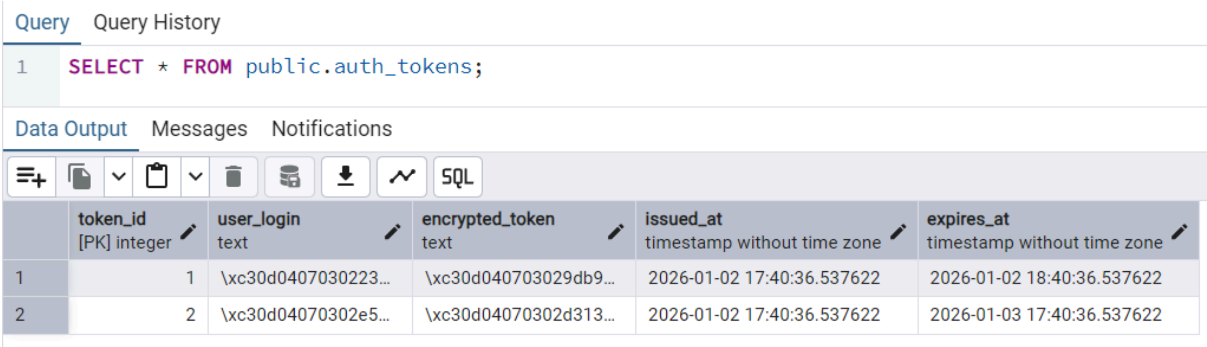
```
VALUES (pgp_sym_encrypt('ivanov_i',
'a44951a9923c54962512e4127234ff5a2f007e692a143fe67cce6e4f403478d6'),
pgp_sym_encrypt('token_jefhgj2356sdffd65f6e5fdvfdf56dff',
'a44951a9923c54962512e4127234ff5a2f007e692a143fe67cce6e4f403478d6'),
    NOW() + INTERVAL '1 hour'
),
(pgp_sym_encrypt('admin_ib',
'a44951a9923c54962512e4127234ff5a2f007e692a143fe67cce6e4f403478d6'),
pgp_sym_encrypt('token_ji4654th8th56ngh48yjh5g4h8jh8h5j4j5gh',
'a44951a9923c54962512e4127234ff5a2f007e692a143fe67cce6e4f403478d6'),
    NOW() + INTERVAL '24 hours'
);
```

Процесс добавления токенов в psql и ответ системы представлены на рисунках 31 и 32.



```
access_control_db=# INSERT INTO public.auth_tokens (user_login, encrypted_token, expires_at)
access_control_db-# VALUES (pgp_sym_encrypt('ivanov_i', 'a44951a9923c54962512e4127234ff5a2f007e692a143fe67cce6e4f403478d
6'),
access_control_db(#     pgp_sym_encrypt('token_jefhgj2356sdffd65f6e5fdvfdf56dff', 'a44951a9923c54962512e4127234ff5a2f007
e692a143fe67cce6e4f403478d6'),
access_control_db(#     NOW() + INTERVAL '1 hour'
access_control_db(# ),
access_control_db-# (pgp_sym_encrypt('admin_ib', 'a44951a9923c54962512e4127234ff5a2f007e692a143fe67cce6e4f403478d6'),
access_control_db(#     pgp_sym_encrypt('token_ji4654th8th56ngh48yjh5g4h8jh8h5j4j5gh', 'a44951a9923c54962512e4127234ff5a
2f007e692a143fe67cce6e4f403478d6'),
access_control_db(#     NOW() + INTERVAL '24 hours'
access_control_db(# );
INSERT 0 2
```

Рисунок 31 – Успешное добавление токенов



```
SELECT * FROM public.auth_tokens;
```

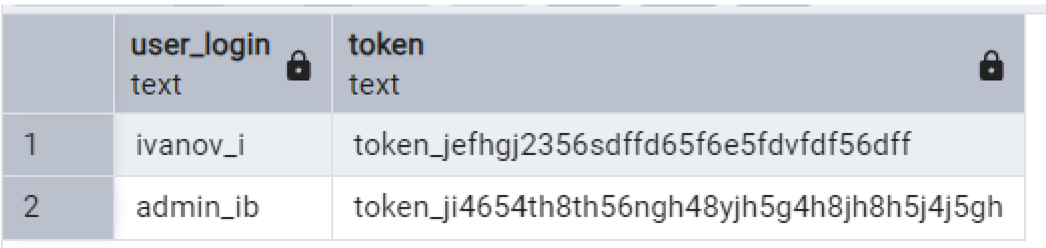
	token_id [PK] integer	user_login text	encrypted_token text	issued_at timestamp without time zone	expires_at timestamp without time zone
1	1	\xc30d0407030223...	\xc30d040703029db9...	2026-01-02 17:40:36.537622	2026-01-02 18:40:36.537622
2	2	\xc30d04070302e5...	\xc30d04070302d313...	2026-01-02 17:40:36.537622	2026-01-03 17:40:36.537622

Рисунок 32 – Демонстрация таблицы с секретными данными

Попытка дешифровки данных:

```
SELECT pgp_sym_decrypt(user_login::bytea,
'a44951a9923c54962512e4127234ff5a2f007e692a143fe67cce6e4f403478d6') AS
user_login, pgp_sym_decrypt(encrypted_token::bytea,
'a44951a9923c54962512e4127234ff5a2f007e692a143fe67cce6e4f403478d6') AS token
FROM public.auth_tokens;
```

Попытка дешифрования токенов представлена на рисунке 33.



	user_login text	token text
1	ivanov_i	token_jefhgj2356sdffd65f6e5fdvfdf56dff
2	admin_ib	token_ji4654th8th56ngh48yjh5g4h8jh8h5j4j5gh

Рисунок 33 – Успешная дешифровка данных

3.3 Задачи по разграничению доступа в БД

Согласно заданию, необходимо создать как минимум 2 групповые роля для каждого из классов потребителей информации. В первой лабораторной были выделены потребители «Администратор ИБ», «Руководитель команды», «Сотрудник», «Аудитор» и «Служба безопасности».

- **Администратор ИБ**

```
CREATE ROLE ib_admin_group WITH
  NOLOGIN
  NOSUPERUSER
  NOCREATEDB
  NOCREATEROLE
  INHERIT;
```

- **Руководитель команды**

```
CREATE ROLE team_lead_group WITH
  NOLOGIN
  NOSUPERUSER
  NOCREATEDB
  NOCREATEROLE
  INHERIT;
```

- **Сотрудник**

```
CREATE ROLE employee_group WITH
  NOLOGIN
  NOSUPERUSER
  NOCREATEDB
  NOCREATEROLE
  INHERIT;
```

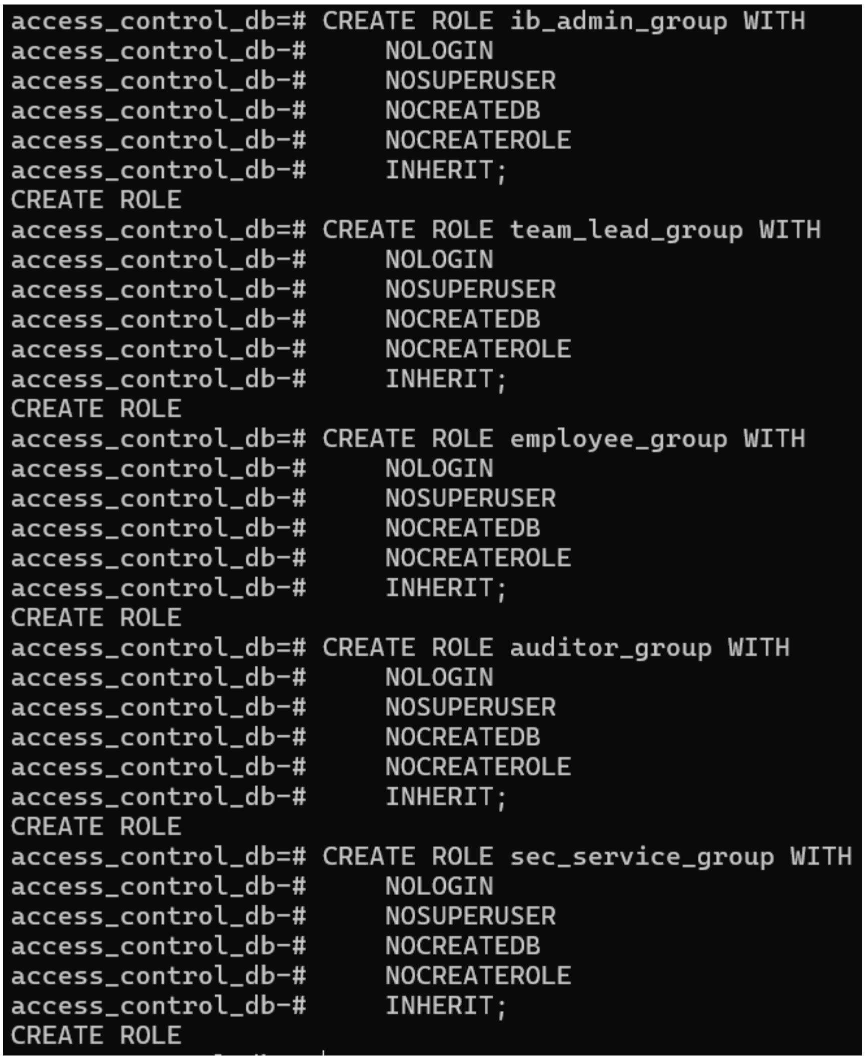
- **Аудитор**

```
CREATE ROLE auditor_group WITH
  NOLOGIN
  NOSUPERUSER
  NOCREATEDB
  NOCREATEROLE
  INHERIT;
```

- **Служба безопасности**

```
CREATE ROLE sec_service_group WITH
  NOLOGIN
  NOSUPERUSER
  NOCREATEDB
  NOCREATEROLE
  INHERIT;
```

Процесс создания ролей в psql и ответ системы представлены на рисунке 34.



```
access_control_db=# CREATE ROLE ib_admin_group WITH
access_control_db-#     NOLOGIN
access_control_db-#     NOSUPERUSER
access_control_db-#     NOCREATEDB
access_control_db-#     NOCREATEROLE
access_control_db-#     INHERIT;
CREATE ROLE
access_control_db=# CREATE ROLE team_lead_group WITH
access_control_db-#     NOLOGIN
access_control_db-#     NOSUPERUSER
access_control_db-#     NOCREATEDB
access_control_db-#     NOCREATEROLE
access_control_db-#     INHERIT;
CREATE ROLE
access_control_db=# CREATE ROLE employee_group WITH
access_control_db-#     NOLOGIN
access_control_db-#     NOSUPERUSER
access_control_db-#     NOCREATEDB
access_control_db-#     NOCREATEROLE
access_control_db-#     INHERIT;
CREATE ROLE
access_control_db=# CREATE ROLE auditor_group WITH
access_control_db-#     NOLOGIN
access_control_db-#     NOSUPERUSER
access_control_db-#     NOCREATEDB
access_control_db-#     NOCREATEROLE
access_control_db-#     INHERIT;
CREATE ROLE
access_control_db=# CREATE ROLE sec_service_group WITH
access_control_db-#     NOLOGIN
access_control_db-#     NOSUPERUSER
access_control_db-#     NOCREATEDB
access_control_db-#     NOCREATEROLE
access_control_db-#     INHERIT;
CREATE ROLE
```

Рисунок 34 – Создание всех ролей

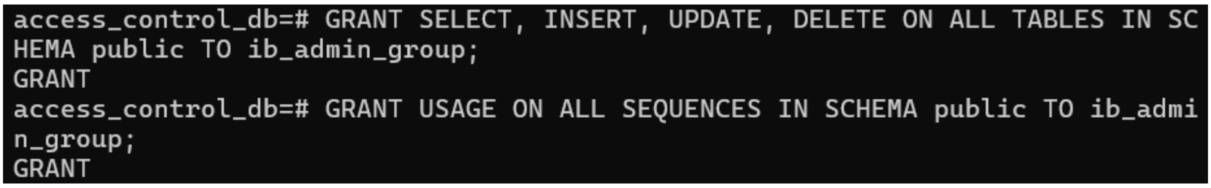
Согласно целям проектирования БД и исходя из принципа представления наименьших привилегий, выделим групповым ролям следующие привилегии:

- **Администратор ИБ**: полный доступ ко всем таблицам и представлениям.

GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO ib_admin_group;

GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO ib_admin_group;

Процесс создания привилегий для роли администратора ИБ в psql и ответ системы представлены на рисунке 35.



```
access_control_db=# GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SC
HEMA public TO ib_admin_group;
GRANT
access_control_db=# GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO ib_admi
n_group;
GRANT
```

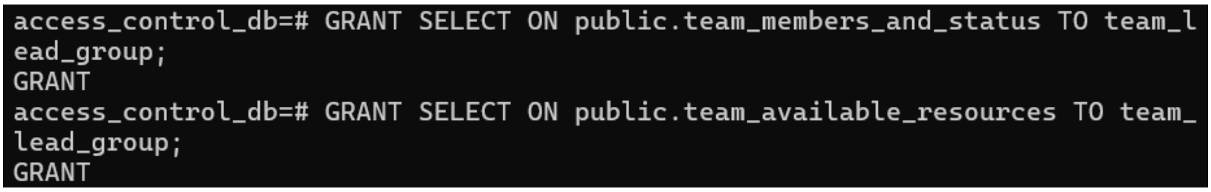
Рисунок 35 – Привилегии для роли администратора ИБ

- **Руководитель команды**: только к своим представлениям.

GRANT SELECT ON public.team_members_and_status TO team_lead_group;

GRANT SELECT ON public.team_available_resources TO team_lead_group;

Процесс создания привилегий для роли руководителя команды в psql и ответ системы представлены на рисунке 36.



```
access_control_db=# GRANT SELECT ON public.team_members_and_status TO team_l
ead_group;
GRANT
access_control_db=# GRANT SELECT ON public.team_available_resources TO team_
lead_group;
GRANT
```

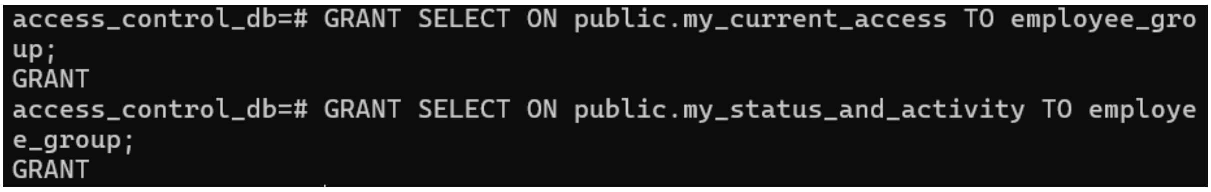
Рисунок 36 – Привилегии для роли руководителя команды

- **Сотрудник**: только к своим данным.

GRANT SELECT ON public.my_current_access TO employee_group;

GRANT SELECT ON public.my_status_and_activity TO employee_group;

Процесс создания привилегий для роли сотрудника в psql и ответ системы представлены на рисунке 37.



```
access_control_db=# GRANT SELECT ON public.my_current_access TO employee_gro
up;
GRANT
access_control_db=# GRANT SELECT ON public.my_status_and_activity TO employe
e_group;
GRANT
```

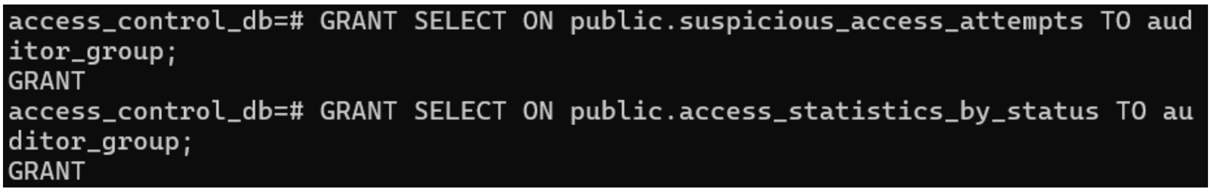
Рисунок 37 – Привилегии для роли сотрудника

- **Аудитор**: только к логам и статистике.

GRANT SELECT ON public.suspicious_access_attempts TO auditor_group;

GRANT SELECT ON public.access_statistics_by_status TO auditor_group;

Процесс создания привилегий для роли аудитора в psql и ответ системы представлены на рисунке 38.



```
access_control_db=# GRANT SELECT ON public.suspicious_access_attempts TO aud
itor_group;
GRANT
access_control_db=# GRANT SELECT ON public.access_statistics_by_status TO au
ditor_group;
GRANT
```

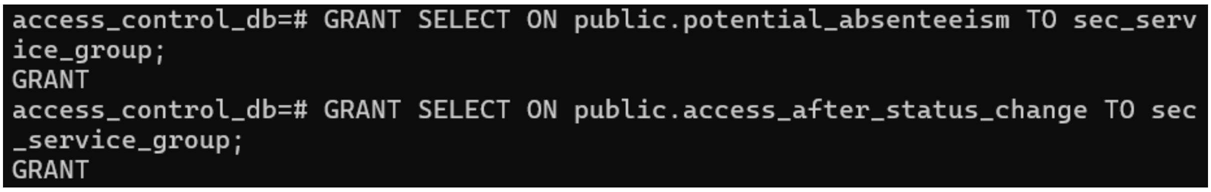
Рисунок 38 – Привилегии для роли аудитора

- **Служба безопасности**: мониторинг без доступа к логам аудита.

GRANT SELECT ON public.potential_absenteeism TO sec_service_group;

GRANT SELECT ON public.access_after_status_change TO sec_service_group;

Процесс создания привилегий для роли представителя службы безопасности в psql и ответ системы представлены на рисунке 39.



```
access_control_db=# GRANT SELECT ON public.potential_absenteeism TO sec_serv
ice_group;
GRANT
access_control_db=# GRANT SELECT ON public.access_after_status_change TO sec
_service_group;
GRANT
```

Рисунок 39 – Привилегии для роли представителя службы безопасности

- Запрет доступа к таблицам логирования и токенов для всех, кроме админа.

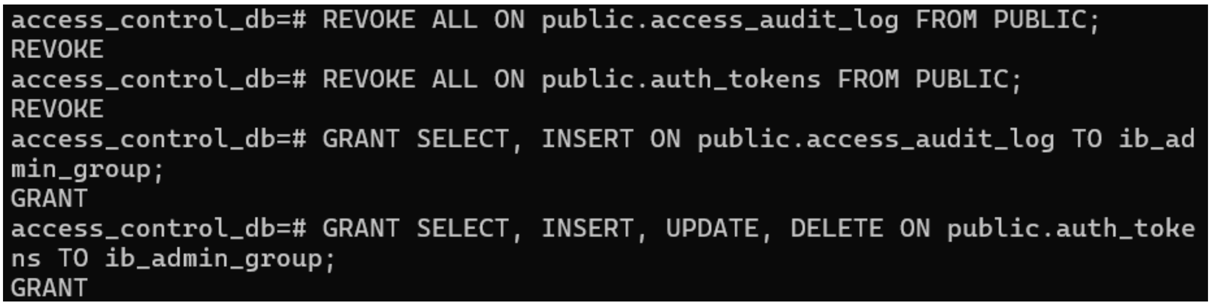
REVOKE ALL ON public.access_audit_log FROM PUBLIC;

REVOKE ALL ON public.auth_tokens FROM PUBLIC;

GRANT SELECT, INSERT ON public.access_audit_log TO ib_admin_group;

```
GRANT SELECT, INSERT, UPDATE, DELETE ON public.auth_tokens TO ib_admin_group;
```

Процесс установки запретов в psql и ответ системы представлены на рисунке 40.



```
access_control_db=# REVOKE ALL ON public.access_audit_log FROM PUBLIC;
REVOKE
access_control_db=# REVOKE ALL ON public.auth_tokens FROM PUBLIC;
REVOKE
access_control_db=# GRANT SELECT, INSERT ON public.access_audit_log TO ib_ad
min_group;
GRANT
access_control_db=# GRANT SELECT, INSERT, UPDATE, DELETE ON public.auth_toke
ns TO ib_admin_group;
GRANT
```

Рисунок 40 – Успешная установка запретов

Теперь создадим по одному пользователю для каждой из групповых ролей. Выделим им привилегии группы и продемонстрируем работу предлагаемой системы разграничения доступа на примере доступа к случайным таблицам.

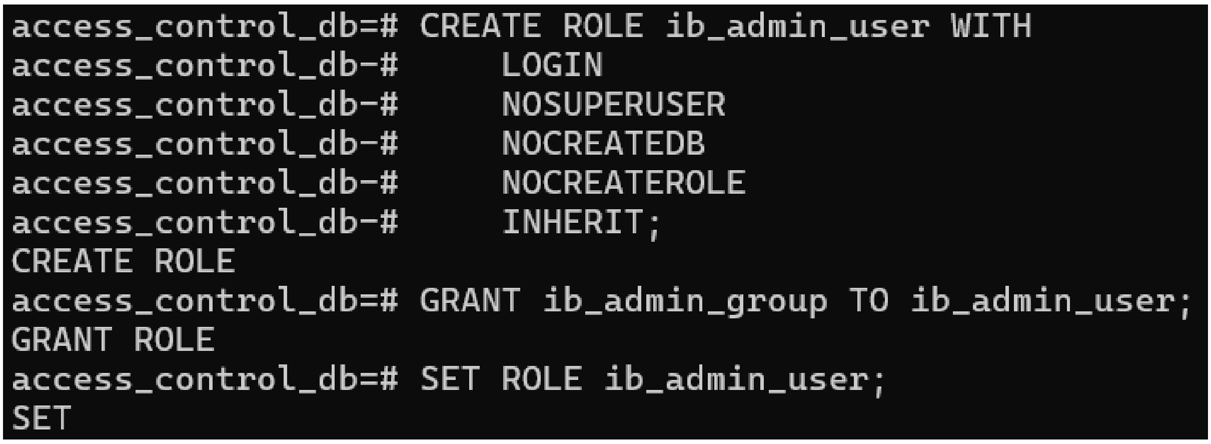
Чтобы протестировать созданных пользователей, необходимо зайти со стороны этих самых ролей. Делается это по следующему шаблону:

```
SET ROLE имя_пользователя;
```

- **Администратор ИБ**

```
CREATE ROLE ib_admin_user WITH
   LOGIN
   NOSUPERUSER
   NOCREATEDB
   NOCREATEROLE
   INHERIT;
GRANT ib_admin_group TO ib_admin_user;
```

Процесс создания пользователя ib_admin_user в psql и ответ системы представлены на рисунке 41.



```
access_control_db=# CREATE ROLE ib_admin_user WITH
access_control_db-#     LOGIN
access_control_db-#     NOSUPERUSER
access_control_db-#     NOCREATEDB
access_control_db-#     NOCREATEROLE
access_control_db-#     INHERIT;
CREATE ROLE
access_control_db=# GRANT ib_admin_group TO ib_admin_user;
GRANT ROLE
access_control_db=# SET ROLE ib_admin_user;
SET
```

Рисунок 41 – Успешное создание пользователя ib_admin_user

- **Руководитель команды**

CREATE ROLE team_lead_user WITH

LOGIN

NOSUPERUSER

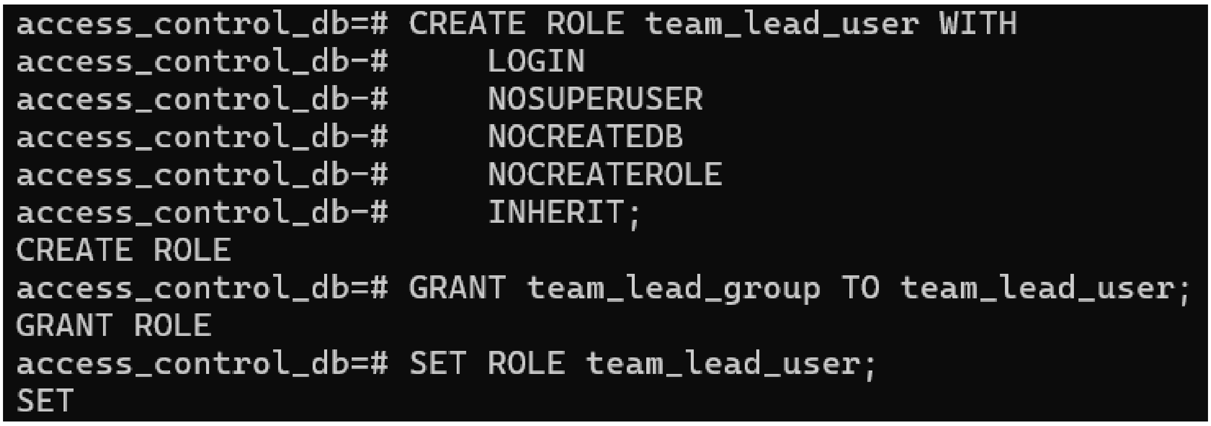
NOCREATEDB

NOCREATEROLE

INHERIT;

GRANT team_lead_group TO team_lead_user;

Процесс создания пользователя team_lead_user в psql и ответ системы представлены на рисунке 42.



```
access_control_db=# CREATE ROLE team_lead_user WITH
access_control_db-#     LOGIN
access_control_db-#     NOSUPERUSER
access_control_db-#     NOCREATEDB
access_control_db-#     NOCREATEROLE
access_control_db-#     INHERIT;
CREATE ROLE
access_control_db=# GRANT team_lead_group TO team_lead_user;
GRANT ROLE
access_control_db=# SET ROLE team_lead_user;
SET
```

Рисунок 42 – Успешное создание пользователя team_lead_user

- **Сотрудник**

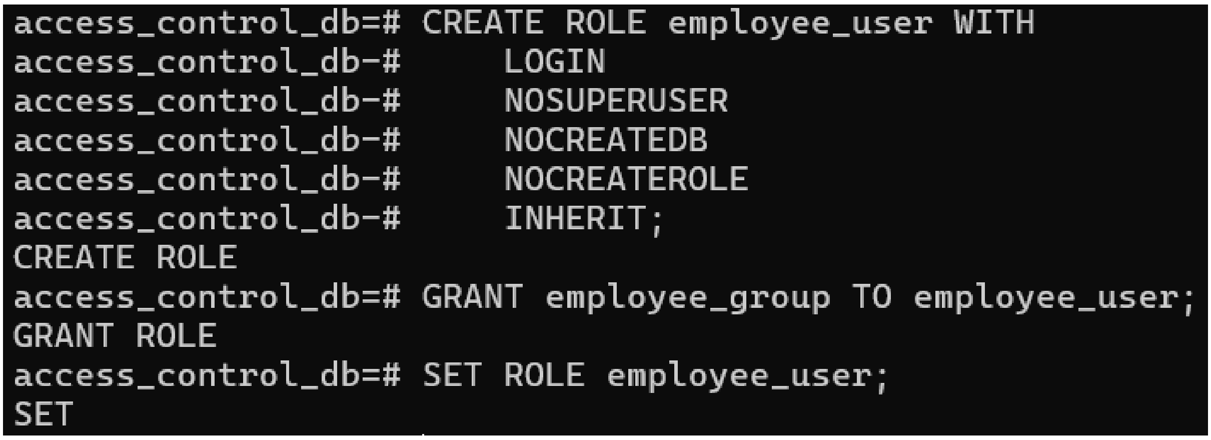
CREATE ROLE employee_user WITH

LOGIN

NOSUPERUSER

```
    NOCREATEDB
    NOCREATEROLE
    INHERIT;
GRANT employee_group TO employee_user;
```

Процесс создания пользователя employee_user в psql и ответ системы представлены на рисунке 43.



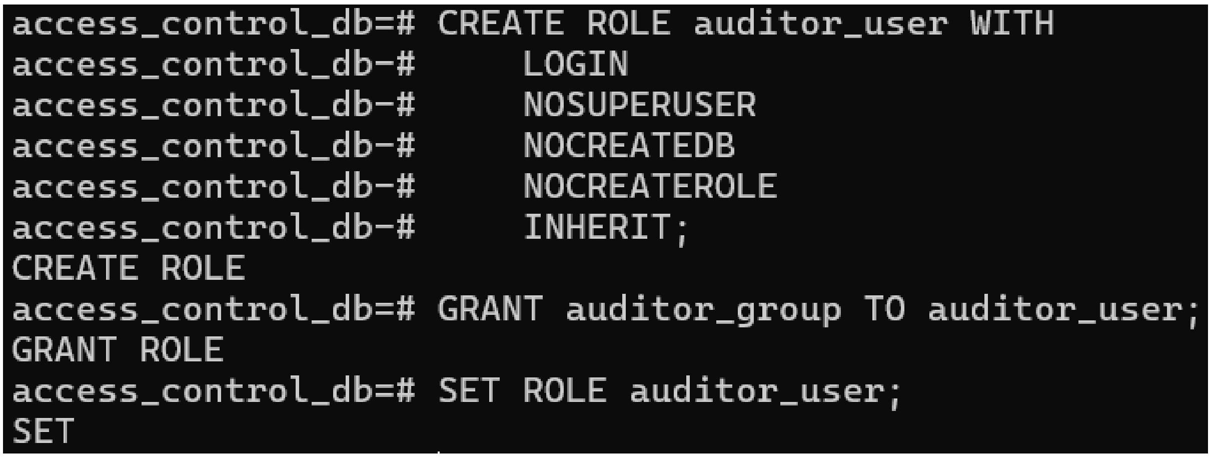
```
access_control_db=# CREATE ROLE employee_user WITH
access_control_db-#     LOGIN
access_control_db-#     NOSUPERUSER
access_control_db-#     NOCREATEDB
access_control_db-#     NOCREATEROLE
access_control_db-#     INHERIT;
CREATE ROLE
access_control_db=# GRANT employee_group TO employee_user;
GRANT ROLE
access_control_db=# SET ROLE employee_user;
SET
```

Рисунок 43 – Успешное создание пользователя employee_user

- **Аудитор**

```
CREATE ROLE auditor_user WITH
    LOGIN
    NOSUPERUSER
    NOCREATEDB
    NOCREATEROLE
    INHERIT;
GRANT auditor_group TO auditor_user;
```

Процесс создания пользователя auditor_user в psql и ответ системы представлены на рисунке 44.



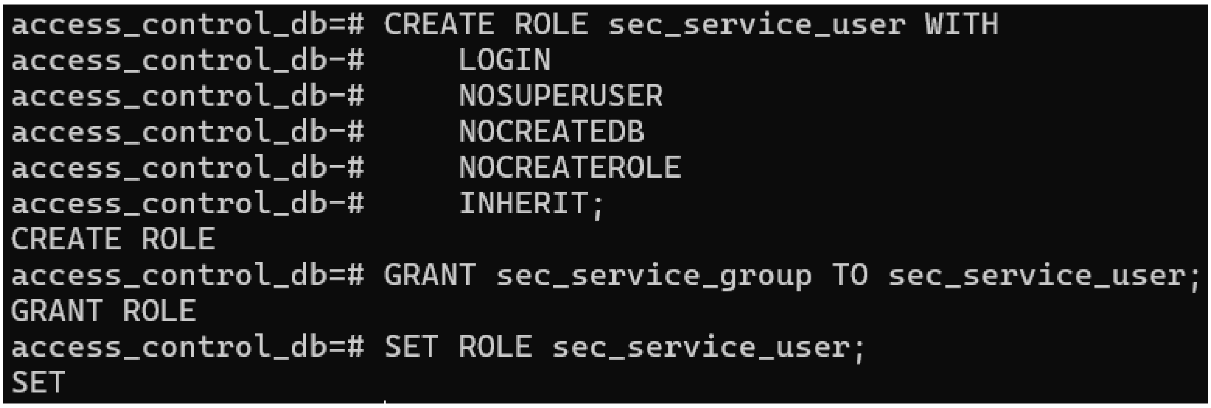
```
access_control_db=# CREATE ROLE auditor_user WITH
access_control_db-#     LOGIN
access_control_db-#     NOSUPERUSER
access_control_db-#     NOCREATEDB
access_control_db-#     NOCREATEROLE
access_control_db-#     INHERIT;
CREATE ROLE
access_control_db=# GRANT auditor_group TO auditor_user;
GRANT ROLE
access_control_db=# SET ROLE auditor_user;
SET
```

Рисунок 44 – Успешное создание пользователя auditor_user

- **Служба безопасности**

```
CREATE ROLE sec_service_user WITH
   LOGIN
   NOSUPERUSER
   NOCREATEDB
   NOCREATEROLE
   INHERIT;
GRANT sec_service_group TO sec_service_user;
```

Процесс создания пользователя sec_service_user в psql и ответ системы представлены на рисунке 45.



```
access_control_db=# CREATE ROLE sec_service_user WITH
access_control_db-#     LOGIN
access_control_db-#     NOSUPERUSER
access_control_db-#     NOCREATEDB
access_control_db-#     NOCREATEROLE
access_control_db-#     INHERIT;
CREATE ROLE
access_control_db=# GRANT sec_service_group TO sec_service_user;
GRANT ROLE
access_control_db=# SET ROLE sec_service_user;
SET
```

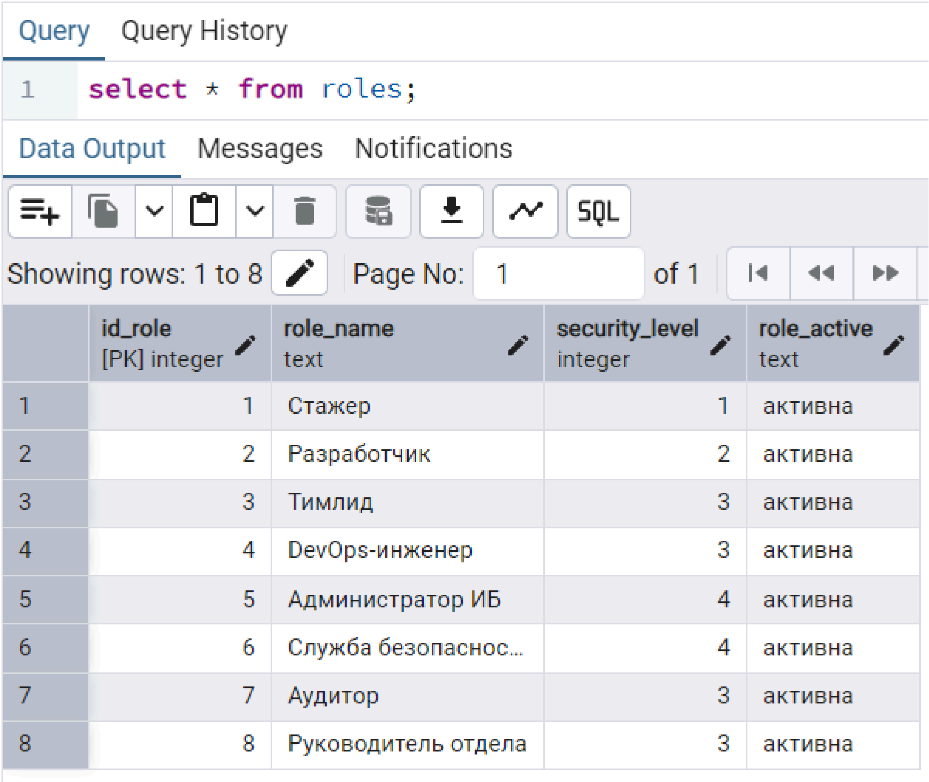
Рисунок 45 – Успешное создание пользователя sec_service_user

Осталось протестировать все созданные роли с помощью пользователей, созданных с помощью этих ролей.

- Администратор ИБ

Администратор ИБ должен обладать полным обзором всей системы. Через представление «Общий обзор прав доступа» он должен видеть полную матрицу доступа: для каждого сотрудника – его логин, ФИО, текущую роль (с уровнем безопасности), название команды и все ресурсы, к которым у команды есть доступ (включая тип ресурса, его название и уровень безопасности). Это позволяет моментально выявить несоответствия: например, если сотрудник с ролью «Стажер» (уровень 1) имеет доступ к «Базе данных клиентов» (уровень 4). Представление «Попытки нарушения политик безопасности» должно показывать все попытки доступа, которые были заблокированы системой: доступ в статусе «Уволен», попытки чтения ресурсов с уровнем выше, чем у роли, и т.д. – с указанием даты, результата и причины отказа. Также администратор должен иметь доступ ко всем остальным созданным таблицам.

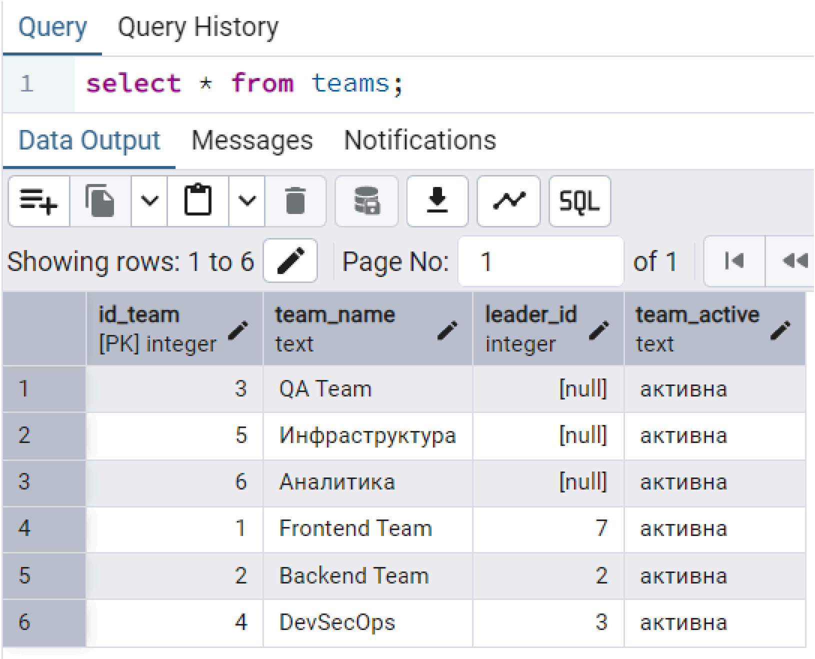
Проверка доступа к таблицам для ib_admin_user представлена на рисунках с 46 до 66.



```
select * from roles;
```

	id_role [PK] integer	role_name text	security_level integer	role_active text
1	1	Стажер	1	активна
2	2	Разработчик	2	активна
3	3	Тимлид	3	активна
4	4	DevOps-инженер	3	активна
5	5	Администратор ИБ	4	активна
6	6	Служба безопаснос...	4	активна
7	7	Аудитор	3	активна
8	8	Руководитель отдела	3	активна

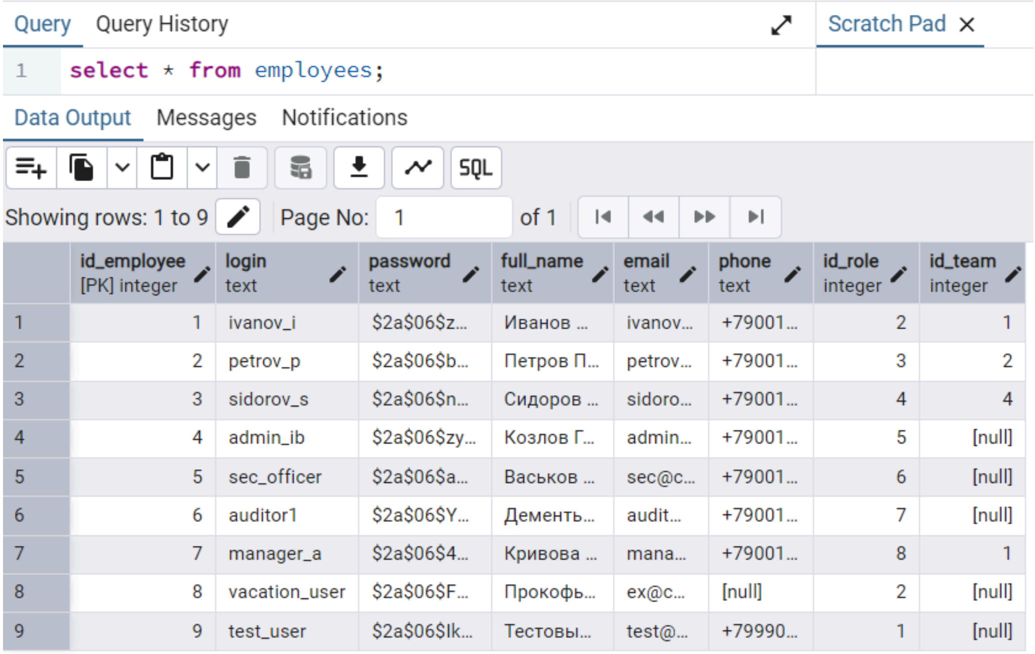
Рисунок 46 – Проверка доступа к таблицам для ib_admin_user (1)


Query
Query History
Data Output
Messages
Notifications
Showing rows: 1 to 6
Page No: 1
of 1

```sql
select * from teams;
```

	id_team [PK] integer	team_name text	leader_id integer	team_active text
1	3	QA Team	[null]	активна
2	5	Инфраструктура	[null]	активна
3	6	Аналитика	[null]	активна
4	1	Frontend Team	7	активна
5	2	Backend Team	2	активна
6	4	DevSecOps	3	активна

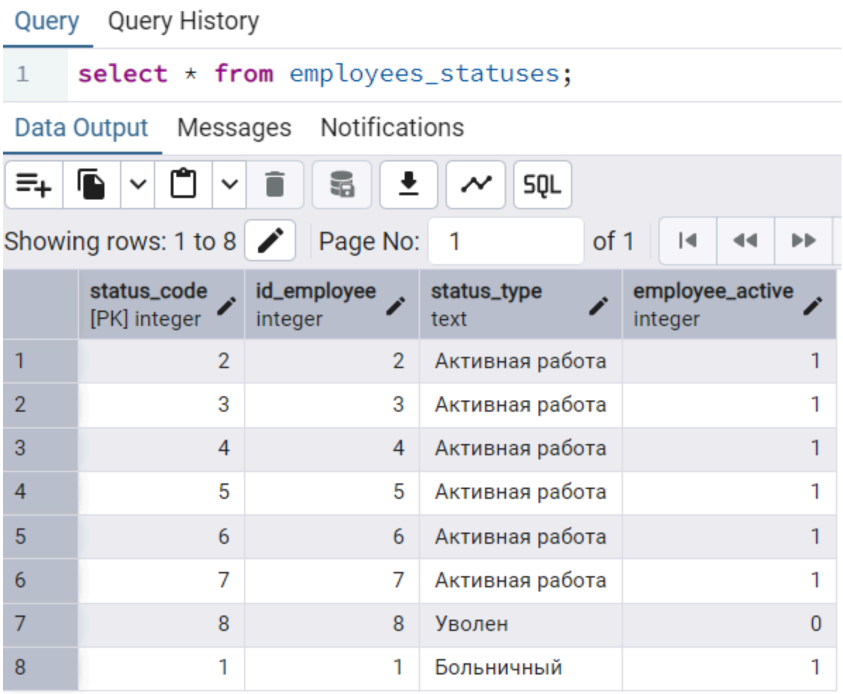
Рисунок 47 – Проверка доступа к таблицам для ib_admin_user (2)


Query
Query History
Scratch Pad
Data Output
Messages
Notifications
Showing rows: 1 to 9
Page No: 1
of 1

```sql
select * from employees;
```

	id_employee [PK] integer	login text	password text	full_name text	email text	phone text	id_role integer	id_team integer
1	1	ivanov_i	$2a$06$z...	Иванов ...	ivanov...	+79001...	2	1
2	2	petrov_p	$2a$06$b...	Петров П...	petrov...	+79001...	3	2
3	3	sidorov_s	$2a$06$n...	Сидоров ...	sidoro...	+79001...	4	4
4	4	admin_ib	$2a$06$zy...	Козлов Г...	admin...	+79001...	5	[null]
5	5	sec_officer	$2a$06$a...	Васьков ...	sec@c...	+79001...	6	[null]
6	6	auditor1	$2a$06$Y...	Дементь...	audit...	+79001...	7	[null]
7	7	manager_a	$2a$06$4...	Кривова ...	mana...	+79001...	8	1
8	8	vacation_user	$2a$06$F...	Прокофь...	ex@c...	[null]	2	[null]
9	9	test_user	$2a$06$Ik...	Тестовы...	test@...	+79990...	1	[null]

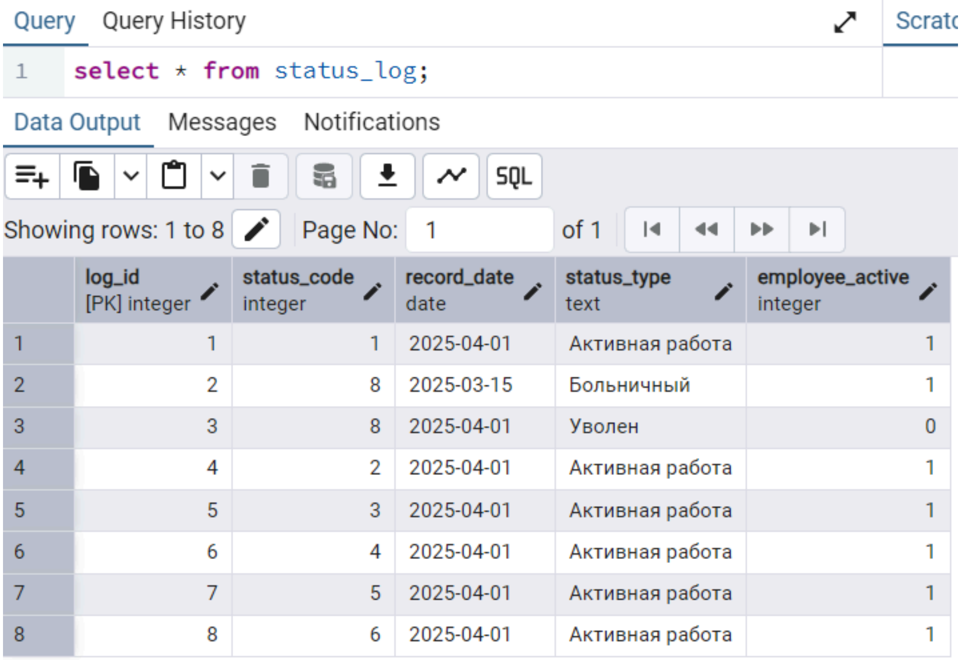
Рисунок 48 – Проверка доступа к таблицам для ib_admin_user (3)


Query
Query History
Data Output
Messages
Notifications
Showing rows: 1 to 8
Page No: 1
of 1

```sql
select * from employees_statuses;
```

	status_code [PK] integer	id_employee integer	status_type text	employee_active integer
1	2	2	Активная работа	1
2	3	3	Активная работа	1
3	4	4	Активная работа	1
4	5	5	Активная работа	1
5	6	6	Активная работа	1
6	7	7	Активная работа	1
7	8	8	Уволен	0
8	1	1	Больничный	1

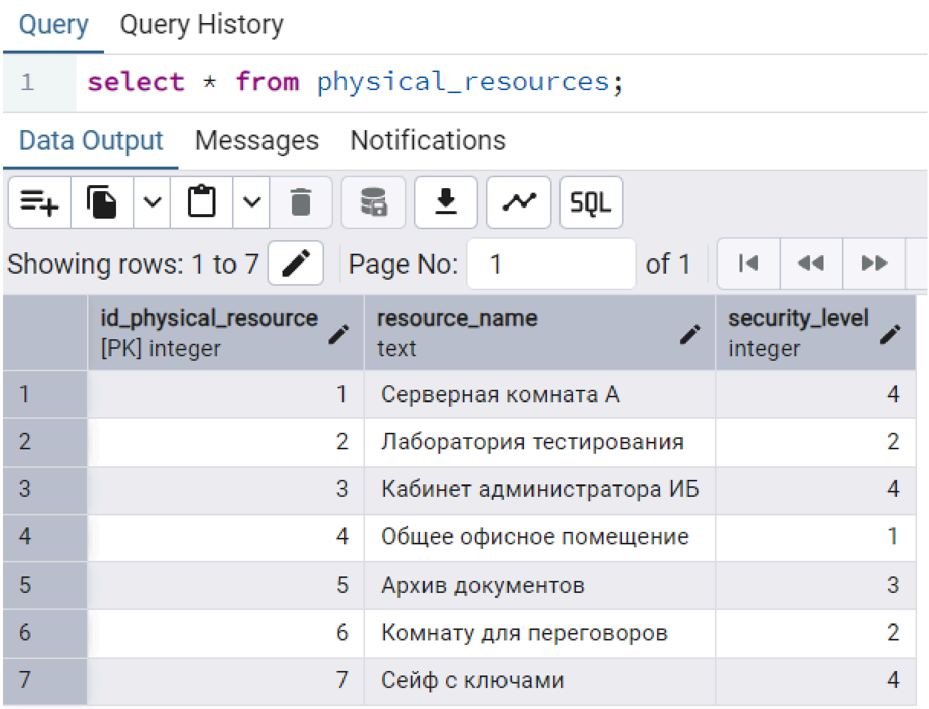
Рисунок 49 – Проверка доступа к таблицам для ib_admin_user (4)


Query
Query History
Scratc
Data Output
Messages
Notifications
Showing rows: 1 to 8
Page No: 1
of 1

```sql
select * from status_log;
```

	log_id [PK] integer	status_code integer	record_date date	status_type text	employee_active integer
1	1	1	2025-04-01	Активная работа	1
2	2	8	2025-03-15	Больничный	1
3	3	8	2025-04-01	Уволен	0
4	4	2	2025-04-01	Активная работа	1
5	5	3	2025-04-01	Активная работа	1
6	6	4	2025-04-01	Активная работа	1
7	7	5	2025-04-01	Активная работа	1
8	8	6	2025-04-01	Активная работа	1

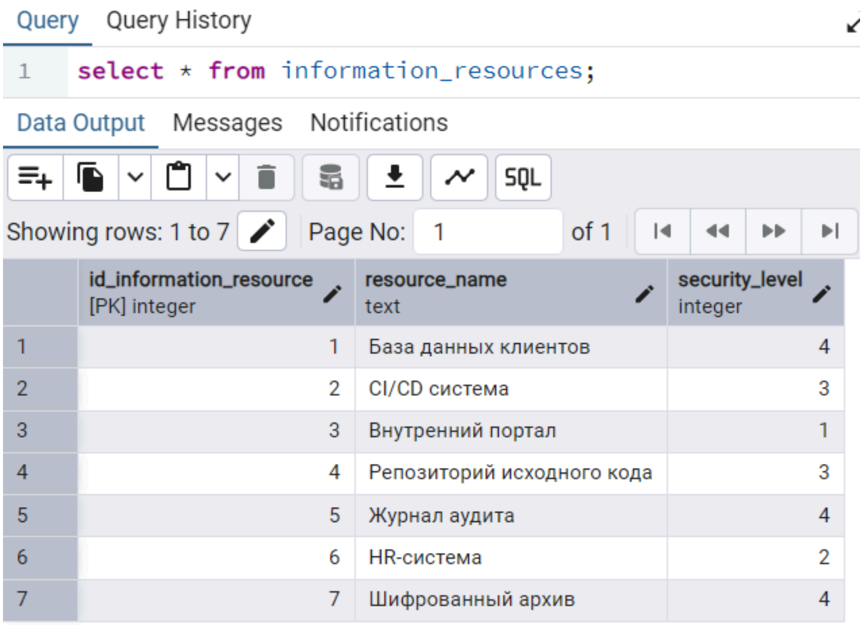
Рисунок 50 – Проверка доступа к таблицам для ib_admin_user (5)


Query
Query History
Data Output
Messages
Notifications
Showing rows: 1 to 7
Page No: 1
of 1

```
select * from physical_resources;
```

	id_physical_resource [PK] integer	resource_name text	security_level integer
1	1	Серверная комната А	4
2	2	Лаборатория тестирования	2
3	3	Кабинет администратора ИБ	4
4	4	Общее офисное помещение	1
5	5	Архив документов	3
6	6	Комнату для переговоров	2
7	7	Сейф с ключами	4

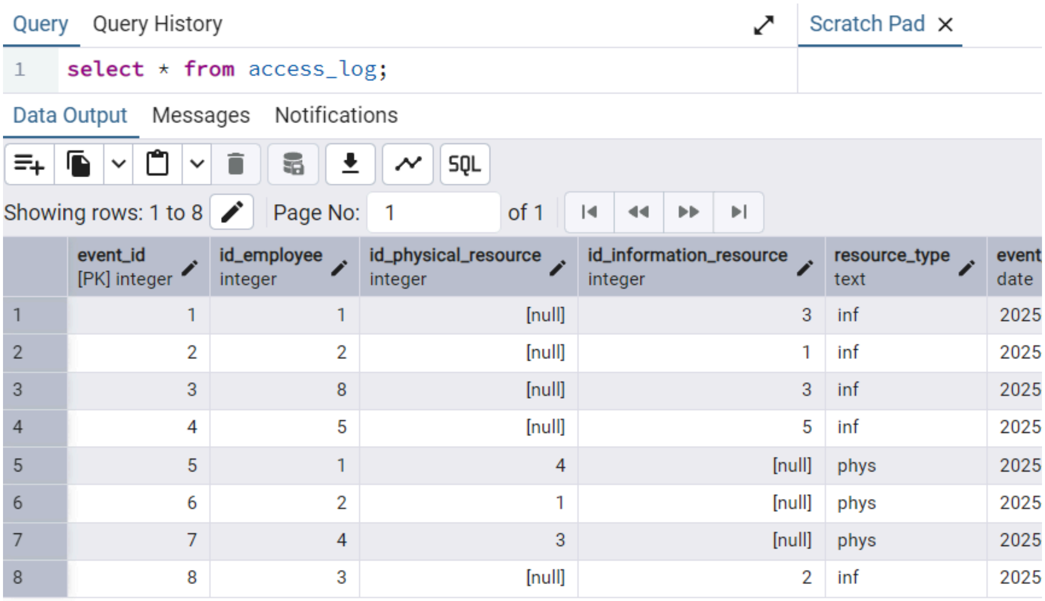
Рисунок 51 – Проверка доступа к таблицам для ib_admin_user (6)


Query
Query History
Data Output
Messages
Notifications
Showing rows: 1 to 7
Page No: 1
of 1

```
select * from information_resources;
```

	id_information_resource [PK] integer	resource_name text	security_level integer
1	1	База данных клиентов	4
2	2	CI/CD система	3
3	3	Внутренний портал	1
4	4	Репозиторий исходного кода	3
5	5	Журнал аудита	4
6	6	HR-система	2
7	7	Шифрованный архив	4

Рисунок 52 – Проверка доступа к таблицам для ib_admin_user (7)

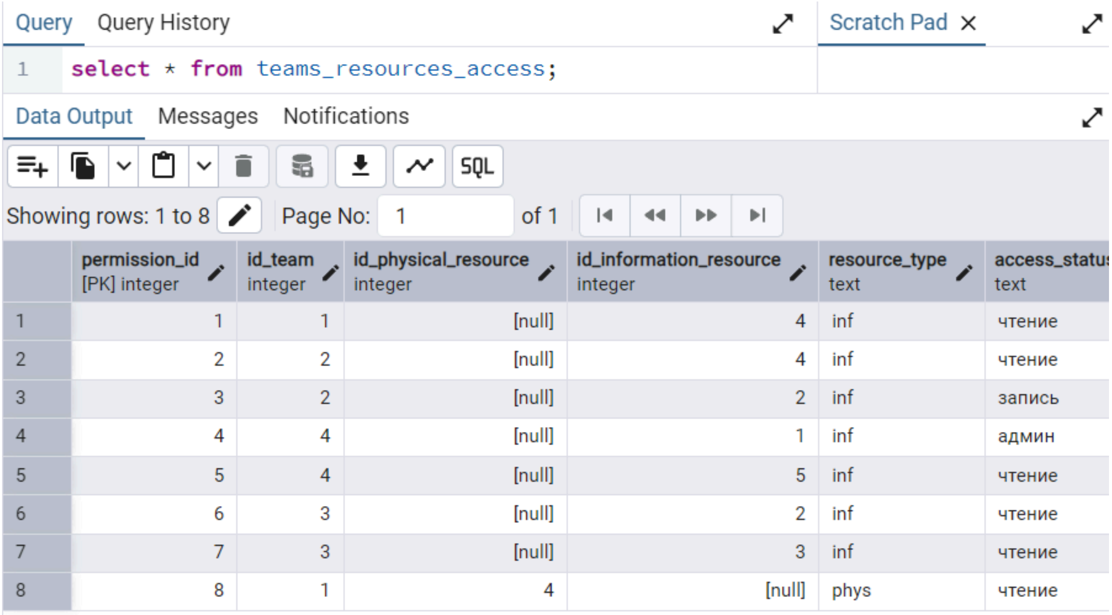

Query
Query History
Scratch Pad
Data Output
Messages
Notifications
SQL

```
select * from access_log;
```

Showing rows: 1 to 8 Page No: 1 of 1

	event_id [PK] integer	id_employee integer	id_physical_resource integer	id_information_resource integer	resource_type text	event date
1	1	1	[null]	3	inf	2025
2	2	2	[null]	1	inf	2025
3	3	8	[null]	3	inf	2025
4	4	5	[null]	5	inf	2025
5	5	1	4	[null]	phys	2025
6	6	2	1	[null]	phys	2025
7	7	4	3	[null]	phys	2025
8	8	3	[null]	2	inf	2025

Рисунок 53 – Проверка доступа к таблицам для ib_admin_user (8)

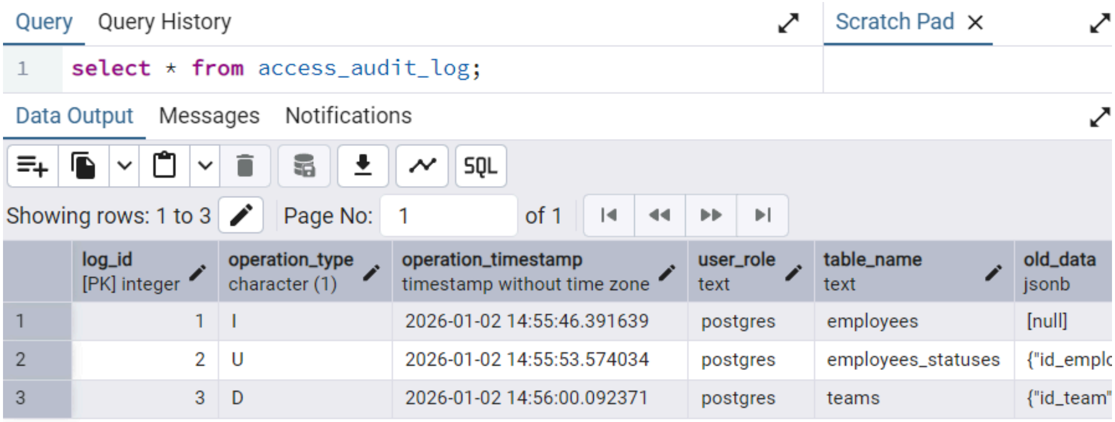

Query
Query History
Scratch Pad
Data Output
Messages
Notifications
SQL

```
select * from teams_resources_access;
```

Showing rows: 1 to 8 Page No: 1 of 1

	permission_id [PK] integer	id_team integer	id_physical_resource integer	id_information_resource integer	resource_type text	access_statu text
1	1	1	[null]	4	inf	чтение
2	2	2	[null]	4	inf	чтение
3	3	2	[null]	2	inf	запись
4	4	4	[null]	1	inf	админ
5	5	4	[null]	5	inf	чтение
6	6	3	[null]	2	inf	чтение
7	7	3	[null]	3	inf	чтение
8	8	1	4	[null]	phys	чтение

Рисунок 54 – Проверка доступа к таблицам для ib_admin_user (9)

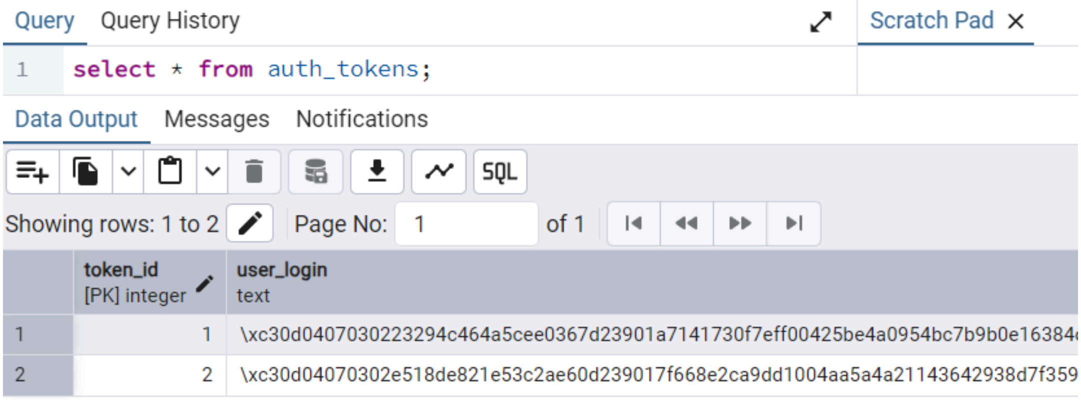

Query
Query History
Scratch Pad
Data Output
Messages
Notifications
SQL

```
select * from access_audit_log;
```

Showing rows: 1 to 3 Page No: 1 of 1

	log_id [PK] integer	operation_type character (1)	operation_timestamp timestamp without time zone	user_role text	table_name text	old_data jsonb
1	1	I	2026-01-02 14:55:46.391639	postgres	employees	[null]
2	2	U	2026-01-02 14:55:53.574034	postgres	employees_statuses	{"id_emplc
3	3	D	2026-01-02 14:56:00.092371	postgres	teams	{"id_team"

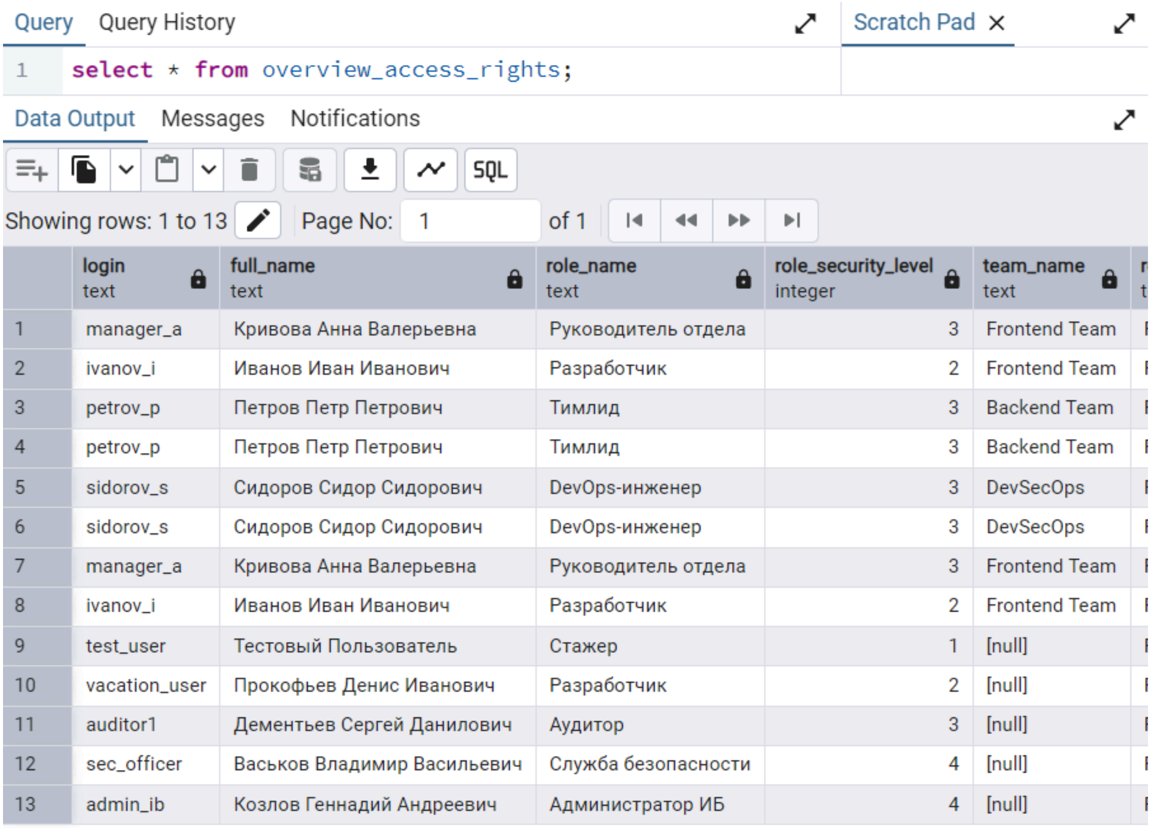
Рисунок 55 – Проверка доступа к таблицам для ib_admin_user (10)


Query
Query History
Scratch Pad
Data Output
Messages
Notifications
Showing rows: 1 to 2
Page No: 1 of 1

```sql
select * from auth_tokens;
```

	token_id [PK] integer	user_login text
1	1	\xc30d0407030223294c464a5cee0367d23901a7141730f7eff00425be4a0954bc7b9b0e16384
2	2	\xc30d04070302e518de821e53c2ae60d239017f668e2ca9dd1004aa5a4a21143642938d7f359

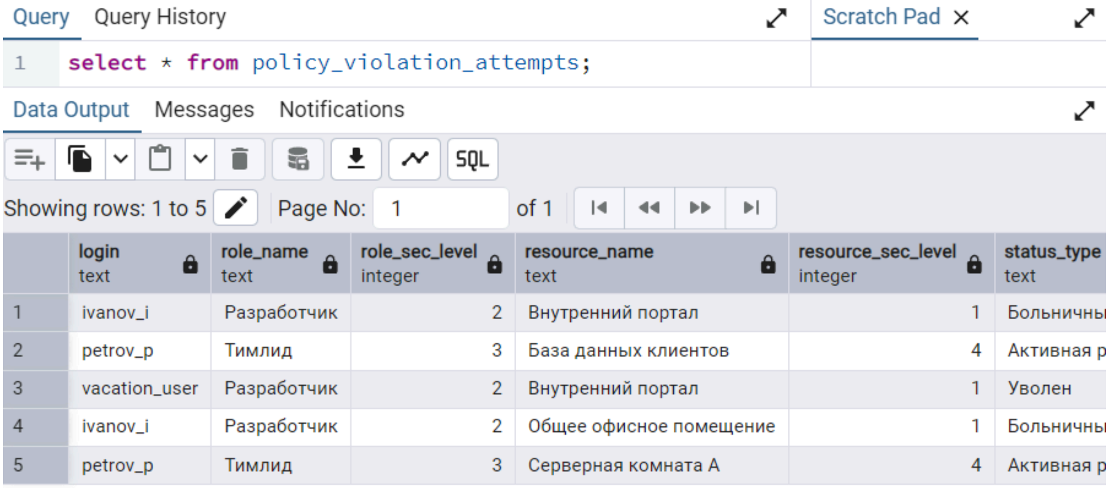
Рисунок 56 – Проверка доступа к таблицам для ib_admin_user (11)


Query
Query History
Scratch Pad
Data Output
Messages
Notifications
Showing rows: 1 to 13
Page No: 1 of 1

```sql
select * from overview_access_rights;
```

	login text	full_name text	role_name text	role_security_level integer	team_name text
1	manager_a	Кривова Анна Валерьевна	Руководитель отдела	3	Frontend Team
2	ivanov_i	Иванов Иван Иванович	Разработчик	2	Frontend Team
3	petrov_p	Петров Петр Петрович	Тимлид	3	Backend Team
4	petrov_p	Петров Петр Петрович	Тимлид	3	Backend Team
5	sidorov_s	Сидоров Сидор Сидорович	DevOps-инженер	3	DevSecOps
6	sidorov_s	Сидоров Сидор Сидорович	DevOps-инженер	3	DevSecOps
7	manager_a	Кривова Анна Валерьевна	Руководитель отдела	3	Frontend Team
8	ivanov_i	Иванов Иван Иванович	Разработчик	2	Frontend Team
9	test_user	Тестовый Пользователь	Стажер	1	[null]
10	vacation_user	Прокофьев Денис Иванович	Разработчик	2	[null]
11	auditor1	Дементьев Сергей Данилович	Аудитор	3	[null]
12	sec_officer	Васьков Владимир Васильевич	Служба безопасности	4	[null]
13	admin_ib	Козлов Геннадий Андреевич	Администратор ИБ	4	[null]

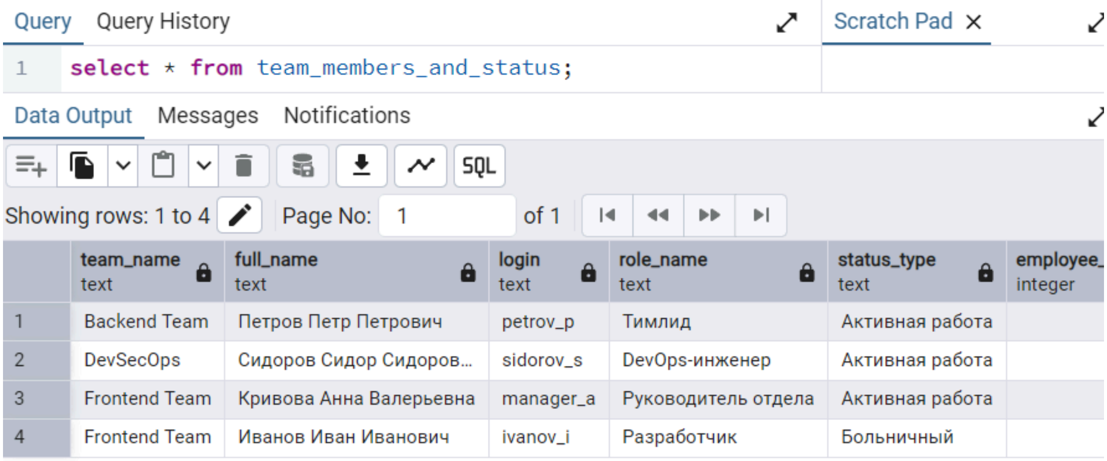
Рисунок 57 – Проверка доступа к таблицам для ib_admin_user (12)


Query
Query History
Scratch Pad
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 5
Page No: 1 of 1

```
select * from policy_violation_attempts;
```

	login text	role_name text	role_sec_level integer	resource_name text	resource_sec_level integer	status_type text
1	ivanov_i	Разработчик	2	Внутренний портал	1	Больничны
2	petrov_p	Тимлид	3	База данных клиентов	4	Активная р
3	vacation_user	Разработчик	2	Внутренний портал	1	Уволен
4	ivanov_i	Разработчик	2	Общее офисное помещение	1	Больничны
5	petrov_p	Тимлид	3	Серверная комната А	4	Активная р

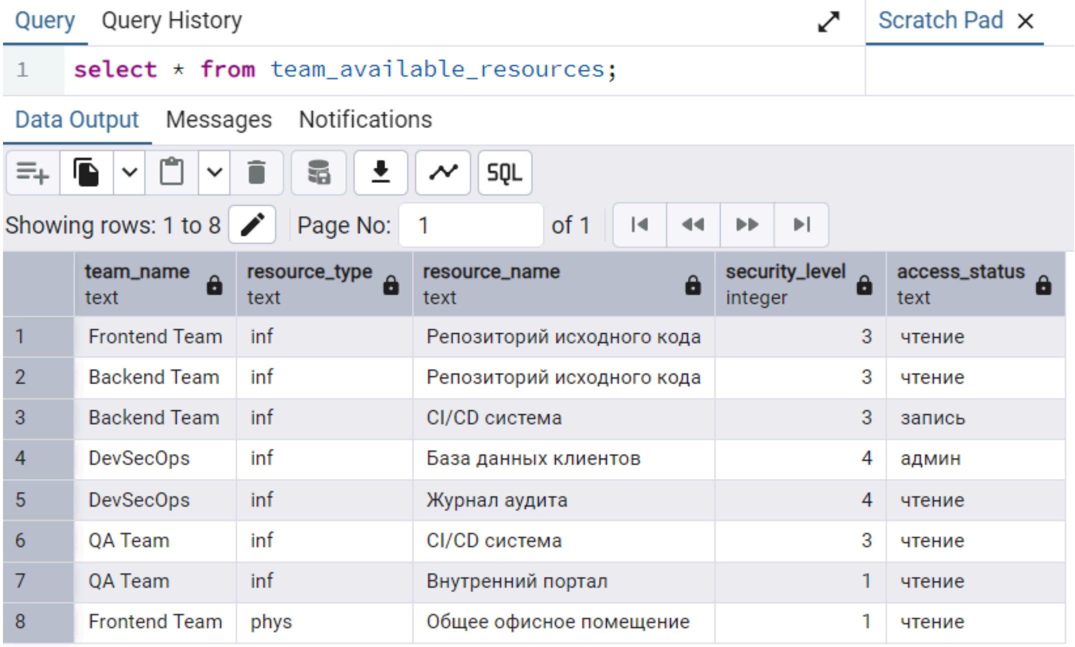
Рисунок 58 – Проверка доступа к таблицам для ib_admin_user (13)


Query
Query History
Scratch Pad
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 4
Page No: 1 of 1

```
select * from team_members_and_status;
```

	team_name text	full_name text	login text	role_name text	status_type text	employee_ integer
1	Backend Team	Петров Петр Петрович	petrov_p	Тимлид	Активная работа	
2	DevSecOps	Сидоров Сидор Сидоров...	sidorov_s	DevOps-инженер	Активная работа	
3	Frontend Team	Кривова Анна Валерьевна	manager_a	Руководитель отдела	Активная работа	
4	Frontend Team	Иванов Иван Иванович	ivanov_i	Разработчик	Больничный	

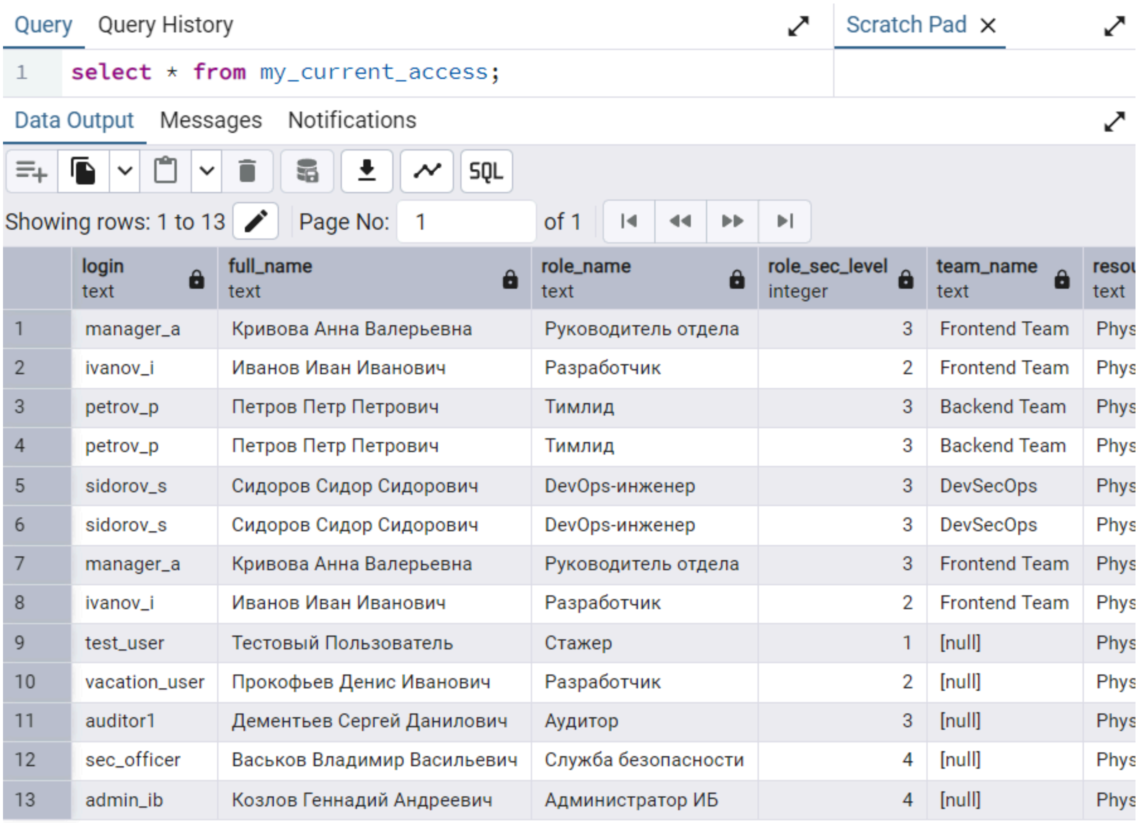
Рисунок 59 – Проверка доступа к таблицам для ib_admin_user (14)


Query
Query History
Scratch Pad
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 8
Page No: 1 of 1

```
select * from team_available_resources;
```

	team_name text	resource_type text	resource_name text	security_level integer	access_status text
1	Frontend Team	inf	Репозиторий исходного кода	3	чтение
2	Backend Team	inf	Репозиторий исходного кода	3	чтение
3	Backend Team	inf	CI/CD система	3	запись
4	DevSecOps	inf	База данных клиентов	4	админ
5	DevSecOps	inf	Журнал аудита	4	чтение
6	QA Team	inf	CI/CD система	3	чтение
7	QA Team	inf	Внутренний портал	1	чтение
8	Frontend Team	phys	Общее офисное помещение	1	чтение

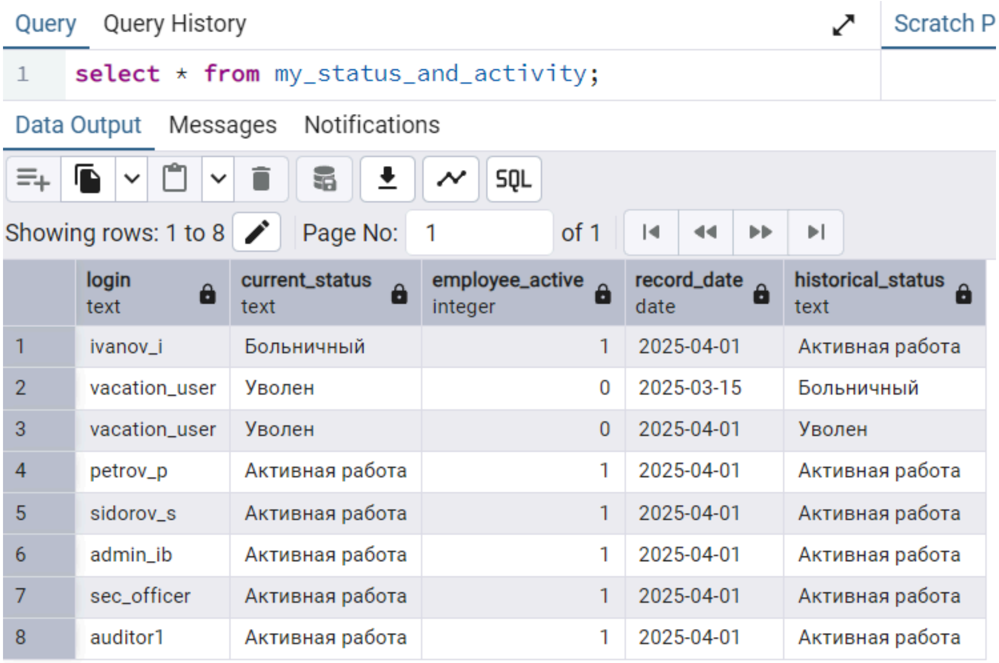
Рисунок 60 – Проверка доступа к таблицам для ib_admin_user (15)


Query
Query History
Scratch Pad
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 13
Page No: 1
of 1

```sql
select * from my_current_access;
```

	login text	full_name text	role_name text	role_sec_level integer	team_name text	resou text
1	manager_a	Кривова Анна Валерьевна	Руководитель отдела	3	Frontend Team	Phys
2	ivanov_i	Иванов Иван Иванович	Разработчик	2	Frontend Team	Phys
3	petrov_p	Петров Петр Петрович	Тимлид	3	Backend Team	Phys
4	petrov_p	Петров Петр Петрович	Тимлид	3	Backend Team	Phys
5	sidorov_s	Сидоров Сидор Сидорович	DevOps-инженер	3	DevSecOps	Phys
6	sidorov_s	Сидоров Сидор Сидорович	DevOps-инженер	3	DevSecOps	Phys
7	manager_a	Кривова Анна Валерьевна	Руководитель отдела	3	Frontend Team	Phys
8	ivanov_i	Иванов Иван Иванович	Разработчик	2	Frontend Team	Phys
9	test_user	Тестовый Пользователь	Стажер	1	[null]	Phys
10	vacation_user	Прокофьев Денис Иванович	Разработчик	2	[null]	Phys
11	auditor1	Дементьев Сергей Данилович	Аудитор	3	[null]	Phys
12	sec_officer	Васьков Владимир Васильевич	Служба безопасности	4	[null]	Phys
13	admin_ib	Козлов Геннадий Андреевич	Администратор ИБ	4	[null]	Phys

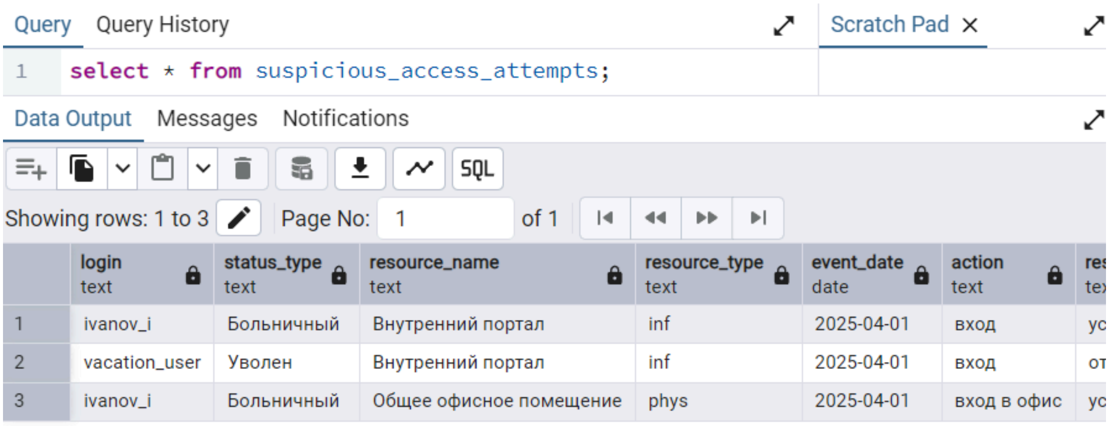
Рисунок 61 – Проверка доступа к таблицам для ib_admin_user (16)


Query
Query History
Scratch P
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 8
Page No: 1
of 1

```sql
select * from my_status_and_activity;
```

	login text	current_status text	employee_active integer	record_date date	historical_status text
1	ivanov_i	Больничный	1	2025-04-01	Активная работа
2	vacation_user	Уволен	0	2025-03-15	Больничный
3	vacation_user	Уволен	0	2025-04-01	Уволен
4	petrov_p	Активная работа	1	2025-04-01	Активная работа
5	sidorov_s	Активная работа	1	2025-04-01	Активная работа
6	admin_ib	Активная работа	1	2025-04-01	Активная работа
7	sec_officer	Активная работа	1	2025-04-01	Активная работа
8	auditor1	Активная работа	1	2025-04-01	Активная работа

Рисунок 62 – Проверка доступа к таблицам для ib_admin_user (17)



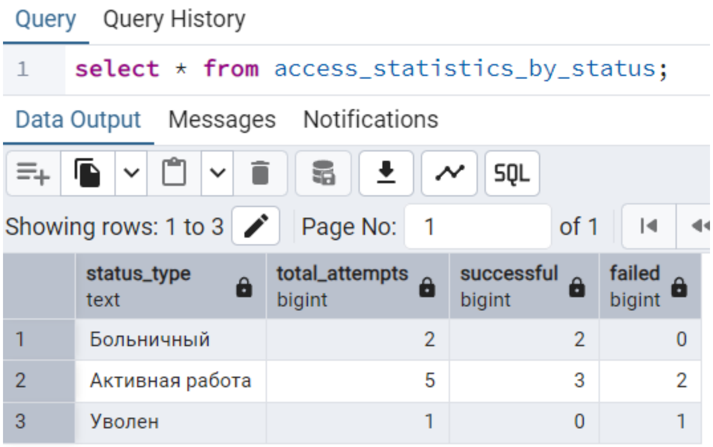
```sql
select * from suspicious_access_attempts;
```

Showing rows: 1 to 3 Page No: 1 of 1

	login text	status_type text	resource_name text	resource_type text	event_date date	action text	res tex
1	ivanov_i	Больничный	Внутренний портал	inf	2025-04-01	вход	ус
2	vacation_user	Уволен	Внутренний портал	inf	2025-04-01	вход	от
3	ivanov_i	Больничный	Общее офисное помещение	phys	2025-04-01	вход в офис	ус

Query
Query History
Scratch Pad
Data Output
Messages
Notifications
SQL

Рисунок 63 – Проверка доступа к таблицам для ib_admin_user (18)



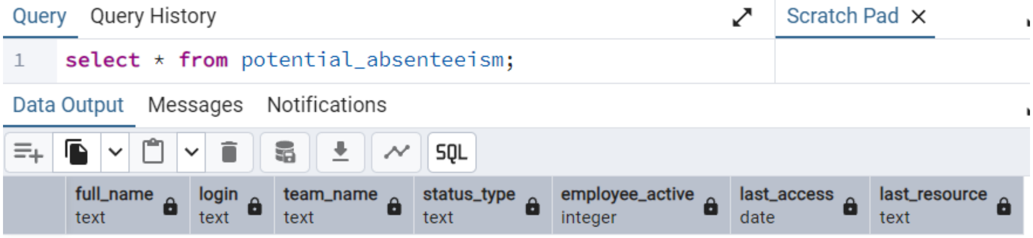
```sql
select * from access_statistics_by_status;
```

Showing rows: 1 to 3 Page No: 1 of 1

	status_type text	total_attempts bigint	successful bigint	failed bigint
1	Больничный	2	2	0
2	Активная работа	5	3	2
3	Уволен	1	0	1

Query
Query History
Data Output
Messages
Notifications
SQL

Рисунок 64 – Проверка доступа к таблицам для ib_admin_user (19)

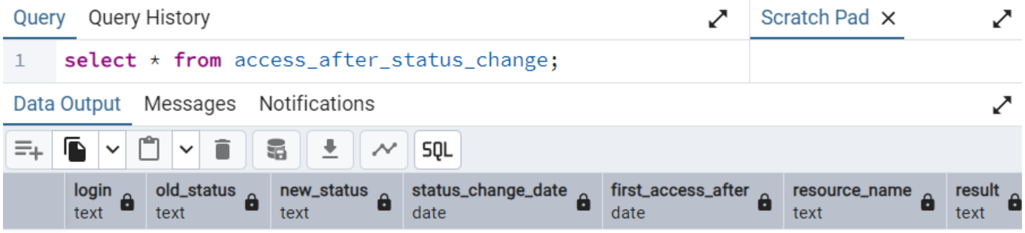


```sql
select * from potential_absenteeism;
```

	full_name text	login text	team_name text	status_type text	employee_active integer	last_access date	last_resource text

Query
Query History
Scratch Pad
Data Output
Messages
Notifications
SQL

Рисунок 65 – Проверка доступа к таблицам для ib_admin_user (20)



```sql
select * from access_after_status_change;
```

	login text	old_status text	new_status text	status_change_date date	first_access_after date	resource_name text	result text

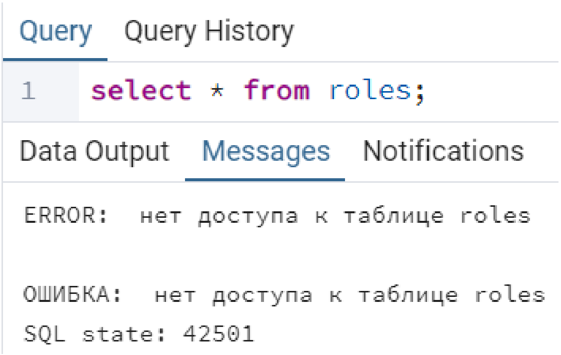
Query
Query History
Scratch Pad
Data Output
Messages
Notifications
SQL

Рисунок 66 – Проверка доступа к таблицам для ib_admin_user (21)

- Руководитель команды

Руководитель должен видеть только данные, относящиеся к его команде. В представлении «Состав и статусы команды» отображаются ФИО, логин, роль и текущий статус каждого члена команды, а также вычисляется признак «прогула»: если статус «Активная работа», но система не зафиксировала активности в течение дня, это помечается как потенциальный прогул. В представлении «Ресурсы, доступные команде» руководитель должен видеть полный список ресурсов (физических и информационных), к которым имеет доступ его команда: название ресурса, его тип, уровень безопасности и текущий статус доступа («чтение», «запись», «админ»). Это позволяет оперативно запрашивать новые права или отзывать доступ у уволенных сотрудников.

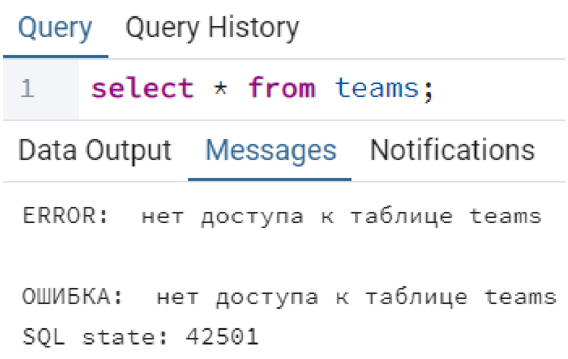
Проверка доступа к таблицам для team_lead_user представлена на рисунках с 67 до 87.



```
select * from roles;
```

```
ERROR:  нет доступа к таблице roles

ОШИБКА:  нет доступа к таблице roles
SQL state: 42501
```

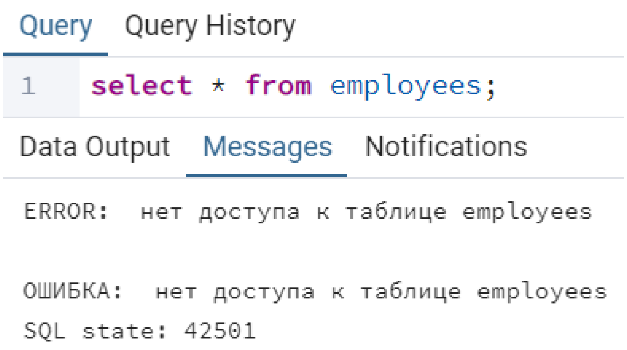
Рисунок 67 – Проверка доступа к таблицам для team_lead_user (1)



```
select * from teams;
```

```
ERROR:  нет доступа к таблице teams

ОШИБКА:  нет доступа к таблице teams
SQL state: 42501
```

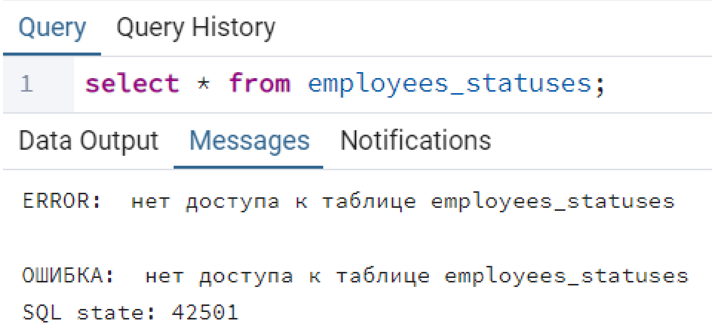
Рисунок 68 – Проверка доступа к таблицам для team_lead_user (2)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from employees;
```

```
ERROR:  нет доступа к таблице employees

ОШИБКА:  нет доступа к таблице employees
SQL state: 42501
```

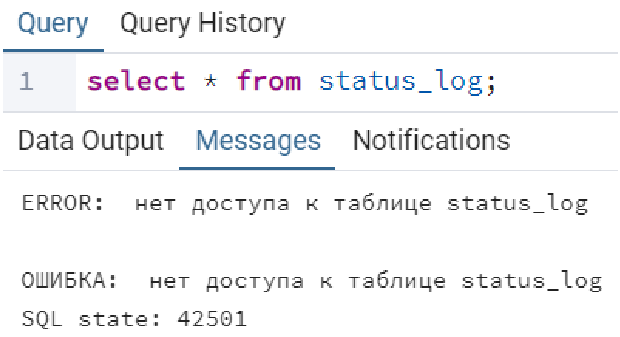
Рисунок 69 – Проверка доступа к таблицам для team_lead_user (3)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from employees_statuses;
```

```
ERROR:  нет доступа к таблице employees_statuses

ОШИБКА:  нет доступа к таблице employees_statuses
SQL state: 42501
```

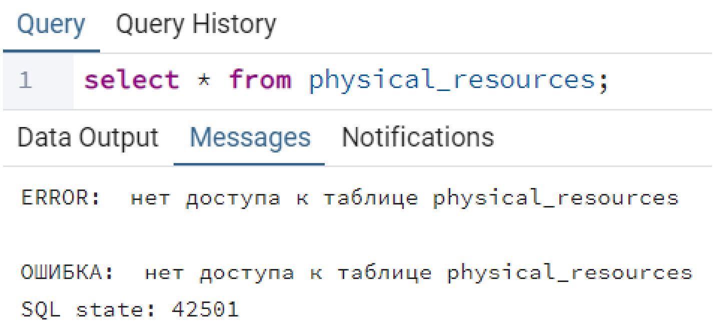
Рисунок 70 – Проверка доступа к таблицам для team_lead_user (4)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from status_log;
```

```
ERROR:  нет доступа к таблице status_log

ОШИБКА:  нет доступа к таблице status_log
SQL state: 42501
```

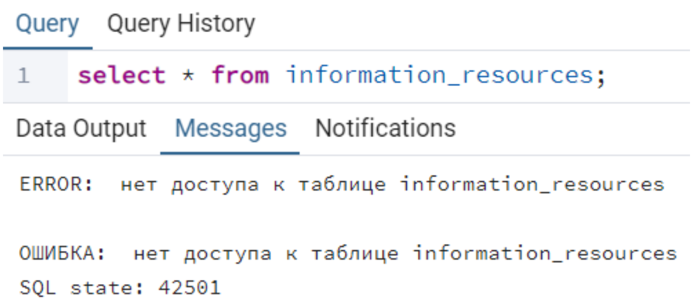
Рисунок 71 – Проверка доступа к таблицам для team_lead_user (5)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from physical_resources;
```

```
ERROR:  нет доступа к таблице physical_resources

ОШИБКА:  нет доступа к таблице physical_resources
SQL state: 42501
```

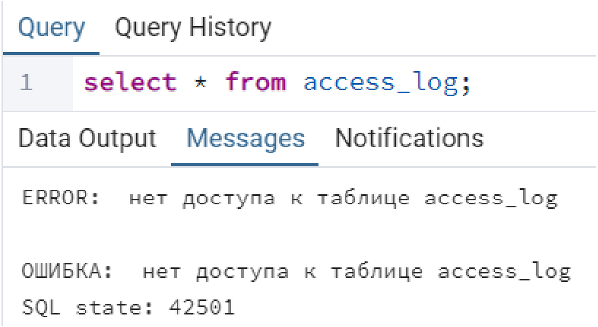
Рисунок 72 – Проверка доступа к таблицам для team_lead_user (6)


Query
Query History
Data Output
Messages
Notifications

```
select * from information_resources;
```

```
ERROR:  нет доступа к таблице information_resources

ОШИБКА:  нет доступа к таблице information_resources
SQL state: 42501
```

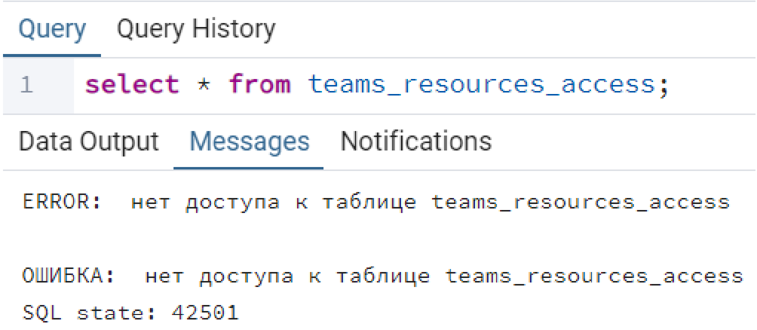
Рисунок 73 – Проверка доступа к таблицам для team_lead_user (7)


Query
Query History
Data Output
Messages
Notifications

```
select * from access_log;
```

```
ERROR:  нет доступа к таблице access_log

ОШИБКА:  нет доступа к таблице access_log
SQL state: 42501
```

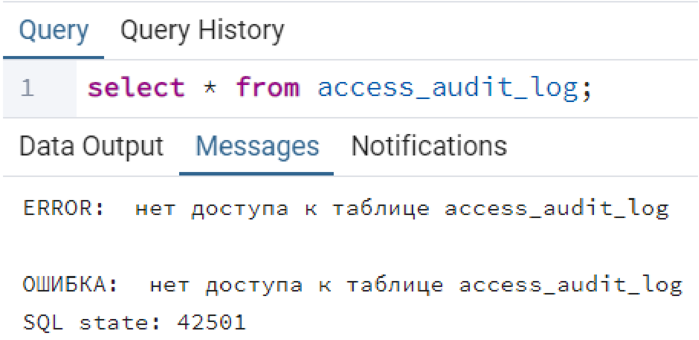
Рисунок 74 – Проверка доступа к таблицам для team_lead_user (8)


Query
Query History
Data Output
Messages
Notifications

```
select * from teams_resources_access;
```

```
ERROR:  нет доступа к таблице teams_resources_access

ОШИБКА:  нет доступа к таблице teams_resources_access
SQL state: 42501
```

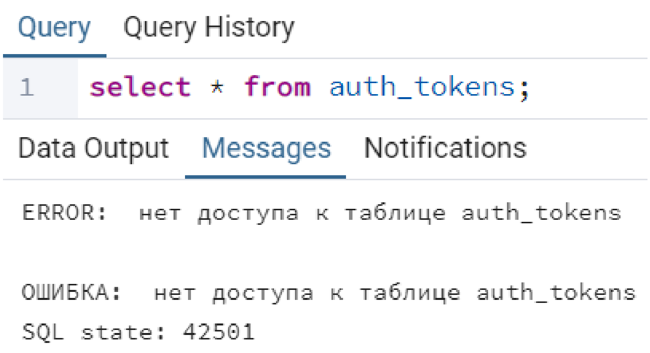
Рисунок 75 – Проверка доступа к таблицам для team_lead_user (9)


Query
Query History
Data Output
Messages
Notifications

```
select * from access_audit_log;
```

```
ERROR:  нет доступа к таблице access_audit_log

ОШИБКА:  нет доступа к таблице access_audit_log
SQL state: 42501
```

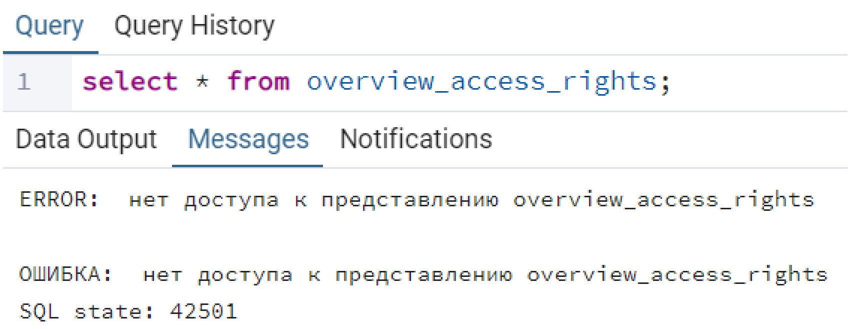
Рисунок 76 – Проверка доступа к таблицам для team_lead_user (10)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from auth_tokens;
```

```
ERROR:  нет доступа к таблице auth_tokens

ОШИБКА:  нет доступа к таблице auth_tokens
SQL state: 42501
```

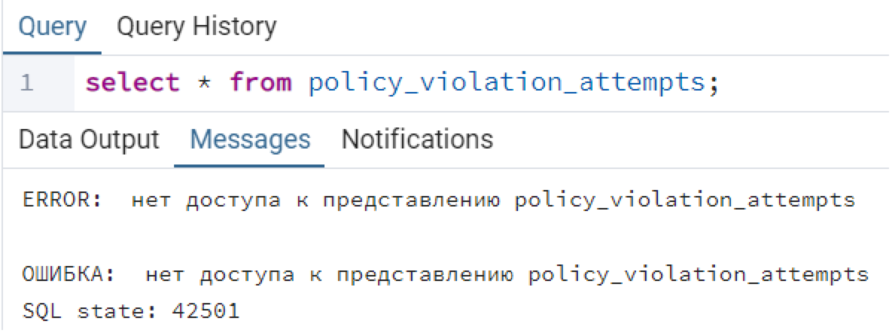
Рисунок 77 – Проверка доступа к таблицам для team_lead_user (11)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from overview_access_rights;
```

```
ERROR:  нет доступа к представлению overview_access_rights

ОШИБКА:  нет доступа к представлению overview_access_rights
SQL state: 42501
```

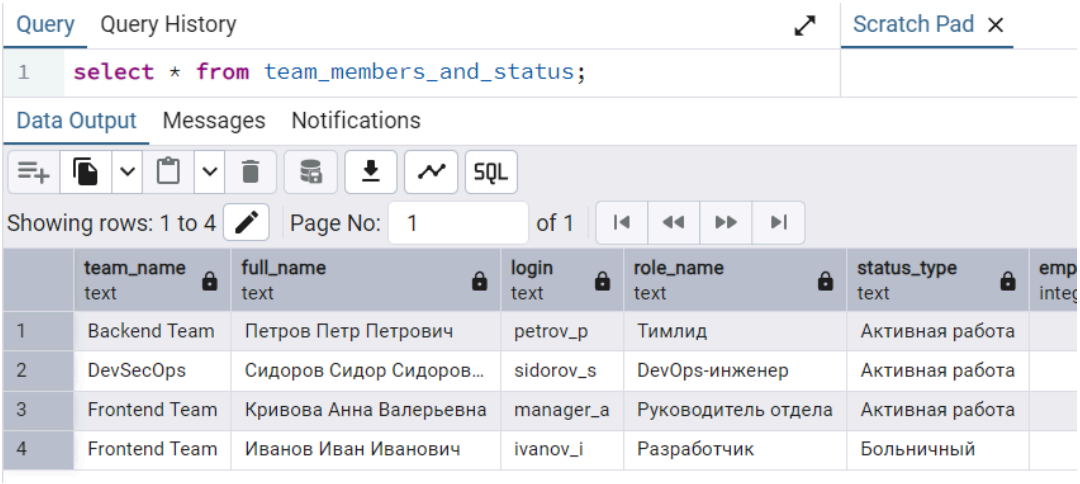
Рисунок 78 – Проверка доступа к таблицам для team_lead_user (12)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from policy_violation_attempts;
```

```
ERROR:  нет доступа к представлению policy_violation_attempts

ОШИБКА:  нет доступа к представлению policy_violation_attempts
SQL state: 42501
```

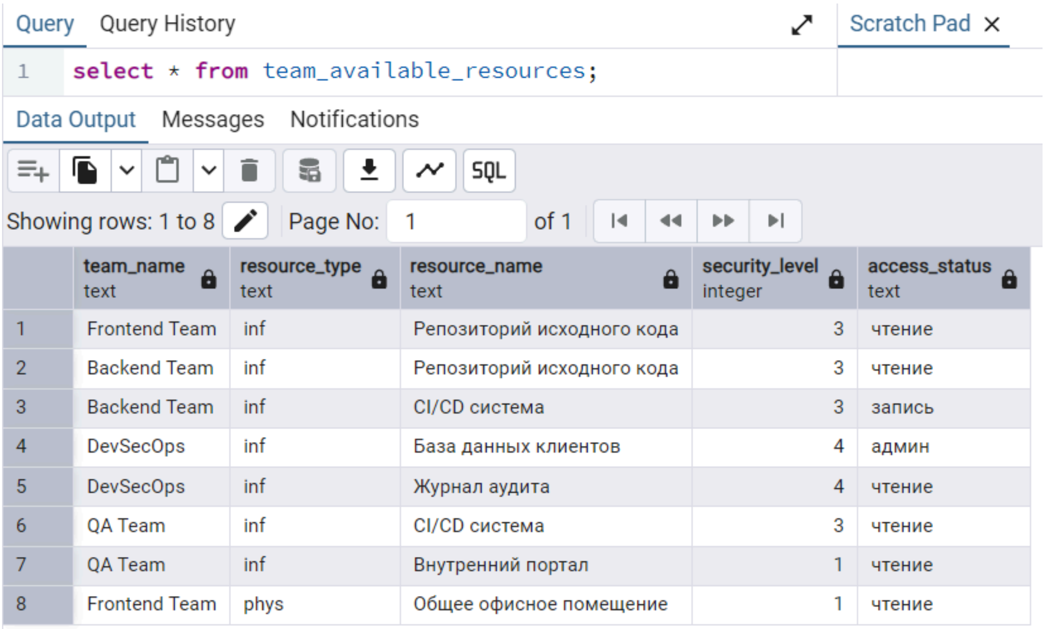
Рисунок 79 – Проверка доступа к таблицам для team_lead_user (13)


Query
Query History
Scratch Pad
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 4
Page No: 1 of 1

```sql
select * from team_members_and_status;
```

	team_name text	full_name text	login text	role_name text	status_type text	emp integ
1	Backend Team	Петров Петр Петрович	petrov_p	Тимлид	Активная работа	
2	DevSecOps	Сидоров Сидор Сидоров…	sidorov_s	DevOps-инженер	Активная работа	
3	Frontend Team	Кривова Анна Валерьевна	manager_a	Руководитель отдела	Активная работа	
4	Frontend Team	Иванов Иван Иванович	ivanov_i	Разработчик	Больничный	

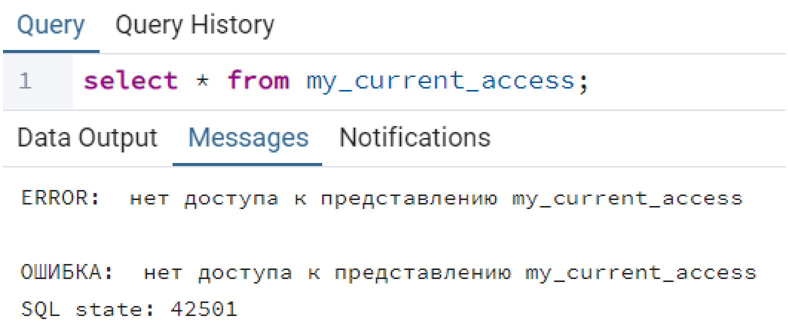
Рисунок 80 – Проверка доступа к таблицам для team_lead_user (14)


Query
Query History
Scratch Pad
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 8
Page No: 1
of 1

```sql
select * from team_available_resources;
```

	team_name text	resource_type text	resource_name text	security_level integer	access_status text
1	Frontend Team	inf	Репозиторий исходного кода	3	чтение
2	Backend Team	inf	Репозиторий исходного кода	3	чтение
3	Backend Team	inf	CI/CD система	3	запись
4	DevSecOps	inf	База данных клиентов	4	админ
5	DevSecOps	inf	Журнал аудита	4	чтение
6	QA Team	inf	CI/CD система	3	чтение
7	QA Team	inf	Внутренний портал	1	чтение
8	Frontend Team	phys	Общее офисное помещение	1	чтение

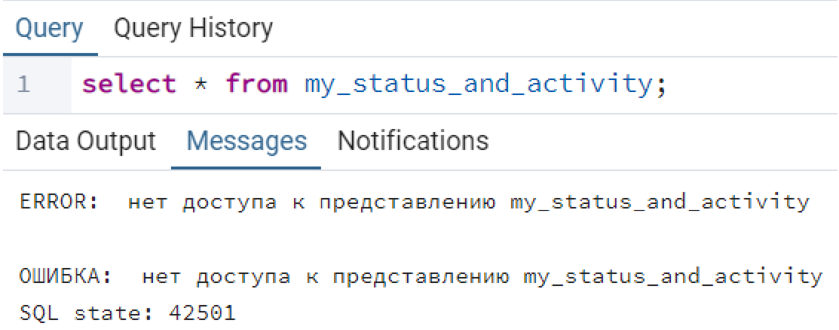
Рисунок 81 – Проверка доступа к таблицам для team_lead_user (15)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from my_current_access;
```

```
ERROR:  нет доступа к представлению my_current_access

ОШИБКА:  нет доступа к представлению my_current_access
SQL state: 42501
```

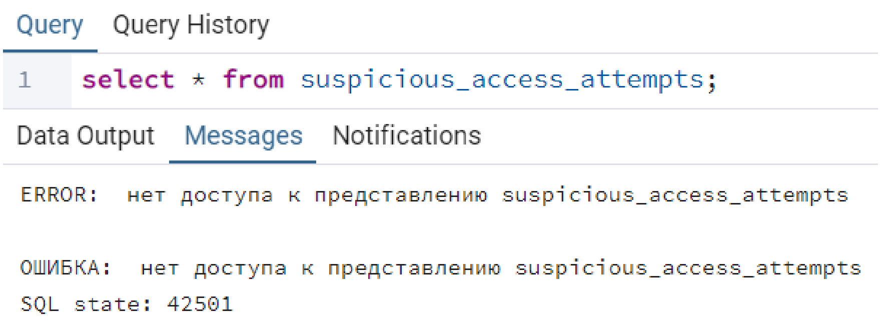
Рисунок 82 – Проверка доступа к таблицам для team_lead_user (16)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from my_status_and_activity;
```

```
ERROR:  нет доступа к представлению my_status_and_activity

ОШИБКА:  нет доступа к представлению my_status_and_activity
SQL state: 42501
```

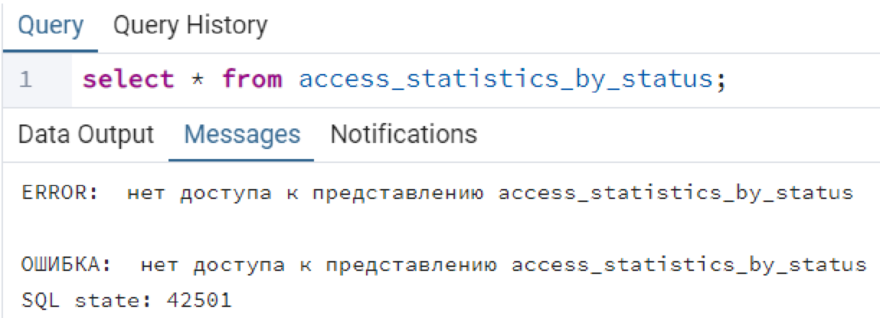
Рисунок 83 – Проверка доступа к таблицам для team_lead_user (17)


Query
Query History
Data Output
Messages
Notifications

```
select * from suspicious_access_attempts;
```

```
ERROR:  нет доступа к представлению suspicious_access_attempts

ОШИБКА:  нет доступа к представлению suspicious_access_attempts
SQL state: 42501
```

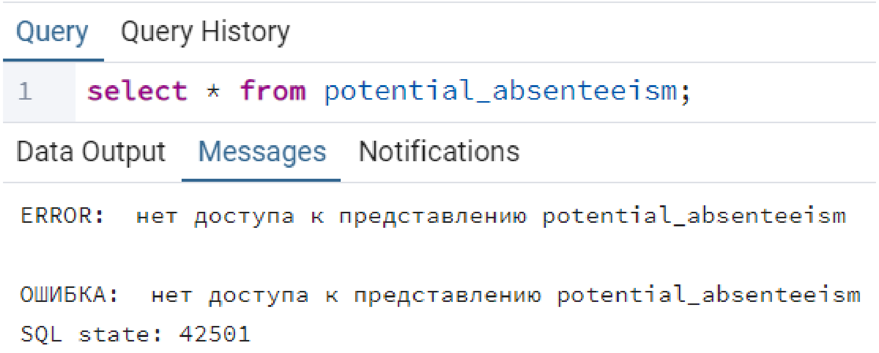
Рисунок 84 – Проверка доступа к таблицам для team_lead_user (18)


Query
Query History
Data Output
Messages
Notifications

```
select * from access_statistics_by_status;
```

```
ERROR:  нет доступа к представлению access_statistics_by_status

ОШИБКА:  нет доступа к представлению access_statistics_by_status
SQL state: 42501
```

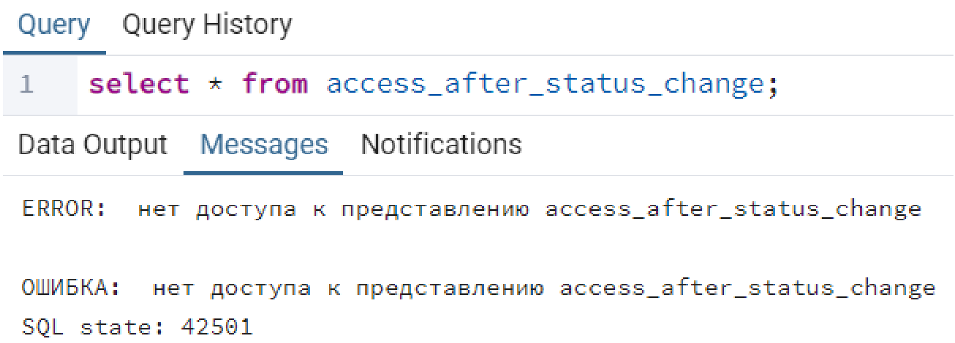
Рисунок 85 – Проверка доступа к таблицам для team_lead_user (19)


Query
Query History
Data Output
Messages
Notifications

```
select * from potential_absenteeism;
```

```
ERROR:  нет доступа к представлению potential_absenteeism

ОШИБКА:  нет доступа к представлению potential_absenteeism
SQL state: 42501
```

Рисунок 86 – Проверка доступа к таблицам для team_lead_user (20)


Query
Query History
Data Output
Messages
Notifications

```
select * from access_after_status_change;
```

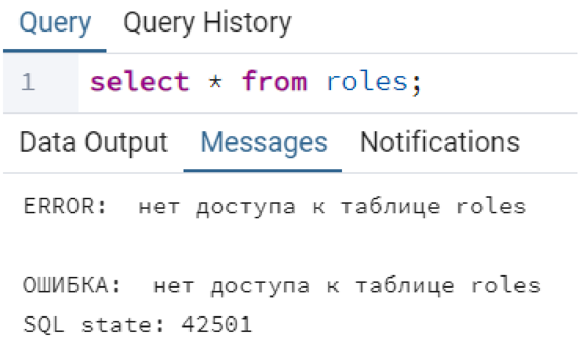
```
ERROR:  нет доступа к представлению access_after_status_change

ОШИБКА:  нет доступа к представлению access_after_status_change
SQL state: 42501
```

Рисунок 87 – Проверка доступа к таблицам для team_lead_user (21)

- Сотрудник

Сотрудник должен получать только персонализированную информацию о себе. В представлении «Мои текущие права доступа» он должен видеть свою роль, ее уровень безопасности, название своей команды (если есть) и список ресурсов, к которым он имеет доступ через команду. В представлении «Мой статус и активность» должны отображаться его текущий статус (например, «Активная работа»), флаг активности за текущий день и история всех смен статусов (например, «01.03.2025 – Больничный» — «05.03.2025 – Активная работа»). Это дает понимание, почему некоторые ресурсы сейчас недоступны (например, при статусе «Отпуск»).

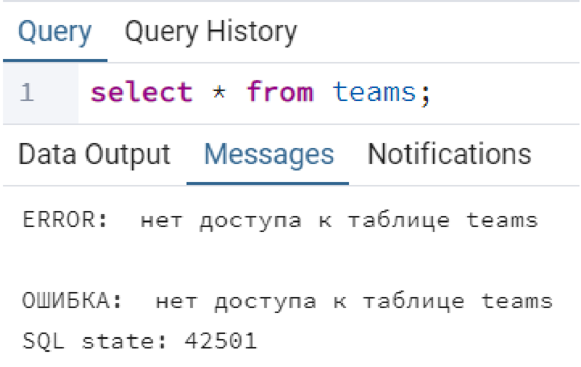
Проверка доступа к таблицам для employee_user представлена на рисунках с 88 до 108.


Query
Query History
Data Output
Messages
Notifications

```
select * from roles;
```

```
ERROR:  нет доступа к таблице roles

ОШИБКА:  нет доступа к таблице roles
SQL state: 42501
```

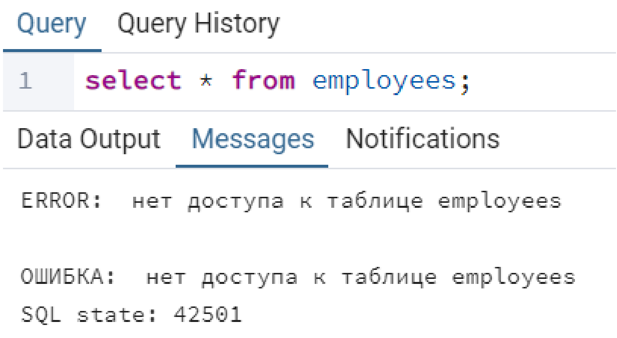
Рисунок 88 – Проверка доступа к таблицам для employee_user (1)


Query
Query History
Data Output
Messages
Notifications

```
select * from teams;
```

```
ERROR:  нет доступа к таблице teams

ОШИБКА:  нет доступа к таблице teams
SQL state: 42501
```

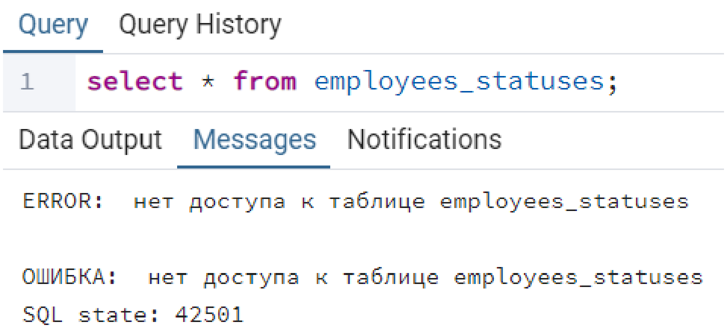
Рисунок 89 – Проверка доступа к таблицам для employee_user (2)


Query
Query History
Data Output
Messages
Notifications

```
select * from employees;
```

```
ERROR:  нет доступа к таблице employees

ОШИБКА:  нет доступа к таблице employees
SQL state: 42501
```

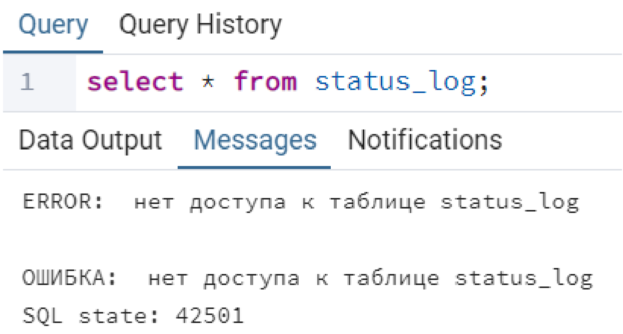
Рисунок 90 – Проверка доступа к таблицам для employee_user (3)


Query
Query History
Data Output
Messages
Notifications

```
select * from employees_statuses;
```

```
ERROR:  нет доступа к таблице employees_statuses

ОШИБКА:  нет доступа к таблице employees_statuses
SQL state: 42501
```

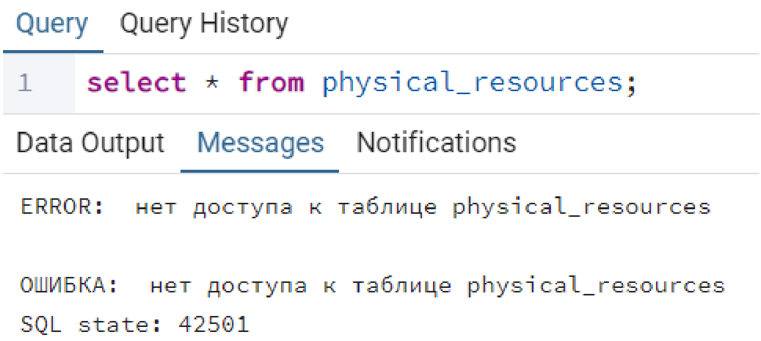
Рисунок 91 – Проверка доступа к таблицам для employee_user (4)


Query
Query History
Data Output
Messages
Notifications

```
select * from status_log;
```

```
ERROR:  нет доступа к таблице status_log

ОШИБКА:  нет доступа к таблице status_log
SQL state: 42501
```

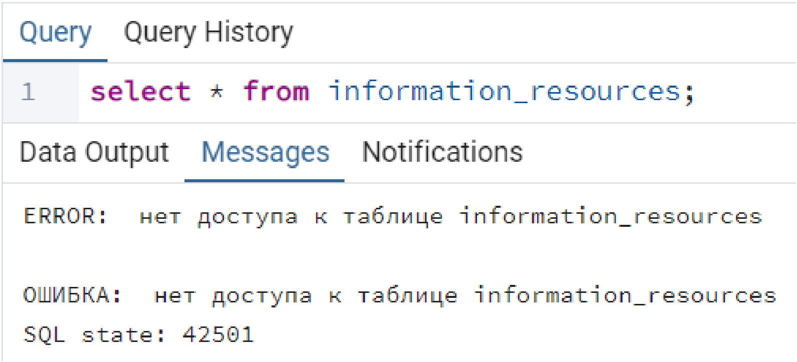
Рисунок 92 – Проверка доступа к таблицам для employee_user (5)


Query
Query History
Data Output
Messages
Notifications

```
select * from physical_resources;
```

```
ERROR:  нет доступа к таблице physical_resources

ОШИБКА:  нет доступа к таблице physical_resources
SQL state: 42501
```

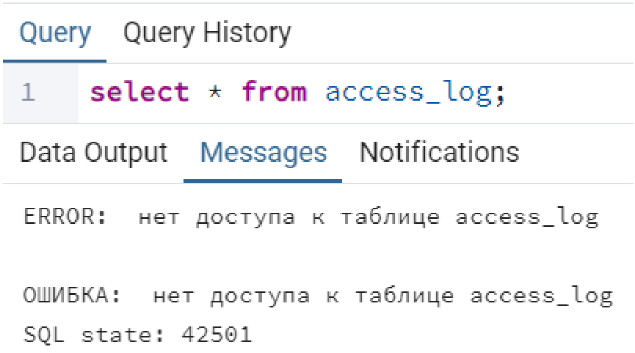
Рисунок 93 – Проверка доступа к таблицам для employee_user (6)


Query
Query History
Data Output
Messages
Notifications

```
select * from information_resources;
```

```
ERROR:  нет доступа к таблице information_resources

ОШИБКА:  нет доступа к таблице information_resources
SQL state: 42501
```

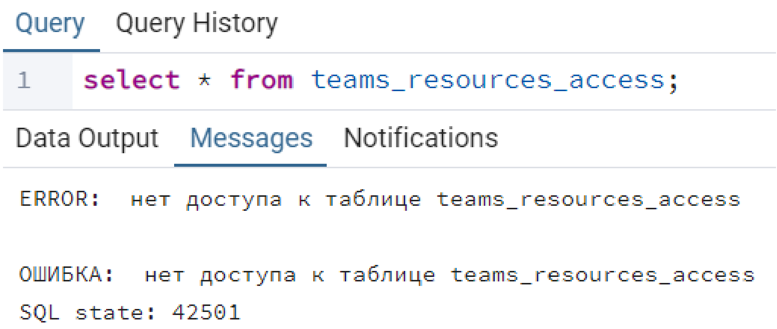
Рисунок 94 – Проверка доступа к таблицам для employee_user (7)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from access_log;
```

```
ERROR:  нет доступа к таблице access_log

ОШИБКА:  нет доступа к таблице access_log
SQL state: 42501
```

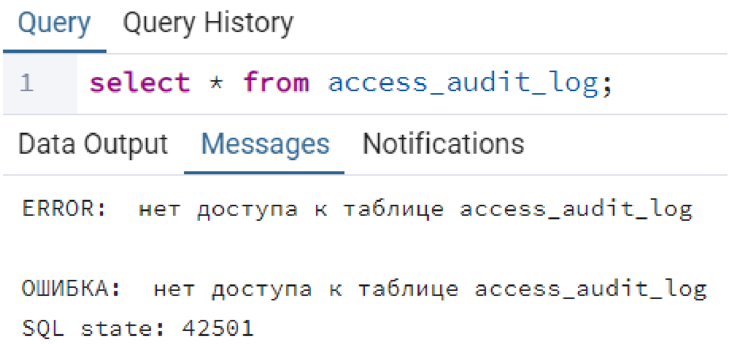
Рисунок 95 – Проверка доступа к таблицам для employee_user (8)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from teams_resources_access;
```

```
ERROR:  нет доступа к таблице teams_resources_access

ОШИБКА:  нет доступа к таблице teams_resources_access
SQL state: 42501
```

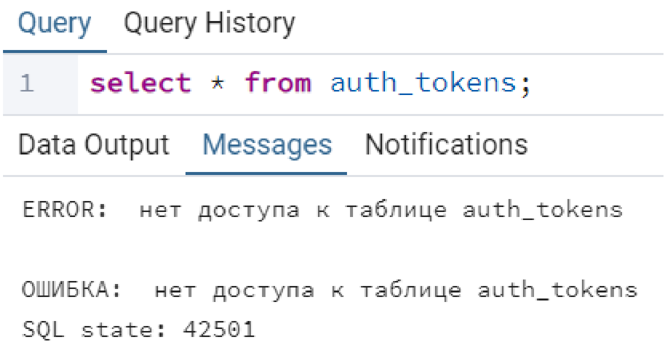
Рисунок 96 – Проверка доступа к таблицам для employee_user (9)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from access_audit_log;
```

```
ERROR:  нет доступа к таблице access_audit_log

ОШИБКА:  нет доступа к таблице access_audit_log
SQL state: 42501
```

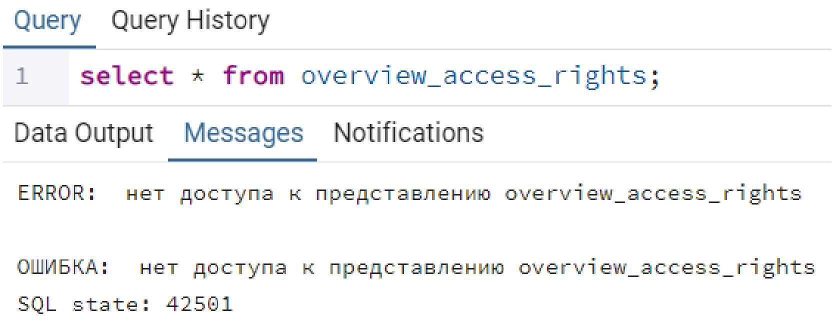
Рисунок 97 – Проверка доступа к таблицам для employee_user (10)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from auth_tokens;
```

```
ERROR:  нет доступа к таблице auth_tokens

ОШИБКА:  нет доступа к таблице auth_tokens
SQL state: 42501
```

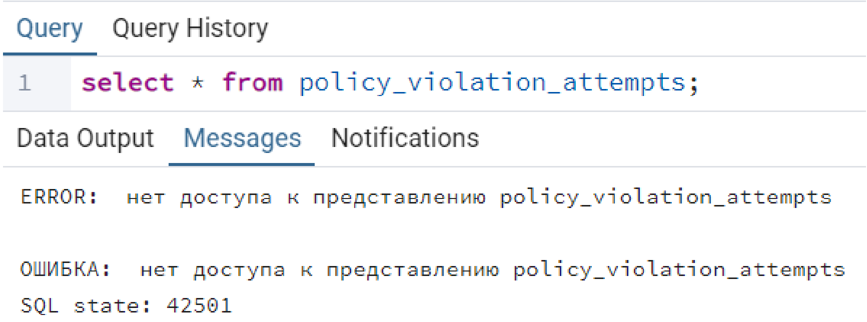
Рисунок 98 – Проверка доступа к таблицам для employee_user (11)


Query
Query History
Data Output
Messages
Notifications

```
select * from overview_access_rights;
```

```
ERROR:  нет доступа к представлению overview_access_rights

ОШИБКА:  нет доступа к представлению overview_access_rights
SQL state: 42501
```

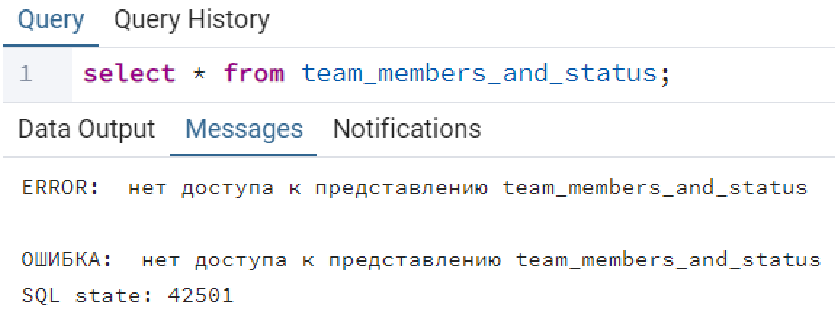
Рисунок 99 – Проверка доступа к таблицам для employee_user (12)


Query
Query History
Data Output
Messages
Notifications

```
select * from policy_violation_attempts;
```

```
ERROR:  нет доступа к представлению policy_violation_attempts

ОШИБКА:  нет доступа к представлению policy_violation_attempts
SQL state: 42501
```

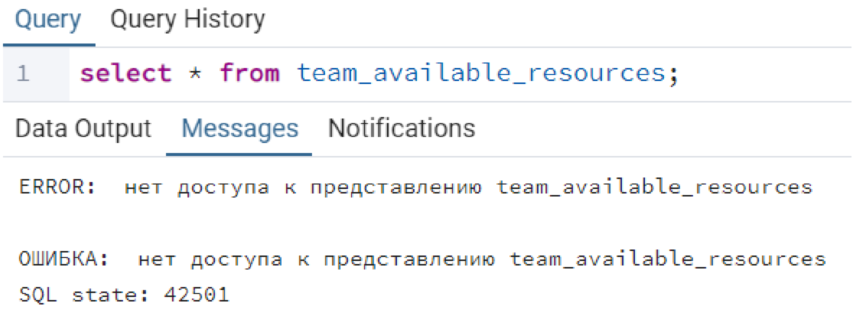
Рисунок 100 – Проверка доступа к таблицам для employee_user (13)


Query
Query History
Data Output
Messages
Notifications

```
select * from team_members_and_status;
```

```
ERROR:  нет доступа к представлению team_members_and_status

ОШИБКА:  нет доступа к представлению team_members_and_status
SQL state: 42501
```

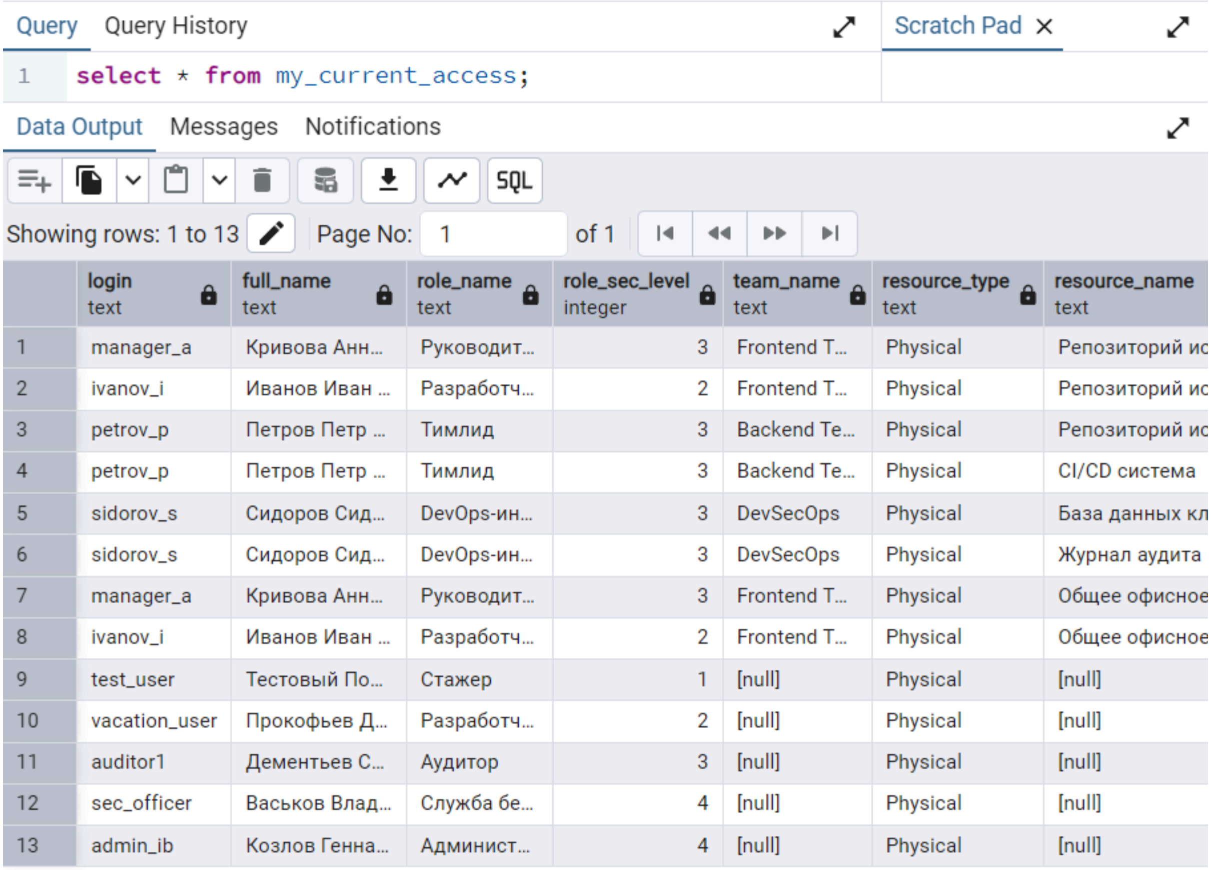
Рисунок 101 – Проверка доступа к таблицам для employee_user (14)


Query
Query History
Data Output
Messages
Notifications

```
select * from team_available_resources;
```

```
ERROR:  нет доступа к представлению team_available_resources

ОШИБКА:  нет доступа к представлению team_available_resources
SQL state: 42501
```

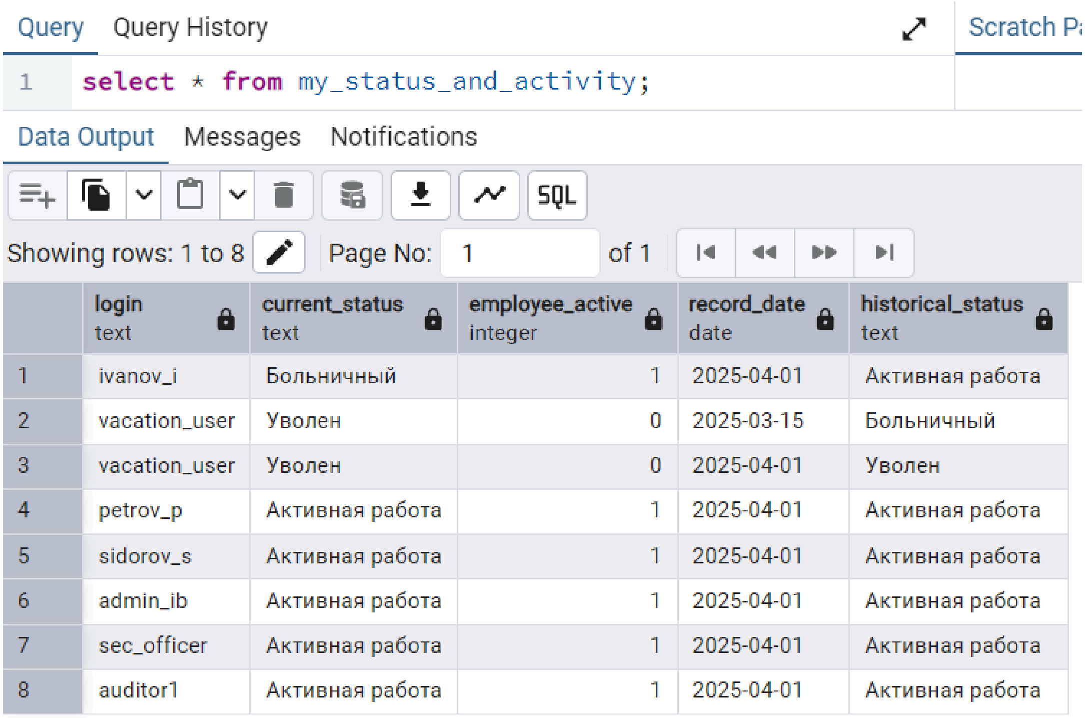
Рисунок 102 – Проверка доступа к таблицам для employee_user (15)


Query
Query History
Scratch Pad
Data Output
Messages
Notifications
Showing rows: 1 to 13
Page No: 1
of 1

```
select * from my_current_access;
```

	login text	full_name text	role_name text	role_sec_level integer	team_name text	resource_type text	resource_name text
1	manager_a	Кривова Анн...	Руководит...	3	Frontend T...	Physical	Репозиторий ис
2	ivanov_i	Иванов Иван ...	Разработч...	2	Frontend T...	Physical	Репозиторий ис
3	petrov_p	Петров Петр ...	Тимлид	3	Backend Te...	Physical	Репозиторий ис
4	petrov_p	Петров Петр ...	Тимлид	3	Backend Te...	Physical	CI/CD система
5	sidorov_s	Сидоров Сид...	DevOps-ин...	3	DevSecOps	Physical	База данных кл
6	sidorov_s	Сидоров Сид...	DevOps-ин...	3	DevSecOps	Physical	Журнал аудита
7	manager_a	Кривова Анн...	Руководит...	3	Frontend T...	Physical	Общее офисное
8	ivanov_i	Иванов Иван ...	Разработч...	2	Frontend T...	Physical	Общее офисное
9	test_user	Тестовый По...	Стажер	1	[null]	Physical	[null]
10	vacation_user	Прокофьев Д...	Разработч...	2	[null]	Physical	[null]
11	auditor1	Дементьев С...	Аудитор	3	[null]	Physical	[null]
12	sec_officer	Васьков Влад...	Служба бе...	4	[null]	Physical	[null]
13	admin_ib	Козлов Генна...	Админист...	4	[null]	Physical	[null]

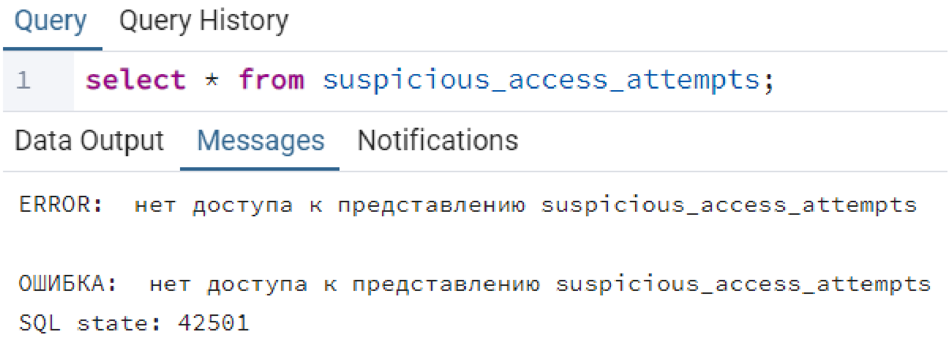
Рисунок 103 – Проверка доступа к таблицам для employee_user (16)


Query
Query History
Scratch Pa
Data Output
Messages
Notifications
Showing rows: 1 to 8
Page No: 1
of 1

```
select * from my_status_and_activity;
```

	login text	current_status text	employee_active integer	record_date date	historical_status text
1	ivanov_i	Больничный	1	2025-04-01	Активная работа
2	vacation_user	Уволен	0	2025-03-15	Больничный
3	vacation_user	Уволен	0	2025-04-01	Уволен
4	petrov_p	Активная работа	1	2025-04-01	Активная работа
5	sidorov_s	Активная работа	1	2025-04-01	Активная работа
6	admin_ib	Активная работа	1	2025-04-01	Активная работа
7	sec_officer	Активная работа	1	2025-04-01	Активная работа
8	auditor1	Активная работа	1	2025-04-01	Активная работа

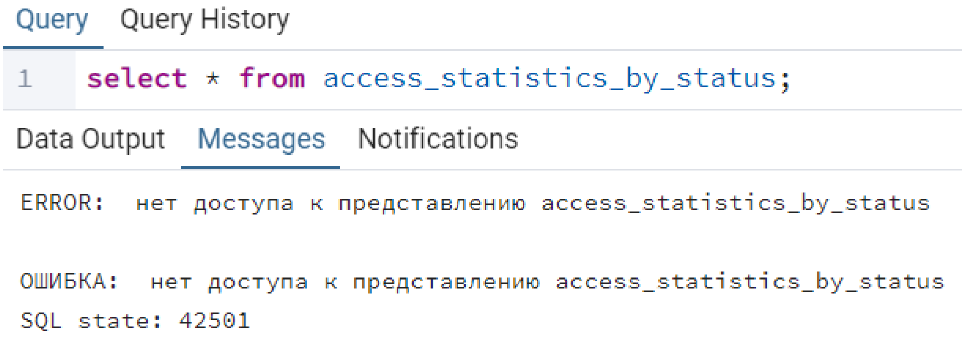
Рисунок 104 – Проверка доступа к таблицам для employee_user (17)


Query
Query History
Data Output
Messages
Notifications

```
select * from suspicious_access_attempts;
```

```
ERROR:  нет доступа к представлению suspicious_access_attempts

ОШИБКА:  нет доступа к представлению suspicious_access_attempts
SQL state: 42501
```

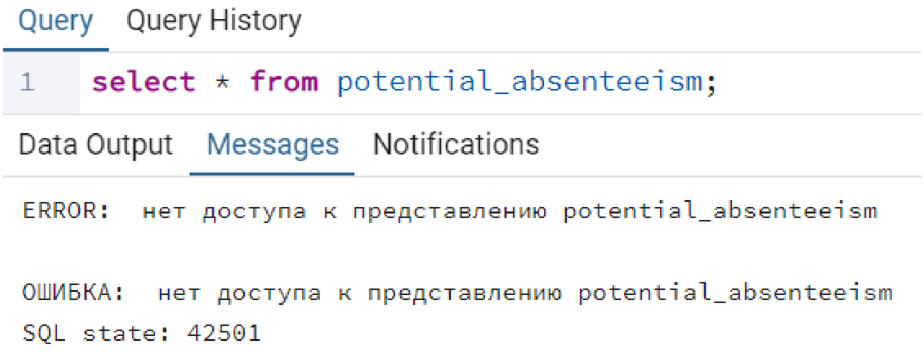
Рисунок 105 – Проверка доступа к таблицам для employee_user (18)


Query
Query History
Data Output
Messages
Notifications

```
select * from access_statistics_by_status;
```

```
ERROR:  нет доступа к представлению access_statistics_by_status

ОШИБКА:  нет доступа к представлению access_statistics_by_status
SQL state: 42501
```

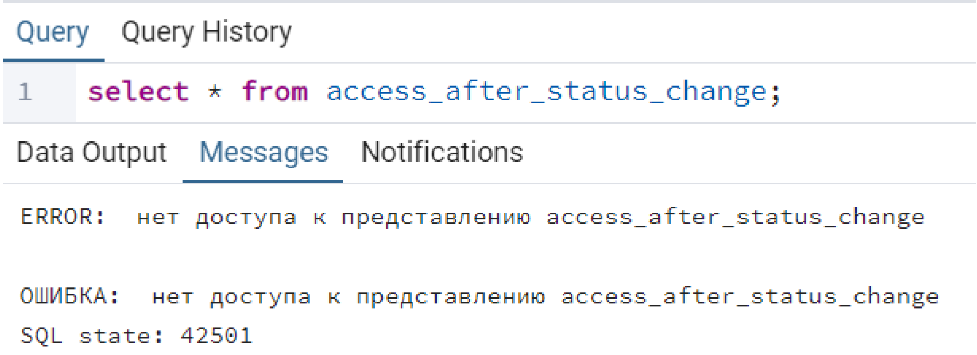
Рисунок 106 – Проверка доступа к таблицам для employee_user (19)


Query
Query History
Data Output
Messages
Notifications

```
select * from potential_absenteeism;
```

```
ERROR:  нет доступа к представлению potential_absenteeism

ОШИБКА:  нет доступа к представлению potential_absenteeism
SQL state: 42501
```

Рисунок 107 – Проверка доступа к таблицам для employee_user (20)


Query
Query History
Data Output
Messages
Notifications

```
select * from access_after_status_change;
```

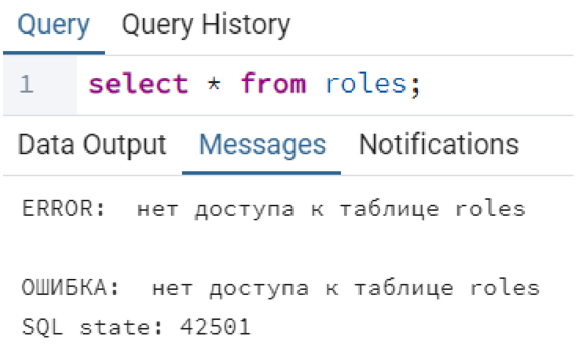
```
ERROR:  нет доступа к представлению access_after_status_change

ОШИБКА:  нет доступа к представлению access_after_status_change
SQL state: 42501
```

Рисунок 108 – Проверка доступа к таблицам для employee_user (21)

— Аудитор

Аудитор не должен видеть персональные данные сотрудников, кроме логина, и не должен иметь доступ к таблицам управления. Его задача – анализ инцидентов. Представление «Подозрительные попытки доступа» должно показывать все попытки доступа, совершенные сотрудниками со статусами, не разрешающими работу («Уволен», «Больничный», «Отпуск»): к какому ресурсу была попытка, когда и с каким результатом. Представление «Статистика доступов по статусам» должно агрегировать данные: сколько всего попыток доступа было совершено за период, сколько из них – успешных и неуспешных, с разбивкой по статусам. Это позволяет оценить эффективность политики статус-контроля.

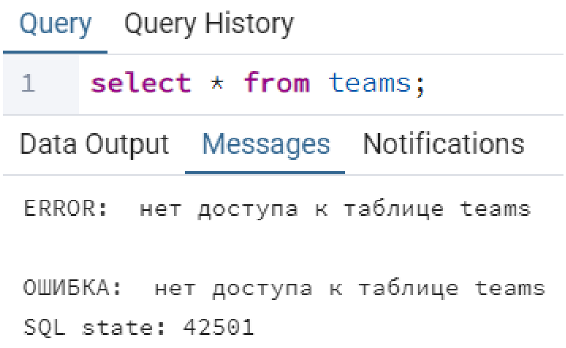
Проверка доступа к таблицам для auditor_user представлена на рисунках с 109 до 129.



```
select * from roles;
```

```
ERROR:  нет доступа к таблице roles

ОШИБКА:  нет доступа к таблице roles
SQL state: 42501
```

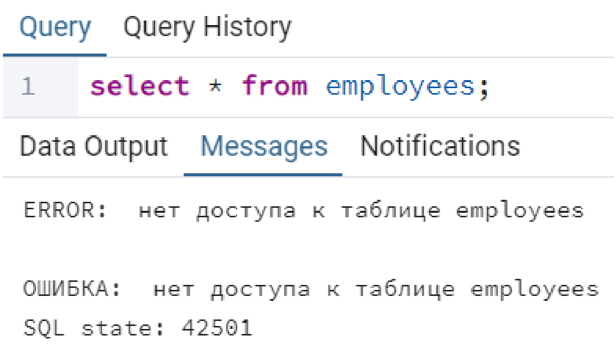
Рисунок 109 – Проверка доступа к таблицам для auditor_user (1)



```
select * from teams;
```

```
ERROR:  нет доступа к таблице teams

ОШИБКА:  нет доступа к таблице teams
SQL state: 42501
```

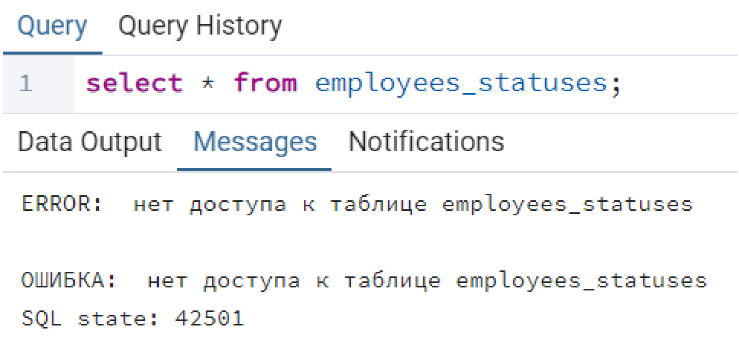
Рисунок 110 – Проверка доступа к таблицам для auditor_user (2)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from employees;
```

```
ERROR:  нет доступа к таблице employees

ОШИБКА:  нет доступа к таблице employees
SQL state: 42501
```

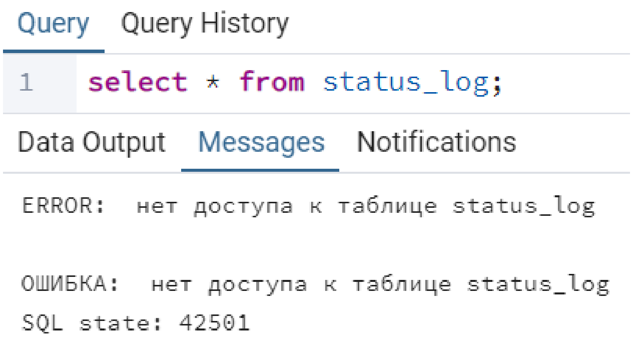
Рисунок 111 – Проверка доступа к таблицам для auditor_user (3)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from employees_statuses;
```

```
ERROR:  нет доступа к таблице employees_statuses

ОШИБКА:  нет доступа к таблице employees_statuses
SQL state: 42501
```

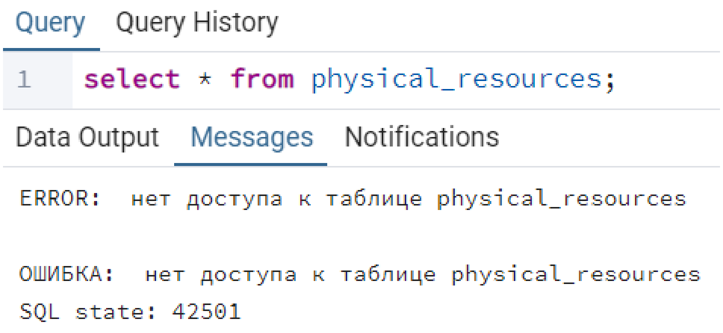
Рисунок 112 – Проверка доступа к таблицам для auditor_user (4)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from status_log;
```

```
ERROR:  нет доступа к таблице status_log

ОШИБКА:  нет доступа к таблице status_log
SQL state: 42501
```

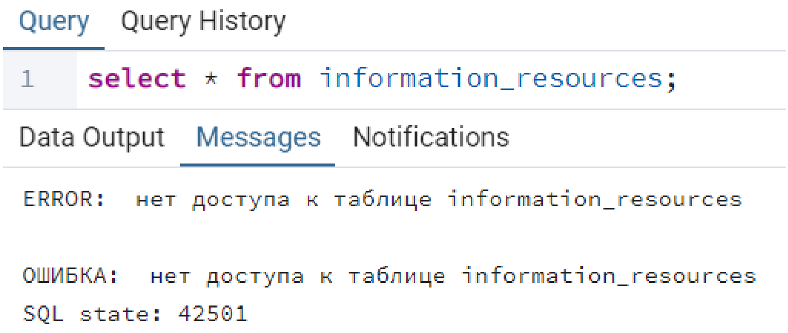
Рисунок 113 – Проверка доступа к таблицам для auditor_user (5)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from physical_resources;
```

```
ERROR:  нет доступа к таблице physical_resources

ОШИБКА:  нет доступа к таблице physical_resources
SQL state: 42501
```

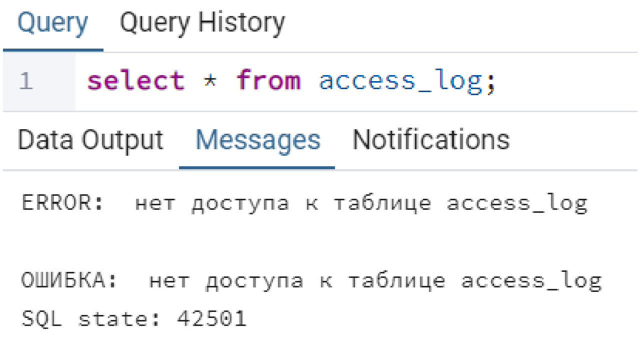
Рисунок 114 – Проверка доступа к таблицам для auditor_user (6)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from information_resources;
```

```
ERROR:  нет доступа к таблице information_resources

ОШИБКА:  нет доступа к таблице information_resources
SQL state: 42501
```

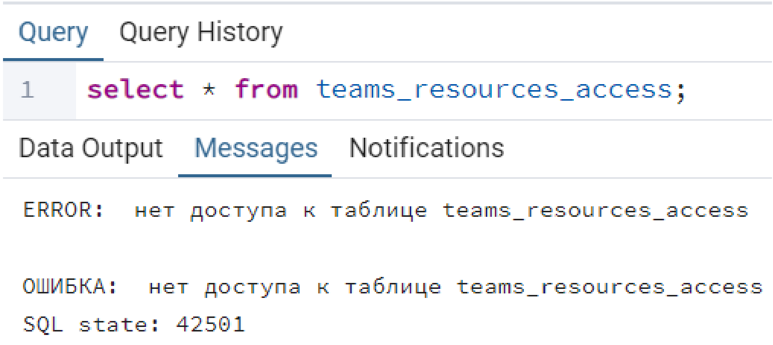
Рисунок 115 – Проверка доступа к таблицам для auditor_user (7)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from access_log;
```

```
ERROR:  нет доступа к таблице access_log

ОШИБКА:  нет доступа к таблице access_log
SQL state: 42501
```

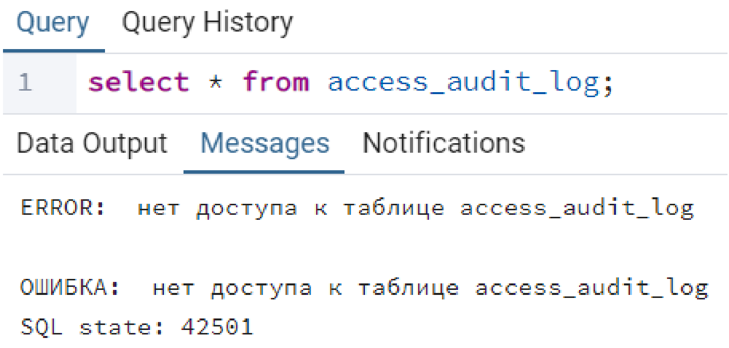
Рисунок 116 – Проверка доступа к таблицам для auditor_user (8)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from teams_resources_access;
```

```
ERROR:  нет доступа к таблице teams_resources_access

ОШИБКА:  нет доступа к таблице teams_resources_access
SQL state: 42501
```

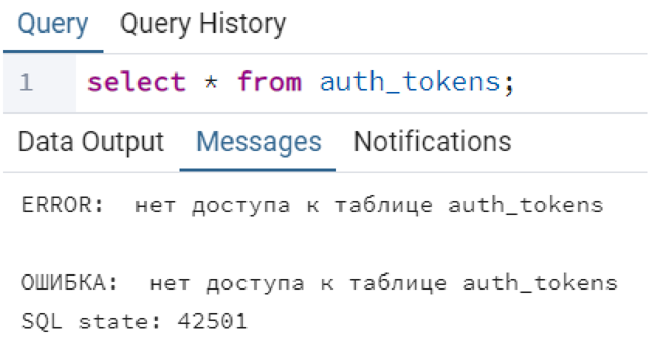
Рисунок 117 – Проверка доступа к таблицам для auditor_user (9)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from access_audit_log;
```

```
ERROR:  нет доступа к таблице access_audit_log

ОШИБКА:  нет доступа к таблице access_audit_log
SQL state: 42501
```

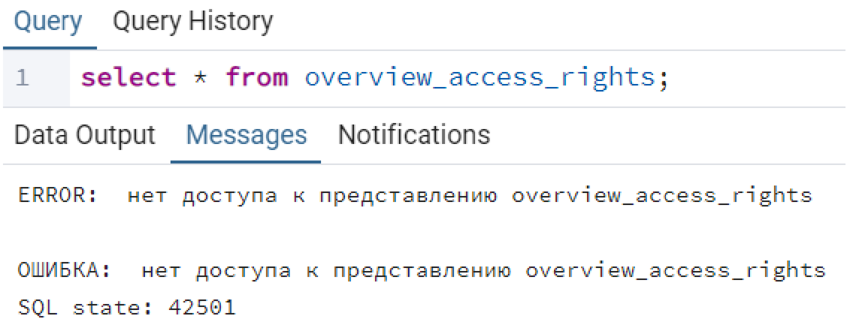
Рисунок 118 – Проверка доступа к таблицам для auditor_user (10)


Query
Query History
Data Output
Messages
Notifications

```
select * from auth_tokens;
```

```
ERROR:  нет доступа к таблице auth_tokens

ОШИБКА:  нет доступа к таблице auth_tokens
SQL state: 42501
```

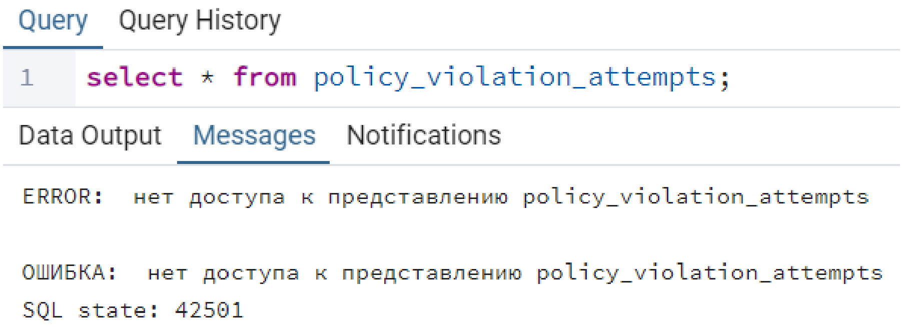
Рисунок 119 – Проверка доступа к таблицам для auditor_user (11)


Query
Query History
Data Output
Messages
Notifications

```
select * from overview_access_rights;
```

```
ERROR:  нет доступа к представлению overview_access_rights

ОШИБКА:  нет доступа к представлению overview_access_rights
SQL state: 42501
```

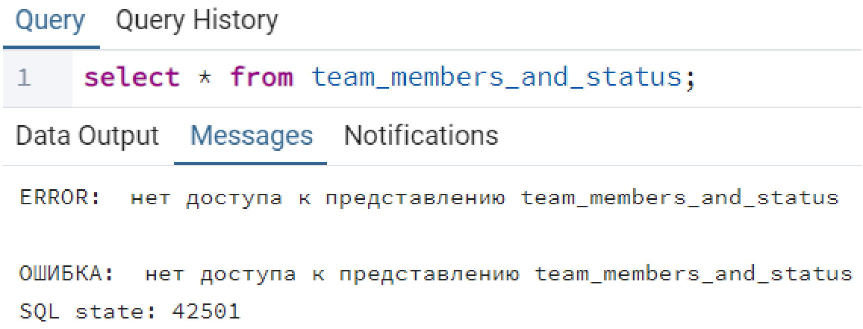
Рисунок 120 – Проверка доступа к таблицам для auditor_user (12)


Query
Query History
Data Output
Messages
Notifications

```
select * from policy_violation_attempts;
```

```
ERROR:  нет доступа к представлению policy_violation_attempts

ОШИБКА:  нет доступа к представлению policy_violation_attempts
SQL state: 42501
```

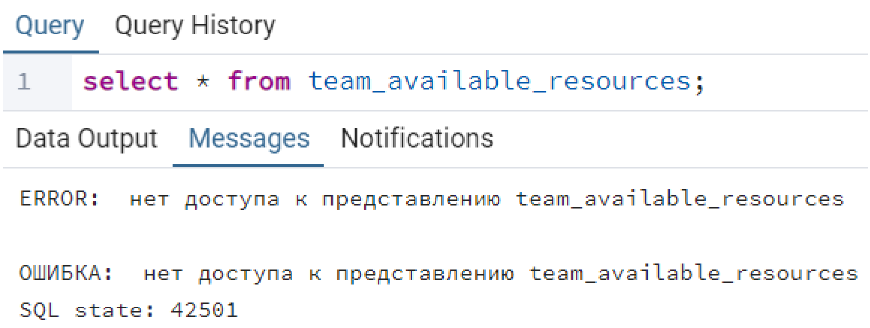
Рисунок 121 – Проверка доступа к таблицам для auditor_user (13)


Query
Query History
Data Output
Messages
Notifications

```
select * from team_members_and_status;
```

```
ERROR:  нет доступа к представлению team_members_and_status

ОШИБКА:  нет доступа к представлению team_members_and_status
SQL state: 42501
```

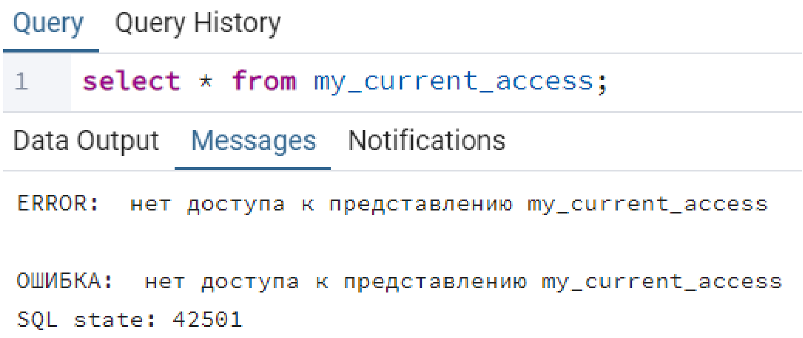
Рисунок 122 – Проверка доступа к таблицам для auditor_user (14)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from team_available_resources;
```

```
ERROR:  нет доступа к представлению team_available_resources

ОШИБКА:  нет доступа к представлению team_available_resources
SQL state: 42501
```

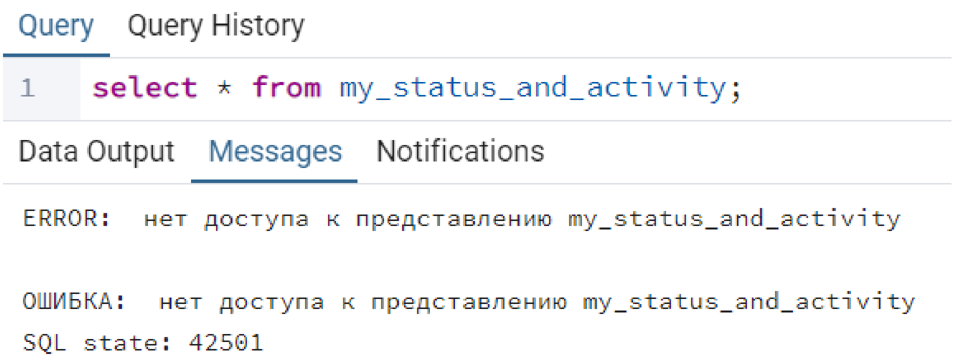
Рисунок 123 – Проверка доступа к таблицам для auditor_user (15)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from my_current_access;
```

```
ERROR:  нет доступа к представлению my_current_access

ОШИБКА:  нет доступа к представлению my_current_access
SQL state: 42501
```

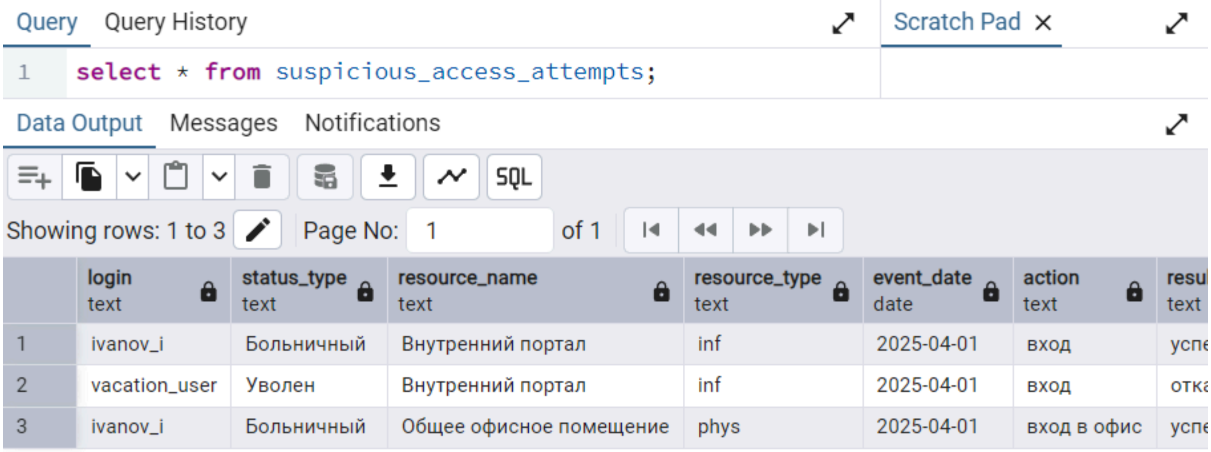
Рисунок 124 – Проверка доступа к таблицам для auditor_user (16)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from my_status_and_activity;
```

```
ERROR:  нет доступа к представлению my_status_and_activity

ОШИБКА:  нет доступа к представлению my_status_and_activity
SQL state: 42501
```

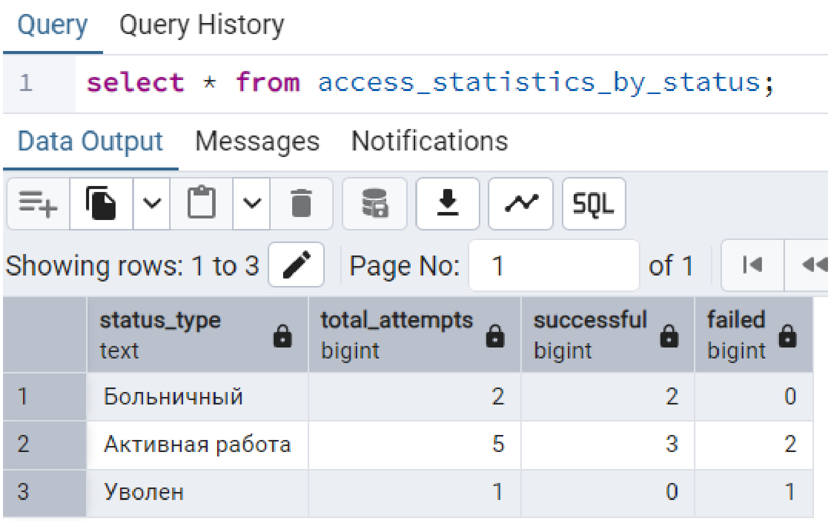
Рисунок 125 – Проверка доступа к таблицам для auditor_user (17)


Query
Query History
Scratch Pad
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 3
Page No: 1
of 1

```sql
select * from suspicious_access_attempts;
```

	login text	status_type text	resource_name text	resource_type text	event_date date	action text	resu text
1	ivanov_i	Больничный	Внутренний портал	inf	2025-04-01	вход	успе
2	vacation_user	Уволен	Внутренний портал	inf	2025-04-01	вход	отка
3	ivanov_i	Больничный	Общее офисное помещение	phys	2025-04-01	вход в офис	успе

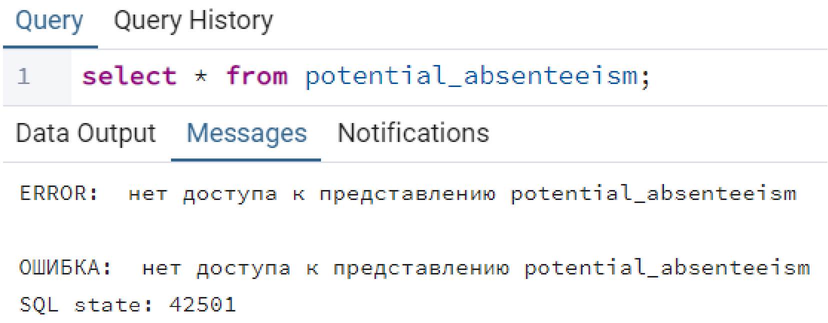
Рисунок 126 – Проверка доступа к таблицам для auditor_user (18)


Query
Query History
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 3
Page No:
of 1

```
select * from access_statistics_by_status;
```

	status_type text	total_attempts bigint	successful bigint	failed bigint
1	Больничный	2	2	0
2	Активная работа	5	3	2
3	Уволен	1	0	1

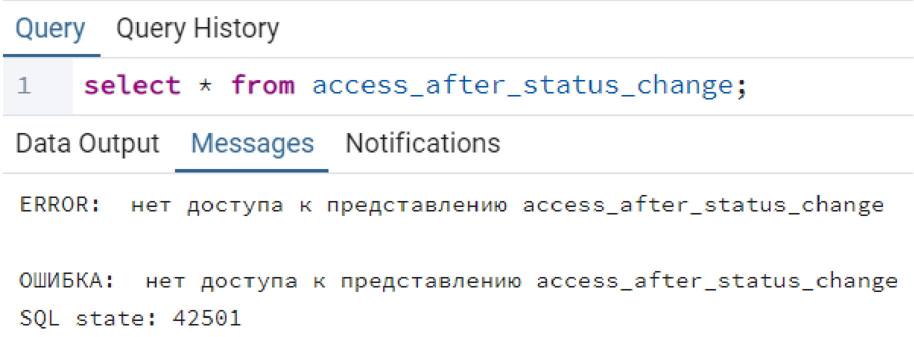
Рисунок 127 – Проверка доступа к таблицам для auditor_user (19)


Query
Query History
Data Output
Messages
Notifications

```
select * from potential_absenteeism;
```

```
ERROR:  нет доступа к представлению potential_absenteeism

ОШИБКА:  нет доступа к представлению potential_absenteeism
SQL state: 42501
```

Рисунок 128 – Проверка доступа к таблицам для auditor_user (20)


Query
Query History
Data Output
Messages
Notifications

```
select * from access_after_status_change;
```

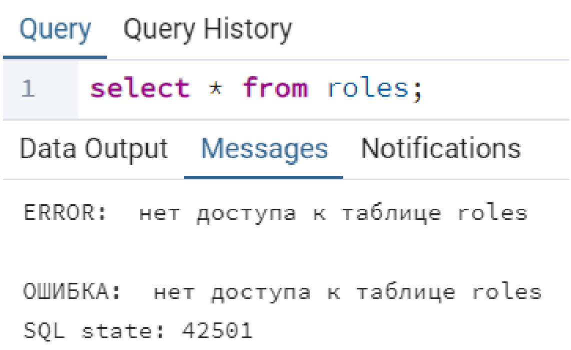
```
ERROR:  нет доступа к представлению access_after_status_change

ОШИБКА:  нет доступа к представлению access_after_status_change
SQL state: 42501
```

Рисунок 129 – Проверка доступа к таблицам для auditor_user (21)

- Служба безопасности

Служба безопасности должен фокусироваться на мониторинге аномалий. В представлении «Потенциальные прогулы» должны отображаться сотрудники с признаком прогула: статус «Активная работа», но отсутствие активности в системе в течение дня. Для каждого такого сотрудника должны указываться его ФИО, команда и последняя зафиксированная активность (например, «доступ к репозиторию 01.04.2025 09:15»). Представление «Доступы после смены статуса» должно фиксировать все попытки доступа, совершенные после смены статуса на критический («Уволен», «Отпуск», «Больничный»): кто, когда, к какому ресурсу пытался получить доступ и с каким результатом. Это помогает выявлять утечки данных или несанкционированные действия бывших сотрудников.

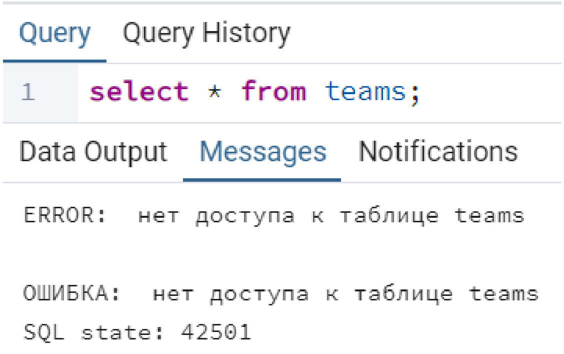
Проверка доступа к таблицам для sec_service_user представлена на рисунках с 130 до 150.


Query
Query History
Data Output
Messages
Notifications

```sql
select * from roles;
```

```
ERROR:  нет доступа к таблице roles

ОШИБКА:  нет доступа к таблице roles
SQL state: 42501
```

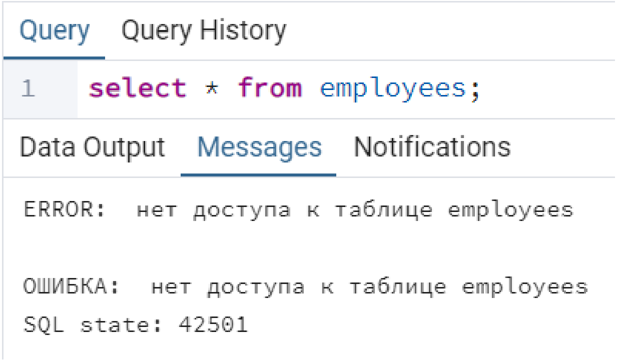
Рисунок 130 – Проверка доступа к таблицам для sec_service_user (1)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from teams;
```

```
ERROR:  нет доступа к таблице teams

ОШИБКА:  нет доступа к таблице teams
SQL state: 42501
```

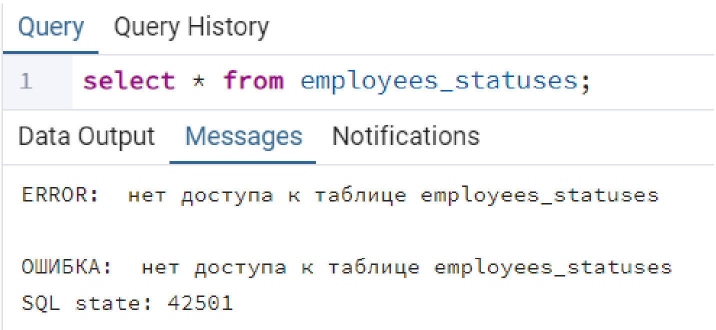
Рисунок 131 – Проверка доступа к таблицам для sec_service_user (2)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from employees;
```

```
ERROR:  нет доступа к таблице employees

ОШИБКА:  нет доступа к таблице employees
SQL state: 42501
```

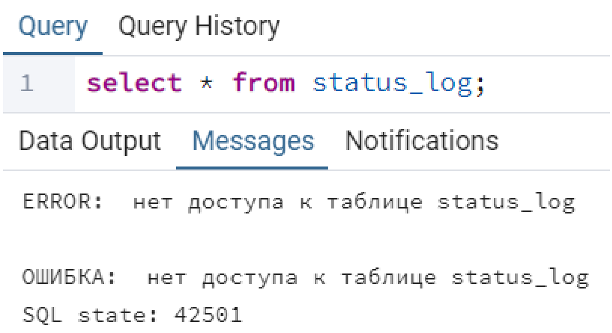
Рисунок 132 – Проверка доступа к таблицам для sec_service_user (3)


Query
Query History
Data Output
Messages
Notifications

```
select * from employees_statuses;
```

```
ERROR:  нет доступа к таблице employees_statuses

ОШИБКА:  нет доступа к таблице employees_statuses
SQL state: 42501
```

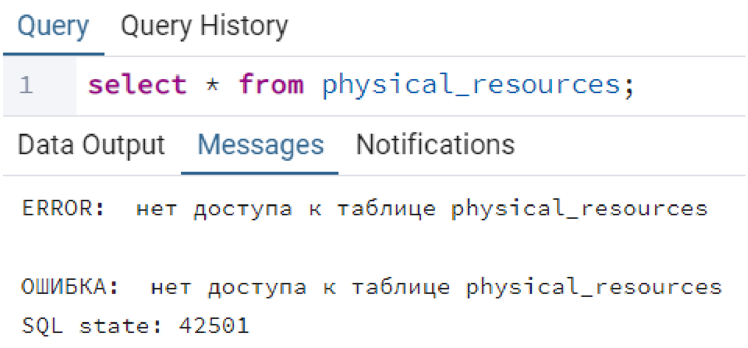
Рисунок 133 – Проверка доступа к таблицам для sec_service_user (4)


Query
Query History
Data Output
Messages
Notifications

```
select * from status_log;
```

```
ERROR:  нет доступа к таблице status_log

ОШИБКА:  нет доступа к таблице status_log
SQL state: 42501
```

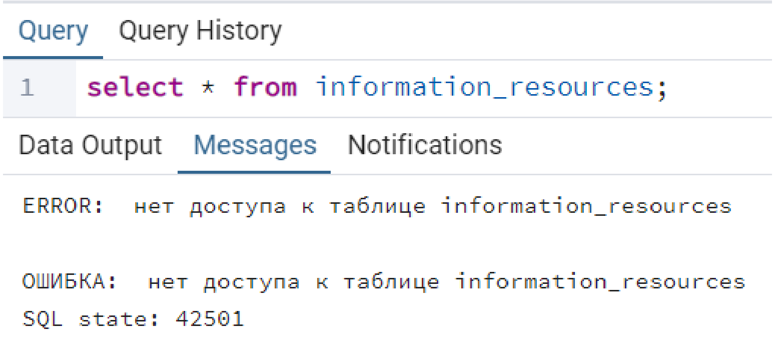
Рисунок 134 – Проверка доступа к таблицам для sec_service_user (5)


Query
Query History
Data Output
Messages
Notifications

```
select * from physical_resources;
```

```
ERROR:  нет доступа к таблице physical_resources

ОШИБКА:  нет доступа к таблице physical_resources
SQL state: 42501
```

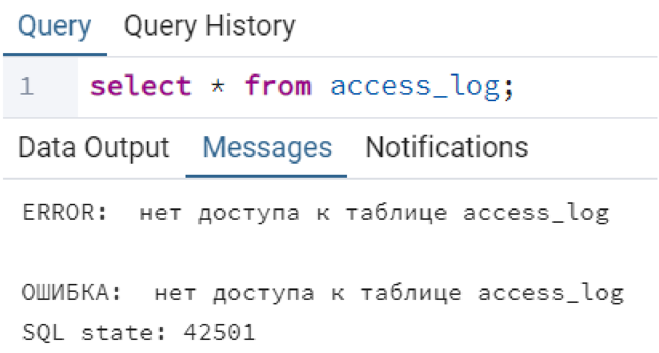
Рисунок 135 – Проверка доступа к таблицам для sec_service_user (6)


Query
Query History
Data Output
Messages
Notifications

```
select * from information_resources;
```

```
ERROR:  нет доступа к таблице information_resources

ОШИБКА:  нет доступа к таблице information_resources
SQL state: 42501
```

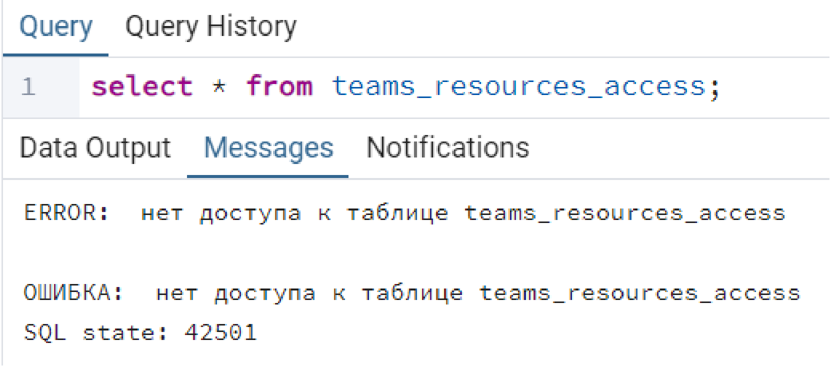
Рисунок 136 – Проверка доступа к таблицам для sec_service_user (7)


Query
Query History
Data Output
Messages
Notifications

```
select * from access_log;
```

```
ERROR:  нет доступа к таблице access_log

ОШИБКА:  нет доступа к таблице access_log
SQL state: 42501
```

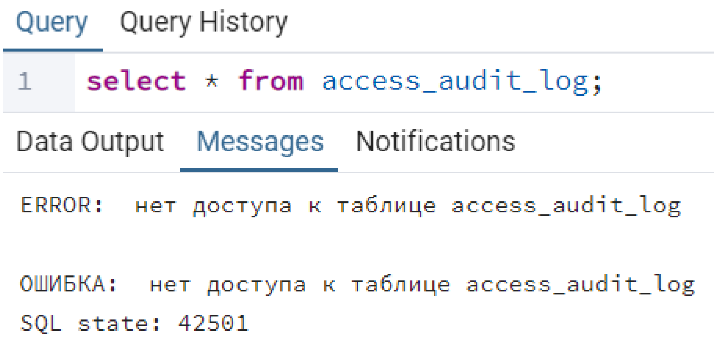
Рисунок 137 – Проверка доступа к таблицам для sec_service_user (8)


Query
Query History
Data Output
Messages
Notifications

```
select * from teams_resources_access;
```

```
ERROR:  нет доступа к таблице teams_resources_access

ОШИБКА:  нет доступа к таблице teams_resources_access
SQL state: 42501
```

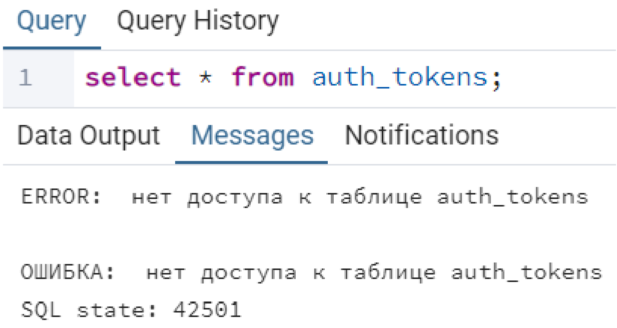
Рисунок 138 – Проверка доступа к таблицам для sec_service_user (9)


Query
Query History
Data Output
Messages
Notifications

```
select * from access_audit_log;
```

```
ERROR:  нет доступа к таблице access_audit_log

ОШИБКА:  нет доступа к таблице access_audit_log
SQL state: 42501
```

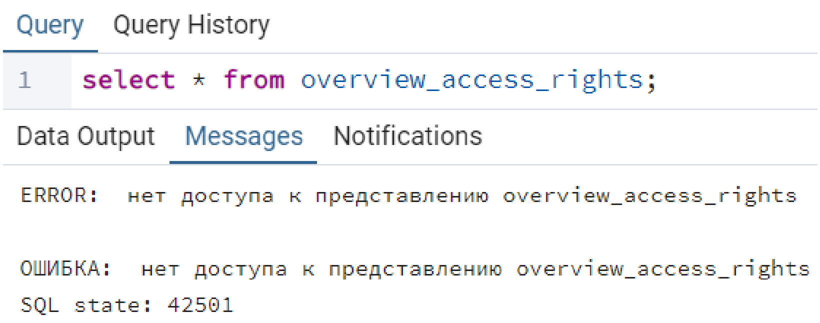
Рисунок 139 – Проверка доступа к таблицам для sec_service_user (10)


Query
Query History
Data Output
Messages
Notifications

```
select * from auth_tokens;
```

```
ERROR:  нет доступа к таблице auth_tokens

ОШИБКА:  нет доступа к таблице auth_tokens
SQL state: 42501
```

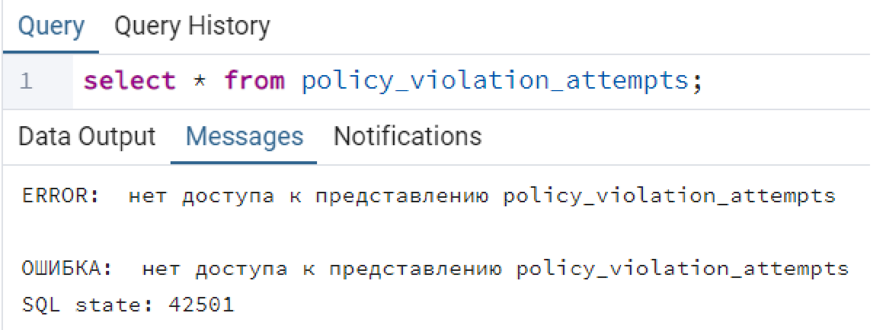
Рисунок 140 – Проверка доступа к таблицам для sec_service_user (11)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from overview_access_rights;
```

```
ERROR:  нет доступа к представлению overview_access_rights

ОШИБКА:  нет доступа к представлению overview_access_rights
SQL state: 42501
```

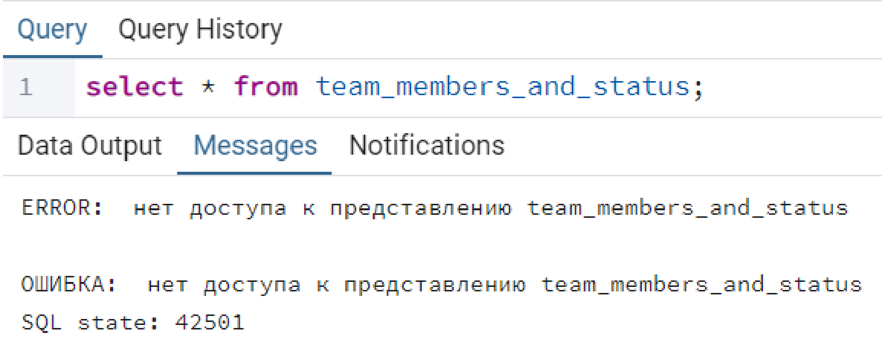
Рисунок 141 – Проверка доступа к таблицам для sec_service_user (12)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from policy_violation_attempts;
```

```
ERROR:  нет доступа к представлению policy_violation_attempts

ОШИБКА:  нет доступа к представлению policy_violation_attempts
SQL state: 42501
```

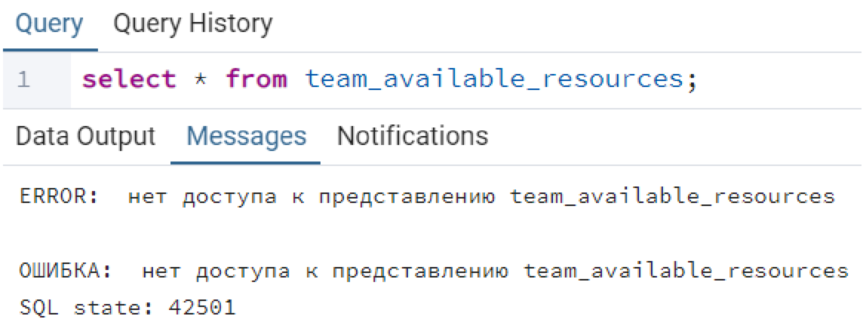
Рисунок 142 – Проверка доступа к таблицам для sec_service_user (13)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from team_members_and_status;
```

```
ERROR:  нет доступа к представлению team_members_and_status

ОШИБКА:  нет доступа к представлению team_members_and_status
SQL state: 42501
```

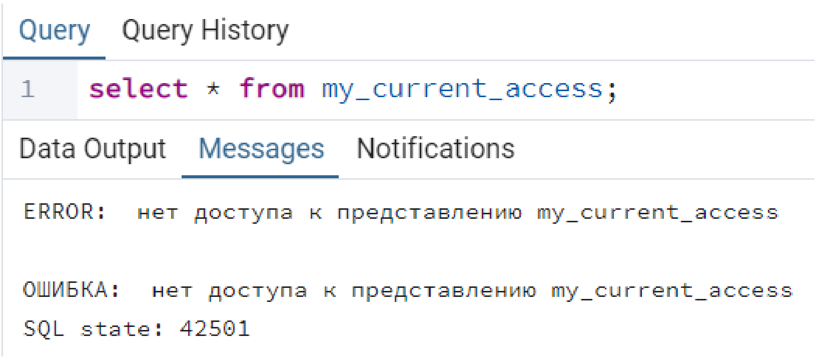
Рисунок 143 – Проверка доступа к таблицам для sec_service_user (14)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from team_available_resources;
```

```
ERROR:  нет доступа к представлению team_available_resources

ОШИБКА:  нет доступа к представлению team_available_resources
SQL state: 42501
```

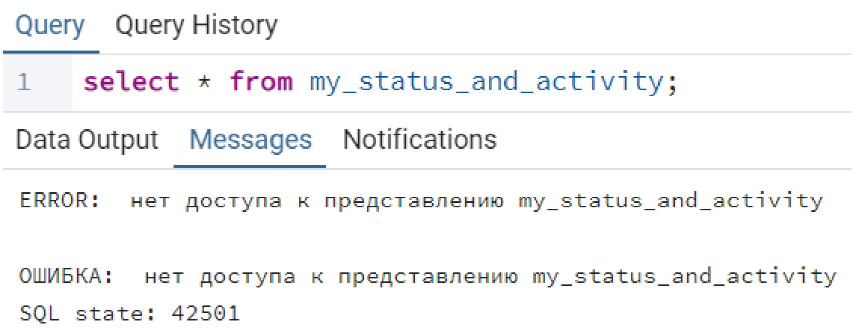
Рисунок 144 – Проверка доступа к таблицам для sec_service_user (15)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from my_current_access;
```

```
ERROR:  нет доступа к представлению my_current_access

ОШИБКА:  нет доступа к представлению my_current_access
SQL state: 42501
```

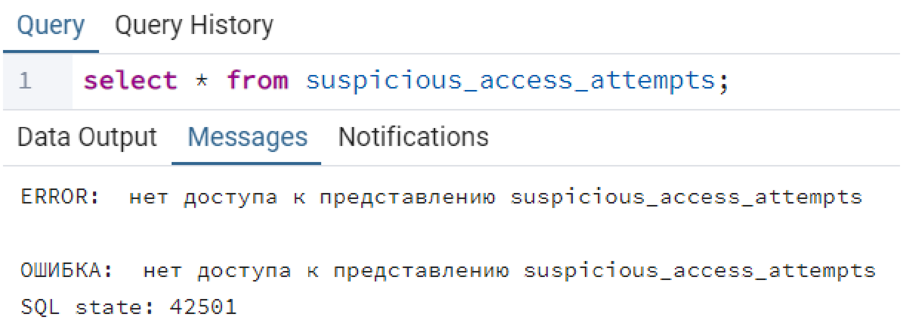
Рисунок 145 – Проверка доступа к таблицам для sec_service_user (16)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from my_status_and_activity;
```

```
ERROR:  нет доступа к представлению my_status_and_activity

ОШИБКА:  нет доступа к представлению my_status_and_activity
SQL state: 42501
```

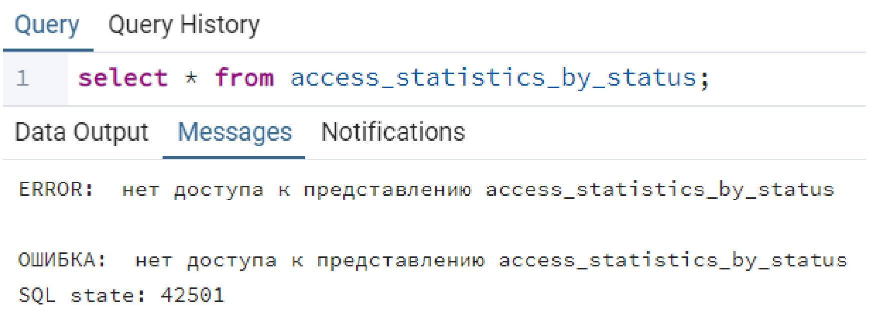
Рисунок 146 – Проверка доступа к таблицам для sec_service_user (17)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from suspicious_access_attempts;
```

```
ERROR:  нет доступа к представлению suspicious_access_attempts

ОШИБКА:  нет доступа к представлению suspicious_access_attempts
SQL state: 42501
```

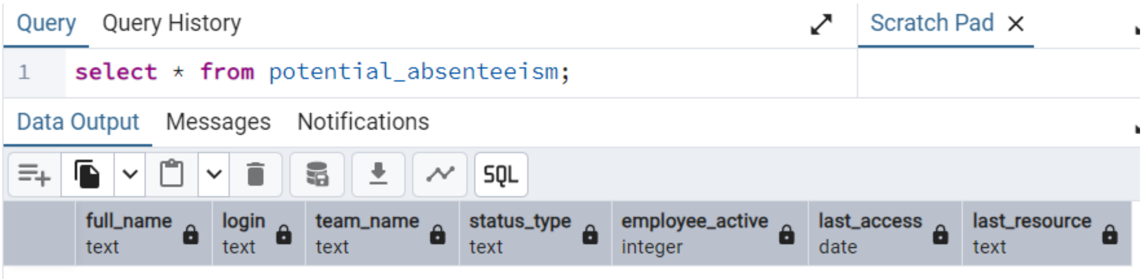
Рисунок 147 – Проверка доступа к таблицам для sec_service_user (18)


Query
Query History
Data Output
Messages
Notifications

```sql
select * from access_statistics_by_status;
```

```
ERROR:  нет доступа к представлению access_statistics_by_status

ОШИБКА:  нет доступа к представлению access_statistics_by_status
SQL state: 42501
```

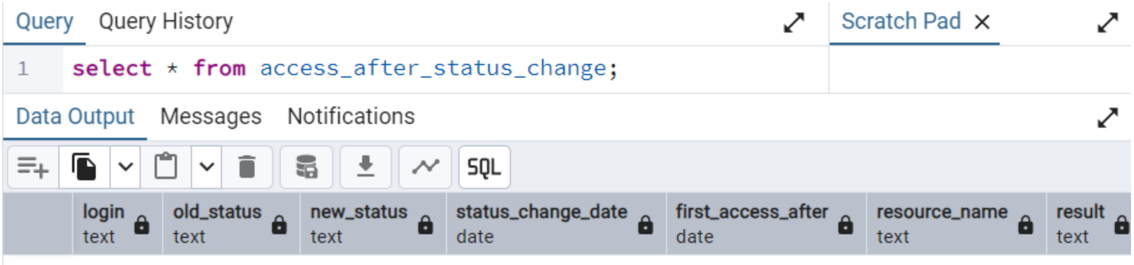
Рисунок 148 – Проверка доступа к таблицам для sec_service_user (19)


Query
Query History
Scratch Pad
Data Output
Messages
Notifications
SQL

```
select * from potential_absenteeism;
```

full_name text	login text	team_name text	status_type text	employee_active integer	last_access date	last_resource text

Рисунок 149 – Проверка доступа к таблицам для sec_service_user (20)


Query
Query History
Scratch Pad
Data Output
Messages
Notifications
SQL

```
select * from access_after_status_change;
```

login text	old_status text	new_status text	status_change_date date	first_access_after date	resource_name text	result text

Рисунок 150 – Проверка доступа к таблицам для sec_service_user (21)

Таблица 1 – Сводная таблица прав доступа ролей

	Администратор ИБ	Руководитель команды	Сотрудник	Аудитор	Служба безопасности
Основные таблицы					
roles	+	–	–	–	–
teams	+	–	–	–	–
employees	+	–	–	–	–
employees_statuses	+	–	–	–	–
status_log	+	–	–	–	–
physical_resources	+	–	–	–	–
information_resources	+	–	–	–	–
access_log	+	–	–	–	–
teams_resources_access	+	–	–	–	–
Логирование и шифроваие					
access_audit_log	+	–	–	–	–
auth_tokens	+	–	–	–	–

Представления					
overview_access_rights	+	–	–	–	–
policy_violation_attempts	+	–	–	–	–
team_members_and_status	+	+	–	–	–
team_available_resources	+	+	–	–	–
my_current_access	+	–	+	–	–
my_status_and_activity	+	–	+	–	–
suspicious_access_attempts	+	–	–	+	–
access_statistics_by_status	+	–	–	+	–
potential_absenteeism	+	–	–	–	+
access_after_status_change	+	–	–	–	+

3.4 Вывод

В ходе выполнения третьего этапа работы была успешно реализована комплексная примитивная система защиты на уровне СУБД PostgreSQL, включающая три ключевых компонента: аудит изменений, шифрование конфиденциальных данных и гибкое разграничение доступа.

Созданная таблица-лог и универсальные триггеры обеспечивают полную прозрачность всех операций DML над основными сущностями, что позволяет отслеживать любые изменения и расследовать инциденты.

Конфиденциальные данные (токены аутентификации) защищены с использованием симметричного шифрования AES-256 через модуль pgcrypto, при этом ключ шифрования не хранится в базе, а восстанавливается из индивидуального пароля суперпользователя, что исключает доступ к секретам даже для администратора СУБД без этого пароля.

Система ролевого доступа, построенная на принципе минимальных привилегий, точно соответствует классам потребителей, выделенным на первом этапе: каждый тип пользователя (администратор ИБ, руководитель команды, сотрудник, аудитор, служба безопасности) получает только те данные, которые необходимы для выполнения его задач, через специально созданные представления, в то время как прямой доступ к служебным таблицам (лог, токены) запрещён для всех, кроме администратора ИБ.

Таким образом, реализованная система обеспечивает целостность, конфиденциальность и контролируемый доступ к данным.

4 РЕАЛИЗАЦИЯ СЕРВИСА ДЛЯ ВЗАИМОДЕЙСТВИЯ С РАЗРАБОТАННОЙ БД

4.1 Технологический стек

Для разработки приложения были выбраны следующие технические средства:

- **Python** – современный, безопасный язык с богатой экосистемой для веб-разработки;
- **PyCharm** – среда программирования;
- **FastAPI** – высокопроизводительный фреймворк с автоматической документацией, поддержкой асинхронности и встроенной валидацией;
- **SQLAlchemy** – мощный ORM для работы с PostgreSQL, поддерживающий сложные запросы и транзакции;
- **JWT** + **HTTPBearer** для безопасной аутентификации в REST API;
- Дополнительные инструменты:
- **python-jose** для работы с JWT-токенами;
- **passlib** и **bcrypt** для хэширования паролей;
- **Jinja2** для шаблонизации HTML;
- **uvicorn** – ASGI-сервер для запуска приложения.

Установка всех необходимых библиотек и фреймворков осуществлялась с помощью текстового файла requirements.txt, содержимое которого следующее:

fastapi==0.104.1 – веб-фреймворк для создания API на Python

uvicorn==0.24.0 – ASGI-сервер для запуска FastAPI приложений

sqlalchemy==2.0.23 – библиотека для работы с базами данных через объекты Python

psycopg2-binary==2.9.9 – нативный драйвер для подключения к PostgreSQL

python-jose[cryptography]==3.3.0 – библиотека для создания и проверки JWT (JSON Web Tokens)

passlib[bcrypt]==1.7.4 – библиотека для безопасного хэширования паролей *python-multipart==0.0.6* – библиотека для обработки multipart/form-data

jinja2==3.1.2 – шаблонизатор для генерации HTML

alembic==1.12.1 – инструмент для управления изменениями схемы БД (миграции БД)

pydantic==2.5.0 – библиотека для проверки и преобразования данных

Запустив данный файл с помощью команды *pip install -r requirements.txt* после предварительного перехода в папку с проектом, установятся все необходимые для работы библиотеки и фреймворки.

Также можно установить все зависимости отдельно:

```
pip install fastapi==0.104.1
pip install uvicorn==0.24.0
pip install sqlalchemy==2.0.23
pip install psycopg2-binary==2.9.9
pip install python-jose[cryptography]==3.3.0
pip install passlib[bcrypt]==1.7.4
pip install python-multipart==0.0.6
pip install jinja2==3.1.2
pip install pydantic==2.5.0
pip install alembic==1.12.1
```

4.2 Архитектура приложения

Все составные части приложения хранятся в папке *access-control-app/*. В этой папке хранятся следующие файлы и папки:

main.py # Основное приложение FastAPI
auth.py # Модуль аутентификации и авторизации
models.py # SQLAlchemy модели (сущности БД)
database.py # Подключение к БД, сессии
schemas.py # Pydantic схемы для валидации
config.py # Конфигурация (маппинг ролей, настройки)
static/ # Статические файлы (CSS, JS)
templates/ # HTML-шаблоны для ролевых дашбордов

Полный код всех составных частей приложения можно будет посмотреть в прикрепленном zip-файле. В каждом файле есть комментарии.

4.3 Реализация многопользовательского доступа

В *config.py* и *auth.py* реализован гибкий маппинг должностей из таблицы roles на роли доступа в приложении:

```
# Маппинг должностей на роли доступа в приложении
# Ключ: название должности в таблице roles.role_name
# Значение: роль доступа в приложении (ib_admin, team_lead, employee, auditor, sec_service)
ROLE_MAPPING = {
    # Административные роли
    "Администратор ИБ": "ib_admin",
    "Служба безопасности": "sec_service",
    "Аудитор": "auditor",

    # Руководящие роли
    "Тимлид": "team_lead",
    "Руководитель отдела": "team_lead",
    "Руководитель проекта": "team_lead",

    # Технические роли (сотрудники)
    "DevOps-инженер": "employee",
    "Разработчик": "employee",
    "Тестировщик": "employee",
    "Аналитик": "employee",
    "Дизайнер": "employee",

    # Начальные позиции
    "Стажер": "employee",
    "Младший разработчик": "employee",

    # По умолчанию для всех остальных должностей
}

# Роль по умолчанию, если должность не найдена в маппинге
DEFAULT_ROLE = "employee"
```

```
# ========== МАППИНГ РОЛЕЙ ==========
ROLE_MAPPING = {
    "Администратор ИБ": "ib_admin",
    "Служба безопасности": "sec_service",
    "Аудитор": "auditor",
    "Тимлид": "team_lead",
    "Руководитель отдела": "team_lead",
    "DevOps-инженер": "employee",
        "Разработчик": "employee",
        "Стажер": "employee",
    }

    DEFAULT_ROLE = "employee"
```

В процессе появления новых ролей сотрудников, администратор информационной безопасности сможет дополнять эту часть кода приложения.

Аутентификация происходит через JWS-токен, чтобы не запрашивать постоянно у сотрудника его логин и пароль. Пароли хранятся в БД в захэшированном виде с использованием *crypt()* PostgreSQL. При успешном входе генерируется JWT-токен с полями sub (логин), role, exp (срок жизни токена составляет 30 минут, после чего система просиз пользователя заново авторизоваться). Токен сохраняется в таблицу *auth_tokens* и передается клиенту через cookie.

```
    @app.post("/login")
    async def login_user(
            request: Request,
            username: str = Form(...),
            password: str = Form(...),
            db: Session = Depends(get_db)
    ):
        """Обработка входа пользователя"""
        # Аутентифицируем пользователя
        employee = auth.authenticate_user(db, username, password)
        if not employee:
            return templates.TemplateResponse("login.html", {
                "request": request,
                "error": "Неверное имя пользователя или пароль"
            })

        # Создаем и сохраняем токен
        token_data = await auth.create_and_store_token(db,
employee)
        access_token = token_data["access_token"]
        access_role = token_data["role"]

        # Создаем HTML с JavaScript для установки cookie и
редиректа
        html_content = f"""
        <!DOCTYPE html>
        <html>
        <head>
            <title>Успешный вход</title>
            <script>
                // Устанавливаем токен в cookie
                document.cookie = "access_token={access_token};
path=/; max-age=1800; SameSite=Lax";

                // Через небольшой таймаут делаем редирект (чтобы
cookie успел установиться)
                setTimeout(function() {{
                    window.location.href =
"/dashboard/{access_role}?token={access_token}";
                }}, 100);
            </script>
        </head>
        <body>
            <div style="text-align: center; margin-top: 100px;">
                <h2>Вход выполнен успешно!</h2>
                <p>Перенаправление на панель управления...</p>
                <p><small>Если перенаправление не произошло, <a
href="/dashboard/{access_role}?token={access_token}">нажмите
сюда</a></small></p>
            </div>
        </body>
        </html>
        """

        return HTMLResponse(content=html_content)
    @app.get("/api/check_token")
    async def check_token(
            request: Request,
            db: Session = Depends(get_db)
    ):
        """Проверяет валидность токена"""
        # Пробуем получить токен из разных источников
        token = None

        # 1. Из Authorization header
        auth_header = request.headers.get("Authorization")
        if auth_header and auth_header.startswith("Bearer "):
            token = auth_header.split(" ")[1]

        # 2. Из cookie
        if not token:
            token = request.cookies.get("access_token")
        # 3. Из query параметра
        if not token:
            token = request.query_params.get("token")

        if not token:
            return JSONResponse(
                status_code=200,  # Используем 200 чтобы фронтенд мог обработать
                content={
                    "valid": False,
                    "message": "Токен не предоставлен",
                    "redirect_url": "/"
                }
            )

        # Проверяем токен
        try:
            employee = await auth.validate_token(db, token)

            if employee:
                return {
                    "valid": True,
                    "message": "Токен действителен",
                    "user": {
                        "login": employee.login,
                        "full_name": employee.full_name,
                        "role": auth.get_user_access_role(employee)
                    },
                    "timestamp": datetime.now().isoformat()
                }
            else:
                return JSONResponse(
                    status_code=200,
                    content={
                        "valid": False,
                        "message": "Токен недействителен или истек",
                        "redirect_url": "/"
                    }
                )

        except Exception as e:
            print(f"Ошибка проверки токена: {e}")
            return JSONResponse(
                status_code=200,
                content={
                    "valid": False,
                    "message": "Ошибка проверки токена",
                    "redirect_url": "/"
                }
            )
```

```
    def verify_password_postgres(db: Session, plain_password: str,
hashed_password: str) -> bool:
        """Проверяет пароль через PostgreSQL crypt()"""
        try:
            query = text("SELECT crypt(:password, :hash) = :hash as
is_valid")
            result = db.execute(
                query,
                {"password": plain_password, "hash":
hashed_password}
            ).fetchone()
            return result.is_valid if result else False
        except Exception as e:
            print(f"Ошибка проверки пароля: {e}")
            return False


    def authenticate_user(db: Session, username: str, password:
str) -> Optional[models.Employee]:
        """Аутентифицирует пользователя"""
        employee =
db.query(models.Employee).filter(models.Employee.login ==
username).first()
        if not employee:
            print(f"Пользователь {username} не найден")
            return None

        if verify_password_postgres(db, password,
employee.password):
            return employee

        print(f"Неверный пароль для {username}")
        return None


    def get_user_access_role(employee: models.Employee) -> str:
        """Определяет роль доступа на основе должности"""
        if not employee or not employee.role:
            return DEFAULT_ROLE

        role_name = employee.role.role_name
        return ROLE_MAPPING.get(role_name, DEFAULT_ROLE)


    def create_jwt_token(login: str, role: str) -> str:
        """Создает JWT токен"""
        expires_delta =
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
        expire = datetime.now() + expires_delta

        to_encode = {
            "sub": login,
            "role": role,
            "exp": int(expire.timestamp())
        }

        return jwt.encode(to_encode, SECRET_KEY,
algorithm=ALGORITHM)


    async def create_and_store_token(db: Session, employee:
models.Employee) -> Dict[str, str]:
        """Создает JWT токен и сохраняет его в БД, удаляя старые
токены пользователя"""
        try:
            # Определяем роль
            access_role = get_user_access_role(employee)

            # УДАЛЯЕМ ВСЕ СУЩЕСТВУЮЩИЕ ТОКЕНЫ ПОЛЬЗОВАТЕЛЯ
            deleted_count = db.query(models.AuthToken).filter(
                models.AuthToken.user_login == employee.login
            ).delete()

            if deleted_count > 0:
                print(f"Удалено {deleted_count} старых токенов для
пользователя {employee.login}")

            # Создаем новый JWT токен
            jwt_token = create_jwt_token(employee.login,
access_role)

            # Создаем запись токена в БД
            expires_at = datetime.now() +
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
```

```
            auth_token = models.AuthToken(
                user_login=employee.login,
                encrypted_token=jwt_token,
                expires_at=expires_at
            )

            db.add(auth_token)
            db.commit()

            return {
                "access_token": jwt_token,
                "token_type": "bearer",
                "role": access_role,
                "employee_id": employee.id_employee
            }

        except Exception as e:
            db.rollback()
            print(f"Ошибка создания токена: {e}")
            raise


    async def validate_token(db: Session, token: str) ->
Optional[models.Employee]:
        """Проверяет токен и возвращает сотрудника"""
        try:
            # 1. Проверяем в БД
            auth_token = db.query(models.AuthToken).filter(
                models.AuthToken.encrypted_token == token,
                models.AuthToken.expires_at > datetime.now()
            ).first()

            if not auth_token:
                print("Токен не найден в БД или истек")
                return None

            # 2. Проверяем JWT
            payload = jwt.decode(token, SECRET_KEY,
algorithms=[ALGORITHM])
            user_login = payload.get("sub")

            if user_login != auth_token.user_login:
                print("Несоответствие ID сотрудника в токене")
                return None
        # 3. Получаем сотрудника
        employee = db.query(models.Employee).filter(
            models.Employee.login == user_login
        ).first()

        return employee

    except JWTError as e:
        print(f"Ошибка JWT: {e}")
        return None
    except Exception as e:
        print(f"Ошибка проверки токена: {e}")
        return None
```

FastAPI позволяет создавать зависимости для проверки ролей:

```
def require_role(required_role: str):
    """Создает dependency для проверки роли"""

    async def role_checker(
            current_user: models.Employee =
Depends(get_current_user),
            db: Session = Depends(get_db)
    ) -> models.Employee:
        user_role = get_user_access_role(current_user)
        if user_role != required_role:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f"Недостаточно прав. Требуется роль:
{required_role}, а у вас: {user_role}"
            )
        return current_user

    return role_checker



# Конкретные зависимости для каждой роли
check_ib_admin = require_role("ib_admin")
check_team_lead = require_role("team_lead")
check_employee = require_role("employee")
check_auditor = require_role("auditor")
check_sec_service = require_role("sec_service")
```

Каждая роль получает уникальный интерфейс (*/dashboard/{role_type}*):

- Администратор ИБ – полный доступ ко всем таблицам, логам, ресурсам;

- Руководитель команды – просмотр состава команды, доступов команды;
- Сотрудник – личная информация, история доступов;
- Аудитор – подозрительные доступы, аудит-логи;
- Служба безопасности – потенциальные прогулы, нарушения.

```
     # ========== ПАНЕЛИ УПРАВЛЕНИЯ ==========

     @app.get("/dashboard/{role_type}",
response_class=HTMLResponse)
     async def dashboard(
            request: Request,
            role_type: str,
            db: Session = Depends(get_db),
                               current_user:    models.Employee    =
Depends(auth.get_current_user)
     ):
        """Панель управления в зависимости от роли"""
        # Проверяем, что роль пользователя совпадает с запрошенной
        user_role = auth.get_user_access_role(current_user)
        if user_role != role_type:
            raise HTTPException(
                status_code=403,
                 detail=f"Доступ запрещен. Ваша роль: {user_role},
запрошена: {role_type}"
            )

        # Подготавливаем контекст
        context = {
            "request": request,
            "user": current_user,
            "user_role": user_role,
            "role_type": role_type
        }

        # Заполняем данными в зависимости от роли
        if role_type == "ib_admin":
            # Администратор ИБ видит все
                                         context["employees"]      =
db.query(models.Employee).limit(100).all()
            context["roles"] = db.query(models.Role).all()
            context["teams"] = db.query(models.Team).all()
                                          context["statuses"]      =
db.query(models.EmployeeStatus).limit(50).all()
                                       context["access_logs"]      =
db.query(models.AccessLog).order_by(models.AccessLog.event_date.de
sc()).limit(
                100).all()
                                 context["physical_resources"]    =
db.query(models.PhysicalResource).all()
                              context["information_resources"]    =
db.query(models.InformationResource).all()
                                        context["audit_logs"]     =
db.query(models.AccessAuditLog).order_by(

models.AccessAuditLog.operation_timestamp.desc()).limit(50).all()

        elif role_type == "team_lead":
            # Руководитель команды видит свою команду
            if current_user.team:
                team_members = db.query(models.Employee).filter(
                    models.Employee.id_team == current_user.id_team
                ).all()
                context["team_members"] = team_members
                context["team"] = current_user.team

                # Доступы команды к ресурсам
                                                   team_access    =
db.query(models.TeamResourceAccess).filter(
                              models.TeamResourceAccess.id_team  ==
current_user.id_team
                ).all()
                context["team_access"] = team_access

        elif role_type == "employee":
            # Сотрудник видит только свою информацию
            context["employee"] = current_user
            if current_user.status:
                context["status"] = current_user.status

            # История доступов сотрудника
            access_history = db.query(models.AccessLog).filter(
                                  models.AccessLog.id_employee   ==
current_user.id_employee

).order_by(models.AccessLog.event_date.desc()).limit(50).all()
            context["access_history"] = access_history

        elif role_type == "auditor":
            # Аудитор видит подозрительные доступы
            suspicious_access = db.query(models.AccessLog).filter(
                models.AccessLog.result == "отказано"

).order_by(models.AccessLog.event_date.desc()).limit(100).all()
            context["suspicious_access"] = suspicious_access

            # Аудит-логи
            audit_logs = db.query(models.AccessAuditLog).order_by(
                models.AccessAuditLog.operation_timestamp.desc()
            ).limit(100).all()
            context["audit_logs"] = audit_logs

        elif role_type == "sec_service":
            # Служба безопасности видит потенциальные проблемы
            # Потенциальные прогулы
                                        potential_absenteeism     =
db.query(models.EmployeeStatus).filter(
                and_(
                     models.EmployeeStatus.status_type == "Активная
работа",
                    models.EmployeeStatus.employee_active == 0
                )
            ).all()
                              context["potential_absenteeism"]    =
potential_absenteeism

            # Последние нарушения
            recent_violations = db.query(models.AccessLog).filter(
                models.AccessLog.result == "отказано"

).order_by(models.AccessLog.event_date.desc()).limit(50).all()
            context["recent_violations"] = recent_violations

                                                              return
templates.TemplateResponse(f"dashboard_{role_type}.html", context)
```

4.4 Реализация сложных SQL-запросов с GROUP BY

Для каждой роли реализованы статистические отчеты с возможностью сортировки и фильтрации.

Для администратора ИБ: статистика по ролям (роли — сотрудники — команды).

Для аудитора: статистика доступа по статусам (статусы — сотрудники — журнал доступа).

Для службы безопасности: статистика нарушений (журнал доступа — типы нарушений — анализ серьезности).

Для руководителя команды: статистика команды (сотрудники команды — роли — журнал доступа).

Для сотрудника: персональная статистика (сотрудник — журнал доступа — анализ успешности).

```
     # ========== СТАТИСТИКА ==========

     @app.get("/stats/{role_type}", response_class=HTMLResponse)
     async def stats_page(
            request: Request,
            role_type: str,
            sort_by: Optional[str] = Query(None),
            order: Optional[str] = Query("desc"),
            start_date: Optional[str] = Query(None),
            end_date: Optional[str] = Query(None),
            db: Session = Depends(get_db),
                               current_user:    models.Employee    =
Depends(auth.get_current_user)
     ):
        """Универсальная страница статистики для всех ролей"""
        user_role = auth.get_user_access_role(current_user)

        # Проверяем доступ
        if user_role != role_type:
               raise HTTPException(status_code=403, detail="Доступ
запрещен")

        context = {
            "request": request,
            "user": current_user,
            "user_role": user_role,
            "role_type": role_type,
            "zip": zip
        }

        # ОБРАБОТКА ВСЕХ РОЛЕЙ В ОДНОМ МЕСТЕ
        if role_type == "employee":
            # Код из employee_stats_page
            total_access = db.query(models.AccessLog).filter(
                                  models.AccessLog.id_employee   ==
current_user.id_employee
            ).count()

            successful_access = db.query(models.AccessLog).filter(
                and_(
                                  models.AccessLog.id_employee  ==
current_user.id_employee,
                    models.AccessLog.result == "успешно"
                )
            ).count()

            failed_access = db.query(models.AccessLog).filter(
                and_(
                                  models.AccessLog.id_employee  ==
current_user.id_employee,
                    models.AccessLog.result == "отказано"
                )
            ).count()

             success_rate = round((successful_access / total_access
* 100) if total_access > 0 else 0, 2)

                                      recent_activities_query     =
db.query(models.AccessLog).filter(
                                  models.AccessLog.id_employee   ==
current_user.id_employee

).order_by(models.AccessLog.event_date.desc()).limit(10).all()

            recent_activities = []
            for activity in recent_activities_query:
                recent_activities.append({
                     "date": activity.event_date.strftime("%Y-%m-%d
%H:%M"),
                                   "title":  f"{activity.action}  -
{activity.result}",
                            "description": activity.reason or "Без
описания"
                })

            # Используем шаблон stats_employee.html для employee
            context.update({
                "stats": {
                    "total_access": total_access,
                    "successful": successful_access,
                    "failed": failed_access,
                    "success_rate": success_rate
                },
                "recent_activities": recent_activities
            })

                                                             return
templates.TemplateResponse("stats_employee.html", context)

        elif role_type == "ib_admin":
                      stats  =  await  get_role_stats(sort_by  or
"employee_count", order or "desc", db, current_user)
            context["stats_data"] = stats
            context["stats_title"] = "Статистика по ролям"
                context["columns"]  =  ["role_name",  "access_role",
"employee_count", "avg_security_level",
                                                   "in_team_count",
"in_team_percentage"]
            context["column_names"] = ["Должность", "Роль доступа",
"Кол-во сотрудников",
                                        "Ср. уровень безопасности",
"В командах", "% в командах"]

                   return  templates.TemplateResponse("stats.html",
context)

        elif role_type == "auditor":
            # Исправляем join в запросе
            query = db.query(
                models.EmployeeStatus.status_type,

func.count(models.AccessLog.event_id).label("total_access"),
                       func.count(case((models.AccessLog.result  ==
"успешно", 1))).label("successful"),
                       func.count(case((models.AccessLog.result  ==
"отказано", 1))).label("failed")
            ).join(models.Employee, models.Employee.id_employee ==
models.AccessLog.id_employee) \
                .join(models.EmployeeStatus,
models.EmployeeStatus.id_employee == models.Employee.id_employee)

            if start_date:
                try:
                    start_date_obj = datetime.strptime(start_date,
"%Y-%m-%d")
                    query =
query.filter(models.AccessLog.event_date >= start_date_obj)
                except ValueError:
                    pass

            if end_date:
                try:
                    end_date_obj = datetime.strptime(end_date,
"%Y-%m-%d")
                    query =
query.filter(models.AccessLog.event_date <= end_date_obj)
                except ValueError:
                    pass

            query =
query.group_by(models.EmployeeStatus.status_type)

            # Сортировка
            if sort_by == "total_access":
                query = query.order_by(desc("total_access") if
order == "desc" else asc("total_access"))
            elif sort_by == "successful":
                query = query.order_by(desc("successful") if order
== "desc" else asc("successful"))
            elif sort_by == "failed":
                query = query.order_by(desc("failed") if order ==
"desc" else asc("failed"))
            elif sort_by == "status_type":
                query = query.order_by(
                    desc(models.EmployeeStatus.status_type) if
order == "desc" else asc(models.EmployeeStatus.status_type)
                )

            stats = query.all()

            result = []
            for stat in stats:
                total = int(stat.total_access) if stat.total_access
else 0
```

```
                            successful = int(stat.successful) if
stat.successful else 0
                failed = int(stat.failed) if stat.failed else 0

                result.append({
                    "status_type": stat.status_type,
                    "total_access": total,
                    "successful": successful,
                    "failed": failed,
                       "success_rate": round((successful / total *
100) if total > 0 else 0, 2),
                    "failure_rate": round((failed / total * 100) if
total > 0 else 0, 2)
                })

            # Сортировка результатов если запрос уже отсортирован
            if sort_by in ["total_access", "successful", "failed",
"success_rate", "failure_rate"]:
                reverse = order == "desc"
                if sort_by == "total_access":
                       result.sort(key=lambda x: x["total_access"],
reverse=reverse)
                elif sort_by == "successful":
                        result.sort(key=lambda x: x["successful"],
reverse=reverse)
                elif sort_by == "failed":
                            result.sort(key=lambda x: x["failed"],
reverse=reverse)
                elif sort_by == "success_rate":
                       result.sort(key=lambda x: x["success_rate"],
reverse=reverse)
                elif sort_by == "failure_rate":
                       result.sort(key=lambda x: x["failure_rate"],
reverse=reverse)

            context["stats_data"] = result
                 context["stats_title"] = "Статистика доступа по
статусам"
              context["columns"] = ["status_type", "total_access",
"successful", "failed",
                                  "success_rate", "failure_rate"]
             context["column_names"] = ["Статус", "Всего попыток",
"Успешных", "Отказано",
                                       "% успеха", "% отказа"]

                   return templates.TemplateResponse("stats.html",
context)

        elif role_type == "team_lead":
            # Получаем статистику команды
            if current_user.id_team and current_user.team:
                # Статистика по членам команды
                team_stats = db.query(
                    models.Employee.id_employee,
                    models.Employee.full_name,
                    models.Role.role_name,

func.count(models.AccessLog.event_id).label("total_access"),
                        func.count(case((models.AccessLog.result ==
"успешно", 1))).label("successful")
                    ).join(models.Role, models.Employee.id_role ==
models.Role.id_role) \
                                            .join(models.AccessLog,
models.AccessLog.id_employee == models.Employee.id_employee,
isouter=True) \
                                .filter(models.Employee.id_team ==
current_user.id_team) \
                             .group_by(models.Employee.id_employee,
models.Employee.full_name, models.Role.role_name) \
                    .all()

                stats_data = []
                for stat in team_stats:
                                total = int(stat.total_access) if
stat.total_access else 0
                             successful = int(stat.successful) if
stat.successful else 0
                    failed = total - successful
                    success_rate = round((successful / total * 100)
if total > 0 else 0, 2)
                     failure_rate = round((failed / total * 100) if
total > 0 else 0, 2)

                    stats_data.append({
                        "team_member": stat.full_name,
                        "role": stat.role_name,
                        "total_access": total,
                        "successful": successful,
                        "failed": failed,
                        "success_rate": success_rate,
                        "failure_rate": failure_rate
                    })
                # Сортировка
                          if  sort_by  in  ["team_member",  "role",
"total_access", "successful", "failed", "success_rate",
                               "failure_rate"]:
                    reverse = order == "desc"
                    if sort_by == "team_member":
                                     stats_data.sort(key=lambda  x:
x["team_member"], reverse=reverse)
                    elif sort_by == "role":
                           stats_data.sort(key=lambda x: x["role"],
reverse=reverse)
                    elif sort_by == "total_access":
                                     stats_data.sort(key=lambda  x:
x["total_access"], reverse=reverse)
                    elif sort_by == "successful":
                                     stats_data.sort(key=lambda  x:
x["successful"], reverse=reverse)
                    elif sort_by == "failed":
                         stats_data.sort(key=lambda x: x["failed"],
reverse=reverse)
                    elif sort_by == "success_rate":
                                     stats_data.sort(key=lambda  x:
x["success_rate"], reverse=reverse)
                    elif sort_by == "failure_rate":
                                     stats_data.sort(key=lambda  x:
x["failure_rate"], reverse=reverse)

                context["stats_data"] = stats_data
                     context["stats_title"]  =  f"Статистика  команды
{current_user.team.team_name}"
                      context["columns"]  =  ["team_member",  "role",
"total_access", "successful", "failed", "success_rate",
                                      "failure_rate"]
                          context["column_names"]  =  ["Сотрудник",
"Должность", "Всего доступов", "Успешных", "Отказано", "% успеха",
                                           "% отказа"]
            else:
                context["stats_data"] = []
                context["stats_title"] = "Статистика команды"
                      context["columns"]  =  ["team_member",  "role",
"total_access", "success_rate"]
                          context["column_names"]  =  ["Сотрудник",
"Должность", "Всего доступов", "% успеха"]

                   return  templates.TemplateResponse("stats.html",
context)

        elif role_type == "sec_service":
            # Статистика нарушений безопасности
            violation_stats = db.query(
                models.AccessLog.action,
func.count(models.AccessLog.event_id).label("count"),
                # Используем выражение для расчета severity как
float
                (func.avg(
                    case((models.AccessLog.result == "успешно", 0),
else_=1)
                ) * 100).label("severity_percent")  # Умножаем
сразу на 100 чтобы получить проценты
            ).filter(models.AccessLog.result == "отказано") \
                .group_by(models.AccessLog.action) \
                .order_by(desc("count")) \
                .all()

            stats_data = []
            for stat in violation_stats:
                count_val = int(stat.count) if stat.count else 0

                # Конвертируем Decimal во float безопасно
                if stat.severity_percent is not None:
                    try:
                        severity_val = float(stat.severity_percent)
                    except (TypeError, ValueError):
                        severity_val = 0.0
                else:
                    severity_val = 0.0

                stats_data.append({
                    "violation_type": stat.action,
                    "count": count_val,
                    "severity": round(severity_val, 2)  # Уже в
процентах
                })

            # Сортировка
            if sort_by in ["violation_type", "count", "severity"]:
                reverse = order == "desc"
                if sort_by == "violation_type":
                    stats_data.sort(key=lambda x:
x["violation_type"], reverse=reverse)
```

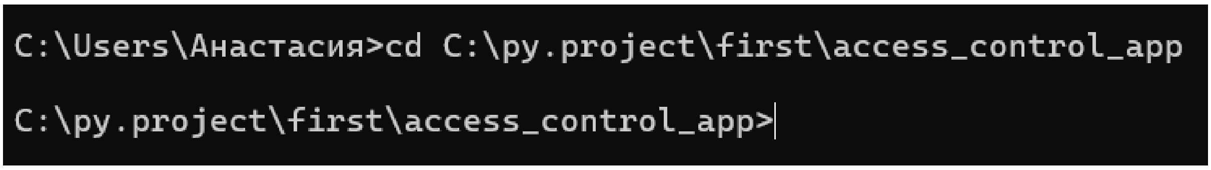
```
                elif sort_by == "count":
                         stats_data.sort(key=lambda x: x["count"],
reverse=reverse)
                elif sort_by == "severity":
                       stats_data.sort(key=lambda x: x["severity"],
reverse=reverse)

            context["stats_data"] = stats_data
                  context["stats_title"] = "Статистика нарушений
безопасности"
                 context["columns"] = ["violation_type", "count",
"severity"]
                    context["column_names"] = ["Тип нарушения",
"Количество", "Серьезность (%)"]

                   return templates.TemplateResponse("stats.html",
context)

        else:
            # Для неизвестных ролей
             raise HTTPException(status_code=404, detail="Страница
статистики не найдена")
```

4.5 Запуск приложение и использование интерфейса

Запуск приложения осуществляется следующим образом (запуск будет осуществляться в операционной системе Windows, а демонстрация приложения – в поисковой системе Chrome).

- Открываем консоль и переходим в папку, в которой лежит проект. Результат на рисунке 151.

cd C:\путь_до_папки_с_приложением\access_control_app

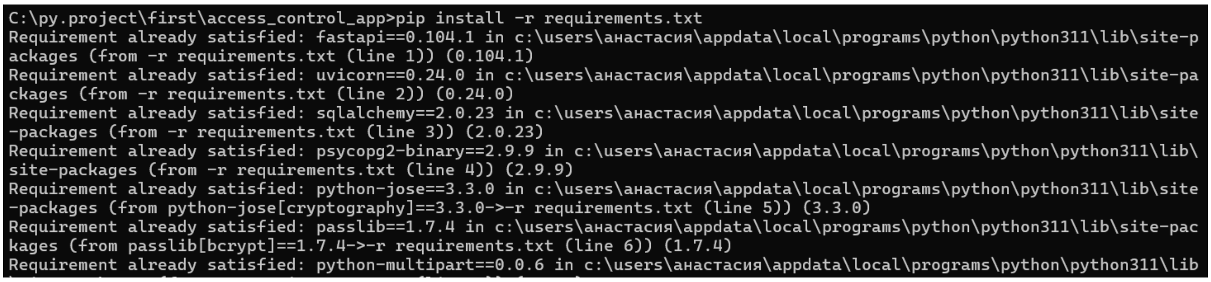


```
C:\Users\Анастасия>cd C:\py.project\first\access_control_app

C:\py.project\first\access_control_app>
```

Рисунок 151 – Переход в папку с приложением

- Устанавливаем зависимости. Результат установки на рисунке 152.

pip install -r requirements.txt

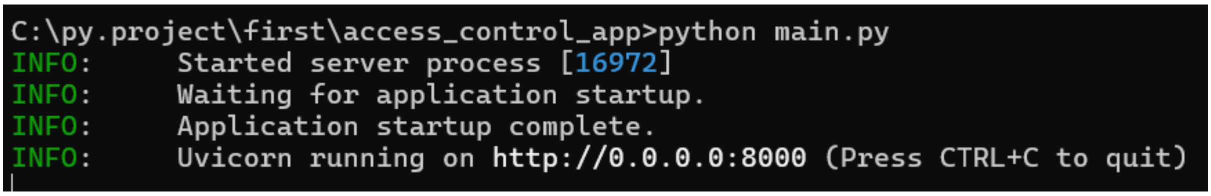


```
C:\py.project\first\access_control_app>pip install -r requirements.txt
Requirement already satisfied: fastapi==0.104.1 in c:\users\анастасия\appdata\local\programs\python\python311\lib\site-p
ackages (from -r requirements.txt (line 1)) (0.104.1)
Requirement already satisfied: uvicorn==0.24.0 in c:\users\анастасия\appdata\local\programs\python\python311\lib\site-pa
ckages (from -r requirements.txt (line 2)) (0.24.0)
Requirement already satisfied: sqlalchemy==2.0.23 in c:\users\анастасия\appdata\local\programs\python\python311\lib\site
-packages (from -r requirements.txt (line 3)) (2.0.23)
Requirement already satisfied: psycopg2-binary==2.9.9 in c:\users\анастасия\appdata\local\programs\python\python311\lib\
site-packages (from -r requirements.txt (line 4)) (2.9.9)
Requirement already satisfied: python-jose==3.3.0 in c:\users\анастасия\appdata\local\programs\python\python311\lib\site
-packages (from python-jose[cryptography]==3.3.0->-r requirements.txt (line 5)) (3.3.0)
Requirement already satisfied: passlib==1.7.4 in c:\users\анастасия\appdata\local\programs\python\python311\lib\site-pac
kages (from passlib[bcrypt]==1.7.4->-r requirements.txt (line 6)) (1.7.4)
Requirement already satisfied: python-multipart==0.0.6 in c:\users\анастасия\appdata\local\programs\python\python311\lib
```

Рисунок 152 – Установка зависимостей

- Запускаем приложение. Результат запуска на рисунке 153.

python main.py



```
C:\py.project\first\access_control_app>python main.py
INFO:     Started server process [16972]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Рисунок 153 – Приложение запущено

- Теперь в браузере переходим по адресу localhost:8000 и видим окно авторизации для пользователей на рисунке 154.

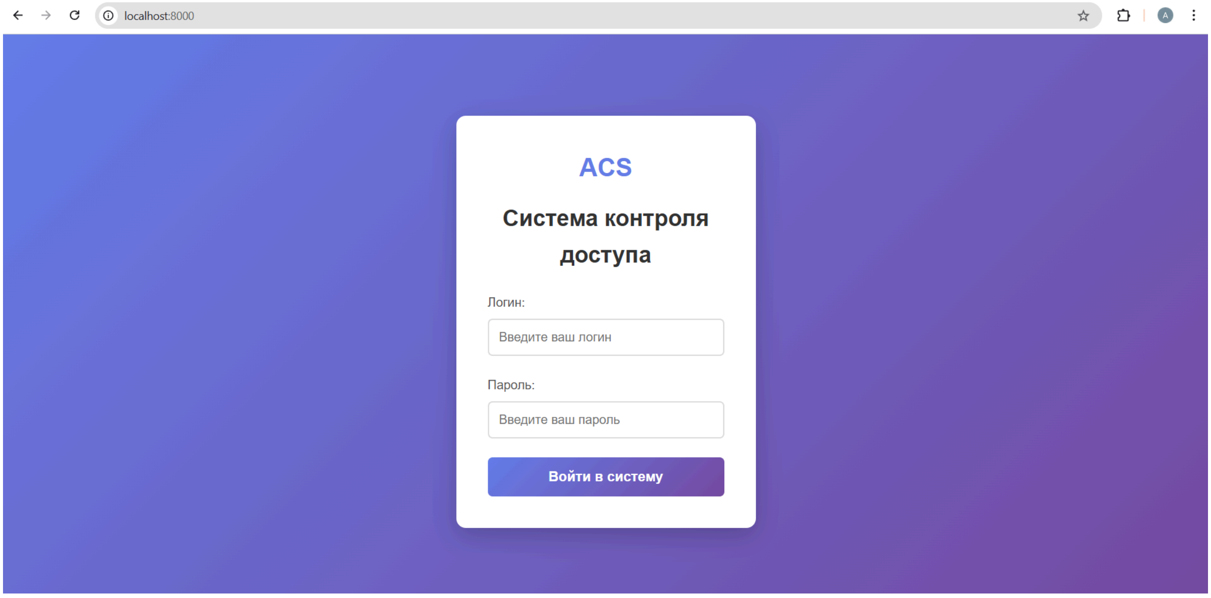
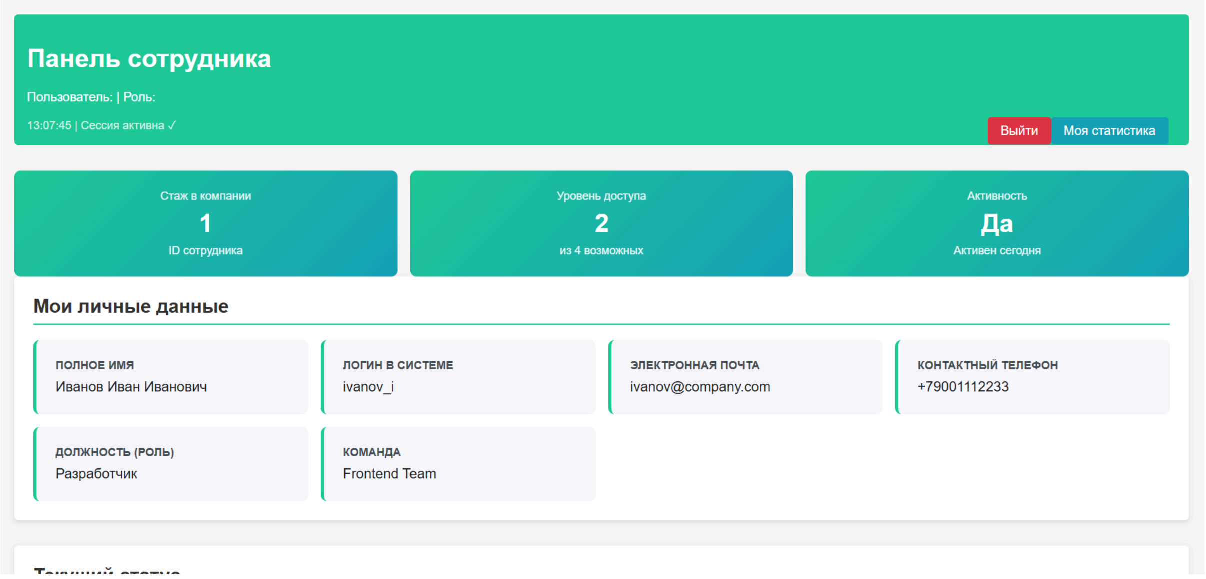

localhost:8000
ACS
Система контроля доступа
Логин:
Введите ваш логин
Пароль:
Введите ваш пароль
Войти в систему

Рисунок 154 – Окно авторизации сотрудников

— Дальше можно войти под любым пользователем из таблицы employees созданной базы данных. Посмотрим, как выглядят интерфейсы для каждой из ролей на рисунках с 155 до 164.


Панель сотрудника
Выйти
Моя статистика
Мои личные данные

Пользователь: | Роль:

13:07:45 | Сессия активна ✓

Стаж в компании
1
ID сотрудника

Уровень доступа
2
из 4 возможных

Активность
Да
Активен сегодня

ПОЛНОЕ ИМЯ: Иванов Иван Иванович

ЛОГИН В СИСТЕМЕ: ivanov_i

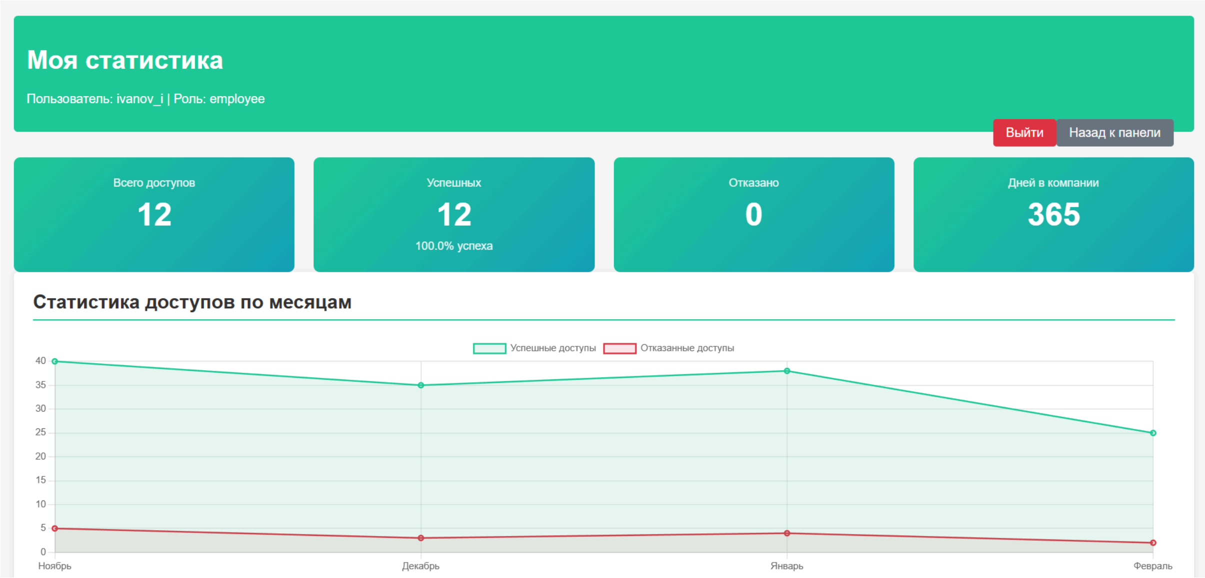
ЭЛЕКТРОННАЯ ПОЧТА: ivanov@company.com

КОНТАКТНЫЙ ТЕЛЕФОН: +79001112233

ДОЛЖНОСТЬ (РОЛЬ): Разработчик

КОМАНДА: Frontend Team

Рисунок 155 – Дашборд сотрудника Иванова Ивана Ивановича


Выйти
Назад к панели
Успешные доступы
Отказанные доступы
40
35
30
25
20
15
10
5
0
Ноябрь
Декабрь
Январь
Февраль

Моя статистика

Пользователь: ivanov_i | Роль: employee

Всего доступов 12

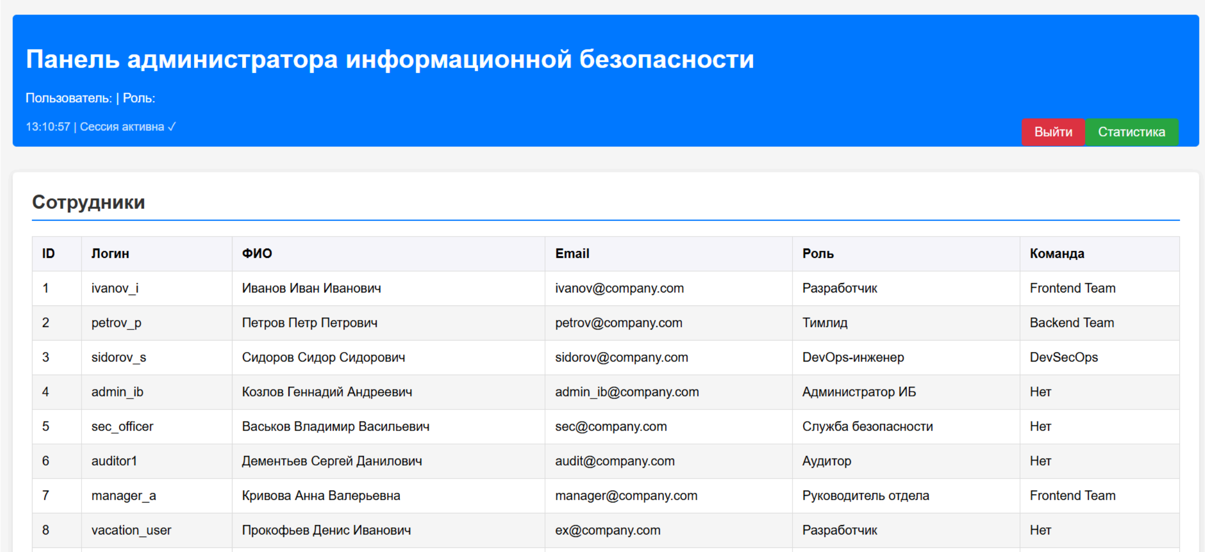
Успешных 12 100.0% успеха

Отказано 0

Дней в компании 365

Статистика доступов по месяцам

Рисунок 156 – Статистика сотрудника Иванова Ивана Ивановича


Выйти
Статистика

Панель администратора информационной безопасности

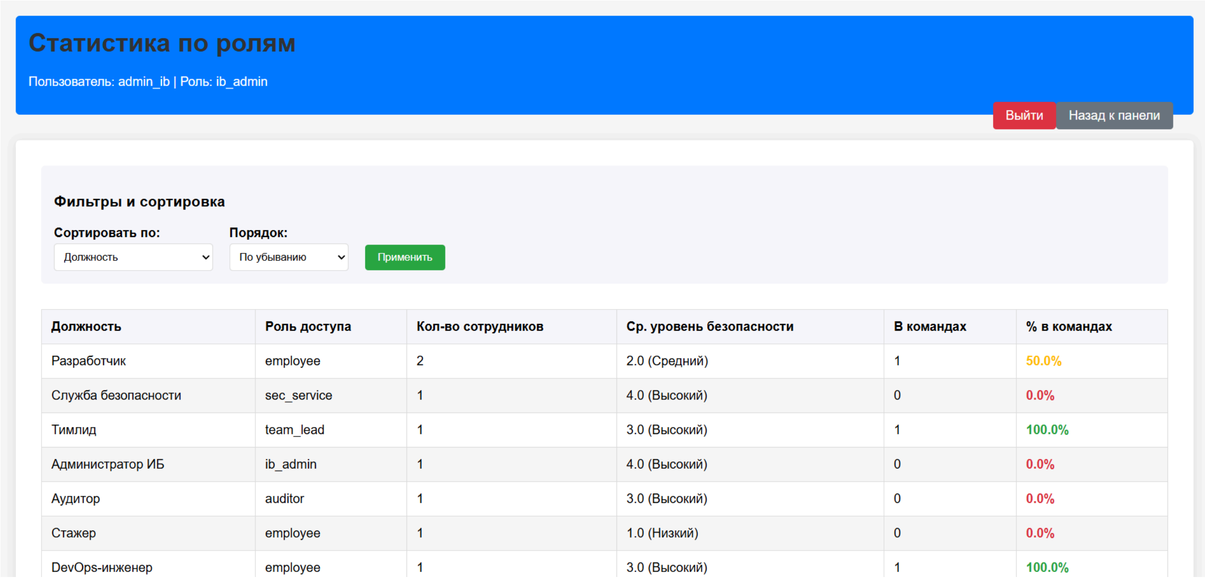
Пользователь: | Роль:

13:10:57 | Сессия активна ✓

Сотрудники

ID	Логин	ФИО	Email	Роль	Команда
1	ivanov_i	Иванов Иван Иванович	ivanov@company.com	Разработчик	Frontend Team
2	petrov_p	Петров Петр Петрович	petrov@company.com	Тимлид	Backend Team
3	sidorov_s	Сидоров Сидор Сидорович	sidorov@company.com	DevOps-инженер	DevSecOps
4	admin_ib	Козлов Геннадий Андреевич	admin_ib@company.com	Администратор ИБ	Нет
5	sec_officer	Васьков Владимир Васильевич	sec@company.com	Служба безопасности	Нет
6	auditor1	Дементьев Сергей Данилович	audit@company.com	Аудитор	Нет
7	manager_a	Кривова Анна Валерьевна	manager@company.com	Руководитель отдела	Frontend Team
8	vacation_user	Прокофьев Денис Иванович	ex@company.com	Разработчик	Нет

Рисунок 157 – Дашборд администратора ИБ Козлова Геннадия Андреевича



Статистика по ролям

Пользователь: admin_ib | Роль: ib_admin

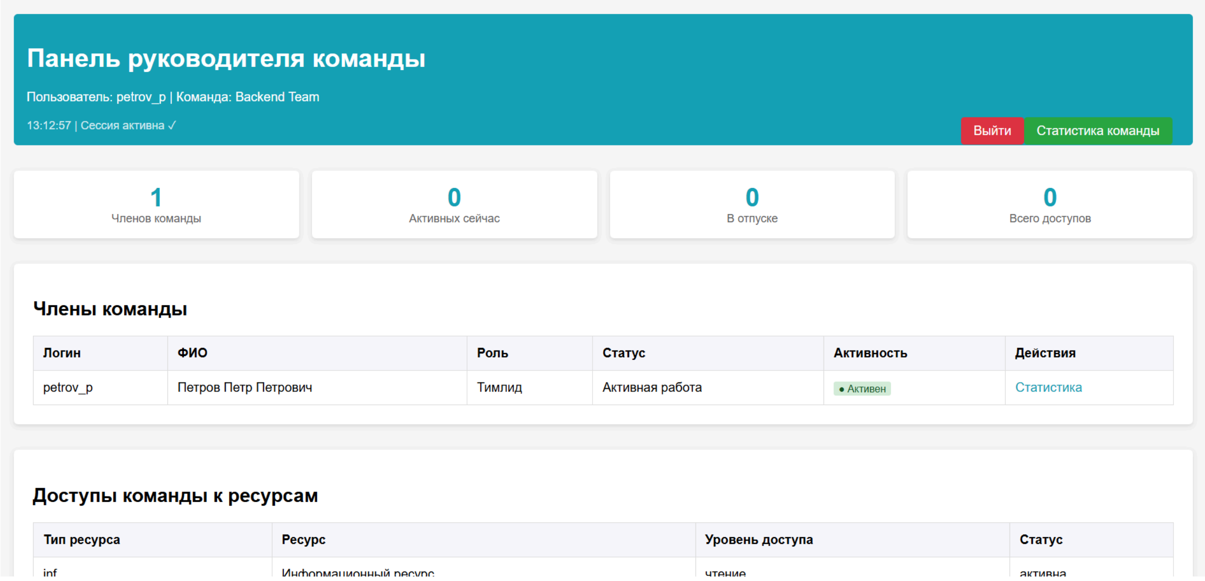
Фильтры и сортировка

Сортировать по: Должность

Порядок: По убыванию

Должность	Роль доступа	Кол-во сотрудников	Ср. уровень безопасности	В командах	% в командах
Разработчик	employee	2	2.0 (Средний)	1	50.0%
Служба безопасности	sec_service	1	4.0 (Высокий)	0	0.0%
Тимлид	team_lead	1	3.0 (Высокий)	1	100.0%
Администратор ИБ	ib_admin	1	4.0 (Высокий)	0	0.0%
Аудитор	auditor	1	3.0 (Высокий)	0	0.0%
Стажер	employee	1	1.0 (Низкий)	0	0.0%
DevOps-инженер	employee	1	3.0 (Высокий)	1	100.0%

Рисунок 158 – Статистика администратора ИБ Козлова Геннадия Андреевича



Панель руководителя команды

Пользователь: petrov_p | Команда: Backend Team

13:12:57 | Сессия активна ✓

1 Членов команды

0 Активных сейчас

0 В отпуске

0 Всего доступов

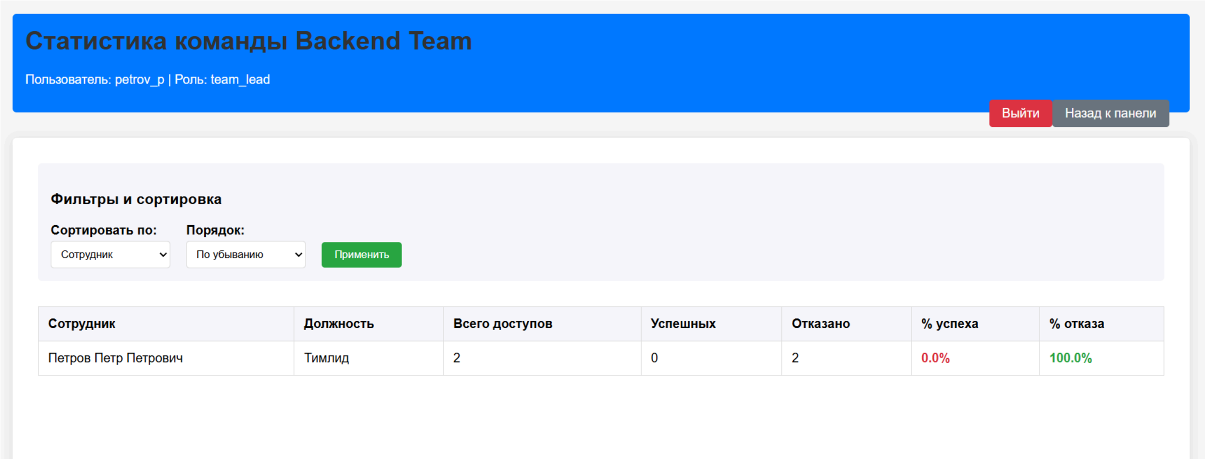
Члены команды

Логин	ФИО	Роль	Статус	Активность	Действия
petrov_p	Петров Петр Петрович	Тимлид	Активная работа	● Активен	Статистика

Доступы команды к ресурсам

Тип ресурса	Ресурс	Уровень доступа	Статус
inf	Информационный ресурс	чтение	активна

Рисунок 159 – Дашборд руководителя команды Петрова Петра Петровича



Статистика команды Backend Team

Пользователь: petrov_p | Роль: team_lead

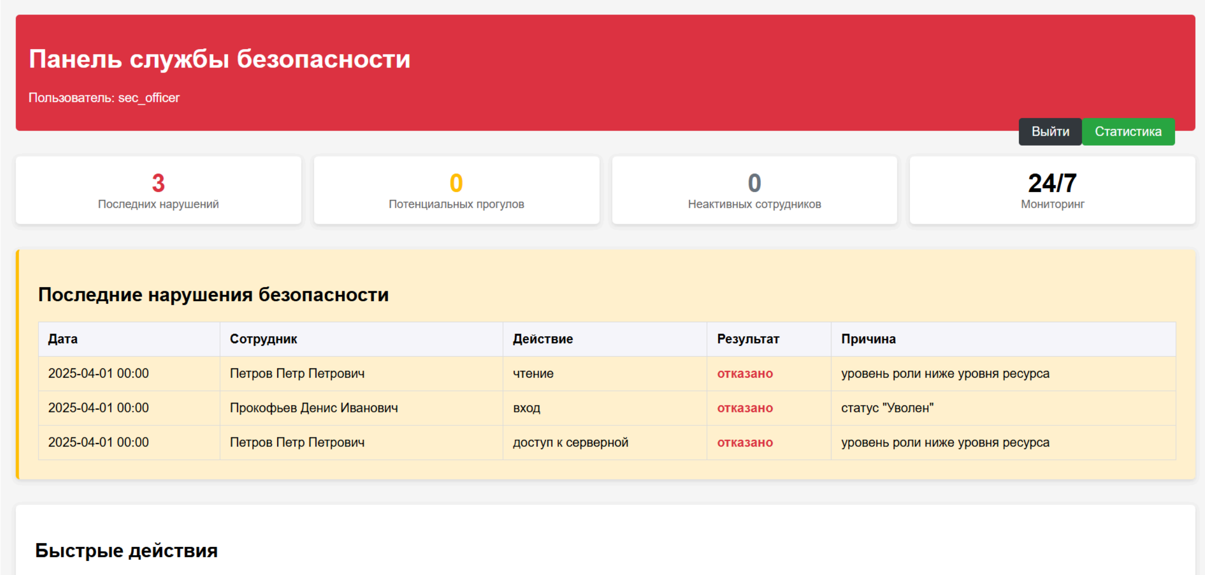
Фильтры и сортировка

Сортировать по: Сотрудник

Порядок: По убыванию

Сотрудник	Должность	Всего доступов	Успешных	Отказано	% успеха	% отказа
Петров Петр Петрович	Тимлид	2	0	2	0.0%	100.0%

Рисунок 160 – Статистика руководителя команды Петрова Петра Петровича



Панель службы безопасности

Пользователь: sec_officer

3 Последних нарушений

0 Потенциальных прогулов

0 Неактивных сотрудников

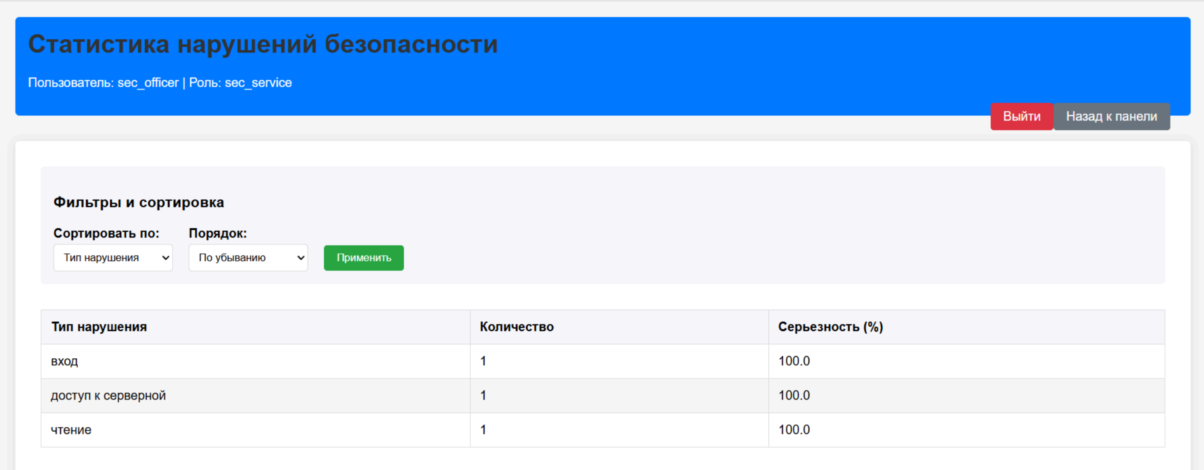
24/7 Мониторинг

Последние нарушения безопасности

Дата	Сотрудник	Действие	Результат	Причина
2025-04-01 00:00	Петров Петр Петрович	чтение	отказано	уровень роли ниже уровня ресурса
2025-04-01 00:00	Прокофьев Денис Иванович	вход	отказано	статус "Уволен"
2025-04-01 00:00	Петров Петр Петрович	доступ к серверной	отказано	уровень роли ниже уровня ресурса

Быстрые действия

Рисунок 161 – Дашборд представителя службы безопасности Васькова Владимира Васильевича


Выйти
Назад к панели
Применить

Статистика нарушений безопасности

Пользователь: sec_officer | Роль: sec_service

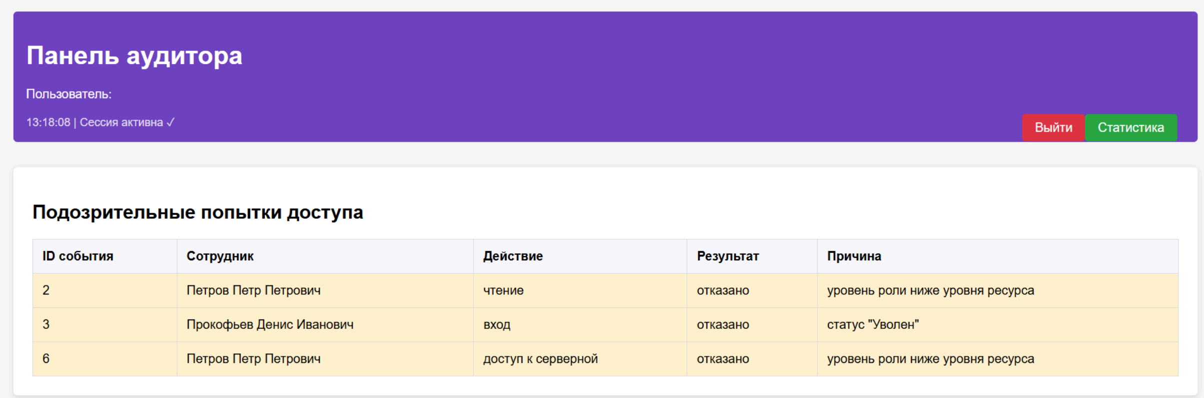
Фильтры и сортировка

Сортировать по: Тип нарушения

Порядок: По убыванию

Тип нарушения	Количество	Серьезность (%)
вход	1	100.0
доступ к серверной	1	100.0
чтение	1	100.0

Рисунок 162 – Статистика представителя службы безопасности Васькова Владимира Васильевича


Выйти
Статистика

Панель аудитора

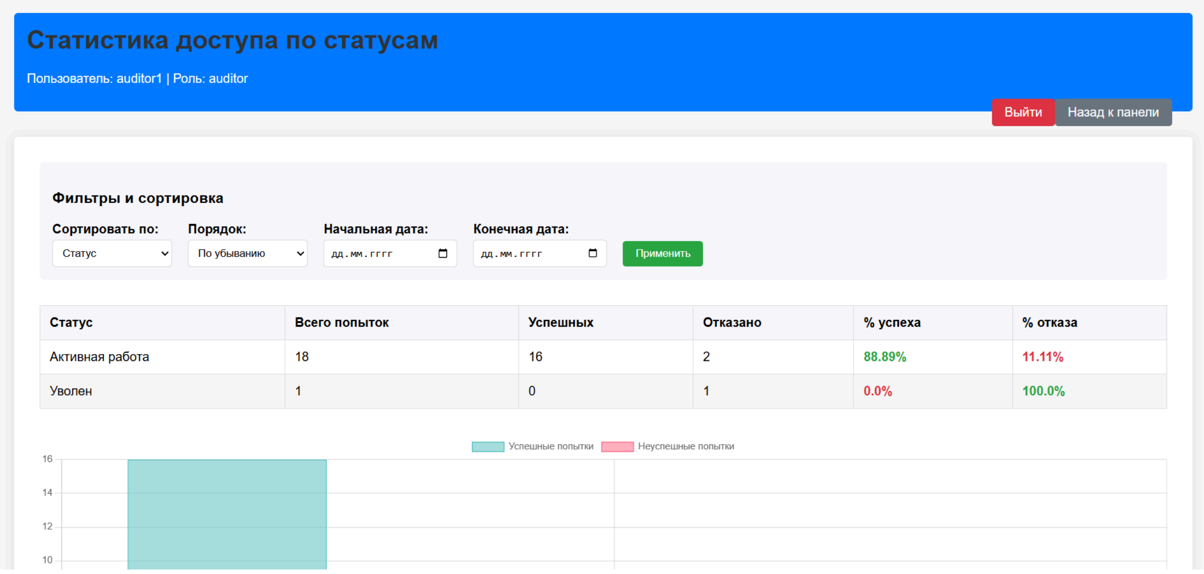
Пользователь:

13:18:08 | Сессия активна ✓

Подозрительные попытки доступа

ID события	Сотрудник	Действие	Результат	Причина
2	Петров Петр Петрович	чтение	отказано	уровень роли ниже уровня ресурса
3	Прокофьев Денис Иванович	вход	отказано	статус "Уволен"
6	Петров Петр Петрович	доступ к серверной	отказано	уровень роли ниже уровня ресурса

Рисунок 163 – Дашборд аудитора Дементьева Сергея Даниловича


Выйти
Назад к панели
Применить
Успешные попытки
Неуспешные попытки
16
14
12
10

Статистика доступа по статусам

Пользователь: auditor1 | Роль: auditor

Фильтры и сортировка

Сортировать по: Статус

Порядок: По убыванию

Начальная дата: дд.мм.гггг

Конечная дата: дд.мм.гггг

Статус	Всего попыток	Успешных	Отказано	% успеха	% отказа
Активная работа	18	16	2	88.89%	11.11%
Уволен	1	0	1	0.0%	100.0%

Рисунок 164 – Статистика аудитора Дементьева Сергея Даниловича

4.6 Вывод

В ходе четвертого этапа работы успешно реализован веб-интерфейс для системы управления доступом с поддержкой многопользовательского режима. Приложение обеспечивает:

- безопасную аутентификацию на основе JWT и ролевую модель доступа;
- персонализированные интерфейсы для 5 категорий пользователей;
- сложную аналитику с использованием GROUP BY, сортировкой и фильтрацией;
- полное соответствие ранее разработанной структуре БД и политикам безопасности;
- масштабируемую архитектуру на основе FastAPI и SQLAlchemy;

Реализация демонстрирует практические навыки работы с современным стеком веб-разработки и интеграции механизмов информационной безопасности на уровне приложения.

5 РЕЗЕРВИРОВАНИЕ БД И ВОССТАНОВЛЕНИЕ ПО КОНТРОЛЬНЫМ ТОЧКАМ

Осуществим резервное копирование базы данных access_control_db, созданную в PostgreSQL, через *pgAdmin 4*.

5.1 Создание резервной копии БД

Для создания резервной копии БД воспользуемся функцией графического интерфейса pgAdmin, которая использует стандартную утилиту PostgreSQL **pg_dump**. Находим в дереве объектов БД, которую необходимо зарезервировать, после нажимаем на нее правой кнопкой мыши и в контекстном меню выбираем **Backup**. Демонстрация описанного представлена на рисунке 165.

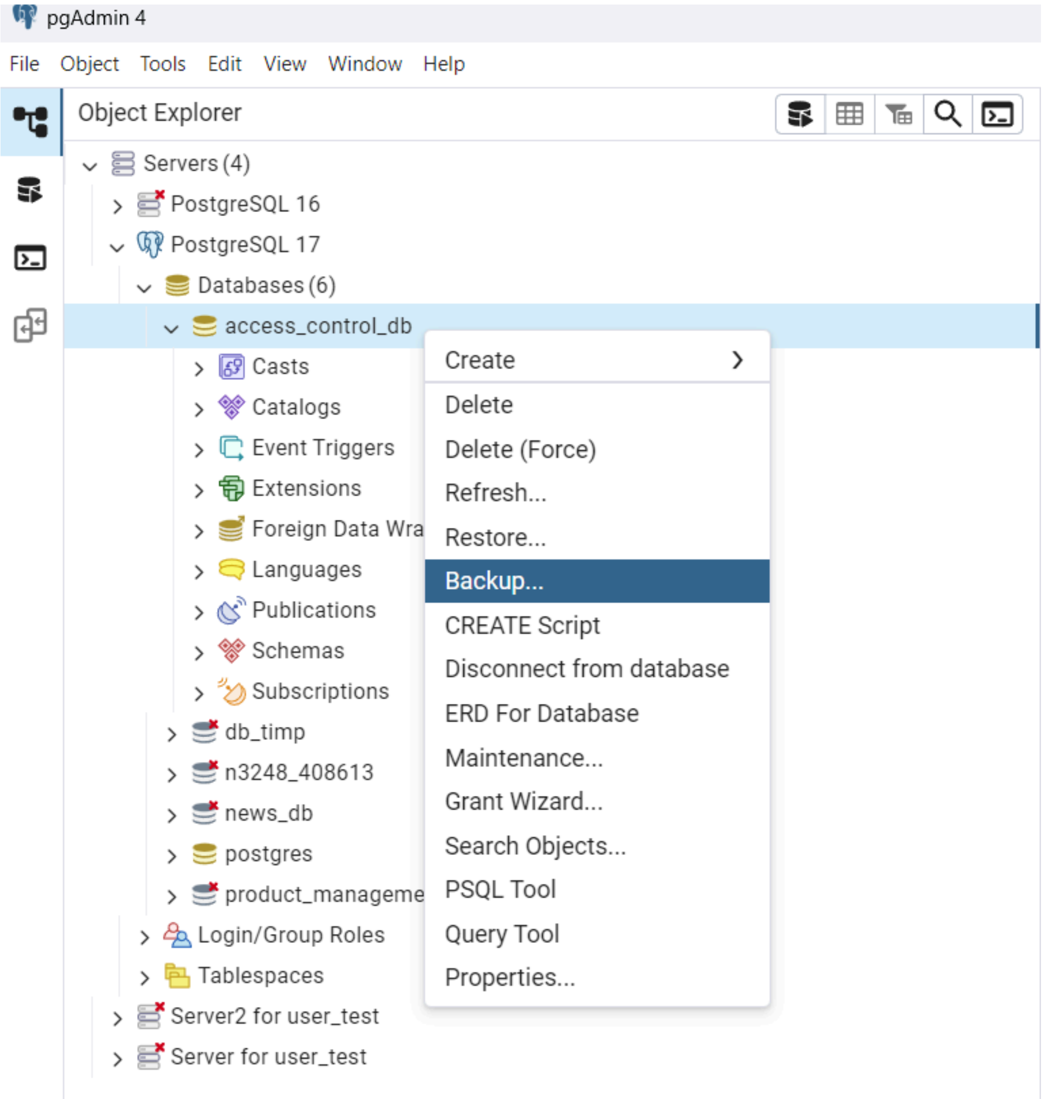
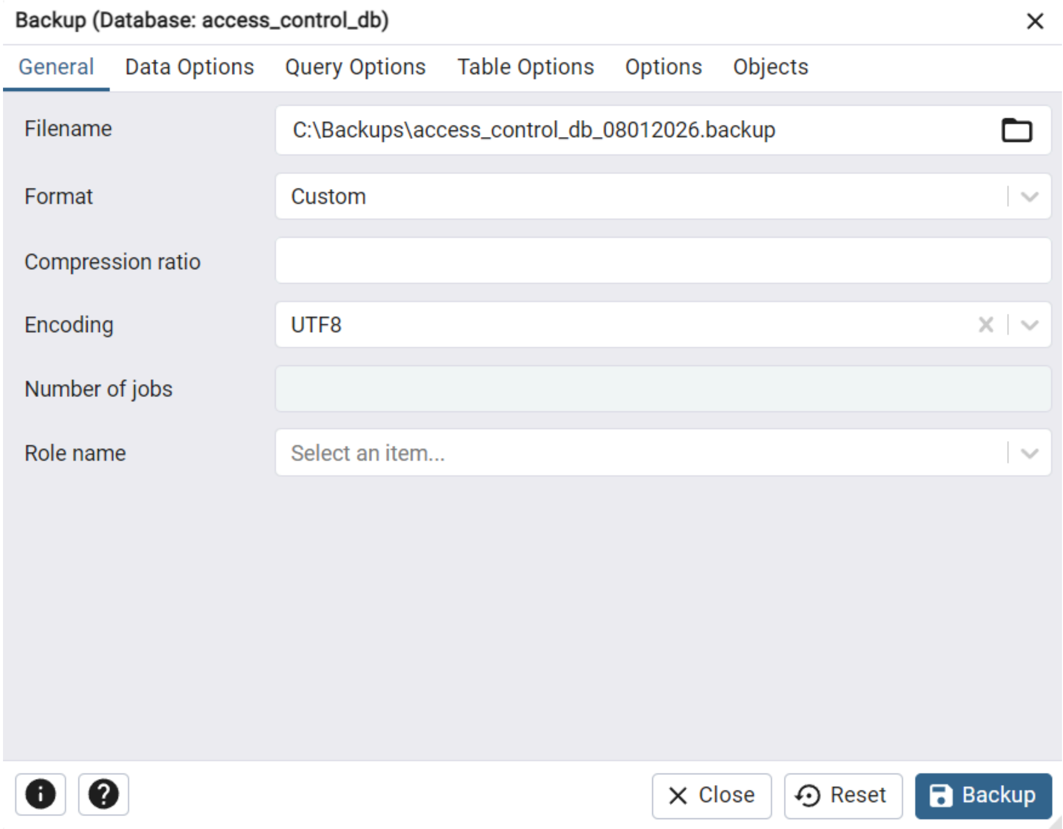

pgAdmin 4
File
Object
Tools
Edit
View
Window
Help
Object Explorer
Servers (4)
PostgreSQL 16
PostgreSQL 17
Databases (6)
access_control_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wra
Languages
Publications
Schemas
Subscriptions
db_timp
n3248_408613
news_db
postgres
product_manageme
Login/Group Roles
Tablespaces
Server2 for user_test
Server for user_test
Create
Delete
Delete (Force)
Refresh...
Restore...
Backup...
CREATE Script
Disconnect from database
ERD For Database
Maintenance...
Grant Wizard...
Search Objects...
PSQL Tool
Query Tool
Properties...

Рисунок 165 – Кнопка создания резервной копии

В открывшемся диалоговом окне (представлено на рисунках 166 и 167):

- Filename: указываем путь и имя для файла резервной копии (я указала C:\Backups\access_control_db_08012026.backup);
- Format: выбираем Custom, так как это тот формат, который создаёт файлы с расширением .backup и подходит для восстановления через pgAdmin или pg_restore;
- Encoding: UTF8;
- В разделе Data Options включим в секции Sections Pre-data, Data и Post-data, чтобы сохранить все.

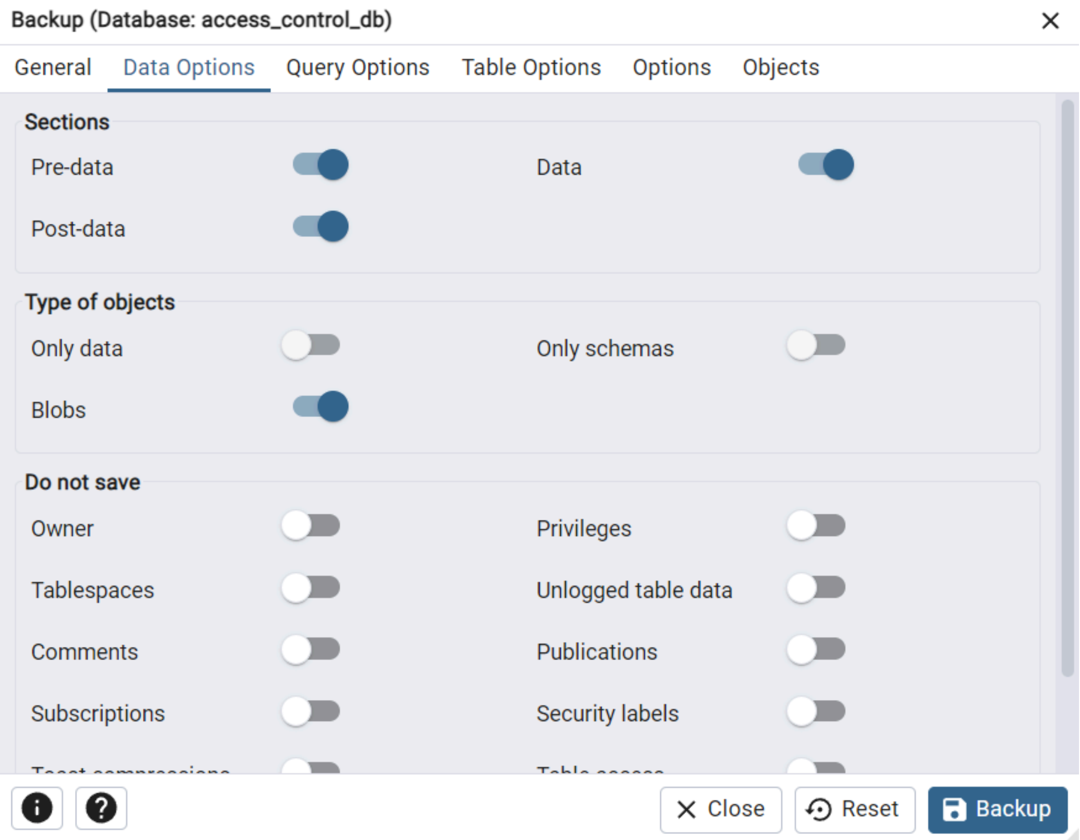


Backup (Database: access_control_db)

Filename: C:\Backups\access_control_db_08012026.backup
Format: Custom
Compression ratio:
Encoding: UTF8
Number of jobs:
Role name: Select an item...

General
Data Options
Query Options
Table Options
Options
Objects
Close
Reset
Backup

Рисунок 166 – Настройка резервной копии (1)


Backup (Database: access_control_db)
General
Data Options
Query Options
Table Options
Options
Objects
Close
Reset
Backup

Sections

Pre-data ☑
Data ☑
Post-data ☑

Type of objects

Only data ☐
Only schemas ☐
Blobs ☑

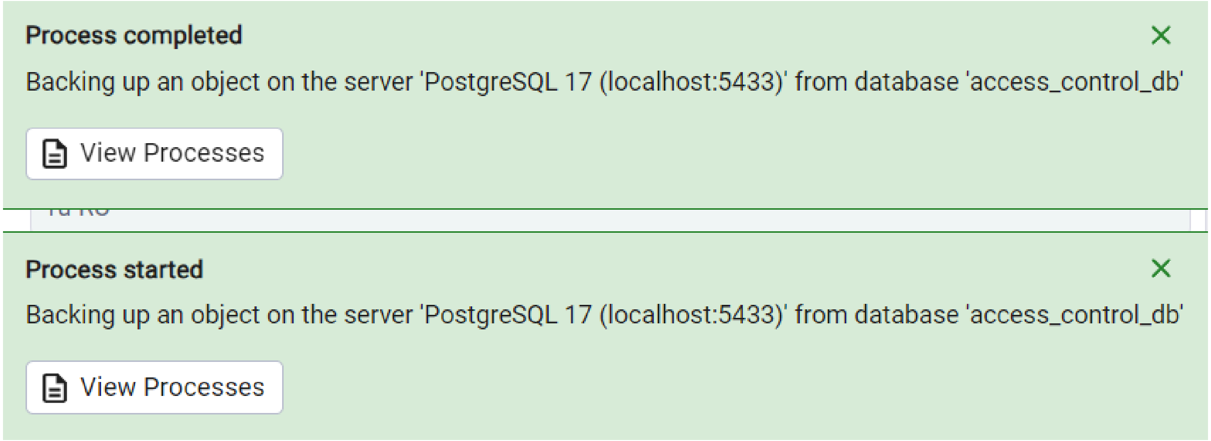
Do not save

Owner ☐
Privileges ☐
Tablespaces ☐
Unlogged table data ☐
Comments ☐
Publications ☐
Subscriptions ☐
Security labels ☐

Рисунок 167 – Настройка резервной копии (2)

После настройки нужно нажать кнопку Backup.

Резервная копия успешно создана. Сообщения представлены на рисунке 168.


View Processes
View Processes

Process completed

Backing up an object on the server 'PostgreSQL 17 (localhost:5433)' from database 'access_control_db'

Process started

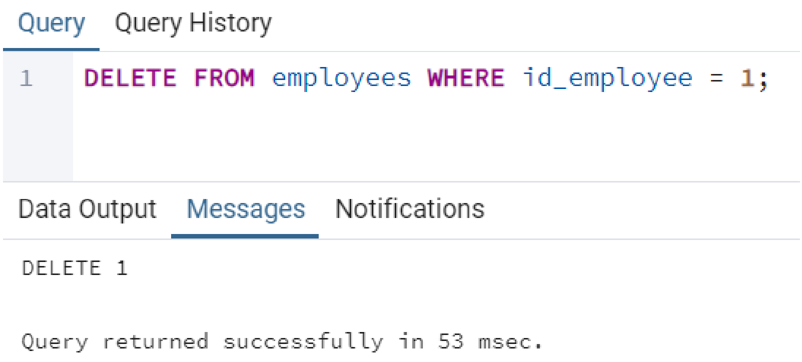
Backing up an object on the server 'PostgreSQL 17 (localhost:5433)' from database 'access_control_db'

Рисунок 168 – Сообщение об успехе

5.2 Внесение изменений в таблицы

Внесем изменения в талицы:

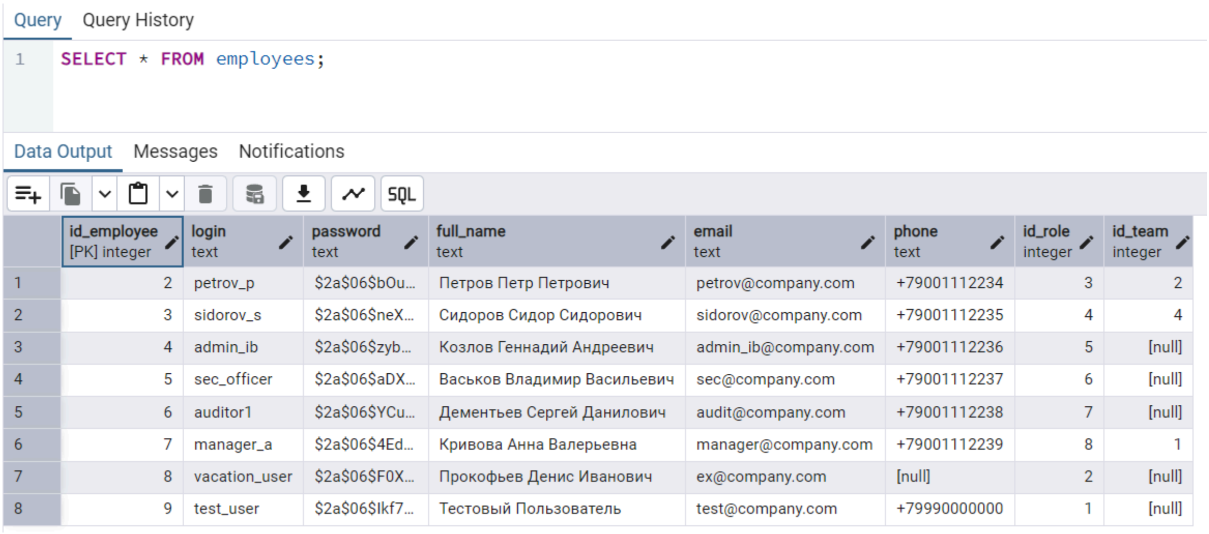
- удалим сотрудника с идентификатором 1 (рисунки 169 и 170): DELETE FROM employees WHERE id_employee = 1;
- создадим новую команду (рисунки 171 и 172): INSERT INTO teams (id_team, team_name, team_active) VALUES (7, 'Temp Backup Team', 'активна');


Query
Query History
Data Output
Messages
Notifications

```
DELETE FROM employees WHERE id_employee = 1;
```

```
DELETE 1

Query returned successfully in 53 msec.
```

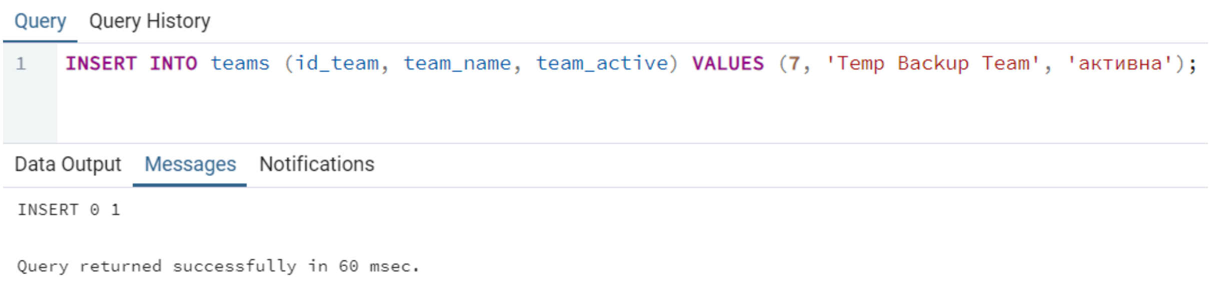
Рисунок 169 – Запрос на удаление сотрудника


Query
Query History
Data Output
Messages
Notifications
SQL

```
SELECT * FROM employees;
```

	id_employee [PK] integer	login text	password text	full_name text	email text	phone text	id_role integer	id_team integer
1	2	petrov_p	$2a$06$bOu...	Петров Петр Петрович	petrov@company.com	+79001112234	3	2
2	3	sidorov_s	$2a$06$neX...	Сидоров Сидор Сидорович	sidorov@company.com	+79001112235	4	4
3	4	admin_ib	$2a$06$zyb...	Козлов Геннадий Андреевич	admin_ib@company.com	+79001112236	5	[null]
4	5	sec_officer	$2a$06$aDX...	Васьков Владимир Васильевич	sec@company.com	+79001112237	6	[null]
5	6	auditor1	$2a$06$YCu...	Дементьев Сергей Данилович	audit@company.com	+79001112238	7	[null]
6	7	manager_a	$2a$06$4Ed...	Кривова Анна Валерьевна	manager@company.com	+79001112239	8	1
7	8	vacation_user	$2a$06$F0X...	Прокофьев Денис Иванович	ex@company.com	[null]	2	[null]
8	9	test_user	$2a$06$Ikf7...	Тестовый Пользователь	test@company.com	+79990000000	1	[null]

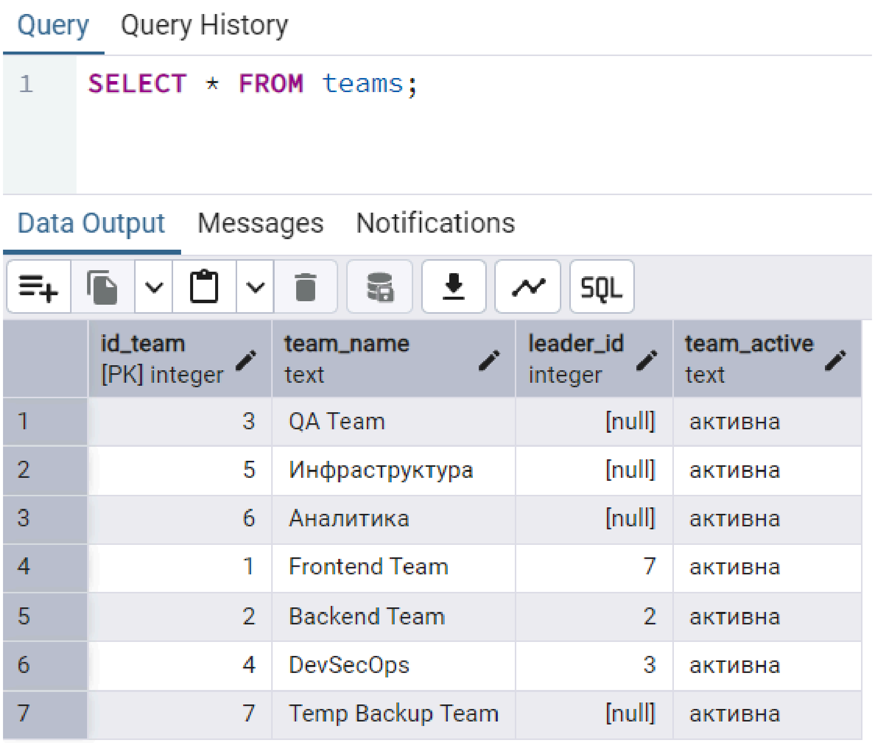
Рисунок 170 – Состояние таблицы employees


Query
Query History
Data Output
Messages
Notifications

```
INSERT INTO teams (id_team, team_name, team_active) VALUES (7, 'Temp Backup Team', 'активна');
```

```
INSERT 0 1

Query returned successfully in 60 msec.
```

Рисунок 171 – Запрос на создание команды


Query
Query History
Data Output
Messages
Notifications
SQL

```
SELECT * FROM teams;
```

	id_team [PK] integer	team_name text	leader_id integer	team_active text
1	3	QA Team	[null]	активна
2	5	Инфраструктура	[null]	активна
3	6	Аналитика	[null]	активна
4	1	Frontend Team	7	активна
5	2	Backend Team	2	активна
6	4	DevSecOps	3	активна
7	7	Temp Backup Team	[null]	активна

Рисунок 172 – Состояние таблицы teams

Изменения таблиц базы данных успешно произведены.

5.3 Процесс отката к контрольной точке

Для безопасного восстановления создадим пустую базу данных *access_control_db_2*.

Для восстановления БД из резервной копии воспользуемся функцией графического интерфейса pgAdmin, которая использует стандартную утилиту PostgreSQL **pg_restore**. Находим в дереве объектов новую БД, нажимаем на нее правой кнопкой мыши и в контекстном меню выбираем **Restore**. Демонстрация описанного представлена на рисунке 173. На рисунке 174 представлено открывшееся диалоговое окно, в котором нужно выбрать восстанавливаемую БД. После настройки нужно нажать кнопку Restore.

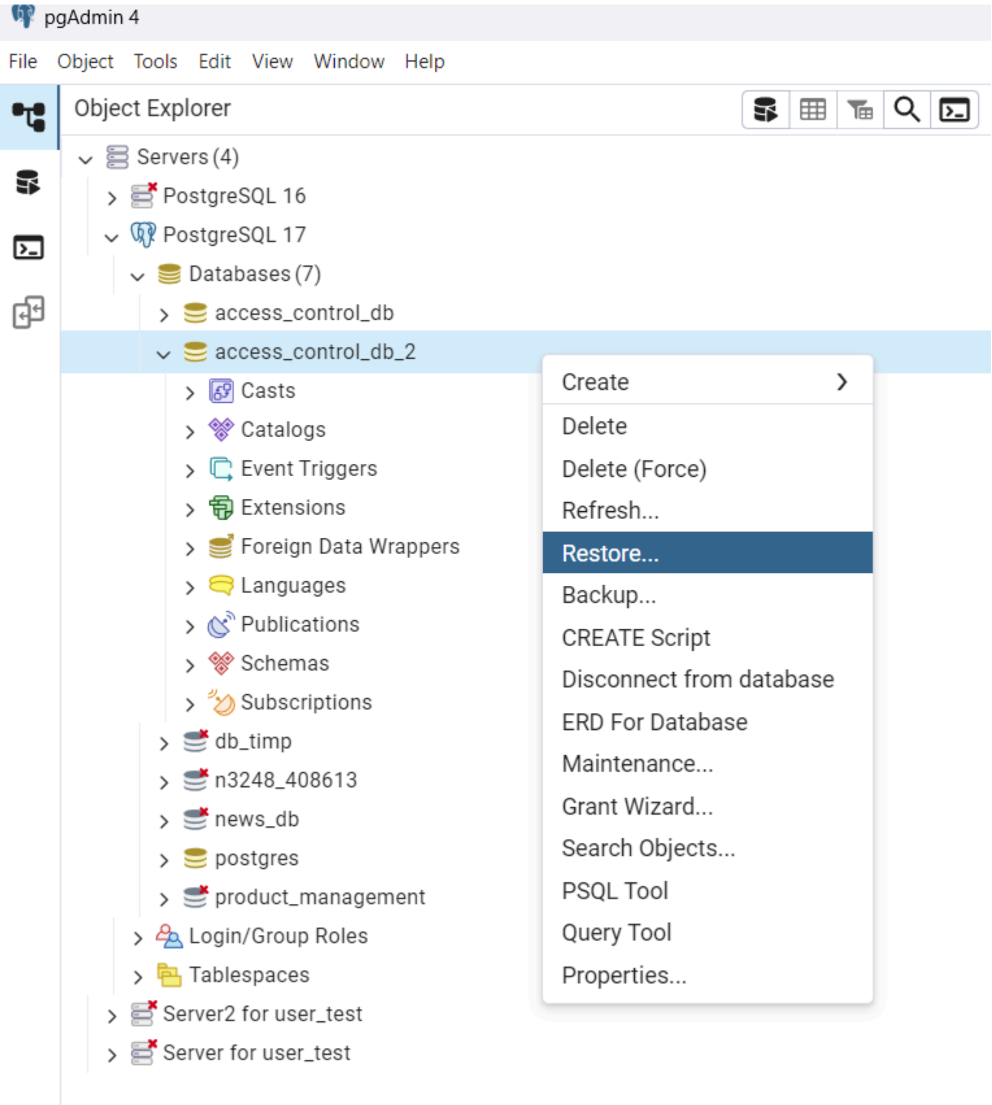
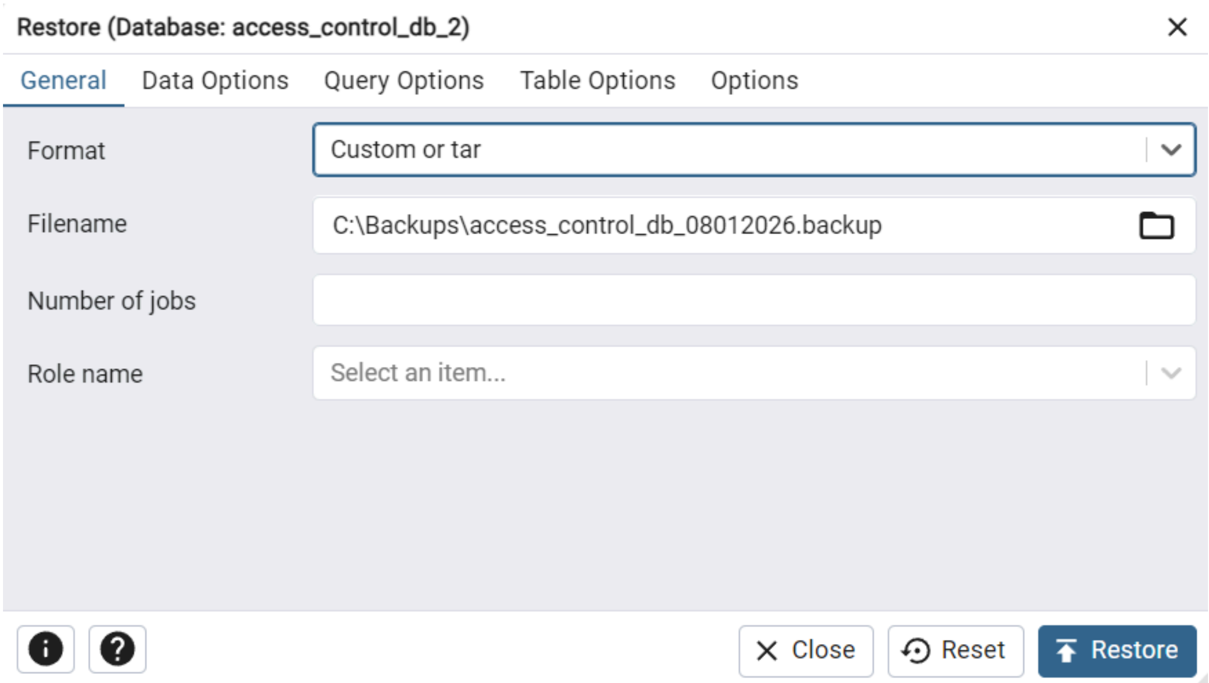

pgAdmin 4
File
Object
Tools
Edit
View
Window
Help
Object Explorer
Servers (4)
PostgreSQL 16
PostgreSQL 17
Databases (7)
access_control_db
access_control_db_2
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas
Subscriptions
db_timp
n3248_408613
news_db
postgres
product_management
Login/Group Roles
Tablespaces
Server2 for user_test
Server for user_test
Create
Delete
Delete (Force)
Refresh...
Restore...
Backup...
CREATE Script
Disconnect from database
ERD For Database
Maintenance...
Grant Wizard...
Search Objects...
PSQL Tool
Query Tool
Properties...

Рисунок 173 – Кнопка восстановления базы данных


Restore (Database: access_control_db_2)

General
Data Options
Query Options
Table Options
Options

Format: Custom or tar

Filename: C:\Backups\access_control_db_08012026.backup

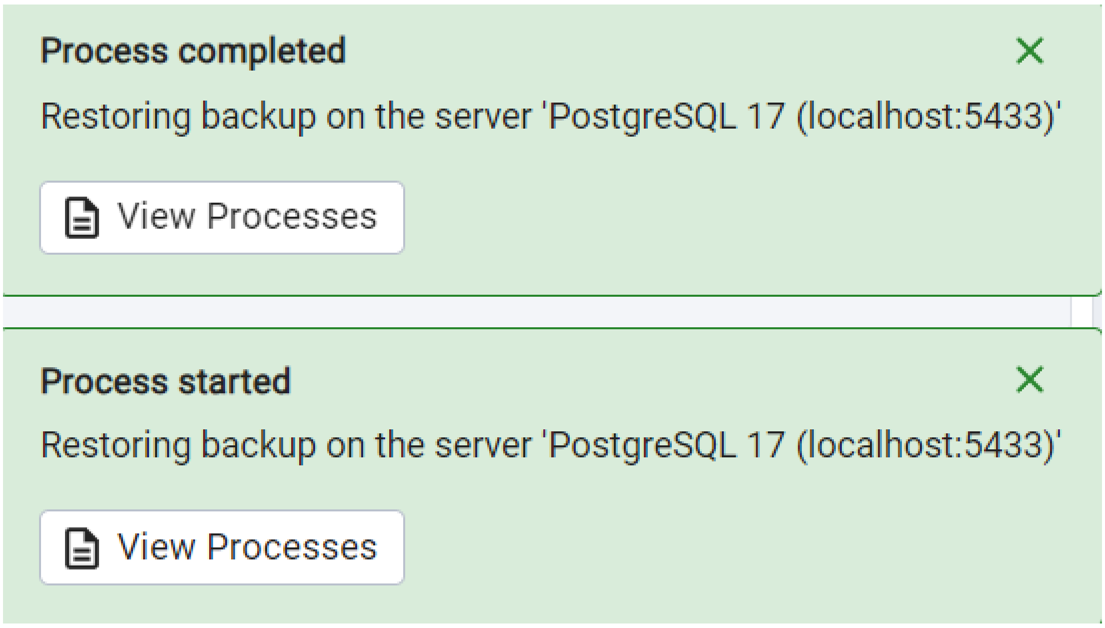
Number of jobs:

Role name: Select an item...

Close
Reset
Restore

Рисунок 174 – Выбор восстанавливаемой БД

Резервная копия успешно восстановлена (рисунок 175).


Process completed

Restoring backup on the server 'PostgreSQL 17 (localhost:5433)'

View Processes

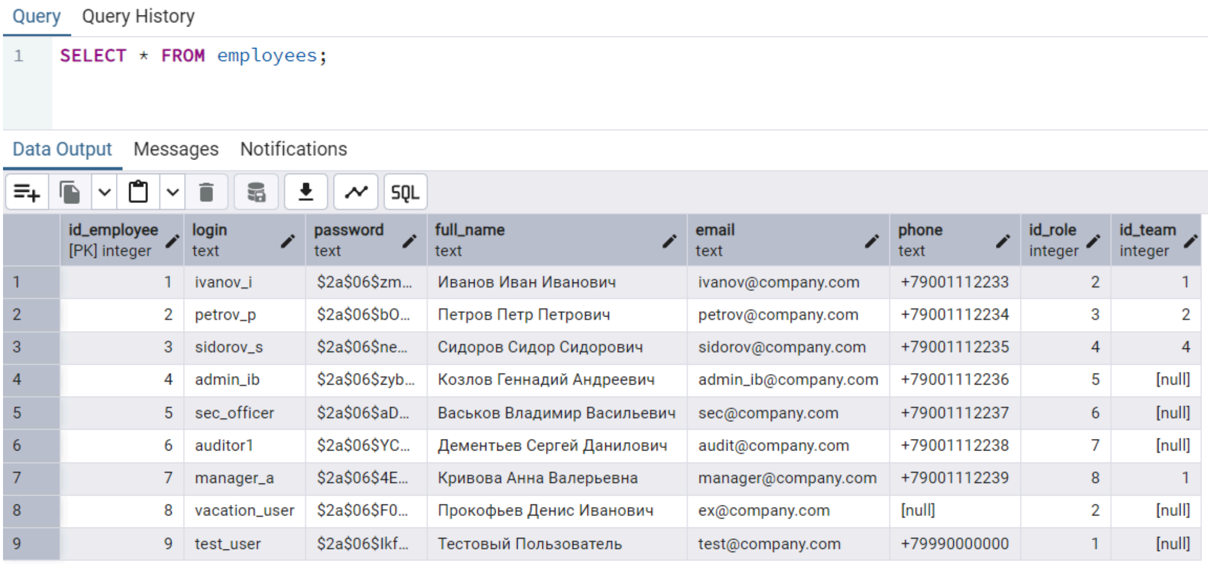
Process started

Restoring backup on the server 'PostgreSQL 17 (localhost:5433)'

View Processes

Рисунок 175 – Успешное восстановление БД

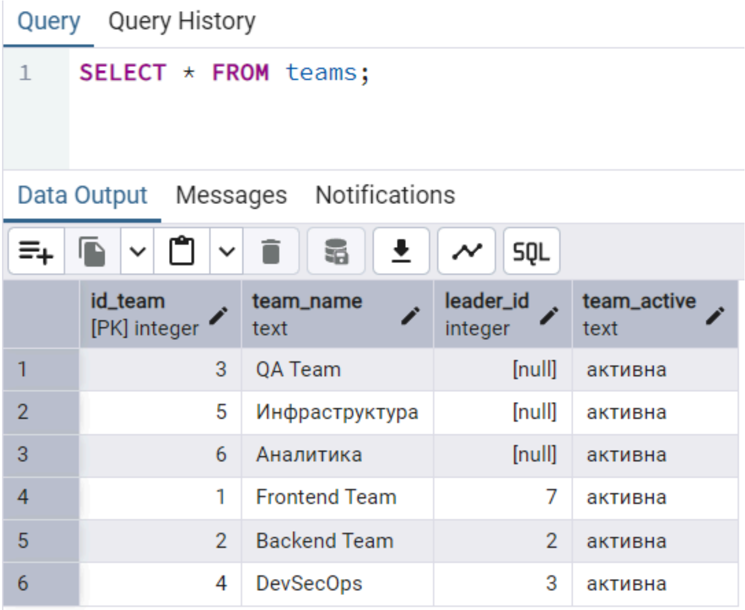
Проверим содержимое таблиц, в которые вносились изменения. Проверка представлена на рисункеах176 и 177.



```sql
SELECT * FROM employees;
```

	id_employee [PK] integer	login text	password text	full_name text	email text	phone text	id_role integer	id_team integer
1	1	ivanov_i	$2a$06$zm...	Иванов Иван Иванович	ivanov@company.com	+79001112233	2	1
2	2	petrov_p	$2a$06$bO...	Петров Петр Петрович	petrov@company.com	+79001112234	3	2
3	3	sidorov_s	$2a$06$ne...	Сидоров Сидор Сидорович	sidorov@company.com	+79001112235	4	4
4	4	admin_ib	$2a$06$zyb...	Козлов Геннадий Андреевич	admin_ib@company.com	+79001112236	5	[null]
5	5	sec_officer	$2a$06$aD...	Васьков Владимир Васильевич	sec@company.com	+79001112237	6	[null]
6	6	auditor1	$2a$06$YC...	Дементьев Сергей Данилович	audit@company.com	+79001112238	7	[null]
7	7	manager_a	$2a$06$4E...	Кривова Анна Валерьевна	manager@company.com	+79001112239	8	1
8	8	vacation_user	$2a$06$F0...	Прокофьев Денис Иванович	ex@company.com	[null]	2	[null]
9	9	test_user	$2a$06$lkf...	Тестовый Пользователь	test@company.com	+79990000000	1	[null]

Рисунок 176 – Состояние таблицы employees



```sql
SELECT * FROM teams;
```

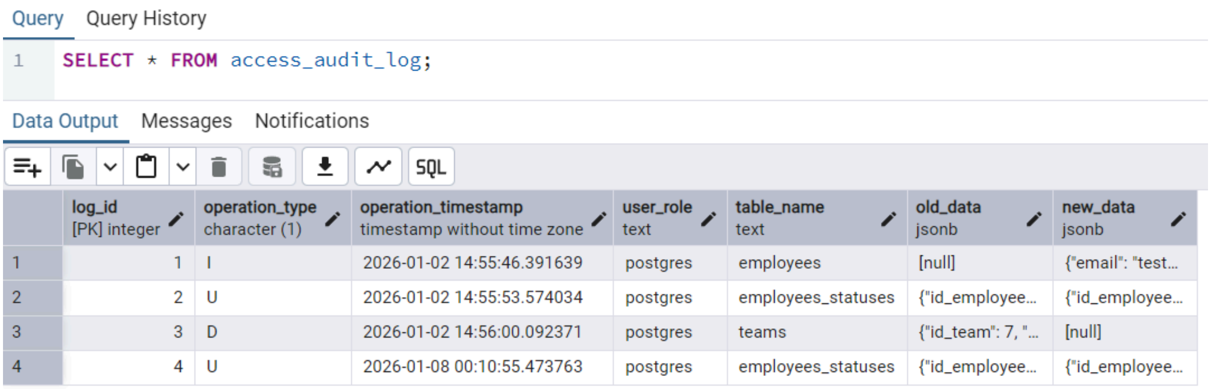
	id_team [PK] integer	team_name text	leader_id integer	team_active text
1	3	QA Team	[null]	активна
2	5	Инфраструктура	[null]	активна
3	6	Аналитика	[null]	активна
4	1	Frontend Team	7	активна
5	2	Backend Team	2	активна
6	4	DevSecOps	3	активна

Рисунок 177 – Состояние таблицы teams

Успешно восстановили зарезервированную БД.

5.4 Анализ откатанных изменений через логирование

Используем таблицу *access_audit_log*, созданную на третьем этапе работы, для анализа изменений. Результат представлен на рисунке 178.



```
SELECT * FROM access_audit_log;
```

	log_id [PK] integer	operation_type character (1)	operation_timestamp timestamp without time zone	user_role text	table_name text	old_data jsonb	new_data jsonb
1	1	I	2026-01-02 14:55:46.391639	postgres	employees	[null]	{"email": "test...
2	2	U	2026-01-02 14:55:53.574034	postgres	employees_statuses	{"id_employee...	{"id_employee...
3	3	D	2026-01-02 14:56:00.092371	postgres	teams	{"id_team": 7, "...	[null]
4	4	U	2026-01-08 00:10:55.473763	postgres	employees_statuses	{"id_employee...	{"id_employee...

Рисунок 178 – Состояние таблицы access_audit_log

Ожидаемо в таблице access_audit_log нету записей о том, что происходило после создания резервной копии, а также о том, что было «откачено» при восстановлении, потому что резервная копия хранит и восстанавливает все данные всех внутренних таблиц. Это является недостатком такой примитивной системы логирования.

Для производственного использования необходимо дополнить систему расширением pgAudit или реализовать двухуровневое логирование (СУБД + приложение) для обеспечения полной трассируемости всех операций с данными.

5.5 Вывод

В ходе лабораторной работы успешно реализована система резервного копирования и восстановления базы данных. Создано автоматическое расписание ежедневных бэкапов с ротацией хранения, продемонстрирован процесс отката изменений через механизм транзакций PostgreSQL. Работа подтвердила важность комплексного подхода к резервированию, включающего не только создание бэкапов, но и мониторинг восстановительных процедур.

ЗАКЛЮЧЕНИЕ

Проведенная работа полностью достигла поставленной цели. Созданный прототип системы является не просто набором отдельных компонентов, а целостным, интегрированным решением, в котором глубоко переплетены вопросы функциональности, удобства использования и, что наиболее важно, информационной безопасности. Принцип Security by Design был последовательно применен на всех этапах: от выбора атрибутов сущностей и проектирования схемы до реализации шифрования, аудита и ролевого доступа как в СУБД, так и в коде приложения.

Практическая значимость работы заключается в демонстрации эффективной методологии построения безопасных информационных систем, основанной на синергии возможностей современной СУБД (PostgreSQL) и прогрессивного фреймворка для разработки back-end (FastAPI). Система обладает высокой степенью масштабируемости и адаптируемости: ролевая модель и представления легко расширяются, архитектура приложения допускает добавление новых модулей, а механизмы аудита и резервного копирования готовы к работе в реальных условиях.

Направления дальнейшего развития системы могут включать:

- внедрение более сложных политик динамического контроля доступа;
- интеграцию с корпоративными системами для автоматического обновления статусов и состава команд;
- развитие панели мониторинга в реальном времени с алертами о подозрительной активности;
- автоматизацию и планирование задач резервного копирования.

Таким образом, в рамках курсовой работы не только были успешно закреплены теоретические знания по информационной безопасности баз данных, но и получен ценный практический опыт создания полноценного, жизнеспособного программного продукта, отвечающего современным требованиям к защите информации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы технологий баз данных: учебное пособие / Б. А. Новиков, Е. А. Горшкова, Н. Г. Графеева; под ред. Е. В. Рогова. — 2-е изд. — М.: ДМК Пресс, 2020. — 582 с.
2. Заколдаев Д.А., Кармановский Н.С., Комаров И.И., Филькова И.А. Методические указания по выполнению научно-исследовательской и выпускной квалификационной работы магистрантов: Учебно-методическое пособие. — Санкт-Петербург: Университет ИТМО, 2024. — 41 с. — экз. Режим доступа: https://books.ifmo.ru/book/2728/metodicheskie_ukazaniya_po_vypolneniyu_nauchno-issledovatelskoy_i_vypusknoy_kvalifikacionnoy_raboty_magistrantov_:_uchebno-metodicheskoe_posobie..htm.